



Lyrene Fernandes da Silva

**Uma Estratégia Orientada a Aspectos
para Modelagem de Requisitos**

Tese de Doutorado

Tese apresentada ao Programa de Pós-Graduação em
Informática do Departamento de Informática da PUC-Rio
como parte dos requisitos parciais para obtenção do título
de Doutor em Ciências.

Orientador: Julio Cesar Sampaio do Prado Leite

Rio de Janeiro,
Março de 2006

Lyrene Fernandes da Silva

**Uma Estratégia Orientada a Aspectos
para Modelagem de Requisitos**

Tese apresentada como requisito parcial para obtenção do grau de Doutor pelo Programa de Pós-graduação em Informática do Departamento de Informática do Centro Técnico Científico da PUC-Rio. Aprovada pela Comissão Examinadora abaixo assinada.

Prof. Julio Cesar Sampaio do Prado Leite

Orientador

Departamento de Informática – PUC-Rio

Prof. Carlos José Pereira de Lucena

Departamento de Informática – PUC-Rio

Prof. Karin Koogan Breitman

Departamento de Informática – PUC-Rio

Prof. Cláudia Werner

COPPE – UFRJ

Prof. Alessandro Fabrício Garcia

Departamento de Computação - Lancaster University

Prof. José Eugenio Leal

Coordenador Setorial do Centro Técnico Científico

Rio de Janeiro, 30 de Março de 2006

Todos os direitos reservados. É proibida a reprodução total ou parcial do trabalho sem autorização da universidade, do autor e do orientador.

Lyrene Fernandes da Silva

Graduou-se em Ciência da Computação na Universidade Federal do Rio Grande do Norte (UFRN) em 1999. Obteve o título de Mestre em Sistemas e Computação na UFRN em 2002. Integrou o quadro de pesquisadores do Laboratório de Engenharia de Software (LES) da PUC-Rio, atuando na área de Engenharia de Software.

Ficha Catalográfica

Silva, Lyrene Fernandes da

Uma estratégia orientada a aspectos para modelagem de requisitos / Lyrene Fernandes da Silva ; orientador: Julio Cesar Sampaio do Prado Leite. Rio de Janeiro : PUC, Departamento de Informática, 2006.

222 f. ; 30 cm

Tese (Doutorado) – Pontifícia Universidade Católica do Rio de Janeiro, Departamento de Informática.

Inclui referências bibliográficas.

1. Informática – Teses. 2. Modelagem de requisitos. 3. Modelo de metas. 4. Early-aspects. 5. Características transversais. 6. Aspectos. 7. Separação e composição de características transversais. 8. Visões. 9. Integração de visões I. Leite, Julio Cesar Sampaio do Prado. II. Pontifícia Universidade Católica do Rio de Janeiro. Departamento de Informática. III. Título.

CDD: 004

À minha família que, mesmo distante,
sempre esteve presente.

Agradecimentos

A Deus, por todas as oportunidades de aprendizado, nas derrotas e nas vitórias.

Ao meu orientador, Professor Julio Cesar Sampaio do Prado Leite pelo conhecimento ensinado e paciência, estando atento às minhas dificuldades e guiando-me quando necessário. Bem como por ter confiado em meu trabalho e ter me dado muitas oportunidades.

À minha família por ser a minha fortaleza e em especial aos meus sobrinhos Luis Eduardo e Filipe, que nasceram durante esta minha fase e mesmo me conhecendo tão pouco, assim como eu a eles, me fizeram inúmeras vezes respirar fundo e continuar esta jornada.

A Leandro Nantes por sua imensa paciência, carinho e atenção.

Às sinceras amizades que fiz durante estes quatro anos de intensivo trabalho. Em especial, por ordem alfabética, a: Adele Malta, Antônio Pádua, Bruno Silvestre, Carolina Felicíssimo, Cláudio Santana, Karin Breitman, Leonardo Matriciano, Miriam Sayão, Renato Maia, Roberta Coelho, Silvana Rossetto, Vagner Sacramento, Valéria Quadros, dentre outros, que mesmo sem saber me fortaleceram nos momentos de tristeza e saudade, sendo diariamente meus companheiros de trabalho e/ou diversão.

Aos funcionários do DI. Em especial a Débora, Emanuelle e Ruth que sempre se mostraram prestativas em esclarecer informações e ajudar com todos os protocolos burocráticos do doutorado e a Zé Carlos e Anderson pelo apoio técnico.

À CAPES e ao CNPq pelos financiamentos concedidos, sem os quais este trabalho não poderia ter sido realizado.

Resumo

Silva, Lyrene Fernandes; Leite, Julio Cesar Sampaio do Prado. Uma Estratégia Orientada a Aspectos para Modelagem de Requisitos. Rio de Janeiro, 2006. 222p. Tese de Doutorado - Departamento de Informática, Pontifícia Universidade Católica do Rio de Janeiro.

Devido ao tamanho e complexidade dos sistemas de software, torna-se cada vez mais difícil manipular os modelos que os representam. É difícil identificar o impacto de mudanças, modificar, ou reutilizar partes da modelagem em outros projetos porque as características do sistema são, na maioria das vezes, muito relacionadas umas às outras. Métodos tradicionais de modelagem e programação utilizam métodos de separação para minimizar este acoplamento, porém eles priorizam apenas uma dimensão das características do sistema (classes, funções, eventos, dentre outras). Características em outras dimensões permanecem espalhadas e entrelaçadas, i.e, elas são características transversais. Nesta tese, propomos que a separação considerando diferentes dimensões, abordado pela programação orientada a aspectos, seja realizada durante a definição de requisitos. Definimos um metamodelo para integração de características transversais que provê um conjunto de atividades e mecanismos para facilitar a modularização, rastreabilidade, modificação e reuso de requisitos. Estes mecanismos são centrados em uma linguagem que define um novo construto para ser utilizado em linguagens de requisitos tradicionais, possibilitando a descrição e composição de características transversais. Mostramos como aplicar este metamodelo utilizando o V-graph, um tipo de modelo de metas, e realizando dois estudos de caso.

Palavras-chave

Modelagem de Requisitos, modelo de metas, *early-aspects*, características transversais, aspectos, separação e composição de características transversais, visões, integração de visões.

Abstract

Silva, Lyrene Fernandes; Leite, Julio Cesar Sampaio do Prado. An Aspect-Oriented Requirements Modeling Strategy. Rio de Janeiro, 2006. 222p. Tese de Doutorado - Departamento de Informática, Pontifícia Universidade Católica do Rio de Janeiro.

Due to the increasing software complexity, it has been difficult to manipulate the models that represent them. It is difficult to identify the impact of changes, to modify, or reuse model parts in other projects because the system concerns are, in general, strongly interconnected. Traditional modeling and programming methods use separation method to minimize this coupling, however they consider only one dimension (classes, functions, events). Therefore, the concerns in other dimensions continue scattered and tangled, i.e., they are crosscutting concerns. In this thesis, we applied aspect-oriented concepts in modeling requirements. We define a metamodel to integrate crosscutting concerns that provides a set of activities and mechanisms to facilitate the requirements modularization, traceability, evolution and reuse. These mechanisms are centered in a language that defines new constructs to be used in traditional requirements languages, allowing the separation and composition of crosscutting concerns. We present two detailed case studies to show the proposed metamodel instantiated to a goal oriented requirements model, V-graph.

Keywords

Requirements modeling, goal models, early-aspect, crosscutting concerns, aspects, separation and composition of crosscutting concerns, views, integration of views.

Sumário

1 Introdução	17
1.1. Problema	20
1.2. Limitações dos Trabalhos Relacionados	21
1.3. Solução Proposta	22
1.4. Contribuições da Tese	24
1.5. Organização da Tese	24
2 Engenharia de Requisitos	26
2.1. Processo	27
2.2. Modelagem	28
2.3. Visões	37
2.4. Evolução	40
2.5. Rastreabilidade	41
2.6. Reuso	42
2.7. Resumo	43
3 Desenvolvimento de Software Orientado a Aspectos	44
3.1. Propriedades da Separação Avançada de Características	45
3.2. AspectJ	47
3.3. Hyper/J	48
3.4. Engenharia de Requisitos Orientada a Aspectos	52
3.4.1. Rashid, Moreira e Brito	52
3.4.2. Baniassad	57
3.4.3. Sousa	58
3.5. Resumo	61
4 Estratégia Orientada a Aspectos para Modelagem de Requisitos	63
4.1. Metamodelo para Integração de Características Transversais	64
4.2. Linguagem de Modelagem de Requisitos Orientada a Aspectos	69
4.2.1. Modelo de Componentes	70

4.2.2. Modelo de <i>Joinpoints</i>	71
4.2.3. Modelo Núcleo: Relacionamento Transversal	71
4.2.4. Diretrizes para Modelagem de Características Separadamente	75
4.3. Utilização da Estratégia – O V-Graph como Modelo de Componentes	76
4.3.1. Separação de Características Transversais	76
4.3.2. Composição de Características Transversais	83
4.3.3. Visualização de Características Transversais	86
4.3.4. Implementação	97
4.4. Aplicação da Estratégia a outros Modelos de Requisitos	101
4.4.1. Cenários	101
4.4.2. Léxico Ampliado da Linguagem (LAL)	104
4.4.3. Sentenças de Requisitos	106
4.5. Processo de Engenharia de Requisitos Utilizando a Estratégia Orientada a Aspectos	108
4.6. Resumo	110
 5 Estudos de Caso	 112
5.1. Contextualização	112
5.2. Sistema Colaborativo para Edição de Léxico e Cenários – C&L	115
5.2.1. Modelagem	115
5.2.2. Análise dos Resultados	129
5.3. Sistema para Divulgação de Eventos – Unical	131
5.3.1. Modelagem	131
5.3.2. Análise dos Resultados	146
5.4. Resumo	149
 6 Conclusão	 151
6.1. Resumo	151
6.2. Contribuições	152
6.3. Comparação com Trabalhos Relacionados	154
6.4. Limitações e Trabalhos Futuros	158
 7 Referências Bibliográficas	 161

Apêndice A Gramática da Linguagem Orientada a Aspectos para Modelagem de Requisitos	168
Apêndice B Implementação dos Mecanismos de Composição e Visualização	171
Apêndice C Especificação do Estudo de Caso – C&L	191
Apêndice D Especificação do Estudo de Caso – Unical	201
Anexo A Documento de Requisitos do Sistema Unical	216

Lista de Figuras

Figura 1. Problemas de espalhamento e entrelaçamento em abordagens (a) funcionais e (b) orientadas a objetos	18
Figura 2. Metamodelo de integração de características transversais	23
Figura 3. Processo de engenharia de requisitos	27
Figura 4. Relação entre requisitos e características	28
Figura 5. Exemplos de templates para documentos de requisitos	29
Figura 6. Espalhamento e entrelaçamento em listas de requisitos – 1º exemplo	31
Figura 7. Espalhamento e entrelaçamento em listas de requisitos – 2º exemplo	32
Figura 8. O modelo de metas <i>V-graph</i> : (a) representação original definida em (Yu, 2004); (b) nossa representação, explicitando todos os relacionamentos possíveis entre os elementos do <i>V-graph</i> .	35
Figura 9. Exemplos de (a) decomposição, (b) contribuição e (c) correlação.	36
Figura 10. Framework de integração de métodos	38
Figura 11. Exemplo de template utilizado para catalogar RNFs	42
Figura 12. a) BNF da linguagem Q7 b) Exemplo utilizando Q7	42
Figura 13. Relação dos atributos de Q7 e os elementos do <i>V-graph</i>	43
Figura 14. Visão geral dos trabalhos sobre desenvolvimento orientado a aspectos	44
Figura 15. Modelo de aspectos	45
Figura 16. Exemplo de aspecto em AspectJ	48
Figura 17. Exemplo de Hyperslice	49
Figura 18. Exemplo de hiper-módulo em Hyper/J	51
Figura 19. Modelo EROA	53
Figura 20. Exemplo de descrição de requisitos no modelo EROA	54
Figura 21. a) Template para RNFs, b) exemplo utilizando o template	55
Figura 22. Exemplo de descrição de requisitos na abordagem multidimensional	56
Figura 23. Exemplo da visão de ações de Theme/Doc	57
Figura 24. Exemplo da visão de ações agrupadas de Theme/Doc	57
Figura 25. Exemplo da visão de temas de Theme/Doc	58
Figura 26. Adaptação do framework NFR	59
Figura 27. Exemplo da modelagem de candidato a aspectos no grafo de RNFs	59

Figura 28. Adaptação do processo dirigido por caso de uso	60
Figura 29. Exemplo de modelagem de candidato a aspectos em casos de uso	60
Figura 30. Metamodelo para integração de características transversais	64
Figura 31. Componentes utilizados para integração de características transversais	65
Figura 32. Analogia entre a estratégia para integração de características transversais durante a definição de requisitos e algumas práticas utilizadas no nível de implementação	66
Figura 33. Exemplo ilustrativo da abordagem tradicional	67
Figura 34. Exemplo ilustrativo - Separação	68
Figura 35. Exemplo ilustrativo - Visualização	68
Figura 36. Sintaxe de LMROA	69
Figura 37 – Modelo conceitual da linguagem de modelagem de requisitos orientada a aspectos	70
Figura 38. Sintaxe do relacionamento transversal	72
Figura 39. Exemplo de <i>pointcut</i>	73
Figura 40. Exemplo de <i>advice</i>	74
Figura 41. Exemplo de <i>intertype declaration</i>	74
Figura 42. Instanciação de LMROA: AOV-graph	77
Figura 43. Sintaxe do <i>V-graph</i>	77
Figura 44. Exemplo do modelo V-graph a) representação gráfica e b) representação em textual	78
Figura 45. Definição dos <i>joinpoints</i> no caso do V-graph como modelo de componentes	78
Figura 46. Exemplo de <i>pointcuts</i>	79
Figura 47. Semântica dos tipos de <i>advice</i> a) before, b) around e c) after	80
Figura 48. Exemplo de <i>advice</i>	80
Figura 49. Exemplo de <i>intertype declarations</i>	81
Figura 50. Exemplo da separação de modelos	81
Figura 51. Exemplo de composição	85
Figura 52. Exemplo base para extração de visões	86
Figura 53. Mecanismo de visualização	87
Figura 54. Modelo de <i>features</i> para configuração de visões de AOV-graph	87
Figura 55. Exemplo de visão com os relacionamentos transversais resumidos e sem o relacionamento de decomposição	88
Figura 56. Exemplo de visão com o relacionamento transversal expandido	88
Figura 57. Exemplo de visão apenas com os elementos dos tipos meta e tarefa	89

Figura 58. Exemplo de visão de uma instância de <i>softmeta</i> , meta, tarefa, tipo ou tópico	90
Figura 59. Exemplo da visão cenário para os grafos Model [requirements] e Edit [scenario]	93
Figura 60. Exemplo da visão matriz de rastreabilidade	94
Figura 61. Exemplo da visão entidade-relacionamento para o grafo Model [requirements model]	95
Figura 62. Exemplo da visão de objetos	96
Figura 63. Implementação da estratégia para integração de características transversais	97
Figura 64. DTD referente à AOV-graph	99
Figura 65. Componentes do mecanismo de composição	99
Figura 66. Componentes do mecanismo de visualização	100
Figura 67. Exemplo de espalhamento e entrelaçamento no modelo de cenários	102
Figura 68. Sintaxe do modelo de componentes: Cenários	102
Figura 69. Exemplo do uso de nossa abordagem no modelo de cenários	104
Figura 70. Exemplo de espalhamento e entrelaçamento no LAL	105
Figura 71. Sintaxe do modelo de componentes: LAL	105
Figura 72. Exemplo do uso de nossa abordagem no Léxico	106
Figura 73. Sintaxe do modelo de componentes: sentenças de requisitos	107
Figura 74. Exemplo do uso de nossa abordagem em sentenças de requisitos	108
Figura 75. Processo de engenharia de requisitos modificado pela estratégia de integração de características transversais	109
Figura 76. Legenda dos elementos de V-graph e AOV-graph	114
Figura 77. Modelagem de C&L em V-graph	116
Figura 78. Modelagem de C&L em AOV-graph	117
Figura 79. Modelo CEL	119
Figura 80. Relacionamento transversal do modelo CEL	119
Figura 81. Modelo CEL após a composição	120
Figura 82. Modelo de segurança	121
Figura 83. Relacionamentos transversais do modelo de segurança	121
Figura 84. Modelo de segurança após a composição	122
Figura 85. Modelo de persistência	123
Figura 86. Relacionamentos transversais do modelo de persistência	123
Figura 87. Modelo de persistência após a composição	123
Figura 88. Modelo de confiabilidade	124
Figura 89. Relacionamentos transversais do modelo de confiabilidade	124

Figura 90. Modelo de confiabilidade após a composição	125
Figura 91. AOV-graph do sistema C&L com os relacionamentos transversais expandidos	127
Figura 92. Diagrama de classes do modelo CEL	128
Figura 93. Visão geral do Unical e seus RNFs	132
Figura 94. V-graph do sistema Unical	133
Figura 95. AOV-graph do sistema Unical	134
Figura 96. Modelo Unical – divulgação de eventos	136
Figura 97. MER para o modelo Unical	136
Figura 98. a) Modelo de performance e b) relacionamento transversal do modelo de performance	137
Figura 99. Visão de cenários do modelo Unical	138
Figura 100. Modelo de usabilidade	138
Figura 101. Interação do modelo de usabilidade com o modelo Unical	139
Figura 102. Relacionamentos transversais do modelo de Usabilidade	140
Figura 103. Modelo Unical após a composição	141
Figura 104. Modelo de usabilidade após a composição	142
Figura 105. Modelo de segurança	143
Figura 106. Relacionamentos transversais do modelo de segurança para o Unical	143
Figura 107. Relacionamentos transversais do modelo de persistência e confiabilidade	144
Figura 108. Relacionamento transversal do modelo de cenários	145
Figura 109. Visão dos interessados do sistema	145
Figura 110. AOV-graph do sistema Unical com os relacionamentos transversais expandidos	148

Lista de Tabelas

Tabela 1. Avaliação de AspectJ e Hyper/J utilizando o modelo de aspecto	47
Tabela 2. Regras de composição	85
Tabela 3. Regras de transformação entre AOV-graph e Cenários	92
Tabela 4. Regras de transformação entre AOV-graph e uma matriz de rastreabilidade	93
Tabela 5. Regras de transformação entre AOV-graph e o modelo de entidade-relacionamento	95
Tabela 6. Regras de transformação entre AOV-graph e o modelo de objetos	96
Tabela 7. Estatísticas do estudo de caso – C&L	129
Tabela 8. Matriz de rastreabilidade - relacionamento transversal do modelo de confiabilidade	130
Tabela 9. Estatísticas do estudo de caso – Unical	146
Tabela 10. Trabalhos relacionados	154
Tabela 11. Comparação de linguagens para integração de características transversais	156

Lista de Abreviaturas

DSOA – Desenvolvimento de Software Orientado a Aspectos

DTD – Document Type Definition

EROA – Engenharia de Requisitos Orientada a Aspectos

HTML – HyperText Markup Language

IEEE – Institute of Electrical and Electronics Engineers

LMROA – Linguagem de Modelagem de Requisitos Orientada a Aspectos

POA – Programação Orientada a Aspectos

PUC-RIO – Pontifícia Universidade Católica do Rio de Janeiro

RF – Requisito funcional

RNF – Requisito não-funcional

UdI – Universo de Informação

URL – Uniform Resource Locator

Web – World Wide Web

XML – Extensible Markup Language

XSL – Extensible Stylesheet Language

XSLT - Extensible Stylesheet Language Transformation

1

Introdução

O princípio de separação de *concerns* permeia todo o processo de desenvolvimento de software. Ele cuida das limitações humanas em tratar a complexidade, permitindo que o desenvolvedor se concentre em um assunto ou característica do sistema por vez (Dijkstra, 1976), e desta forma melhora sua compreensão sobre o sistema e aumenta a capacidade para evolução e reuso.

Não há uma clara definição para *concern*. Algumas definições possíveis são: “é uma propriedade observável desejada”, “é uma *feature*”, “é alguma responsabilidade”, “é um subproblema” (Tekinerdođan, 2004). Neste trabalho, chamamos *concerns* de características¹. Elas podem ser qualquer característica (instâncias de: requisito, dado, função, restrição, objeto, custo, dentre outras) do sistema ou do domínio, que vale a pena analisar isoladamente ou em conjunto com outras.

Em Engenharia de Software, o princípio de separação de características está relacionado à decomposição e modularização (Parnas, 1972); sistemas de software complexos devem ser decompostos em unidades modulares menores, cada uma tratando uma única característica. Como definido pela programação modular, independentemente do método utilizado, um módulo é uma unidade compilável e integrável para formar o programa, e possui as seguintes propriedades (Staa, 2000):

- encapsulamento – o módulo deve tornar invisível para os módulos clientes sua implementação interna, disponibilizando seus serviços apenas por sua interface;

¹ Em (Piveta, 2004) a tradução dada para “*concerns*” é “interesses”. Nós não adotamos esta tradução porque consideramos que “interesses” em engenharia de requisitos têm uma conotação de que há alguém interessado, podendo, algumas vezes, ser confundida com “pontos de vista”. Neste trabalho, queremos ter uma conotação mais ampla e independente de haver ou não um interessado, daí escolhemos a palavra “característica” e utilizamos o termo “características transversais” como tradução para “*crosscutting concerns*”

- acoplamento – o módulo deve ter menor acoplamento com os demais, i.e., menor dependência; e
- coesão – os elementos que constituem um módulo devem ter um forte inter-relacionamento, devendo relacionar-se a um único conceito ou característica.

As propriedades de encapsulamento, acoplamento e coesão são essenciais para gerenciar a complexidade de partes do sistema e dele como um todo. Entretanto, cada método de modelagem e programação utilizado no desenvolvimento impõe uma maneira dominante de decomposição e modularização, explicitando algumas características e ocultando outras, igualmente importantes, tais como objetos, funções, tempo, dados, dentre outras (Tarr, 1999).

Na Figura 1, Chavez (2004) ilustra os problemas de espalhamento e entrelaçamento causados pelas abordagens procedural e orientada a objetos. Na Figura 1a, o mesmo dado pode ser utilizado em diferentes funções (procedimentos), e cada função trata de diferentes dados. Na Figura 1b, o mesmo comportamento (método) é desempenhado em diferentes componentes e cada componente desempenha comportamentos diferentes.

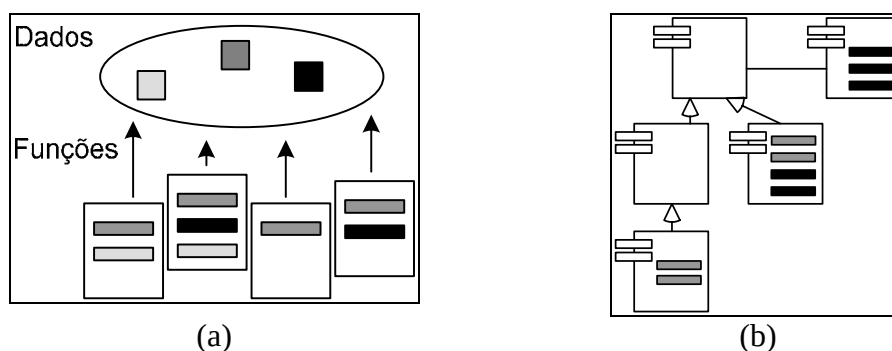


Figura 1. Problemas de espalhamento e entrelaçamento em abordagens (a) funcionais e (b) orientadas a objetos

Tarr (1999) argumenta que a decomposição dominante, seja funcional ou orientada a objetos, influencia negativamente a rastreabilidade, evolução e reuso, e argumenta sobre a importância de termos abordagens que tratem, simultaneamente, da separação de características em múltiplas dimensões. Desta forma, pode-se abordar os problemas de espalhamento e entrelaçamento que são ocasionados pela utilização de modelos parciais de decomposição (objetos, funções, dentre outros).

Além disto, um sistema é naturalmente composto de diferentes

características e entendê-las em conjunto é igualmente importante. Se por um lado precisamos separar as características, por outro precisamos compô-las. Muitas vezes estas características são fortemente relacionadas, entrelaçadas e/ou sobrepostas, influenciando ou restringindo umas às outras, sendo chamadas de **características transversais** (Tekinerdođan, 2004). Isto torna difícil a separação e análise das partes do sistema bem como a análise do impacto que umas exercem sobre as outras.

Para melhorar a separação e composição de características transversais, nos últimos anos, houve um investimento no paradigma de orientação a aspectos (Kiczales, 1997). Linguagens de programação orientadas a aspectos dão suporte à composição de características separadas e encapsuladas em “aspectos”², tornando a tarefa de composição transparente para o programador. Entretanto, tratar características transversais somente no nível de implementação não é suficiente porque muitas decisões de planejamento e de desenho³ já foram tomadas sem levar em consideração esta natureza transversal (Nuseibeh, 2004). Além disso, muitas vezes já não se tem mais o rastro entre os “aspectos” e o que deu origem a eles, dificultando o entendimento de onde mudanças em requisitos afetam.

Assim, têm surgido linguagens e métodos de modelagem considerando estes novos elementos e relacionamentos de desenho, permitindo a separação e composição de características transversais em um nível mais alto de abstração (Chavez, 2004). Identificar, separar e compor características transversais de alto nível de abstração provê: melhor entendimento do problema e da solução, rastreabilidade entre os diversos modelos construídos durante o desenvolvimento e entre as diversas características, reusabilidade de soluções de maior granularidade, e diminuição da distância entre as fases de desenvolvimento (Rashid, 2004; Baniassad, 2004; Brito, 2004).

Além disto, os fundamentos de separação e composição de características transversais inspiram o desenvolvimento de ferramentas aptas a extrair diferentes características de modelos projetados em qualquer linguagem de programação ou de desenho. Neste sentido, podemos prover, de maneira menos intrusiva, os

² Neste Capítulo, utilizamos o termo “aspecto” entre aspas, num sentido geral, referindo elementos utilizados em linguagens de programação para modularizar características transversais.

³ Nesta tese, utilizamos o termo “desenho” como tradução para o termo “design”, no sentido da atividade de desenvolvimento que resulta no projeto arquitetural e detalhado.

benefícios da separação e composição de características aos métodos e técnicas já utilizados para desenvolvimento de software. Desta forma, a separação e composição sugerem que podemos decompor o sistema utilizando abordagens diferentes ou que podemos decompor utilizando uma única abordagem, mas estamos aptos a visualizar e manipular outras formas de decomposição.

Esta tese tem por objetivo prover mecanismos de separação, composição e visualização de características transversais para melhorar o entendimento, manutenibilidade e reusabilidade de modelos de requisitos. Na Seção 1.1 explicamos o problema a ser tratado nesta tese. Na Seção 1.2 apresentamos as limitações das abordagens existentes. Na Seção 1.3 resumimos a solução proposta nesta tese. Na Seção 1.4 citamos as contribuições oferecidas. Na Seção 1.5 descrevemos como o restante desta tese está organizado.

1.1. Problema

Em documentos de requisitos, seja considerando um conjunto de modelos ou considerando apenas um tipo de modelo de requisitos, percebemos o espalhamento e entrelaçamento de muitas características. Na maioria das vezes, estas características são classificadas e separadas (decompostas) de maneiras diferentes, baseadas, por exemplo: em se são requisitos funcionais, não funcionais ou inversos; nas pessoas que estão interessadas no serviço, que o requisitaram, ou que o implementam; em opiniões dadas pelos requerentes; ou no tipo de serviço que oferecem; dentre outras.

A classificação e separação de características são úteis para facilitar a leitura do modelo utilizado e identificar nele as partes importantes em cada momento do processo, seja para elicitar novas características, validar com o usuário, analisar conflitos ou simplesmente entregar a especificação aos desenvolvedores. Entretanto, muitas vezes estas características estão transversais; por exemplo, um requisito funcional oferece um tipo de serviço e pode ter sido requisitado por diferentes pessoas interessadas nele e cada pessoa pode dar uma opinião complementar ou conflitante sobre o que ele é.

Neste caso, uma única categorização não é suficiente para satisfazer às necessidades dos engenheiros de requisitos, e utilizar diferentes categorizações requer maior trabalho em identificar as partes que interessam em cada momento

do processo de definição. As informações estão entrelaçadas e espalhadas, e quanto mais tentamos torná-las menos entrelaçadas mais as espalhamos, e vice-versa.

Se considerarmos o processo de requisitos isoladamente, os problemas de espalhamento e entrelaçamento em documentos de requisitos trazem sérias dificuldades para o entendimento de cada requisito, para manipulação do documento, para a reutilização de partes da especificação e para a modificação nos requisitos. Estes problemas trazem dificuldades ainda maiores se consideramos o processo de desenvolvimento de software inteiro, pois este depende da boa elaboração dos requisitos para o efetivo desenvolvimento e aceitação do produto (Sommerville, 2005).

O problema abordado nesta tese é o espalhamento e entrelaçamento de características transversais em modelos de requisitos. Consideramos que eles são problemas naturais, decorrentes da complexidade dos sistemas e, portanto, que é necessário sabermos amenizar seus danos para podermos conviver com eles.

1.2. Limitações dos Trabalhos Relacionados

Durante a modelagem de requisitos, a separação e composição de características têm sido tratadas parcialmente por abordagens tradicionais, tais como:

- Separação - definição de linguagens que permitem a descrição de requisitos funcionais (RFs), requisitos não funcionais (RNFs), ou ambos, seqüência de ações, intencionalidade, influência entre requisitos, interação entre interessados e funcionalidades, dentre outros; descrição de opiniões de cada interessado no sistema (pontos de vista) (Finkelstein, 1992, Nuseibeh, 1994); descrição modular dos requisitos, principalmente de RNFs, que são catalogados de acordo com algum método, para posterior reutilização (Cysneiros, 2003).
- Composição - frameworks para integração de modelos (Nuseibeh, 1994); e processos para integração de RNFs e RFs (Cysneiros, 2001).

Estas abordagens de separação não são suficientes porque permitem apenas uma forma de decomposição de requisitos e não deixam explícitas as características transversais, nem como elas interagem umas com as outras.

Enquanto que estas abordagens de composição se baseiam apenas nos construtos das linguagens de modelagem para realizar a integração entre modelos, entre opiniões ou entre RFs e RNFs; elas não prevêm a necessidade de compor características transversais aos construtos das linguagens. Além disto, estas abordagens são primordialmente ou de separação, ou de composição. Elas não abrangem as duas atividades porque a separação não prevê a descrição de como compor os modelos separados.

Com o crescimento do interesse pelo desenvolvimento orientado a aspectos, têm surgido abordagens de separação que se preocupam, também, com a composição. É possível descrever características transversais separadamente, porque há mecanismos para juntá-las (Baniassad, 2004; Moreira, 2005; Rashid, 2002; Sousa, 2003, 2004a, 2004b). Entretanto, estes mecanismos não são automatizados (Baniassad, 2004; Sousa, 2003, 2004), são restritos a integração de RNFs (Rashid, 2002), não permitem a visualização separada das características após a composição, ou são específicos para um único tipo de linguagem de modelagem (Baniassad, 2004; Moreira, 2005; Rashid, 2002; Sousa, 2003, 2004a, 2004b).

Nesta tese, mostramos a necessidade de trabalhos que envolvam a separação, a composição e a visualização como uma abordagem mínima para modelagem de requisitos. Nossa abordagem consiste em um modelo para integração de características transversais, que pode ser utilizado por diferentes linguagens de modelagem de requisitos para abordar os problemas de espalhamento e entrelaçamento de características transversais.

1.3. Solução Proposta

Para abordar os problemas de espalhamento e entrelaçamento de características durante a definição de requisitos, propomos um metamodelo de integração de características transversais. Este modelo é fundamentado nas idéias implementadas nas linguagens de programação orientadas a “aspectos”: AspectJ (Kiczales, 2001) e Hyper/J (Tarr, 2000).

Assim como nestas linguagens, o modelo de integração dá apoio à separação e composição de características transversais, por prover elementos para modelagem de características separadamente e um mecanismo de composição

automático que facilita a inclusão e a exclusão de características da especificação. Além disto, o modelo de integração dá apoio à extração de diferentes visões (visualização) dos modelos separados e compostos. Nesta etapa do desenvolvimento, a visualização é essencial porque facilita a análise e especificação do problema, bem como a definição do sistema, veja Figura 2.

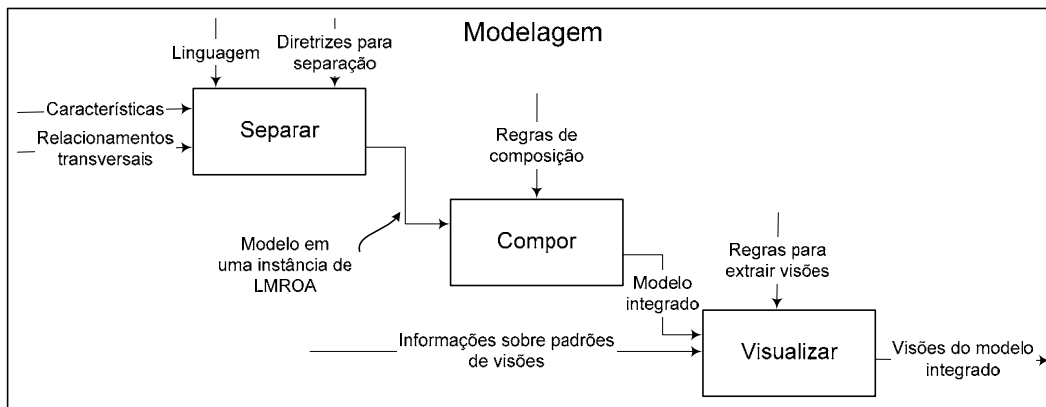


Figura 2. Metamodelo de integração de características transversais

Para realizar a separação e a composição, definimos uma linguagem de modelagem de requisitos orientada a “aspectos” (LMROA). LMROA oferece um novo tipo de relacionamento, denominado “relacionamento transversal”, para estender linguagens de modelagem de requisitos. O relacionamento transversal é utilizado para registrar a interação entre características, indicando que elas se espalham e/ou se entrelaçam às demais e registrando como tais características influenciam ou modificam umas as outras.

A composição é realizada por um mecanismo que processa as informações contidas no relacionamento transversal, propagando as modificações nele descritas. Cada construto do relacionamento transversal indica uma maneira de modificar o modelo e também novas características, por meio das quais pode-se extrair visões. A visualização é responsável pela geração de diferentes visões do modelo integrado, permitindo que o engenheiro de requisitos possa analisar partes ou a modelagem inteira, sob diferentes perspectivas.

Os mecanismos de composição e visualização dependem da linguagem de modelagem adotada. Desta forma, aplicamos a extensão definida por LMROA ao modelo de metas V-graph (Yu, 2004), e assim, instanciamos os mecanismos de composição e visualização.

1.4. Contribuições da Tese

Por abordar os problemas de entrelaçamento e espalhamento em modelos de requisitos, nossa abordagem provê facilidades para que os engenheiros de requisitos modelem características transversais e suas interações, e entendam, analisem, modifiquem e reusem partes do modelo de requisitos. Dentre os produtos gerados estão: metamodelo para integração de características transversais; linguagem de modelagem de requisitos orientada a “aspectos” (LMROA); extensão do modelo de metas V-graph; regras para extração de um conjunto de visões que podem ser geradas a partir do V-graph; e um conjunto preliminar de características transversais reusáveis.

Além disso, consideramos que nossa abordagem provê a infra-estrutura básica para pesquisarmos a natureza de características transversais, além daquelas originadas em requisitos não funcionais. Desta forma, contribuímos para o desenvolvimento orientado a aspectos. No Capítulo 6, detalhamos como estas contribuições são alcançadas em nossa abordagem.

1.5. Organização da Tese

O restante desta tese está organizado da seguinte maneira. No **Capítulo 2** apresentamos os conceitos de engenharia de requisitos necessários para o entendimento do contexto e problema abordados nesta tese. No **Capítulo 3**, relatamos os fundamentos da separação de características, soluções utilizadas no nível de implementação e abordagens em engenharia de requisitos orientada a aspectos, que contribuíram para a criação de nossa estratégia.

No **Capítulo 4**, descrevemos nossa estratégia para os problemas de entrelaçamento e espalhamento de características transversais durante a modelagem de requisitos – um metamodelo para integração de características transversais. Detalhamos a utilização deste modelo com o V-graph, mostrando os benefícios desta abordagem e sua implementação. Além disto, apresentamos como esta estratégia pode ser utilizada com outras linguagens de modelagem e qual o seu impacto no processo de engenharia de requisitos como um todo.

No **Capítulo 5**, apresentamos os estudos de caso realizados para demonstrar e validar nossa abordagem. Finalmente, no **Capítulo 6**, apresentamos as contribuições e limitações de nossa abordagem, uma comparação com os trabalhos relacionados, bem como os trabalhos futuros e as tendências apontadas pelo tema desta tese.

No Apêndice A, apresentamos a linguagem de modelagem de requisitos orientada a aspectos, utilizando os padrões BNF e DTD. No Apêndice B, apresentamos a implementação dos componentes do mecanismo de composição e visualização. Nos Apêndices C e D, resumimos a modelagem e algumas visões de nossos estudos de caso. No Anexo A, apresentamos o documento de requisitos utilizado em um dos estudos de caso.

2

Engenharia de Requisitos

Um dos principais objetivos da engenharia de requisitos é melhorar a modelagem de sistemas e a capacidade de analisá-los, possibilitando maior entendimento de suas características antes da implementação (Robinson 2003). É seu papel realizar a interação entre requisitantes⁴ e desenvolvedores, entre “o que” deve ser feito e “como” deve ser feito. É necessário nesta etapa, elicitar, analisar conflitos, validar, priorizar, modificar e reusar requisitos, rastreá-los considerando sua origem, os componentes arquiteturais e o código que os implementa, dentre outras tarefas.

A reutilização, evolução e rastreabilidade de requisitos estão intimamente relacionados à habilidade de gerenciar interações entre requisitos, que por sua vez, está relacionada à habilidade de separar e compor características, representando-as em modelos. Visto que, geralmente, um único tipo de modelo não é suficiente para explicitar todas as características do sistema, diferentes modelos são utilizados, tornando as informações espalhadas e entrelaçadas, também, em diferentes visões. Em engenharia de requisitos o uso de visões é extremamente importante porque com este artifício consegue-se representar e visualizar os requisitos, focando as características de interesse a cada momento do processo. O uso de visões também é uma maneira de abordar a separação de características.

Assim, a engenharia de requisitos tem abordado o princípio de separação de características por meio de métodos de modelagem em conjunto com o uso de visões, com o intuito de prover facilidades para o reuso, rastreabilidade e evolução. Na Seção 2.1, resumimos o processo de definição de requisitos. Na Seção 2.2, apresentamos a modelagem de requisitos e mostramos a presença dos problemas de espalhamento e entrelaçamento. Na Seção 2.3, apresentamos como visões são utilizadas para separação de características. Na Seção 2.4, 2.5 e 2.6 definimos, respectivamente, os conceitos de evolução, rastreabilidade e reuso.

⁴ O termo requisitante está sendo utilizado para representar as pessoas que requisitam serviços ou impõem restrições, tais como usuários e clientes.

2.1. Processo

O processo de definição de requisitos pode ser definido, resumidamente, por três atividades: elicitação, modelagem e análise (Leite, 1988), veja Figura 3. Geralmente, estas atividades ocorrem simultânea e incrementalmente, num processo evolutivo que dura todo o processo de desenvolvimento de software (Jacobson, 1999).

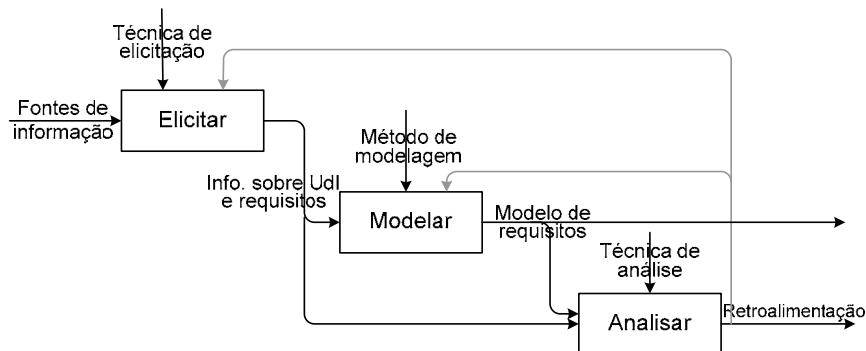


Figura 3. Processo de engenharia de requisitos

- Elicitação – o engenheiro de requisitos utiliza alguma ou um conjunto de técnicas para descobrir (elicitar) os requisitos do sistema a ser desenvolvido, incluindo as informações do UdI (universo de informação) que restringem este sistema. Algumas das técnicas mais conhecidas estão relacionadas a técnicas de leitura de documentos, observação, entrevistas, reuniões, questionários, antropologia, participação ativa dos atores e engenharia reversa (Goguen, 1994).
- Modelagem – as informações obtidas durante a elicitação são registradas e organizadas em modelos de requisitos tais como, casos de uso (Jacobson, 1992), cenários e léxico (Leite, 1997), entidade relacionamento (Chen, 1976), SADT (Ross, 1977), DFD (Marco, 1979), dentre outros. A construção destes modelos exige maior aprofundamento no conhecimento sobre o UdI, sobre as necessidades dos usuários e também informações técnicas. Isto remete à atividade de análise, sendo necessário analisar as informações registradas para descobrir erros e omissões, sendo muitas vezes necessário retornar à elicitação para esclarecer, acrescentar ou corrigir o conhecimento adquirido.
- Análise – além da análise de erros e omissões o processo de definição de requisitos possibilita a análise sob diferentes perspectivas tais como,

viabilidade, custo, tempo, prioridades, reuso, completude, corretude, variabilidade, evolução, dentre outras.

As atividades de elicitação e análise estão fora do escopo desta tese. Porém, no Capítulo 4, mostramos qual o impacto de nossa abordagem nestas atividades.

2.2. Modelagem

A modelagem de requisitos é a atividade central no processo de engenharia de requisitos porque dela resulta o produto principal deste processo: o documento de requisitos, o documento no qual os desenvolvedores devem se basear para construir a arquitetura e o código do sistema (Jacobson, 1999; Sommerville, 2000; Kotonya, 1998b). Vários modelos são necessários para compor o documento de requisitos porque cada um deles enfatiza apenas parte dos aspectos necessários para que os desenvolvedores entendam o que o sistema sendo construído deve prover e o seu contexto.

Requisitos são sentenças que indicam as necessidades dos interessados (Kotonya, 1998b). Eles são, geralmente, categorizados como: requisitos funcionais, i.e, aqueles que representam a funcionalidade do sistema; e não-funcionais, i.e, aqueles que restringem os requisitos funcionais. Eles podem estar associados a diferentes dados, serviços, *features*, restrições, níveis de abstração, representações, dentre outros, veja Figura 4.

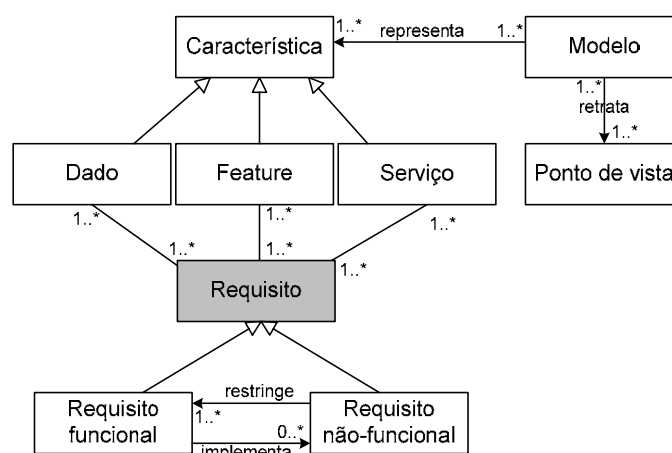


Figura 4. Relação entre requisitos e características

Em (IEEE Std. 830; Sommerville, 2000; Volere, 2006) são definidos *templates* para o documento de requisitos. Eles apresentam estruturas para organizar os requisitos no documento, veja na Figura 5. De maneira geral, um

documento de requisitos deve definir, no mínimo, um glossário de termos do Udl (em cinza claro na Figura 5) e uma lista de sentenças de requisitos, geralmente, em linguagem natural não estruturada e organizada de diferentes maneiras (em cinza escuro na Figura 5).

Volere (2006)	IEEE Std. 830
PROJETO 1. O propósito do produto 2. Clientes, fregueses e outros interessados 3. Usuários do produto RESTRIÇÕES DO PROJETO 4. Restrições obrigatórias 5. Convenções para nomes e definições 6. Fatos relevantes e suposições REQUISITOS FUNCIONAIS 7. O escopo do trabalho 8. O escopo do produto 9. Requisitos funcionais e de dados REQUISITOS NÃO-FUNCIONAIS 10. Requisitos de amigabilidade 11. Requisitos de usabilidade 12. Requisitos de performance 13. Requisitos operacionais 14. Requisitos de manutenibilidade e portabilidade 15. Requisitos de segurança 16. Requisitos culturais e políticos 17. Requisitos legais ASSUNTOS DE PROJETO 18. Assuntos em aberto 19. Soluções de prateleira 20. Novos problemas 21. Tarefas 22. Agenda 23. Riscos 24. Custos 25. Documentação para o usuário e treinamento 26. Requisitos para as próximas versões 27. Idéias para soluções	1. Introdução 1.1 Propósito 1.2 Escopo 1.3 Definições, acronimos e abreviações 1.4 Referencias 1.5 Visão geral 2. Descrição global 2.1 Perspectivas do produto 2.2 Funções do produto 2.3 Características dos usuários 2.4 Restrições 2.5 Suposições e dependências 3. Requisitos específicos Apêndices
	Sommerville (2005)
	1. Prefácio 2. Introdução 3. Glossário 4. Requisitos do usuário 5. Arquitetura do sistema 6. Requisitos dos sistema 7. Modelos do sistema 8. Evolução do sistema 9. Apêndices 10. Índice

Figura 5. Exemplos de templates para documentos de requisitos

Espalhamento e Entrelaçamento de Características

Podemos considerar que ambos, glossário e lista, são modelos de requisitos. No primeiro, enfatizam-se dados e restrições a eles, no segundo enfatizam-se serviços, restrições e exceções. Informações sobre os dados estão detalhadas no primeiro modelo e aparecem no segundo modelo como recursos ou atores dos serviços modelados. Enquanto informações sobre os serviços estão detalhadas no segundo modelo que muitas vezes omite detalhes sobre os dados, sendo que, algumas restrições impostas aos serviços são restrições decorrentes de restrições aos dados. Nenhum dos dois modelos traz toda a informação necessária ou dá igual importância às características, eles devem ser utilizados em conjunto. Esta separação é realizada para que o engenheiro de requisitos seja capaz de se

concentrar ora em serviços ora em dados, mas amplia os problemas de espalhamento e entrelaçamento e tem o preço de ter que manter dois modelos atualizados e consistentes.

Consideremos, então, apenas um dos modelos por vez, por exemplo, a lista de sentenças de requisitos. Tomemos duas situações extremas:

1) cada requisito representa apenas um serviço aplicado a um tipo de dado, obedecendo às diretrizes para escrever requisitos (Alexander, 2002) ou as diretrizes de decomposição modular (Staa, 2000). Consideramos que o conceito de modularização em uma lista de requisitos indica que cada requisito deve retratar um forte relacionamento de seus elementos constituintes e representar uma única funcionalidade, sendo pouco dependente de outros requisitos; e

2) cada requisito é escrito livremente, podendo fazer referência às restrições de serviço e dados a que ele se aplica.

Na primeira situação, temos uma lista de requisitos ampla em que os requisitos se encontram entrelaçados apenas por um tipo de serviço ou um tipo de dado. Entretanto, cada tipo de dado e serviço está espalhado por muitos requisitos, veja Figura 6⁵. Por exemplo: operações sobre o dado “evento” estão espalhadas nos requisitos de números 1, 2.8.3, 4.3, 7.1 e 7.3; as operações de “adicionar”, “apagar” e “modificar” estão espalhadas nos requisitos de números 1, 2 e 3; nos requisitos de números 2.8.3 e 2.8.3.1 os dados sobre “eventos” e “categorias” estão entrelaçados, dentre outros.

⁵ Este exemplo foi desenvolvido a partir do documento de requisitos do sistema Unical (Unical, 2006), um dos nossos estudos de caso apresentado no Capítulo 5. Estas sentenças de requisitos foram reescritas com base no documento original.

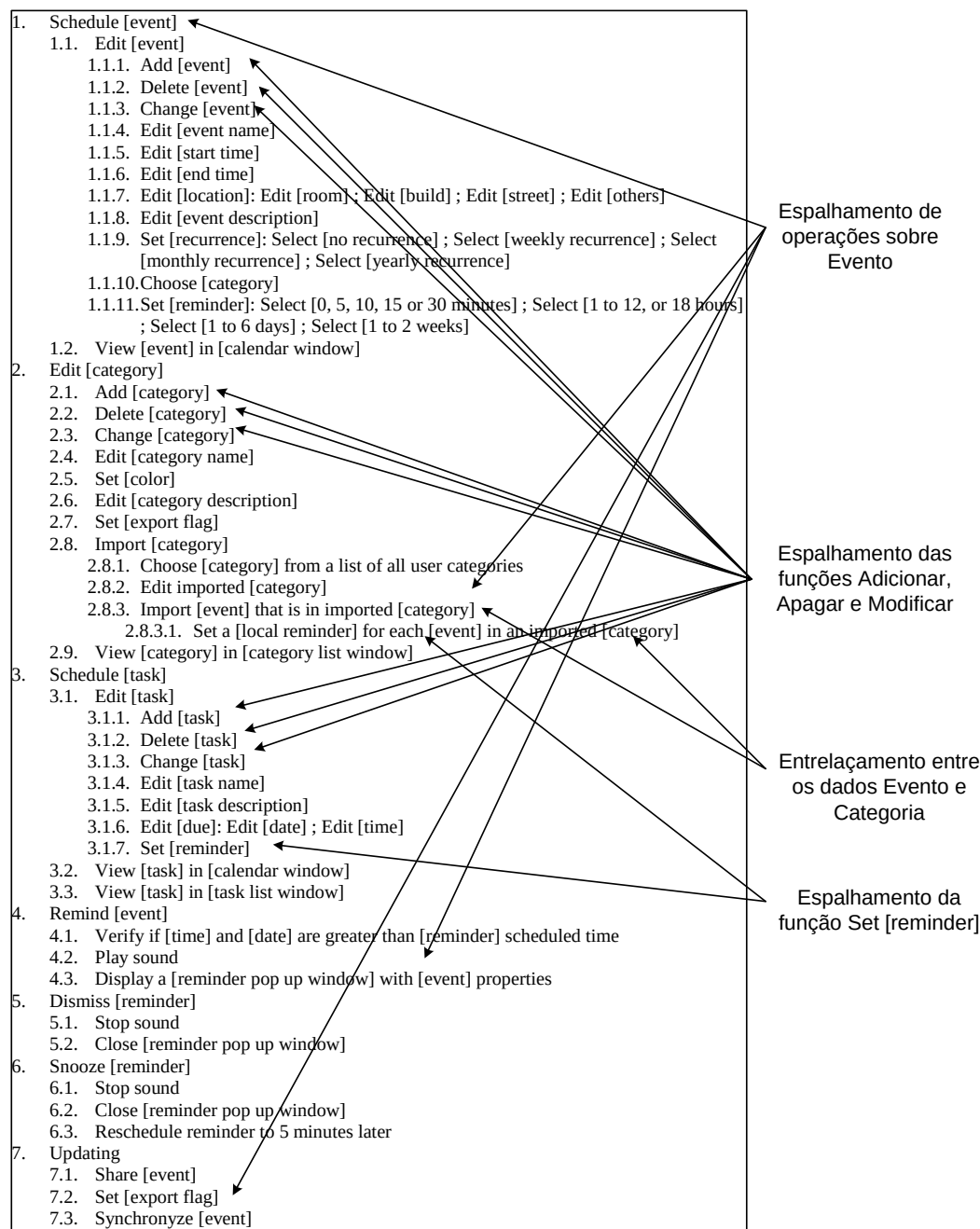


Figura 6. Espalhamento e entrelaçamento em listas de requisitos – 1º exemplo

Por outro lado, na segunda situação, podemos ter uma lista de requisitos menor, e os tipos de dados ou os tipos de serviços não estão muito espalhados, mas cada requisito está tratando de mais de um serviço e um dado, ou seja, as informações estão mais entrelaçadas que no primeiro exemplo, veja Figura 7⁶. Por exemplo: os requisitos de números 4.2.2, 4.2.5 e 4.2.6 contém as operações “adicionar”, “apagar” e “modificar” espalhadas e entrelaçadas; os requisitos de

⁶ Este exemplo consiste em uma parte do documento de requisitos original do sistema Unical (Unical, 2006).

números 4.2.5.2.3, 4.2.5.2.2 e 4.2.6.2.3 descrevem a mesma restrição ao formato de “tempo”; dentre outros.

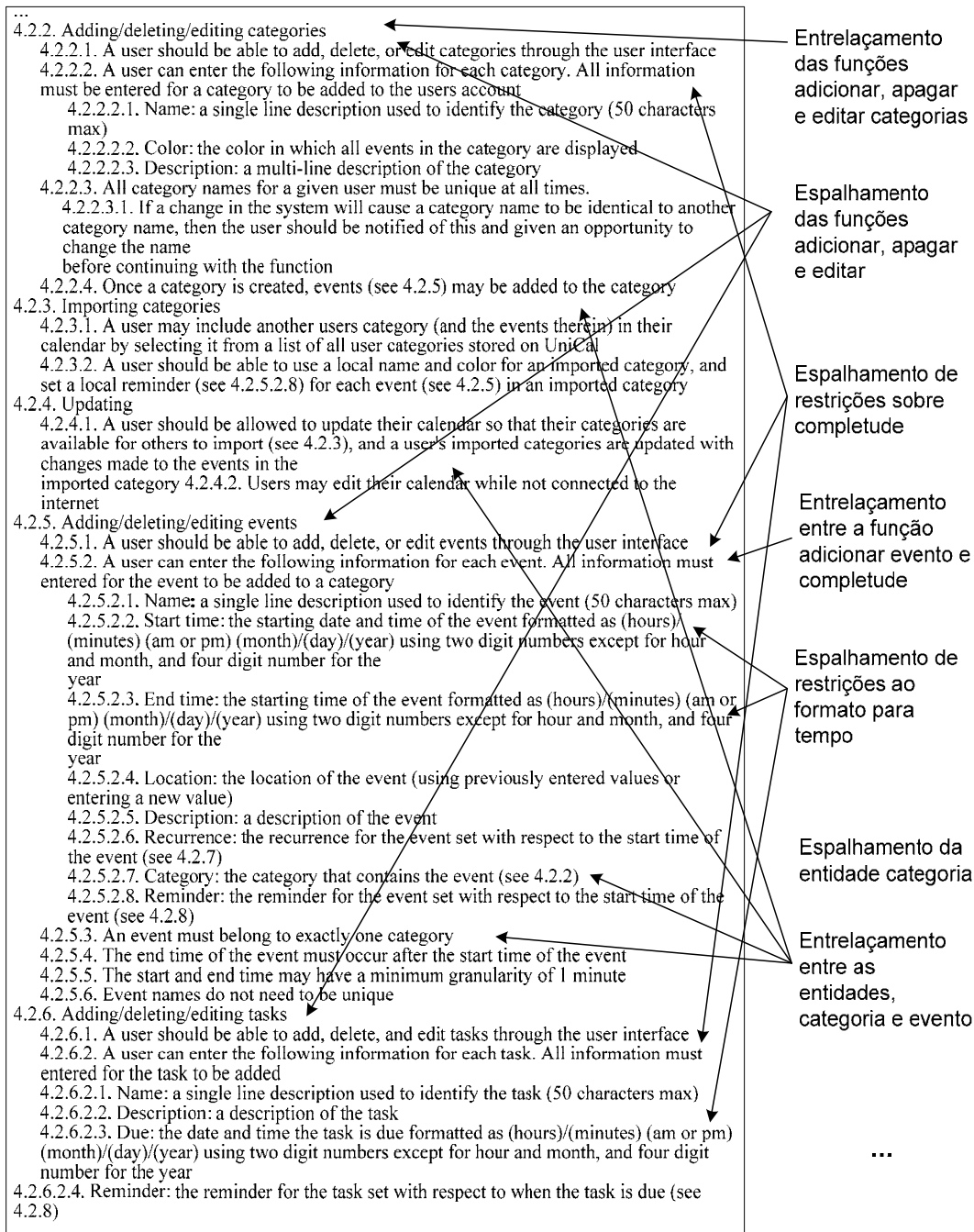


Figura 7. Espalhamento e entrelaçamento em listas de requisitos – 2º exemplo

Em ambos os casos, há os problemas de espalhamento e entrelaçamento. Na segunda situação a dificuldade em manipular o documento pode ser ainda maior porque há maior probabilidade dos requisitos serem confusos e ambíguos. Entretanto, aplicar os conceitos de modularização, apesar de diminuir o problema de entrelaçamento, pode aumentar o espalhamento, como visto no primeiro exemplo. Principalmente porque não há uma única maneira de separar os

requisitos, ora é interessante separá-los como requisitos funcionais, não funcionais e inversos, ora é interessante separá-los por tipo de serviço (por exemplo, tudo que pode ser configurado no sistema), por tipo de dado (por exemplo, tudo que diz respeito a evento), por interessados (por exemplo, tudo que o usuário administrador pode realizar), por relevância, por complexidade, por custo, dentre outros.

Na Figura 6 e Figura 7, exemplificamos os problemas de espalhamento e entrelaçamento em listas de sentenças de requisitos, que são menos estruturadas e modularizadas. Entretanto, estes problemas também ocorrem em modelos de requisitos semi-formais ou estruturados, tais como modelos de metas e cenários. Desta forma, consideramos que é necessário haver uma maneira de realizar não só a separação, mas também o controle das interações entre as partes separadas, sendo possível visualizá-las de maneiras distintas.

Nesta tese, utilizamos modelos de metas (apresentados a seguir) para representar os requisitos do sistema e separar características transversais, e para ter maior controle das interações, utilizamos alguns fundamentos definidos por trabalhos sobre desenvolvimento orientado a aspectos (apresentados no Capítulo 3).

Modelos de Metas

Modelos de metas surgiram com o objetivo de modelar, também, a razão da existência dos requisitos. Desta maneira, o engenheiro consegue identificar a solução mais adequada para o problema em questão. Metas trazem à tona requisitos e regras de negócio normalmente omitidos, e que muitas vezes, levam ao fracasso do produto (Lamsweerde 01; Chung, 2000). Em (Mylopoulos, 1992), *softmetas* são propostas como meio para modelar e analisar RNFs em modelos de metas.

Modelos de metas representam explicitamente requisitos funcionais e não funcionais através da decomposição de metas (Giorgini, 2002; Yu, 2004). Esta decomposição indica como satisfazer uma determinada meta, indicando a razão pela qual as submetas são necessárias. Na literatura há algumas variações de modelos de metas tais como i^* (Yu, 95), Kaos (Lamsweerde, 2001) e V-graph (Yu, 2004). Em geral, eles usam árvores *and/or* para representar a decomposição

de metas e definir um espaço de soluções alternativas para satisfazer a meta raiz. Na verdade, modelos de metas são grafos e não árvores (Chung, 2000) porque muitos elementos das árvores contribuem para satisfação de elementos em diferentes ramificações.

Como modelos de metas explicitam vários tipos de informação em diferentes níveis de abstração, é possível realizar diferentes tipos de análise, tais como: análise de obstáculos - explora os possíveis obstáculos para a satisfação de uma meta (Lamsweerde, 2000); análise qualitativa de metas - permite atribuir valores qualitativos aos relacionamentos entre metas e ajuda a formalizar e pensar sobre estes relacionamentos (Giorgini, 2002); propagação de rótulo - propaga os rótulos em uma árvore de metas, tornando visível conflitos de interesses e objetivos (Giorgini, 2002); análise de variabilidade (Gonzáles, 2004; Park, 2004)- possibilita a análise de soluções alternativas; análise do *fan-in* e *fan-out* de cada elemento para identificar candidatos a aspectos (Yu, 2004), dentre outros.

V-graph, o modelo de metas que utilizamos nesta tese, utiliza: três tipos de elementos, *softmetas*, metas e tarefas; e dois tipos de relacionamentos, contribuição e correlação. Nós escolhemos este modelo porque seus elementos e relacionamentos nos oferecem maneiras diferentes de separar as características do sistema sem muita dificuldade.

V-graph

V-graph é um tipo de modelo de metas, proposto em (Yu, 2004) como uma simplificação do modelo RNF (Chung, 2000) para demonstrar uma abordagem de identificação de candidatos a aspectos em requisitos. O nome *V-graph* originou-se da disposição entre os seus três elementos constituintes, sendo o elemento tarefa o vértice inferior que operacionaliza metas e *softmetas* correlacionadas. A seguir explicamos os conceitos e relacionamentos do V-graph; para facilitar a explanação, fizemos algumas alterações (Figura 8b) na ilustração original (Figura 8a).

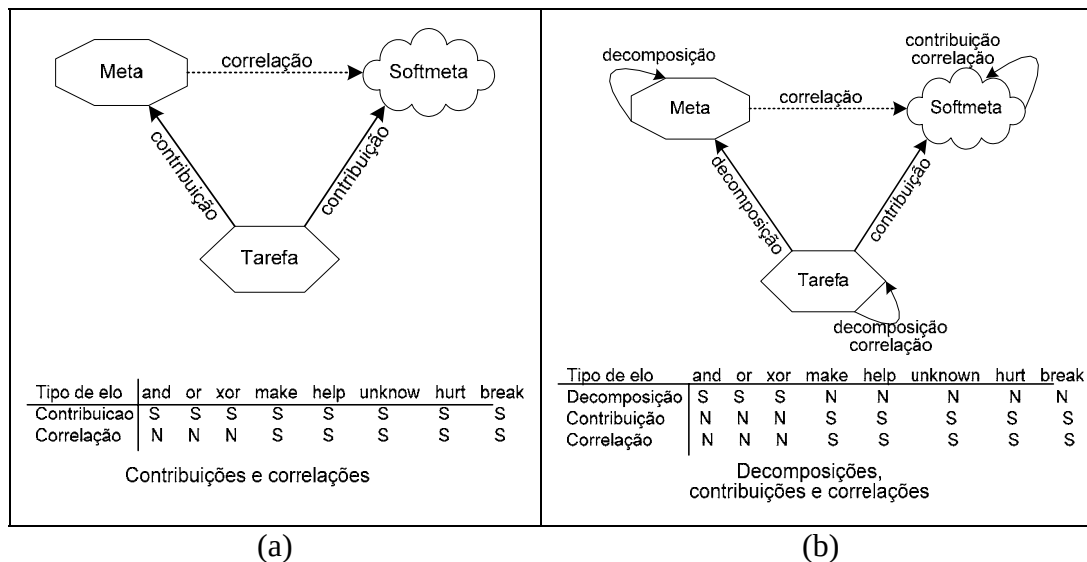


Figura 8. O modelo de metas *V-graph*: (a) representação original definida em (Yu, 2004); (b) nossa representação, explicitando todos os relacionamentos possíveis entre os elementos do *V-graph*.

V-graph é composto de três elementos: *softmetas*, metas e tarefas. Cada elemento é composto de duas partes: tipo e tópicos. O tipo descreve uma função genérica ou um requisito não-funcional genérico. O tópico captura a informação contextual para o elemento; esta informação contextual é similar ao *subject* definido em (Zave, 1997). Por exemplo, “segurança” e “verificação” são os tipos dos elementos “Segurança na transmissão de dados”, “Segurança no armazenamento de dados”, “Verificação do modelo de requisitos” e “Verificação das leis trabalhistas”, enquanto que “transmissão de dados”, “armazenamento de dados”, “modelo de requisitos” e “leis trabalhistas” são os tópicos.

Este modelo permite a descrição de nós intencionais (metas e *softmetas*) e nós operacionais (tarefas). Usualmente, *softmetas* representam RNFs, metas representam macro-requisitos, regras ou objetivos do negócio, e tarefas representam RFs que operacionalizam metas e *softmetas*. Metas e *softmetas* se distinguem pelos conceitos de *satisfy* e *satisfice* (Mylopoulos, 1992), respectivamente, i.e.: uma meta é satisfeita totalmente (*satisfy*) através do conjunto de submetas e tarefas nas quais ela se decompõe; e uma *softmeta* é suficientemente satisfeita (*satisfice*) pelas metas, *softmetas* e tarefas que contribuem ou estão correlacionadas positivamente a ela. Os possíveis relacionamentos entre metas, *softmetas* e tarefas são:

Decomposição – elos de decomposição podem ter rótulo AND, XOR ou OR. AND indica que o subelemento é obrigatório, OR que ele é opcional e XOR

que pode ocorrer apenas um dos subelementos. Eles podem indicar refinamento ou agregação, decompondo os tipos ou tópicos de cada elemento. O sentido da decomposição é do elemento decomposto para o elemento pai, podendo ocorrer entre $subsoftmeta \rightarrow softmeta$, $submeta \rightarrow meta$, $subtarefa \rightarrow tarefa$ e $tarefa \rightarrow meta$. Normalmente, utilizamos o elo de decomposição quando o subelemento está semântica e diretamente associado ao elemento-pai. Na Figura 9a, ilustramos alguns relacionamentos de decomposição.

Contribuição – elos de contribuição podem ter os rótulos *make(++)*, *help(+)*, *unknown(?)*, *hurt(-)* e *break(--)*. *Make* e *help* indicam contribuição positiva, *unknown* indica que há um relacionamento, mas é desconhecido se positivo ou negativo, e *hurt* e *break* indicam contribuição negativa. A contribuição é utilizada com o mesmo sentido da decomposição, mas ocorre entre $subsoftmeta \rightarrow softmeta$ e $tarefa \rightarrow softmeta$ devido à natureza *fuzzy* do elemento *softmeta*. Na Figura 9b, ilustramos alguns relacionamentos de contribuição.

Correlação – elos de correlação também podem ter os rótulos *make(++)*, *help(+)*, *unknown(?)*, *hurt(-)* e *break(--)*. Entretanto, a correlação é utilizada para representar relacionamentos de contribuição horizontal entre diferentes árvores ou subárvores, indicando menor acoplamento do que os elos de decomposição e contribuição. A correlação é baseada no conceito de “conflito e harmonia” indicando a interferência positiva ou negativa entre árvores de metas aparentemente desconexas. A correlação pode ocorrer entre $softmeta \rightarrow softmeta$, $meta \rightarrow softmeta$, e $tarefa \rightarrow tarefa$. A correlação entre $tarefa \rightarrow tarefa$ ocorre somente quando existe uma correlação entre os elementos pais de ambas as tarefas. Na Figura 9c ilustramos alguns relacionamentos de correlação.

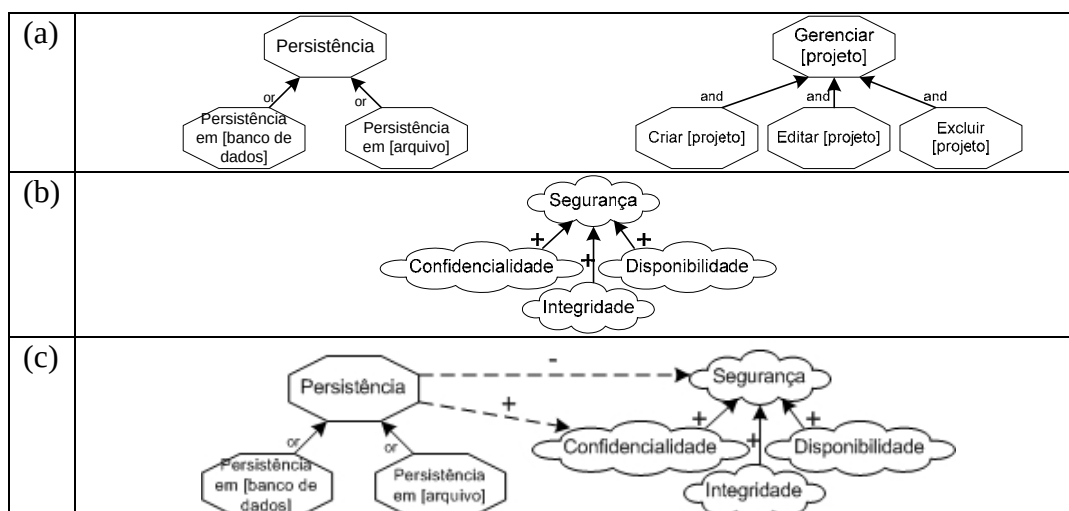


Figura 9. Exemplos de (a) decomposição, (b) contribuição e (c) correlação.

Em (Yu, 2004), os relacionamentos existentes entre tarefas são utilizados na identificação de aspectos-meta (goal aspects). As tarefas que contribuem ou estão correlacionadas a várias *softmetas* e metas indicam espalhamento e entrelaçamento ou que podem se espalhar por vários componentes em fases posteriores, se este comportamento não for percebido a tempo de escolher uma modularização melhor para elas.

Os elementos do V-graph nos possibilitam a extração de várias visões: visão de dados, através dos tópicos; visão de objetos, através dos tópicos e tipos; influência direta ou indireta que uma característica exerce sobre as outras, através dos relacionamentos; visão funcional, através dos tipos; dentre outras.

2.3. Visões

Uma visão é uma representação do sistema inteiro sob a perspectiva de um conjunto de características relacionadas (IEEE St.1471). O uso de visões é uma maneira de separar diferentes características para focar uma por vez. Elas ajudam no entendimento e elaboração de soluções (Sommerville, 2000), sendo, assim, necessárias durante todo o processo de desenvolvimento.

Na literatura, muitas vezes os termos em inglês *viewpoints*, *views* e *point-of-view* são utilizados com significados diferentes ou semelhantes (Leite, 1996). Há problemas, também, quando os traduzimos para o português, pois passam a ser representados pelas expressões “pontos de vista”, “visões” e “perspectivas”. Nesta tese adotamos o termo “visão” num sentido amplo que pode significar todas ou uma das três categorias definidas em (Leite, 1996):

- Visões como opiniões (pontos de vista) – no contexto social, cada um dos interessados tem suas próprias premissas, prioridades e experiências e lidam com os problemas de maneiras diferentes. Assim, é necessário conhecer, comparar e negociar as diferentes opiniões ou diferentes maneiras de ver a mesma coisa, por exemplo: qual a opinião do gerente, do comprador e do vendedor sobre uma compra?
- Visões como serviços (característica) – a idéia de particionar o sistema em um conjunto de serviços que podem ser conectados de diferentes maneiras provê o desenvolvimento baseado em componentes, por exemplo, pagamento de contas, vendas, segurança, dentre outros;

- Visões como modelos (perspectivas) – no contexto de engenharia de software várias técnicas baseadas em linguagens têm sido propostas para retratar o sistema parcialmente, por exemplo, modelo entidade-relacionamento, casos de uso e diagramas de seqüência. Assim, é importante detectar problemas de consistência e completude entre modelos.

Estas categorias não representam conjuntos disjuntos. Normalmente, utilizamos **modelos** (perspectivas) para representar **serviços** de acordo com a **opinião** (ponto de vista) de um ou mais interessados. Além disto, os modelos, por si só, explicitam um tipo de informação ocultando outros, então temos modelos que enfatizam funções, outros, dados, outros enfatizam seqüência de atividades e assim por diante. Desta forma, além dos serviços estarem naturalmente entrelaçados e espalhados entre si, eles ainda encontram-se espalhados e entrelaçados entre as diferentes opiniões e também nos diferentes elementos representativos de cada modelo.

Integração de Visões

Se por um lado, a utilização de visões ajuda a gerenciar a complexidade da modelagem, por outro ela leva ao espalhamento e entrelaçamento de informações. Assim, são necessários mecanismos de integração, seja por meio da integração de serviços, modelos ou de opiniões (resolução de conflitos).

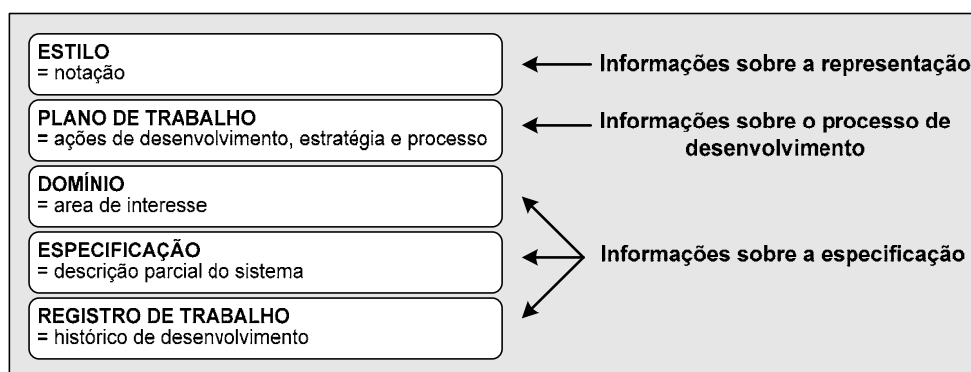


Figura 10. Framework de integração de métodos

Em (Nuseibeh, 2004) é definido um framework para integração de modelos (métodos de modelagem). Este framework determina a especificação das seguintes informações (veja a Figura 10): 1) estilo – indica a notação utilizada; 2) plano de trabalho – indica atividades, estratégias e processos para construir a visão – regras de transformação entre os métodos; 3) domínio – indica a área de

interesse; 4) especificação – é o modelo em si; e 5) registro de trabalho – indica o estado e histórico da especificação em desenvolvimento. As informações descritas em (1) e (2) são informações genéricas, aplicadas a todas as instâncias do mesmo método. As informações em (3), (4) e (5) deste framework são específicas para cada instância do modelo.

Este framework pode ser utilizado para especificação das três categorias de visões (opiniões, modelos e serviços), mas seu foco é a integração de modelos, pois cada instância do framework especifica uma representação. É possível integrar duas instâncias que possuem a mesma notação, e que representam opiniões ou serviços diferentes, mas todo o processo de relacionar os dois serviços tem que ser refeito para cada instância, ou seja, não reutilizado do framework. O principal propósito deste trabalho é dar apoio à checagem de consistência entre diferentes modelos e ao gerenciamento de inconsistências, possibilitando a reutilização das informações sobre a representação e o processo de desenvolvimento.

Já em (Cysneiros, 2001), é definido um processo para integrar RNFs a RFs (serviços). Ele utiliza os construtos do modelo de entidade-relacionamento (MER), diagrama de classes, e léxico ampliado da linguagem (LAL) (Leite, 1990), para realizar esta composição. O LAL deve conter todos os símbolos representativos de RF e RNFs, com os relacionamentos entre símbolos representados em linguagem natural através do elemento “impacto”. O processo consiste em identificar no LAL todos os símbolos relacionados a cada RNF e para cada um destes símbolos, acrescentar ao modelo entidade-relacionamento ou de classe uma referência para aquele RNF. Depois disto, é necessário verificar se esta adição implica em mudanças maiores nos modelos.

Em resumo, o trabalho definido em (Cysneiros, 2001) foca apenas a adição de RNFs a modelos de RFs sem preocupação em observar os problemas de espalhamento e entrelaçamento naturais a muitos RNFs. Desta forma, uma vez que o RNF é integrado aos vários elementos do MER ou diagrama de classes, não há nenhuma facilidade para que o engenheiro os veja separados ou perceba o entrelaçamento e espalhamento ocasionados.

A sobreposição e os conflitos existentes entre diferentes opiniões (integração de pontos de vista) são abordados por técnicas de análise de conflitos

(Kotonya, 1996; Leite, 1989, 1991), e estão fora do escopo de nosso trabalho. Nesta tese estamos focados em visões como serviços e como modelos:

- considerando-as serviços, queremos prover uma maneira mais eficiente de separá-las sem perder a habilidade de gerenciar a interseção entre elas (relacionamento ou interação); e
- considerando-as modelos, queremos ser hábeis a visualizar os serviços e suas interações por meio de diferentes representações (modelos).

2.4. Evolução

Devido à natureza volátil dos requisitos, é necessário estar preparado para mudanças, ou evolução. A evolução de requisitos ocorre em dois sentidos: no sentido do desenvolvimento de software, mudando do nível alto de abstração para a implementação, i.e., de requisitos para código; e no sentido da melhoria contínua para atender as expectativas dos requisitantes, sofrendo alterações decorrentes de erros/omissões ou para atender novas necessidades (Lehman, 1996; Leite, 1997).

Em ambos os sentidos, conhecer e gerenciar as **interações** entre requisitos é extremamente importante para:

- a decomposição e modularização das características do sistema porque indica o acoplamento e coesão entre as partes envolvidas; e
- para a análise do impacto de mudanças, tanto entre requisitos no mesmo nível de abstração quanto entre requisitos e artefatos de software em níveis de abstração diferentes (Robinson 2003).

Interações entre requisitos são relacionamentos que indicam dependência, decomposição, complementação, conflito, dentre outros (Robinson, 2003). Além de acoplamento e coesão estes relacionamentos podem indicar a dificuldade de modificar requisitos ou parte deles e de manipular (incluir, modificar, excluir) os elementos textuais ou gráficos que os representam em cada modelo de requisitos. Desta forma, se um requisito tem alto acoplamento e é difícil de manipular, consideramos, nesta tese, que ele é complexo. Utilizamos o termo “complexidade” como definido em (Ferreira, 2004): a qualidade de abranger muitos elementos ou partes, de ser observado sob diferentes aspectos, ou de ser composto por

elementos com naturezas distintas. A gerência de interações está associada à rastreabilidade de requisitos.

2.5. Rastreabilidade

Como definido em (Sommerville, 2000), rastreabilidade é a propriedade de uma especificação de requisitos que reflete a habilidade de encontrar requisitos relacionados. Sommerville (2005) define três tipos de informações a serem rastreadas:

- Para a fonte (origem), também chamada de pré-rastreabilidade, liga os requisitos aos interessados que os propuseram e à razão de algumas escolhas (*rationale*) (Dutoit, 2001). Quando uma mudança é proposta, esta informação é utilizada para encontrar e consultar os interessados sobre tais mudanças;
- Entre requisitos, registra a dependência entre requisitos (interações entre requisitos). Esta informação é utilizada para avaliar quais requisitos são afetados por uma mudança e quais as subseqüentes mudanças necessárias;
- Para o projeto, também chamada de pós-rastreabilidade, liga os requisitos aos módulos da arquitetura e implementação. Esta informação é utilizada para avaliar qual o impacto que mudanças nos requisitos exercem nos módulos que os implementam e/ou estão relacionados.

As informações de rastreabilidade são, geralmente, representadas por matrizes de rastreabilidade (Davis, 1990; Sommerville, 2000; Gotel, 1994) ou mesmo pelas linguagens de modelagem de requisitos, tais como: cenários (Leite, 1997); casos de uso (Jacobson, 1992); modelos de entidade relacionamento; modelos de metas (Chung, 2000; Lamsweerde, 2001); dentre outros.

Nesta tese, abordamos, principalmente, a rastreabilidade no sentido de interações entre requisitos. Gerenciando as interações entre requisitos ou entre elementos que os representam é possível analisar, descobrir e agrupar elementos que estão mais coesos e menos acoplados e assim, reutilizar tais agrupamentos.

2.6.
Reuso

O reuso durante a definição de requisitos ocorre, principalmente, em relação ao conhecimento sobre requisitos não funcionais (RNFs) ou quando desenvolvendo uma família de produtos. Requisitos não funcionais são características ou restrições impostas ao comportamento de um sistema. Estas características, normalmente, dão origem ou restringem diversas funcionalidades. Por se encontrarem espalhadas e entrelaçadas nos requisitos de um sistema, são consideradas características transversais (Leite, 2004; Rashid, 2003).

Em (Cysneiros, 2003; Sommerville, 2000) é proposta a catalogação de RNFs para que possam ser reutilizados. Em (Sommerville, 2000), relata-se como taxonomias de RNFs podem ajudar na identificação de requisitos ocultos e tácitos. Estas taxonomias são utilizadas para lembrar e guiar os engenheiros na definição de requisitos. Em (Cysneiros, 2003), é proposto um esquema para catalogar e compartilhar RNFs utilizando grafos de metas. Esta abordagem é orientada a domínio, onde cada domínio é definido pelas facetas: tipo, lista de tipos relacionados, lista de operacionalizações e tópico, veja Figura 11.

<Domain, Facet type, Facet list of related types, Facet list of operationalizations, Facet topic>
Ex.: (traceability, {safety, performance, security, reliability}, {make a historic of places, store the last location, store all the times thing has moved, store time of last move, store all the times thing was changed, ...}, things)

Figura 11. Exemplo de template utilizado para catalogar RNFs

<pre>START := Advice* Advice := [When] [Who] Why [What] [Where] [How] [HowMuch] When:= " (" Expr ")" " => " Who:= "<" id ">" "::" Why:= id What := "[" id {"," id}* "]" Where:= "<=" Pointcut {"," Pointcut }* How:= '{' BoolOp Advice* '}' HowMuch:= "=>" Effect {"," Effect }* Expr:= "true" "false" id Expr BoolOp Expr Effect:= HowMuchOp [Who "::"] Why ["[" What "]"] Pointcut:= HowMuchOp [("*" Who) "::"] ("*" Why) [("[" "*" "]" What)] BoolOp := "&" " " HowMuchOp:= "++" "+" "-" "--" "?"</pre>	<pre>Persistence { AND "Persistence in" [DB] { AND "initiate [DB]" "verify if is connected" [DB] Connect [DB] Disconnect [DB] } => + mro "Make registry operations" { AND include [data] select [data] delete [data] update [data] } "Persistence in" [file] => + "Make registry operations" } => + Authentication</pre>
(a)	(b)

Figura 12. a) BNF da linguagem Q7 b) Exemplo utilizando Q7

Em (Leite, 2005), os autores propõem um processo para viabilizar o reuso de RNFs. Esta abordagem define a linguagem Q7, baseada no framework 5W2H (What, Where, When, While, Who, How e How much), para armazenar e recuperar requisitos de uma biblioteca (veja a Figura 12).

Os elementos desta linguagem registram as informações sobre os RNFs e sobre como relacioná-los a RFs, veja na Figura 13: *Why* – registra a intencionalidade, a razão para utilizar o RNF; ele ajuda na seleção de um grafo parcial; *Who* – registra o requisito destino (*target*) do RNF, a quem ele se aplica; *What* – registra as partes do RNF que devem ser aplicadas no requisito destino; *Where* – registra o ponto específico no requisito destino onde o RNF deve ser aplicado; *When* – registra pré-condições que precisam ser atendidas antes que operacionalizações (*How*) possam ser aplicadas em um ponto (*Where*); *How* – registra o refinamento do RNF através de RFs; *How much* – registra o impacto de aplicar as operacionalizações (*How*).

Who	What	Why	When	Where	How	How much
Função	tópico	Tipo	Claim	<i>pointcuts</i>	operacionalizações	elos de contribuição

Figura 13. Relação dos atributos de Q7 e os elementos do V-graph

Este processo utiliza dois V-graphs como entrada, o primeiro recuperado da biblioteca e o segundo representando a aplicação onde o RNF será inserido, e gera um terceiro grafo de metas com as informações integradas. Esta composição é realizada manualmente analisando onde (elemento *where*) na representação funcional o RNF deve ser ligado, e o que (elemento *what*) deve ser incluído. O trabalho sugere que esta tarefa pode ser automatizada, consistindo de um mecanismo similar ao *weaver* de linguagens de programação orientadas a aspectos (Capítulo 3). Este trabalho motivou nossa idéia de definir um mecanismo automatizado de composição de características transversais. No Capítulo 6, apresentamos as diferenças e semelhanças entre essa abordagem e a nossa.

2.7. Resumo

Neste Capítulo descrevemos o contexto e escopo desta tese, a engenharia de requisitos. Mostramos como os problemas de espalhamento e entrelaçamento atrapalham as atividades durante a definição de requisitos e as consequências destes problemas para o desenvolvimento de software como um todo. Além disto, apresentamos os conceitos de reuso, rastreabilidade, evolução e como aqueles problemas têm sido tratados pela utilização de diferentes visões.

3

Desenvolvimento de Software Orientado a Aspectos

O desenvolvimento de sistemas orientado a aspectos (DSOA) caracteriza uma nova sub-área da engenharia de software preocupada em desenvolver métodos, técnicas e ferramentas que dêem apoio a todas as fases do desenvolvimento. Ela leva em consideração a nova geração de tecnologias usadas para promover melhor separação de características, tais como programação orientada a aspectos e programação orientada a assuntos (*subjects*). Assim como quando surgiu o desenvolvimento orientado a objetos, DSOA surgiu depois que linguagens de programação atingiram um nível de maturidade suficiente para que haja a preocupação em propagar os novos conceitos e fundamentos para as etapas iniciais do desenvolvimento.

Na Figura 14, resumimos como os trabalhos em DSOA (utilizados nesta tese) estão relacionados entre si: o DSOA envolve trabalhos em *Early-aspects*, programação orientada a aspectos e programação orientada a assuntos; trabalhos em *Early-aspects* consideram trabalhos em engenharia de requisitos orientada a aspectos e trabalhos em desenho orientado a aspectos; AspectJ e HyperJ são exemplos de linguagens/ferramentas que possibilitam a programação orientada a aspectos (POA). Em (Brichau, 2005), encontramos um extenso relatório sobre linguagens de POA existentes.

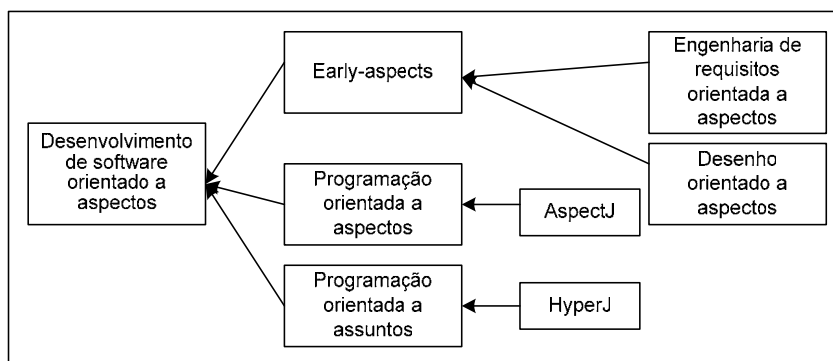


Figura 14. Visão geral dos trabalhos sobre desenvolvimento orientado a aspectos

Neste Capítulo apresentamos os principais conceitos sobre desenvolvimento orientado a aspectos que embasam esta tese. Na Seção 3.1, descrevemos as

propriedades da separação avançada de características. Nas Seções 3.2 e 3.3, apresentamos as linguagens AspectJ e Hyper/J. Na Seção 3.4, apresentamos algumas abordagens em engenharia de requisitos influenciadas pelo crescente interesse neste novo paradigma. Na Seção 3.5, resumimos como estas abordagens nos influenciaram.

3.1. Propriedades da Separação Avançada de Características

Abordagens para separação avançada de características, tais como, programação adaptativa (Lieberherr, 1994; Lieberherr, 1996), filtros de composição (Aksit, 1992) e programação orientada a assuntos (Harrison, 1993) foram responsáveis pela consolidação da programação orientada a aspectos, num sentido mais amplo. Em (Chavez, 2003, 2004), um modelo de aspectos é definido para esclarecer e unificar os conceitos que estas abordagens utilizam. Neste modelo de aspectos, Chavez (2003, 2004) define os seguintes elementos como necessários para que uma linguagem seja considerada orientada a aspectos, veja Figura 15⁷.

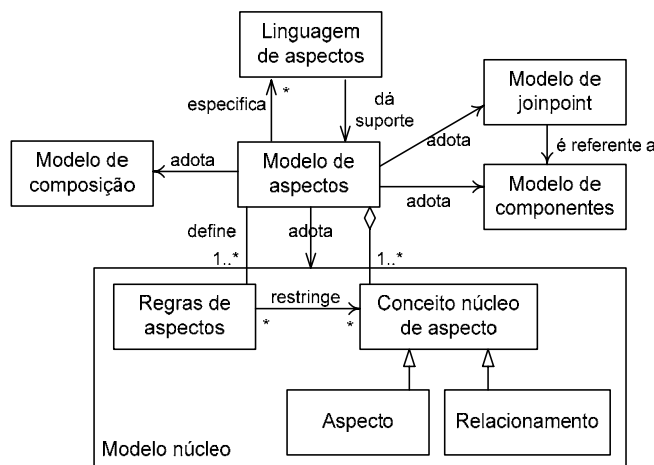


Figura 15. Modelo de aspectos

- Modelo de componentes – representa o modelo conceitual que define os tipos de componentes, regras e princípios gerais que ajudam a pensar sobre o problema e decompô-lo. Por exemplo, o modelo de objetos;
- Modelo de *joinpoint* – representa o modelo conceitual que define os elementos do modelo de componentes que podem ser afetados por um

⁷ Esta figura é uma tradução da figura original definida em (Chavez, 2003, 2004).

“aspecto”. O modelo de *joinpoint* é, assim, dependente do modelo de componentes adotado. Ele pode denotar um elemento relacionado à estrutura estática (contexto léxico) ou dinâmica (contexto dinâmico), por exemplo, métodos e atributos do modelo de objetos;

- Modelo núcleo (core) – representa o modelo conceitual que define: “aspectos” como uma abstração para modularizar características transversais e “relacionamento” (crosscutting) como um mecanismo de composição de aspectos e componentes. “Aspectos” especificam um conjunto de *joinpoints* (interfaces transversais – onde corta) e modificações (*features* transversais – o que é transversal) para serem aplicadas neles. “Relacionamento” é o relacionamento que liga um aspecto a um ou mais componentes, afetando sua estrutura e comportamento.
- Modelo de composição (weaving) – representa o modelo conceitual que define os tipos de mecanismos de composição. A composição é um processo que relaciona “aspectos” e componentes especificados nos *joinpoints*;
- Suporte à dicotomia – dicotomia é a clara distinção e separação do que são componentes e aspectos;
- Suporte a algum tipo de quantificação sobre a estrutura ou comportamento dos componentes (quantification) – definida como a habilidade de escrever declarações unitárias e separadas que atingem muitos lugares não-locais em um código;
- Suporte à transparência (obliviousness) – assume-se que os aspectos são invisíveis para os componentes, i.e., é a idéia de que os componentes não precisam conhecer nem ser preparados para receber as mudanças providas pelos aspectos.

Em (Chavez, 2003, 2004) é apresentada uma avaliação de algumas linguagens em relação a este modelo de aspectos. Na Tabela 1, ilustramos um resumo desta avaliação para as linguagens AspectJ e Hyper/J, e nas seções seguintes descrevemos estas duas linguagens.

Tabela 1. Avaliação de AspectJ e Hyper/J utilizando o modelo de aspecto

	AspectJ	Hyper/J
Modelo de componentes	modelo de objetos	modelo de objetos
Componente	Objeto	objeto
Linguagem de componentes	Java	Java
Modelo de <i>joinpoint</i>		
Joinpoints dinâmicos	pontos em execução	--
Joinpoints estáticos	classes	classes, campos, métodos, interfaces
Modelo núcleo		
Aspecto	aspect	<i>hyperslice?</i>
Interface transversal	<i>Pointcuts</i>	Hypermodule
Feature transversal	introduction, <i>advice</i>	campos, métodos
Modelo de composição	estático e dinâmico	estático
Dicotomia	sim	não
Quantificação	sim	sim
Transparência	sim	sim

3.2. AspectJ

O termo programação orientada a aspectos foi inicialmente utilizado entre 1995 e 1996 em um projeto da *Xerox Palo Alto Research Center* supervisionado por Gregor Kiczales (Lopes, 2004). Este grupo estava desenvolvendo algumas ferramentas de propósito específico (AML, RG, DJ) que influenciaram o surgimento da linguagem AspectJ como uma linguagem de **propósito geral** (Lopes, 2004).

AspectJ é uma extensão à linguagem Java (AspectJ, 2006). Nela, a separação é realizada através da inserção de uma nova abstração denominada “aspecto” à programação orientada a objetos. A composição é realizada através de um combinador denominado *weaver*, que é responsável por pré-processar o código orientado a objetos, inserindo ou modificando os objetos com o comportamento dos aspectos.

Além dos métodos e atributos, um aspecto é composto de *pointcuts*, *advice* e *intertype declarations*. Eles podem alterar a estrutura estática ou dinâmica de um programa. A estrutura estática é alterada adicionando, por meio das *intertype declarations*, membros (atributos, métodos ou construtores) a uma classe, modificando, assim, a hierarquia do sistema. A alteração na estrutura dinâmica de

um programa ocorre por meio dos *joinpoints*, que são selecionados por *pointcuts*, e pela adição de comportamentos (*advice*) antes ou depois dos *joinpoints*.

Um *pointcut* indica os pontos onde um determinado comportamento será inserido. Ele é descrito por um nome e um corpo. O corpo indica os *joinpoints* onde serão aplicados o *advice* associado, podendo haver vários *joinpoints* aninhados através dos operadores *or*, *and* e *not*. Cada *pointcut* está associado a um ou mais *advice*. *Advice* descreve o comportamento (trecho de código) a ser inserido nos *joinpoints*. Existem três formas principais de *advice*, são elas: *before*, *around* e *after*. Como seus nomes sugerem, *before* executa antes do *joinpoint*, *after* executa depois e *around* executa antes e depois, alternativamente, substituindo o comportamento do *joinpoint*.

Na Figura 16 (Brichau, 2005), exemplificamos os elementos de um aspecto na linguagem AspectJ. No aspecto abstrato FFDC são definidos três *pointcuts* (1) e um *advice* (2), que especifica a emissão de uma exceção (3) antes dos pontos afetados..

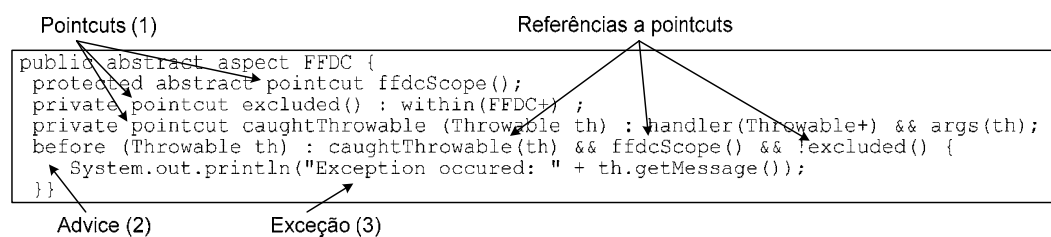


Figura 16. Exemplo de aspecto em AspectJ

Nossa abordagem utiliza os conceitos de *pointcut*, *advice* e *intertype declaration* para representar, em modelos de requisitos, como características transversais afetam umas as outras. No Capítulo 4, apresentamos nossa abordagem.

3.3. Hyper/J

Hyper/J originou-se da abordagem de separação multidimensional de características (Tarr, 1999), uma evolução da programação orientada a assuntos (Harrison, 1993). Com o objetivo de prover uma flexível e incremental separação, modularização e integração de artefatos de software baseados em diferentes características (Ossher, 2001; Hyper/J, 2006).

A programação orientada a assuntos foi proposta em 1993 como uma extensão do paradigma de orientação a objetos para abordar o problema de separar visões alternativas de uma classe ou um conjunto de classes. Harrison e Ossher (1993) definem a noção de especificar, separadamente, hierarquias diferentes de classes. Cada hierarquia implementaria uma característica do sistema e a composição delas seria realizada para construir o sistema, ou sistemas diferentes.

Assuntos são conjuntos de classes ou partes de classes que formam uma hierarquia para modelar uma característica. A combinação de hierarquias, escrita através de regras de composição, produz novos assuntos e é realizada por uma ferramenta de composição (Kaplan, 1996). Desta forma, o programador pode criar vários modelos, cada um encapsulando uma única característica, e assim, gerenciar os problemas de espalhamento e entrelaçamento de características.

Por volta de 1999, a programação orientada a assuntos evoluiu para separação multidimensional de características (SMC) (Ossher, 2001). O termo “multidimensional” significa “múltiplos critérios para decomposição”. SMC provê um meio para que os desenvolvedores decomponham o sistema, utilizando diferentes critérios (dimensões) ao mesmo tempo e de maneira não-invasiva. Desta forma, SMC trata da decomposição dominante.

Hyper/J é uma ferramenta desenvolvida pela IBM T.J. Watson Research Center que provê SMC para a linguagem Java (HyperJ, 2006). Em oposição a AspectJ, Hyper/J não modifica ou estende a linguagem Java. Hyper/J provê uma linguagem paralela para especificação de *hyperslices*, sendo usada juntamente ao código para gerar um novo conjunto de unidades integradas.

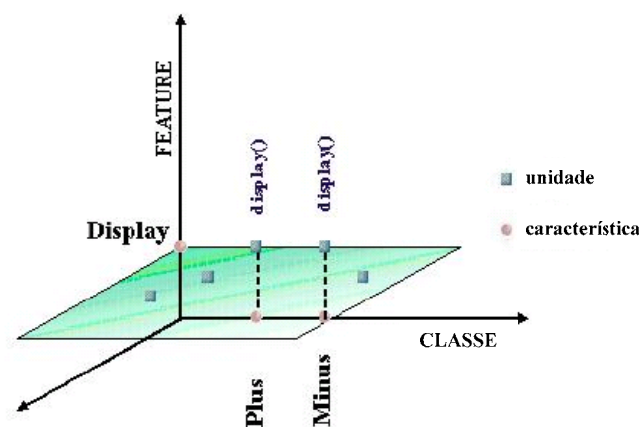


Figura 17. Exemplo de Hyperslice

Um *hyperslice* é um módulo que encapsula uma *feature*. Na Figura 17 (Chavez, 2004), o *hyperslice* denominado “display” é representado pelo plano que corta cinco unidades (representadas pelos quadrados). Destas cinco unidades, duas são os métodos “display” das classes “minus” e “plus”.

Para usar a ferramenta Hyper/J é necessário oferecer os seguintes arquivos (Chavez, 2004):

- Hiper-espaco – um arquivo que consiste de uma lista de classes Java com as quais o desenvolvedor está trabalhando e deseja aplicar Hyper/J.
- Mapeamento de características – um ou mais arquivos descrevendo quais unidades das classes Java (do hiper-espaco) endereçam cada *feature*.
- Hiper-módulo – um arquivo descrevendo quais *hyperslices* (*features*) devem ser integrados e como a integração deve ocorrer, incluindo a estratégia de composição e expressões de ordem. Um relacionamento de integração indica que unidades de correspondência nos diferentes *hyperslices* devem ser integradas umas com as outras.

Unidades de correspondência são unidades (interfaces, classes, métodos, atributos, dentre outros) relacionadas segundo algum critério e são compostas (misturadas ou sobre-escritas) dentro de uma nova unidade de software. As unidades de correspondência são identificadas utilizando uma das seguintes estratégias de composição: *mergeByName*, *nonCorrespondingMerge* e *overrideByName*. Pode-se ainda indicar a ordem em que a integração deve ocorrer através do relacionamento *Order*; a combinação de unidades, mesmo que seus nomes não sejam os mesmos, através do relacionamento *Equate*; e o relacionamento *Bracket* indica que um conjunto de métodos deve ser agrupado de maneira que quando invocado será precedido ou seguido por outros métodos.

Na Figura 18 (Brichau, 2005), exemplificamos a especificação de um hiper-módulo, que contém os *hyperslices*: *kernel*, *check*, e *display*. Estes *hyperslices* são integrados utilizando a estratégia *mergeByName*. O relacionamento utilizando *equate* indica uma correspondência entre as operações *process*, *check_process* e *display_process* das características *Kernel*, *Check* e *Display*, respectivamente. O resultado da composição são novos arquivos de classes, que podem ser utilizados na execução.

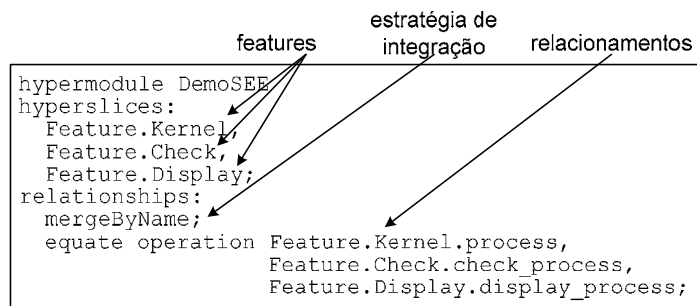


Figura 18. Exemplo de hiper-módulo em Hyper/J

Desta maneira, Hyper/J provê (HyperJ, 2006):

- Flexível separação e modularização de características – sistemas Java podem ser escritos com ou sem a separação de características multidimensional. Hyper/J possibilita a identificação, separação e manipulação de características, podendo ser utilizado apenas quando for necessário, durante a evolução.
- Composição – o mecanismo de composição sintetiza e integra tipos diferentes de características, e provê a habilidade de realizar *mix-and-match* e *plug-and-play* não invasivo, tratando alguns problemas de desenvolvimento, tais como: criação de extensões e configurações de software, sem modificar o código fonte original; customização e integração de sistemas e componentes reusáveis; facilitação do desenvolvimento em vários times, cada um trabalha em modelos independentes; manutenção e rastreabilidade entre todos os artefatos de software.
- Evolução e remodelarização não-invasiva e sob demanda – por facilitar a separação utilizando diferentes critérios, utilizando uma linguagem em paralelo a Java, Hyper/J permite efetuar mudanças no código à medida que novas características e melhores formas de decompô-las surgem.

Nossa abordagem é inspirada em Hyper/J da seguinte maneira: provemos um mecanismo de separação menos intrusivo, podendo ser utilizado de acordo com a demanda; e extraímos visões segundo diferentes critérios a partir da especificação estática (descrição léxica), tal como a composição realizada em Hyper/J. No Capítulo 4, Seção 4.2.3 e Seção 4.3.3, respectivamente, apresentamos estas propriedades em nossa abordagem.

3.4.

Engenharia de Requisitos Orientada a Aspectos

Early-aspects é uma nova sub-área da engenharia de software que está interessada em prover técnicas e métodos para sistematicamente identificar, separar, representar e compor características transversais nas fases de desenvolvimento anteriores à implementação (Rashid, 2002). Ainda não se tem uma clara definição para “aspectos” no processo de definição de requisitos (Bakker, 2005; Nuseibeh, 2004). A maioria das abordagens tem trabalhado com o conceito de “candidato a aspectos”, denotando características transversais, e em muitas destas, elas são somente RNFs.

Em (Chitchyan, 2005a) é apresentado um apanhado das abordagens para engenharia de requisitos que se preocupam com a separação e composição de características transversais. Os autores definem alguns critérios para comparar estas abordagens e avaliar novos nichos de pesquisa na área. Os critérios são, suporte à: identificação, composição, análise de interações e tomada de decisões, rastreabilidade, mapeamento, evolução e escalabilidade. Em (Chitchyan, 2005a, 2005b), a comparação entre estas abordagens é apresentada e é relatado que a identificação de características tem sido o tópico mais pesquisado, enquanto os demais critérios não têm recebido a devida atenção.

Como os trabalhos neste tópico apresentam idéias ainda em evolução, nesta seção não apresentamos conceitos consolidados, mas sim, as principais abordagens na área. Tentamos organizá-las por autor ou grupo de pesquisa, seguindo a ordem cronológica de publicação. Estes trabalhos nos inspiraram e foram decisivos para definição de nossa abordagem.

3.4.1.

Rashid, Moreira e Brito

Em (Rashid, 2002), é definido um modelo genérico para engenharia de requisitos orientada a aspectos (modelo EROA). Este modelo explicita a necessidade de tratar características transversais cedo, e primeiramente define que estas características são “candidatas a aspectos”. Neste trabalho, apenas RNFs em um alto nível de abstração são identificados como candidatos a aspectos.

A identificação de candidatos a aspectos é realizada por identificar em uma matriz requisitos versus características (*concerns*) quais as características que cruzam mais de um requisito. Posteriormente, cada candidato a aspecto é analisado em mais detalhe para a identificação de conflitos e estabelecimento de prioridades. Por fim, deve-se especificar o impacto de aspectos candidatos em termos de duas dimensões: i) mapeamento – indica como ele é mapeado para fases posteriores do desenvolvimento (função, decisão arquitetural ou aspecto); e ii) influência – estabelece quais as atividades do ciclo de desenvolvimento são afetadas pelo candidato a aspectos (requisitos, arquitetura, projeto ou manutenção).

Em (Rashid, 2003), o modelo EROA é melhorado por incluir as atividades de composição e tratamento de conflitos, veja Figura 19. A composição determina como candidatos a aspectos influenciam ou restringem requisitos, através de regras de composição. O tratamento de conflitos é realizado através do preenchimento de uma matriz de contribuição entre aspectos candidatos, e atribuição de pesos quando um contribui negativamente para a realização do outro.

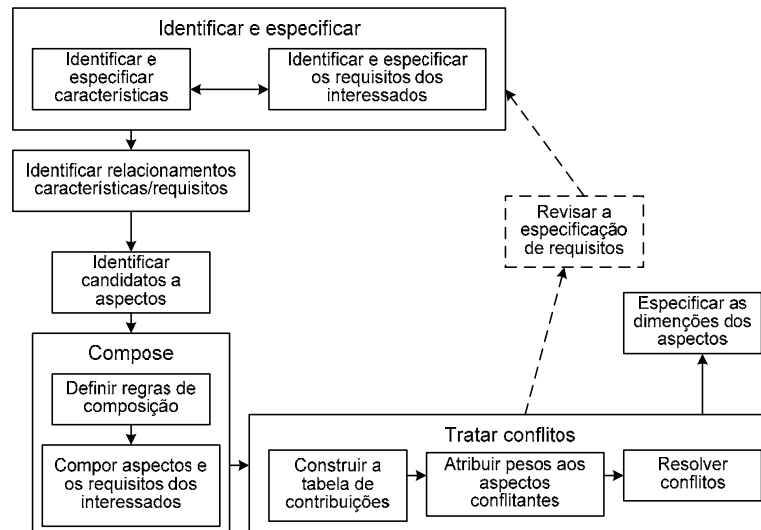


Figura 19. Modelo EROA

Neste trabalho, Rashid (2003) define uma instância do modelo EROA baseada em pontos de vista (Finkelstein, 1996). Nesta instância, a identificação de características transversais é realizada por meio de uma matriz que apresenta pontos de vista (visões como opiniões) versus requisitos. Se um requisito cruza vários pontos de vista então ele é um candidato a aspectos. Requisitos são escritos na forma de sentenças em linguagem natural não estruturada. XML é utilizado

para especificar tanto sentenças de requisitos, quanto candidatos a aspectos e regras de composição, veja um exemplo na Figura 20.

Os três formatos de descrição, ilustrados na Figura 20 representam uma maneira formal de descrever os requisitos e as composições. Entretanto:

- são utilizadas sentenças de requisitos não estruturadas, não permitindo a composição entre partes do requisito;
- duas representações diferentes são utilizadas para os requisitos de pontos de vista e requisitos que são candidatos a aspectos, dificultando o reuso;
- a composição não pode ser realizada entre candidatos a aspectos ou entre pontos de vista;
- o formato utilizado para especificar regras de composição, apesar de ser simples, torna a especificação mais extensa porque não provê uma maneira simplificada de escrever que uma característica transversal afeta vários pontos (propriedade de quantificação, definida na Seção 3.1), assim, para cada ponto tem que ser especificada uma regra.

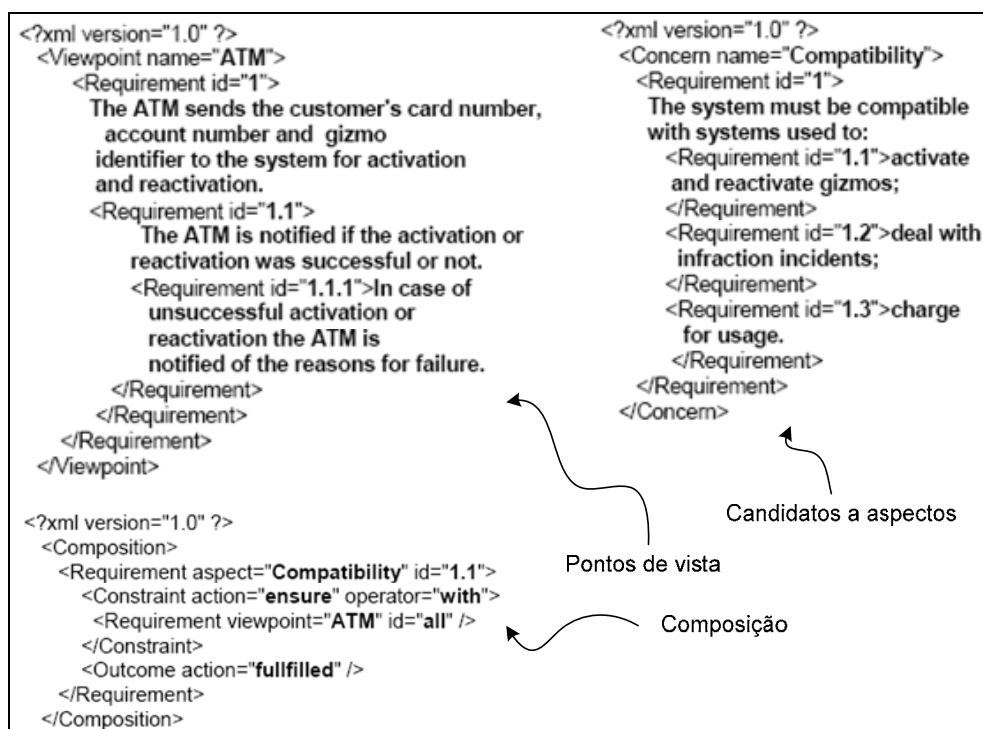


Figura 20. Exemplo de descrição de requisitos no modelo EROA

Em (Moreira, 2002) é definido um processo para identificação e especificação de RNFs que cortam RFs, bem como a integração entre eles representada por modelos de casos de uso e diagramas de sequência. Na Figura 21a, ilustramos o *template* utilizado para especificar RNFs e na Figura 21b há um

exemplo ilustrativo. As informações contidas nas linhas *Where* e *Requirements* do *template* (Figura 21) são utilizadas para identificar se o RNF corta vários casos de uso. Se sim ele é considerado um RNF transversal.

Os conceitos (operadores) *overlapping*, *overriding* e *wrapping* são utilizados para representar, respectivamente, mudança, sobreposição ou encapsulamento do requisito afetado. No diagrama de casos de uso os RNFs transversais são representados por casos de uso estereotipados, os detalhes da composição são mostrados em diagramas de sequência.

Em (Brito, 2004), Brito amplia a abordagem definida em (Moreira, 2002), utilizando o framework NFR para identificar e especificar RNFs. Além disto, são definidos os operadores *enabling*, *disabling*, *pure interleaving* e *full synchronization*, inspirados nos operadores LOTOS, para registrar a ordem em que os RNFs transversais devem ser aplicados.

Name The name of the quality attribute	Name Security
Description Executive description	Description Restricts the access to the system and to the data within the system
Focus A quality attribute can affect the system (i.e. the end product) or the development process	Focus System
Source Source of information (e.g. stakeholders, documents)	Source Stakeholders
Decomposition Quality attributes can be decomposed into simpler ones. When all (sub) quality attributes are needed to achieve the quality attribute, we have an AND relationship. If not all the sub quality attributes are necessary to achieve the quality attribute, we have an OR relationship	Decomposition Integrity and Confidentiality. Both have an AND relationship with Security
Priority Expresses the importance of the quality attribute for the stakeholders. A priority can be MAX, HIGH, LOW and MIN	Priority MAX
Obligation Can be optional or mandatory	Obligation Mandatory
Influence Activities of the software process affected by the quality attribute	Influence Design, system architecture and implementation
Where List of the actors influenced by the quality attribute and also a list of models (e.g. use cases and sequence diagrams) requiring the quality attribute	Where Actors: VehicleOwner, Bank Use cases: PayBill, RegisterVehicle
Requirements Requirements describing the quality attribute	Requirements The system must: 1. protect the vehicle's owner registration data 2. guarantee integrity in the data transmitted to the bank 3. guarantee integrity on data changed/queried by the operator 4....
Contribution Represents how the quality attribute affects other quality attributes. This contribution can be positive (+) or negative (-)	Contribution (-) to response time and (+) to multiuser and compatibility.

(a)

(b)

Figura 21. a) Template para RNFs, b) exemplo utilizando o template

Assim, as abordagens em (Brito, 2004; Moreira, 2002) consideram apenas RNFs como características transversais e as informações informais contidas no *template* são utilizadas apenas como guia para o engenheiro construir manualmente os diagramas; nenhum suporte ferramental é proposto.

Em (Moreira, 2005a, 2005b), os autores apresentam uma abordagem denominada multi-dimensional, para explicitar a necessidade de considerar RFs tanto quanto RNFs como candidatos a aspectos. Nesta abordagem define-se:

- um espaço de meta-características (*meta-concerns*) onde características concretas, específicas de uma aplicação, podem ser derivadas baseando-se nas *features* específicas de um domínio de problema;
- a noção de interseção composicional, possibilitando a identificação de conjuntos de características como base (foco), de maneira a observar as interações (*trade-offs*) entre as outras características; e
- a utilização da análise destas interações para prover um guia de mapeamento destas estruturas para a arquitetura.

Esta abordagem utiliza os formatos de descrição para regras de composição e para características definidos em (Rashid, 2003), mas substitui a descrição de pontos de vista, por uma descrição de meta-características, veja um exemplo na Figura 22. As regras de composição definidas em (Rashid, 2003) são utilizadas. A diferença é que em (Moreira, 2005a, 2005b) uma característica pode restringir qualquer outra característica e não apenas pontos de vista como em (Rashid, 2003).

```

<?xml version="1.0" ?>
<MetaConcern name="InformationRetrieval">
  <Description>The operation of accessing information from a computer system</Description>
  <Examples>Database retrieval, Multimedia retrieval</Examples>
  <Relationships>Availability, Mobility, InformationUpdate</Relationships>
</MetaConcern>

<?xml version="1.0" ?>
<Concern name="InformationRetrieval">
  <Requirement id="1">It should be possible to retrieve information from the system.
    <Requirement id="1.1">It should be possible to access information about the attractions</Requirement>
    <Requirement id="1.2">It should be possible to access information about the current location</Requirement>
  </Requirement>
  <Requirement id="1.3">It should be possible to obtain a list of available preset tours</Requirement>
</Concern>

<?xml version="1.0" ?>
<Composition>
  <Requirement concern="InformationRetrieval" id="all" />
  <Constraint action="provide" operator="during">
    <Requirement concern="Customisability" id="all"/>
    <Requirement concern="Navigation" id="1"/>
    <Requirement concern="Mobility" id="1" children="include"/>
    <Requirement concern="InformationUpdate" id="all"/>
  </Constraint>
  <Outcome action="fulfilled"/>
</Requirement>
</Composition>

```

Meta-concern

Concern

Composição

Figura 22. Exemplo de descrição de requisitos na abordagem multidimensional

3.4.2. Baniassad

Em (Baniassad, 2004) é definida uma abordagem para análise (Theme/Doc) e desenho (Theme/UML) baseados no conceito de “temas” (*themes*). Um tema é um conjunto de estruturas e comportamentos que representam uma *feature*. O modelo de temas define dois tipos de temas, base e transversais. Os temas transversais são aqueles que compartilham estruturas e/ou comportamentos com outros temas, e são, assim, considerados “aspectos”.

Theme/Doc provê visões e suporte ferramental para identificação e representação de características transversais. As visões de ações (*action view*), de ações agrupadas (*clipped action view*) e a visão de temas (*theme view*) são geradas semi-automaticamente através da análise léxica de um documento de requisitos. Na Figura 23 e Figura 24, os losangos representam os verbos das sentenças de requisitos e as elipses representam as sentenças onde eles aparecem. Por meio das interações entre as ações visualizadas na Figura 23, e análise das sentenças de requisitos, o engenheiro percebe que a ação “logged” corta as ações “register”, “unregister” e “give”, retratando este relacionamento transversal com um elo cinza na Figura 24.

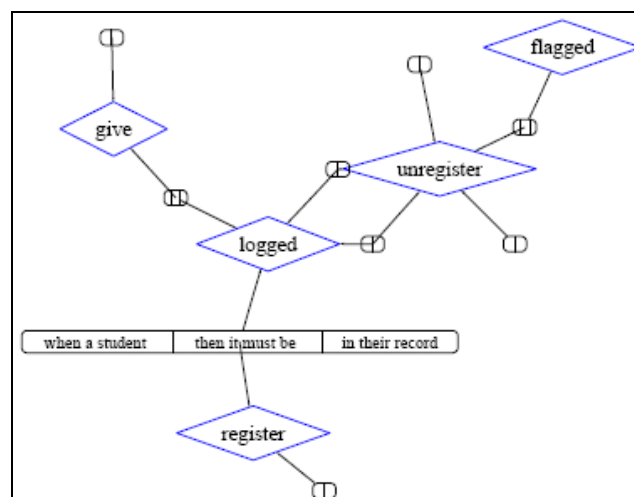


Figura 23. Exemplo da visão de ações de Theme/Doc

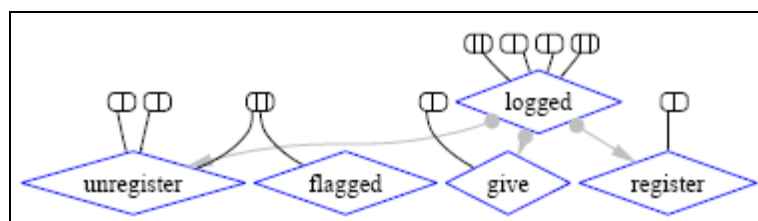


Figura 24. Exemplo da visão de ações agrupadas de Theme/Doc

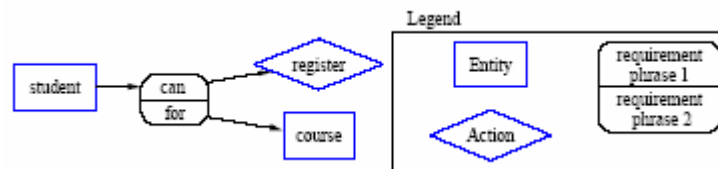


Figura 25. Exemplo da visão de temas de Theme/Doc

Estas visões ajudam o engenheiro a analisar as interações entre requisitos, e assim, separá-los em dois grupos. Um grupo onde os requisitos são auto-contidos, i.e, nenhum requisito de um conjunto interage com requisitos de outro conjunto, e o segundo grupo onde os conjuntos possuem requisitos que interagem com os de outros conjuntos, i.e, são transversais. A visão de temas (Figura 25) é utilizada para guiar o engenheiro na modelagem da arquitetura usando Theme/UML, que consiste de algumas extensões para os diagramas UML.

A geração destas visões é semi-automática e o engenheiro precisa fornecer outras informações além das sentenças de requisitos, desta forma mudanças em requisitos podem demandar algum re-trabalho ao engenheiro. Além disto, nenhum mecanismo para composição de características transversais é proposto.

3.4.3. Sousa

Em (Sousa, 2003), propõe-se uma adaptação do framework NFR (Mylopoulos, 1992). Esta adaptação modifica algumas atividades do framework, para incluir a composição e o mapeamento de RNFs transversais, veja a Figura 26. Os operadores de composição definidos em (Moreira, 2002), representando os conceitos *overlapping*, *overriding* e *wrapping*, são utilizados para adicionar a semântica dos candidatos a aspectos aos relacionamentos dos grafo de RNFs no nível de operacionalização, e não no nível abstrato como em (Moreira, 2002). Veja o exemplo ilustrado na Figura 27 (Sousa, 2003).

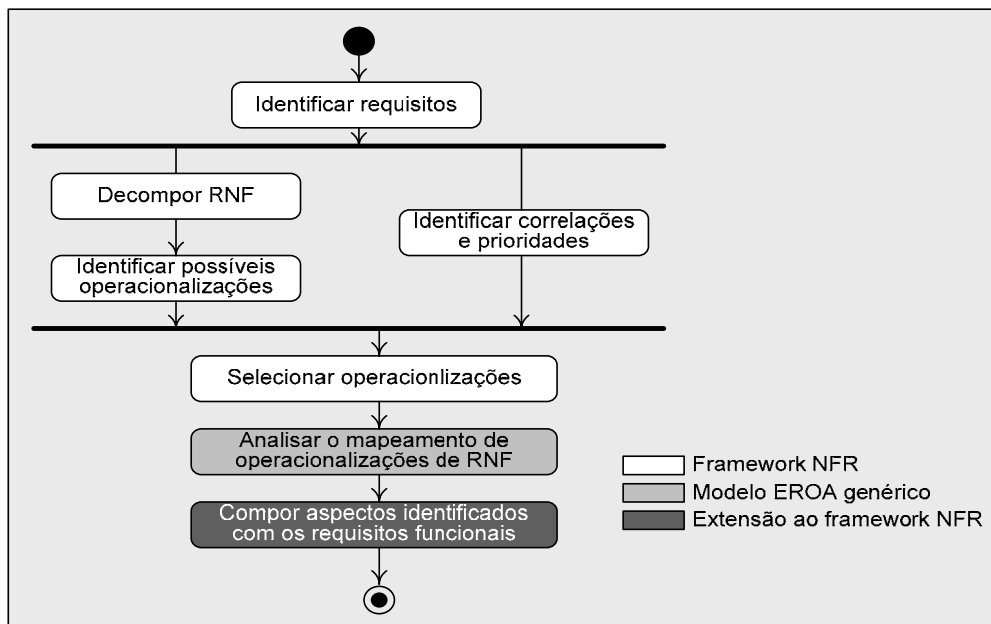


Figura 26. Adaptação do framework NFR

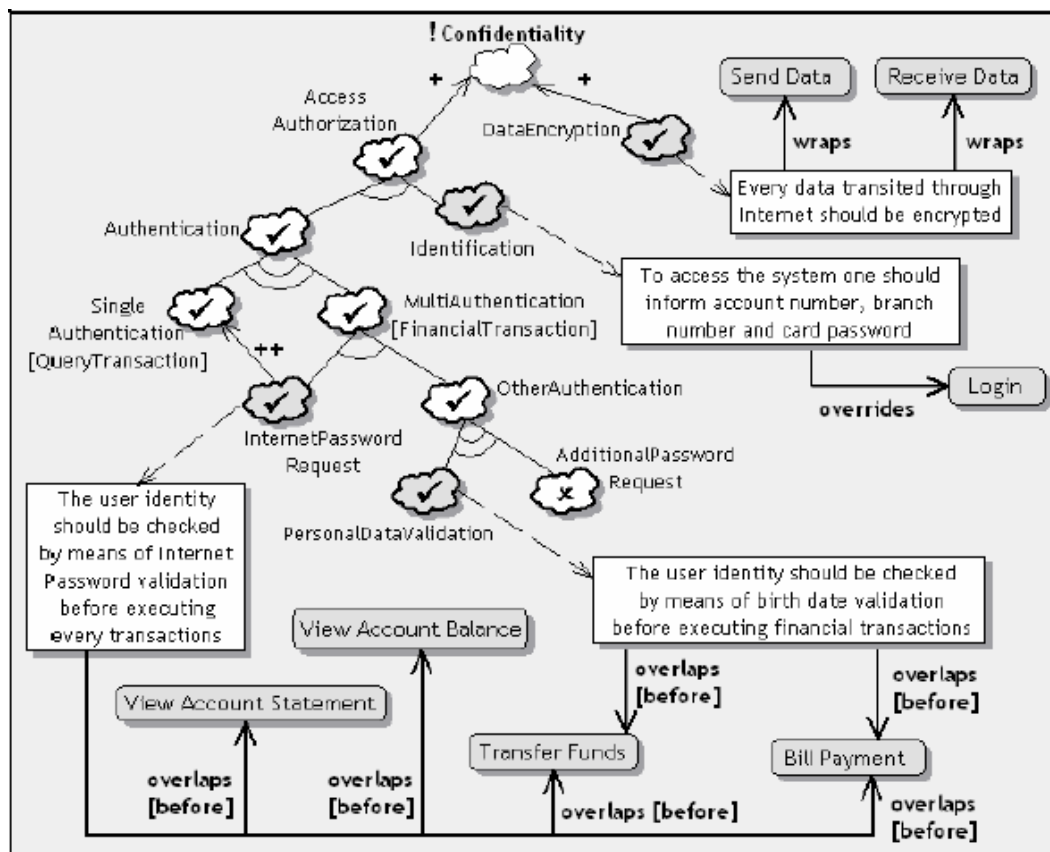


Figura 27. Exemplo da modelagem de candidato a aspectos no grafo de RNFs

Em (Sousa, 2004a, 2004b) é proposta a integração da abordagem de desenvolvimento dirigida por caso de usos (Jacobson, 1999) e o framework NFR (Chung, 2000), para dar suporte à separação de características transversais. O processo criado consiste em modelar RNFs e casos de uso, separadamente, e

então, especificar em tabelas como os RNFs afetam os casos de uso. Com base nesta especificação, o engenheiro pode incluir as operacionalizações de RNFs nos diagramas de casos de uso, relacionando-os com “*extend*”. Além disto, os autores propõem: mudar o rótulo “*extend*” dos casos de uso para “*crosscut*” quando ele relaciona uma característica transversal; e usar um *template*, similar ao *template* dos casos de uso, para descrever as operacionalizações dos RNFs. Veja o exemplo ilustrado na Figura 29.

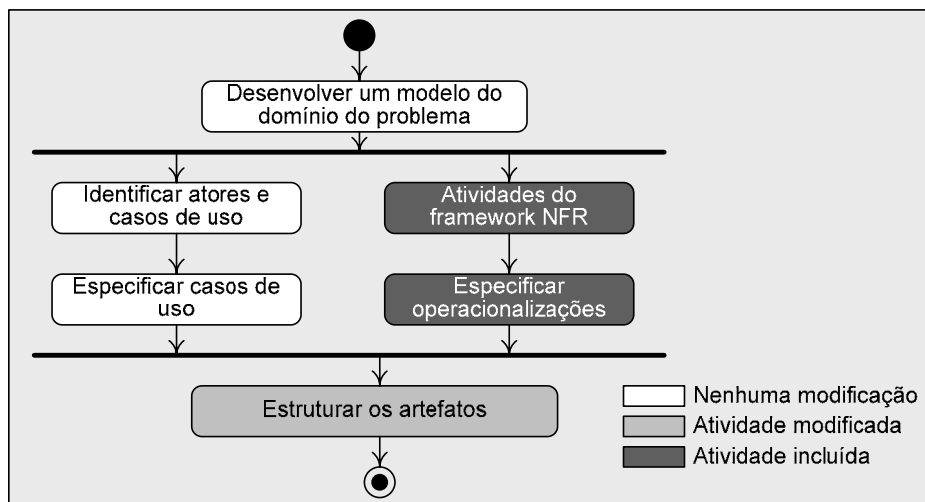


Figura 28. Adaptação do processo dirigido por caso de uso

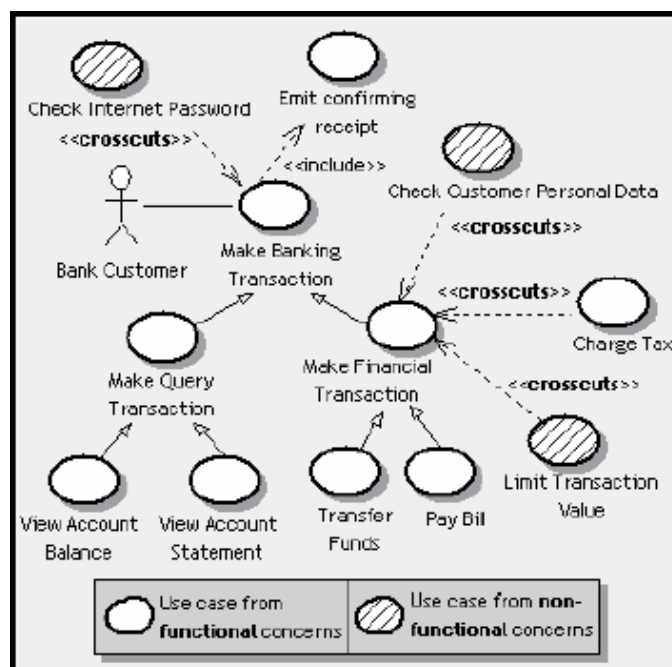


Figura 29. Exemplo de modelagem de candidato a aspectos em casos de uso

As abordagens definidas em (Sousa, 2003, 2004a, 2004b) trazem a preocupação em considerar as operacionalizações de RNFs, ao invés de RNFs abstratos. São definidas algumas diretrizes para identificar, integrar e mapear

características transversais para fases posteriores, mas a linguagem para retratar características transversais é apenas diagramática, criada manualmente com apoio das descrições informais de composição. Não há facilidades de ferramentas para modelagem dos artefatos propostos (diagrama de casos de uso, *template* de casos de uso, tabela de composição, diagrama de colaboração), sendo difícil manter o rastro entre eles.

3.5. Resumo

Neste Capítulo descrevemos os conceitos principais que influenciaram o desenvolvimento de nossa abordagem para integração de características transversais durante a engenharia de requisitos. Na Seção 3.1, descrevemos as principais propriedades da separação avançada de características e o meta-modelo, definido em (Chavez, 2003, 2004), para abordagens orientadas a aspectos.

Na Seção 3.2 e 3.3, apresentamos as linguagens de programação orientada a aspectos, AspectJ e Hyper/J. AspectJ nos influenciou em definir um relacionamento transversal, como elemento da linguagem de modelagem, para registrar como algumas características se espalham e se entrelaçam a outras. Ao mesmo tempo, Hyper/J nos influenciou com a idéia de ser menos intrusivo e registrar grupos de elementos de acordo com diferentes critérios, possibilitando que a decomposição dominante da linguagem de modelagem seja utilizada, mas que outras maneiras de separação possam coexistir em um outro espaço.

Hyper/J e AspectJ são consideradas, respectivamente, abordagens simétricas e assimétricas (Harrison, 2002). Não realizamos uma avaliação aprofundada sobre a simetria de nossa abordagem, mas consideramos que nossa abordagem não pode ser classificada integralmente como simétrica nem como assimétrica. Ela apresenta simetria nos elementos porque não há distinção entre os elementos que estão transversais e aqueles que não estão, mas define um novo tipo de relacionamento que deve ser utilizado para explicitamente relacionar elementos que estão transversais. Por outro lado, nossa abordagem apresenta assimetria no relacionamento porque explicitamente define os pontos que afetam e são afetados pelas características transversais.

Na Seção 3.4, resumimos as principais abordagens em engenharia de requisitos orientada a aspectos. Todos estes trabalhos inspiraram a idéia para

nossa proposta. Entretanto, eles estão focados em definir métodos e técnicas que ajudam o desenvolvimento de software orientado a aspectos, tentando promover o paradigma de aspectos em todas as fases de desenvolvimento, enquanto nós estamos primeiramente interessados em ajudar a engenharia de requisitos por utilizar técnicas e ferramentas inspiradas naquelas de programação, e, só então, ajudar à engenharia de requisitos orientada a aspectos. Nosso foco é a modelagem e composição de características transversais, com o propósito de fornecer diferentes visões do modelo integrado, facilitando a inclusão e exclusão de diferentes características da modelagem de requisitos. Uma comparação detalhada entre estes trabalhos e o nosso é apresentada no Capítulo 6.

4

Estratégia Orientada a Aspectos para Modelagem de Requisitos

Como descrevemos no Capítulo 2, durante a definição de requisitos os problemas de espalhamento e entrelaçamento de características dificultam a modelagem, rastreabilidade, evolução e a reutilização de requisitos. Visto que estes problemas têm sido abordados durante a implementação, por meio de linguagens de programação orientadas a aspectos, desenvolvemos uma estratégia baseada nos fundamentos deste paradigma para prover facilidades à modelagem de requisitos.

Esta estratégia consiste em um metamodelo para integração de características transversais. Este metamodelo define um conjunto mínimo de atividades para modelar requisitos considerando características transversais, e mecanismos para efetivá-las. A integração é centrada na definição de um elemento de modelagem que permite a descrição explícita de como as características estão entrelaçadas e espalhadas em modelos de requisitos. Este elemento é definido como parte de uma linguagem de modelagem de requisitos orientada a aspectos (LMROA), que pode ser instanciada para a utilização de diferentes métodos de modelagem.

A seguir, na Seção 4.1, apresentamos o metamodelo para integração de características transversais e seus componentes. Na Seção 4.2, definimos a linguagem de modelagem de requisitos orientada a aspectos. Na Seção 4.3, detalhamos como utilizar esta estratégia aplicando-a ao modelo V-Graph. Na Seção 4.4, descrevemos como aplicar nossa estratégia aos modelos de cenários (Leite, 1997), sentenças de requisitos ou léxico (Leite, 1990). Na Seção 4.5, apresentamos como esta estratégia afeta as atividades de elicitação e análise do processo de engenharia de requisitos. Na Seção 4.6, resumimos nossa abordagem.

4.1.

Metamodelo para Integração de Características Transversais

O metamodelo para integração de características transversais é uma estratégia para modelagem de requisitos que provê elementos para registrar, controlar e analisar a interação transversal entre eles. Desta forma, a estratégia oferece ao engenheiro uma nova maneira de raciocinar e modelar os requisitos e oferece, também, mecanismos que facilitam a visualização, análise e alteração do documento sendo criado (Silva, 2005c; 2005d).

A estratégia consiste de três atividades, denominadas de Separar, Compor e Visualizar, veja Figura 30. Colocamos estas três atividades como sendo essenciais durante o processo de modelagem de requisitos porque elas possibilitam, respectivamente, a modularização, a junção das partes modularizadas para formar o sistema e a análise de visões (opiniões, serviços e modelos) dos módulos e do sistema inteiro.

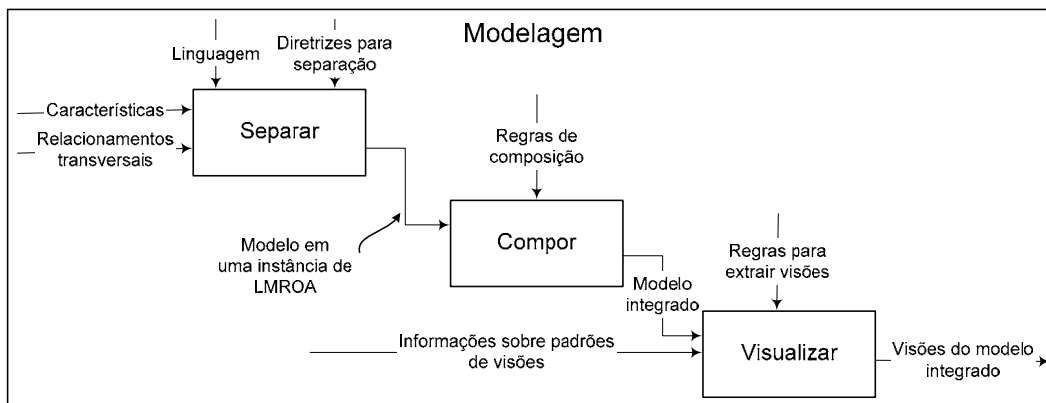


Figura 30. Metamodelo para integração de características transversais

Para Separação, provemos uma linguagem e diretrizes para que os requisitos sejam modelados levando em consideração sua natureza transversal, ou seja, explicitando quais e como alguns requisitos têm tendência a estar espalhados ou entrelaçados aos outros. Esta informação sobre transversalidade é registrada em um novo tipo de relacionamento, adicionado como uma extensão ao modelo de requisitos. Colocando em evidência a transversalidade dos requisitos podemos mais cuidadosamente observá-la, e assim entender como os requisitos afetam uns aos outros.

A Composição é um processo automatizado que combina os requisitos inicialmente modelados de maneira separada. Esta combinação é realizada pela interpretação da informação contida no relacionamento transversal e aplicação de

regras de composição. As regras de composição definem como os elementos do relacionamento transversal são transformados, i.e., sua semântica, propagando a informação nele contida.

O modelo resultante da composição e informações sobre quais visões podem ser extraídas de suas informações são necessários para a atividade de Visualização. Esta atividade provê alguns modelos parciais do modelo integrado, de maneira a facilitar o entendimento das características do sistema e da composição delas. Nós consideramos que, no momento da modelagem, o mais interessante para o desenvolvedor não é ter apenas a representação do sistema completo, mas sim, diferentes visões parciais deste mesmo modelo.

As atividades de separação, composição e visualização compõem juntamente às atividades de elicitação e análise o processo iterativo de definição de requisitos. Na Seção 4.5 descrevemos qual o impacto de nossa estratégia nas atividades de elicitação e análise.

Componentes da Estratégia

Nossa estratégia para integração de características transversais define: uma linguagem de modelagem de requisitos orientada a aspectos (LMROA), diretrizes de como utilizá-la, um mecanismo de composição e um mecanismo de visualização, veja na Figura 31. Os mecanismos utilizam regras de composição e de transformação que dependem dos construtos de LMROA. Desta forma, a separação, a composição e a visualização são centradas na linguagem de modelagem de requisitos orientada a aspectos.

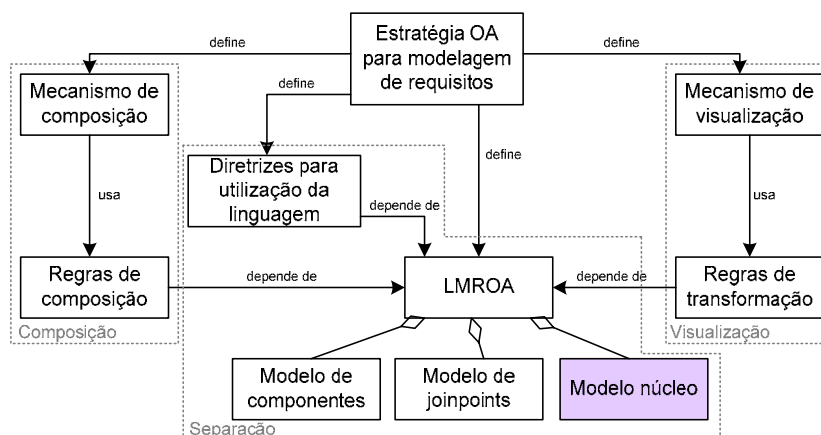


Figura 31. Componentes utilizados para integração de características transversais

LMROA é composta por: qualquer linguagem de modelagem de requisitos (modelo de componentes), tais como cenários, casos de uso ou modelo de metas; um relacionamento transversal (modelo núcleo), que foi definido com base em alguns conceitos da linguagem AspectJ; e um modelo de *joinpoints* que define como os modelos de componentes e núcleo estão relacionados. LMROA e as diretrizes para sua utilização constituem a infra-estrutura necessária para a atividade de separação, na qual as atividades de composição e visualização se baseiam.

É importante ressaltar que nos baseamos no metamodelo definido por Chavez (2003, 2004) para definir LMROA. Assim, utilizamos os nomes “modelo de componentes” e “modelo de joinpoints” com a mesma semântica descrita em (Chavez, 2003, 2004) apesar de estarmos trabalhando em um nível de abstração diferente.

Fazendo analogia às práticas utilizadas no nível de implementação temos que, veja a Figura 32:

- a atividade de separação é equivalente à modularização provida por linguagens de programação com sua sintaxe, semântica e diretrizes para melhor implementar diferentes características;
- a atividade de composição é equivalente ao *weaver* ou compilador das linguagens de programação; e
- o mecanismo de visualização é equivalente à realização de consultas (pattern match) em busca de diferentes características no código dos componentes (como acontece em Hyper/J) ou à visão representada pelo comportamento do programa em execução (como acontece em AspectJ).

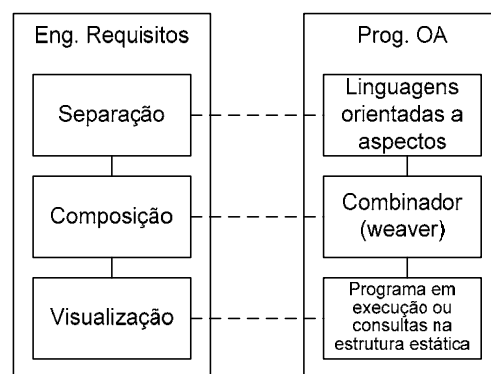


Figura 32. Analogia entre a estratégia para integração de características transversais durante a definição de requisitos e algumas práticas utilizadas no nível de implementação

A seguir, apresentamos um pequeno exemplo para ilustrar as atividades de Separação, Composição e Visualização. Detalhes de cada uma delas são apresentados nas próximas Seções.

Exemplo

Considere um sistema para edição de cenários, em que há dois principais grupos de requisitos: os específicos para edição, e os relacionados à persistência. Na Figura 33, representamos estes grupos de requisitos em V-graph, sem utilizar nossa abordagem. Neste grafo, podemos observar que as tarefas “Include [data]”, “Update [data]”, “Exclude [data]” e “Select [data]” afetam várias tarefas do grafo “Model [requirements]” (veja os relacionamentos em negrito). Estes relacionamentos dificultam a visibilidade do modelo e têm que ser desenhados, analisados, inspecionados e mantidos pelo engenheiro de requisitos quando o modelo muda.

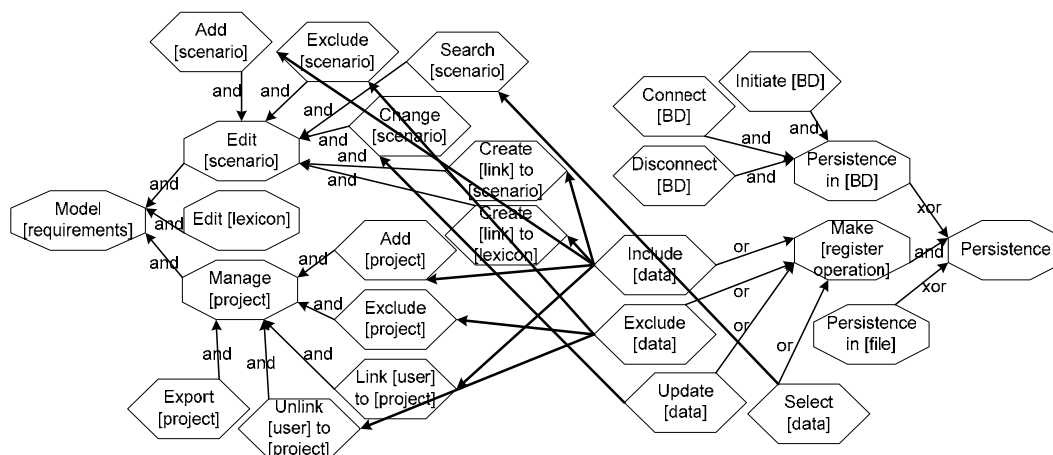


Figura 33. Exemplo ilustrativo da abordagem tradicional

A linguagem de modelagem e os mecanismos propostos por nossa abordagem possibilitam a separação, composição e visualização da seguinte maneira.

A atividade de separação é responsável por possibilitar a modelagem destes dois grupos de requisitos separadamente, bem como a modelagem de como cada um deles afeta os demais, por meio de relacionamentos transversais (elos em negrito na Figura 34 e descrição). Diretrizes indicando como e quando realizar esta separação são apresentadas na Seção 4.2.4.

O mecanismo de composição é responsável por processar estes modelos e realizar a propagação das informações contidas nos relacionamentos transversais, gerando, então o modelo ilustrado na Figura 33, o modelo complexo mostrado

anteriormente. Entretanto, com nossa abordagem, o responsável por desenhar e propagar mudanças nestes relacionamentos é o mecanismo de composição.

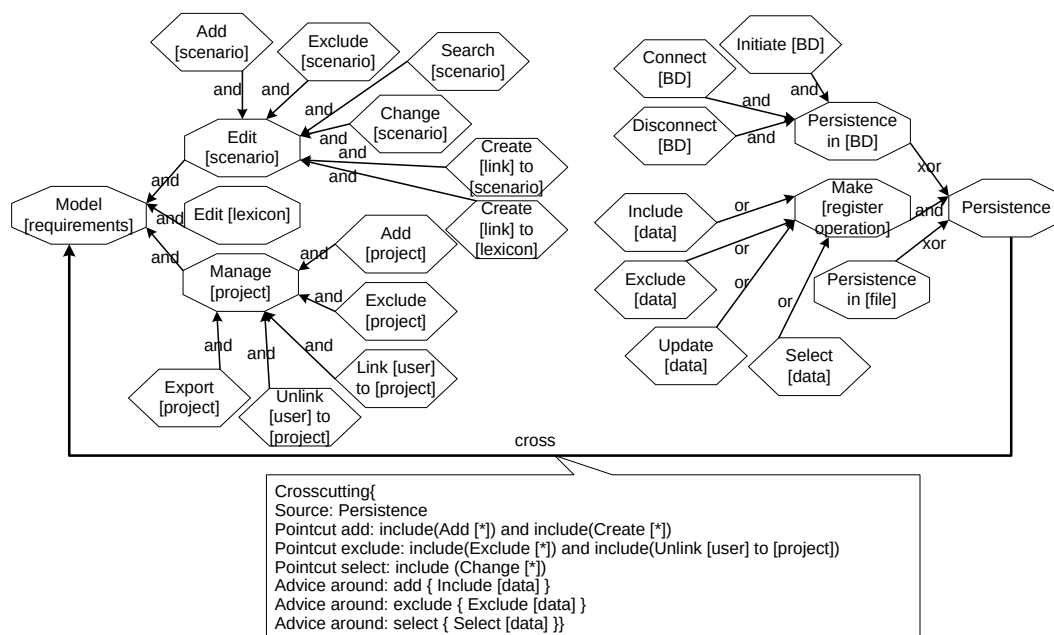


Figura 34. Exemplo ilustrativo - Separação

Além disso, com o mecanismo de visualização, o engenheiro de requisitos pode extrair diferentes visões, parciais ou totais do modelo integrado, por exemplo, matrizes de rastreabilidade, mapa de tópicos (*topic map*), como a confidencialidade se espalha ou se entrelaça às demais características, dentre outras. Na Figura 35, mostramos uma visão parcial do modelo integrado (ilustrado na Figura 33), focando o serviço de persistência.

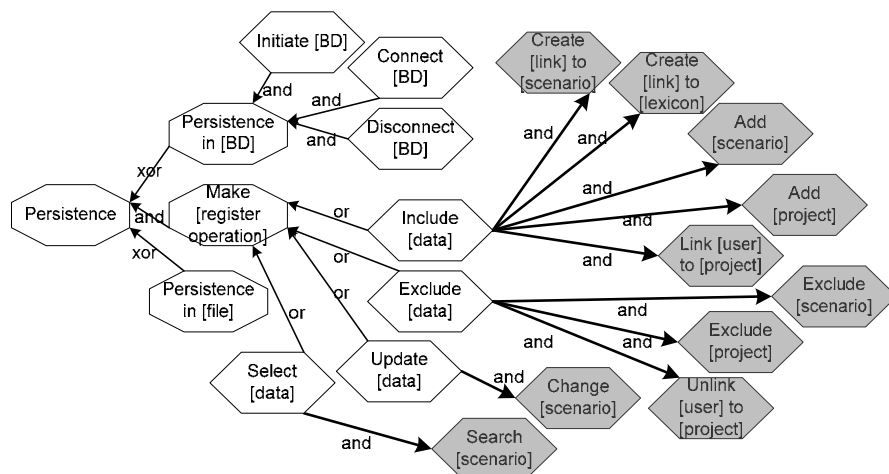


Figura 35. Exemplo ilustrativo - Visualização

Por deixar a informação de transversalidade centralizada no relacionamento transversal e deixar a cargo do mecanismo de composição espalhar esta informação:

- facilitamos a **modelagem** e **análise**, pois o engenheiro de requisitos pode se concentrar em um conjunto de requisitos por vez, não precisando manipular modelos grandes e com muitas interações. Isto diminui a complexidade de cada modelo;
- promovemos o **reuso**, porque alguns grupos de requisitos podem ser reaproveitados em outros projetos, sendo necessário modificar, apenas, a maneira na qual eles afetam os demais grupos;
- facilitamos a **rastreabilidade** entre requisitos, pois as informações de transversalidade estão centralizadas no relacionamento transversal. Estas informações são mais difíceis de rastrear quando utilizando a abordagem tradicional porque estão relacionadas a características que afetam ou são afetadas por muitas outras; e
- provemos facilidades para **modificar** os requisitos. Visto que eles foram modelados separadamente e o impacto que eles causam também está separado, é mais fácil modificar ambas as informações, pois fica a cargo do mecanismo de composição propagar estas mudanças.

No Capítulo 5, ilustramos estas contribuições nos estudos de caso. A seguir, na Seção 4.2, apresentamos LMROA e detalhamos seus modelos constituintes.

4.2.

Linguagem de Modelagem de Requisitos Orientada a Aspectos

Estruturalmente, a linguagem de modelagem de requisitos orientada a aspectos (LMROA) consiste das seguintes partes: 1) a definição dos elementos da linguagem de modelagem de requisitos, i.e., o modelo de componentes da linguagem; 2) a definição dos elementos do modelo núcleo, neste caso, retratados pelo relacionamento transversal; e 3) o modelo de *joinpoints*.

- | |
|--|
| <ol style="list-style-type: none"> 1. <LMROA>:= <model> 2. <model>:= <component><relationship> 3. <relationship>:= <crosscuttingRel> relacionamentos do modelo de componentes 4. <component>:= componentes do modelo de componentes 5. <joinpoint>:= elementos do modelo de componentes |
|--|

Figura 36. Sintaxe de LMROA

Na Figura 36 e Figura 37, ilustramos os detalhes destes modelos. Os elementos em cinza representam os pontos fixos da linguagem, enquanto os elementos em branco representam pontos a serem instanciados.

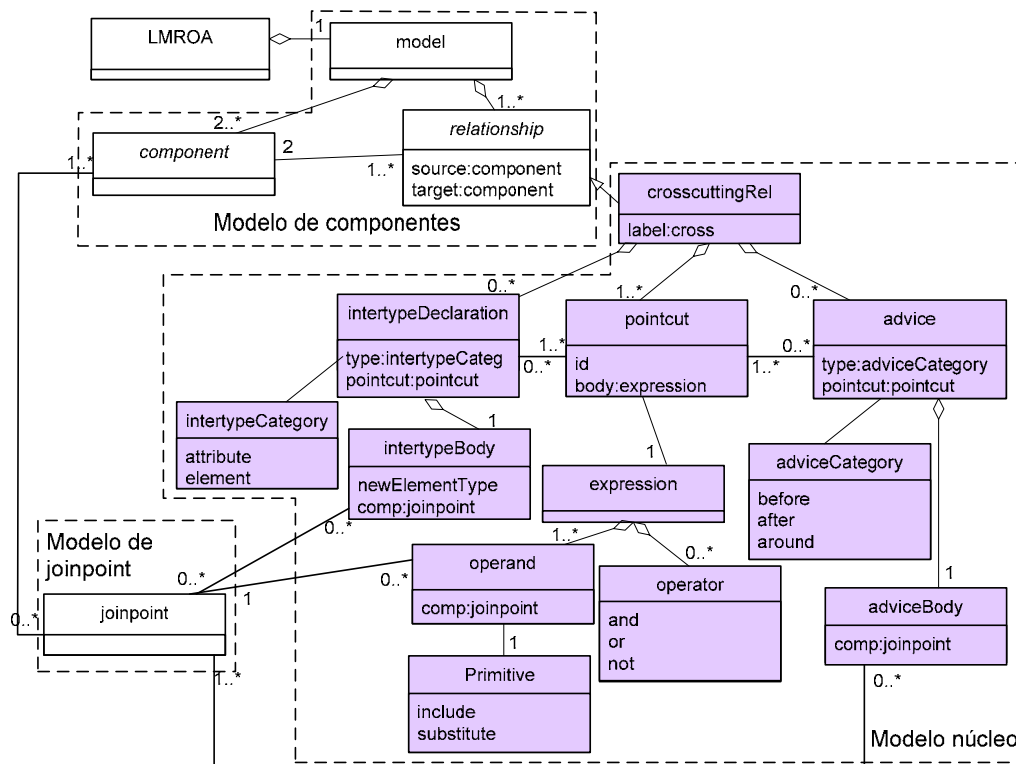


Figura 37 – Modelo conceitual da linguagem de modelagem de requisitos orientada a aspectos

4.2.1. Modelo de Componentes

No modelo conceitual de LMROA (Figura 37), o modelo de componentes representa uma linguagem de modelagem de requisitos descrita por componentes e relacionamentos (linhas 1 e 2 da Figura 36). Desta forma, *relationship* e *component* são elementos que devem ser instanciados (linhas 3-4 da Figura 36) de acordo com o modelo de requisitos a ser adotado, para dar origem a uma nova linguagem de modelagem de requisitos orientada a aspectos. Alguns exemplos de modelos de requisitos que podem ser instâncias de nosso modelo de componentes são os modelos de cenários, casos de uso, léxico, e os modelos de dependência e razão estratégica do i*, dentre outros. Na Seção 4.3, mostramos como instanciar o modelo de componentes.

4.2.2. Modelo de *Joinpoints*

Joinpoints são elementos do modelo de componentes que podem ser afetados por aspectos (Chavez, 2003, 2004). Em linguagens de implementação, eles podem ser estáticos ou dinâmicos: um *joinpoint* estático é uma localização na estrutura de um elemento, enquanto um *joinpoint* dinâmico é uma localização na execução de um programa. Visto que, na maioria das vezes, linguagens de modelagem de requisitos são especificações não executáveis, i.e., estáticas, utilizamos apenas *joinpoints* estáticos.

Na Figura 36, o construto *joinpoint* (na linha 5) não tem valor definido porque depende do modelo de componentes a ser utilizado. Na Seção 4.3, instanciamos o modelo de *joinpoints* para o V-graph. Na descrição do relacionamento transversal, a seguir, referimos estes elementos por *joinpoint*.

4.2.3. Modelo Núcleo: Relacionamento Transversal

O relacionamento transversal representa, em nossa linguagem, o que o aspecto representa em AspectJ. Ele é um novo tipo de relacionamento a ser adicionado ao modelo de componentes (linha 3 da Figura 36). Optamos por não utilizar o conceito de Aspecto (Kiczales, 2001) ou candidato a aspecto (Rashid, 2002) como uma entidade de primeira ordem porque consideramos que desta maneira:

- 1) a extensão realizada é menos intrusiva à linguagem de modelagem de requisitos escolhida. Assim, as técnicas e diretrizes já definidas para a linguagem de modelagem continuam sendo válidas para todos os requisitos do sistema, podendo ou não estar entrelaçados e espalhados;

- 2) retardamos a decisão e descrição das informações relativas à transversalidade de requisitos. Isto significa que não é necessário saber a priori que uma característica é transversal. Quando percebemos esta propriedade, substituímos alguns relacionamentos por um relacionamento transversal, mas a estrutura e organização dos requisitos permanecem as mesmas. Por exemplo, a Figura 33, apresentada anteriormente, poderia ter sido criada pelo engenheiro, que ao perceber o espalhamento das atividades de “Persistence” modificaria o modelo,

trocando as dez contribuições por um relacionamento transversal (mostrado na Figura 34).

3) facilitamos o reuso de características que em alguns projetos têm esta propriedade de transversalidade e em outros não a têm, i.e, consideramos que características transversais, na verdade, “estão” transversais. Por isso, representamos todas elas, transversal ou não, da mesma maneira, o que diferencia é se ela interage com outras características usando um relacionamento transversal ou não; e

4) separamos as informações referentes a “o quê” é transversal, representada pelos elementos do modelo de componentes, das referentes a “como” é transversal, representadas no relacionamento transversal.

Como ilustrado na Figura 37, o relacionamento transversal tem o rótulo “cross” e herda as informações de *origem* e *destino* (*pointcuts*) de *relacionamento*. Além destas, a informação referente à transversalidade é descrita por meio de *pointcuts*, *advice* e *intertype declarations*.

```

1. <crosscuttingRel> := crosscutting <crosscutting_label>{
2.     source=<joinpoint> <pointcut><advice><intertype_declaration>}
3. <pointcut> := pointcut (<name>; <pointcut_id>) : <pointcut_expression> |
4.     <pointcut> <pointcut>
5. <pointcut_expression> := <operand> |
6.     not <operand> |
7.     <pointcut_expression> and <pointcut_expression> |
8.     <pointcut_expression> or <pointcut_expression>
9. <operand> := <primitive> (<joinpoint>) | <primitive> (<regular_expression>)
10. <primitive> := include | substitute
11. <regular_expression> := <value>; “<joinpoint>”; <attribute_type>; <path>
12. <attribute_type> := id | name
13. <advice> := advice <advice_type>: <a_it_expression> {<advice_body>} |
14.     <advice> <advice>
15. <advice_type> := after | before | around
16. <a_it_expression> := <pointcut_id> |
17.     not <pointcut_id> |
18.     <a_it_expression> and <a_it_expression> |
19.     <a_it_expression> or <a_it_expression>
20. <intertype_declaration> := intertype <intertype_type>: <a_it_expression>
21.     {<intertype_declaration_body>} |
22.     <intertype_declaration> <intertype_declaration>
23. <intertype_type> := attribute | element
24. <advice_body> := <joinpoint> | <advice_body>; <advice_body>
25. <intertype_declaration_body> := new_element_type {<new_element_type>} |
26.     <joinpoint> |
27.     <intertype_declaration_body>; <intertype_declaration_body>
28. <new_element_type> := <name>=<value> | <new_element_type>; <new_element_type>
29. <crosscutting_label>:=cross

```

Figura 38. Sintaxe do relacionamento transversal

Na Figura 38, apresentamos a sintaxe do relacionamento transversal e a seguir detalhamos cada um de seus construtos. Utilizamos os nomes *pointcut*, *advice* e *intertype declaration* para ajudar o leitor a fazer uma associação direta a

elementos conhecidos no nível de desenho e implementação. Entretanto, para nossa abordagem, adaptamos a semântica destes elementos para que eles representem a transversalidade de características no nível de engenharia de requisitos.

Pointcuts

Pointcuts indicam os elementos afetados por uma determinada característica. Cada *pointcut* é definido por um nome e uma expressão (linhas 3-4 da Figura 38). Expressões consistem em sentenças que usam operandos, operadores e primitivas (linhas 5-8). Os operandos são elementos cujos tipos estão associados aos *joinpoints* (linha 9). Os operadores são utilizados para agrupar um ou mais *joinpoints*, eles são OR, AND e NOT.

As primitivas definem uma ação a ser realizada no *pointcut* (linha 10). Em nosso caso, definimos os seguintes tipos de primitivas: *include* - adiciona um conjunto de elementos ao local especificado; e *substitute* - substitui os elementos especificados. A primitiva *substitute* pode ser utilizada para apagar elementos quando o corpo do *advice* ou *intertype declaration* associado é vazio.

Na Figura 39, ilustramos três *pointcuts*, denominados *add*, *exclude* e *select*. Cada expressão regular representa um operando associado à primitiva *include*. Nas expressões regulares é possível definir: a expressão (<value>); qual o tipo de elemento (“<joinpoint>”), neste caso são os tipos definidos em *joinpoints*; qual o atributo do elemento (<attribute_type>), podendo ser o identificador ou nome; e opcionalmente, qual o caminho (<path>) dos elementos a serem consultados. Operadores *and* são utilizados para agrupar os operandos.

Nome do pointcut	Primitiva	Expressão regular	Operador	Identificador
Pointcut add:	include	(Add [*])	and	include(Create [*])
Pointcut exclude:	include	(Exclude [*])	and	include(Unlink [user] to [project])
Pointcut select:	include	(Change [*])		

Figura 39. Exemplo de *pointcut*

Advice

Um *advice* define quais elementos do modelo de origem se espalham ou se entrelaçam nos *pointcuts*. Ele registra o conjunto de elementos que representa um comportamento ou estrutura que se repete. Cada *advice* é formado por um tipo

(*after*, *before* e *around*) que indica a maneira como ele afeta os *pointcuts*; uma expressão de *pointcuts*, indicando quais são os *pointcuts* onde ele será aplicado; e o corpo que define o conjunto de elementos que se espalha ou se entrelaça nos *pointcuts*.

Na Figura 40, ilustramos três exemplo de *advice* do tipo *around*. O corpo de cada *advice* define os identificadores dos *pointcuts* (*add*, *exclude* e *select*) a serem afetados e as operações a serem incluídas ou substituídas (Include [data], Exclude [data] e Select [data], respectivamente).

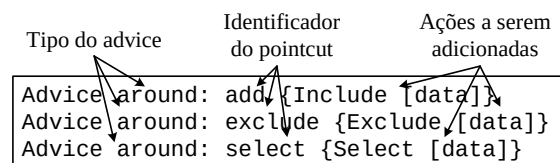


Figura 40. Exemplo de *advice*

Intertype declarations

Intertype declarations são utilizadas para adicionar novos tipos de dados (*attribute*) ou de elementos (*element*) ao modelo de requisitos escolhido, indicando que estes novos elementos são decorrentes da junção de duas características. Utilizar *intertype declarations* oferece informações para o mecanismo de visualização sobre novas visões a serem geradas, porque representam o interesse por estes novos elementos e pela interação deles com os demais.

Assim como o *advice*, cada *intertype declaration* é definida através de uma expressão de *pointcuts* indicando onde ela será aplicada, e um corpo que define os novos tipos de dados ou elementos a serem incluídos. Na Figura 41, ilustramos uma *intertype declaration*. Assim como no *advice*, o corpo da *intertype declaration* define os identificadores dos *pointcuts* a serem afetados (*add*, *exclude* e *select*) e novos tipos a serem incluídos no modelo (*actor=participant_user*).

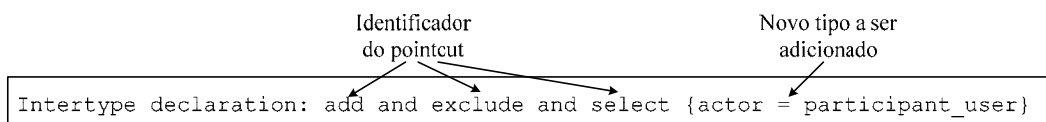


Figura 41. Exemplo de *intertype declaration*

4.2.4.

Diretrizes para Modelagem de Características Separadamente

A identificação de características transversais é na maioria das vezes realizada com base na experiência e intuição do desenvolvedor (Sousa, 2004b), tanto durante a definição de requisitos quanto nas atividades de desenho e implementação. Nesta tese, consideramos que qualquer característica pode ou não “estar” transversal. Não é necessário saber disto com antecedência porque elas são modeladas da mesma maneira (utilizando os elementos do modelo de componentes). É necessário sim, saber quando separá-las e quando utilizar o relacionamento transversal:

- Quando separá-las ? - qualquer característica que atenda a pelo menos uma das propriedades a seguir pode ser modelada separadamente:
 - Ela está repetida muitas vezes no modelo, ou possui alto *fan-in*;
 - Sua existência é decorrente da integração de requisitos semanticamente distintos;
 - Ela representa um ponto de variabilidade, podendo ou não estar presente no sistema, sem impedir a realização de seu objetivo global;
 - Ela pode ser reutilizada em outros domínios;
 - Ela é muito complexa, estando associada a muitas variáveis;
 - Ela é independente das demais, representando um conceito bem delimitado;
- Quando utilizar o relacionamento transversal ? - de maneira geral, nos três primeiros casos acima é recomendado o uso de relacionamentos transversais. Contudo, nossa abordagem considera que estas situações surgem durante o processo de modelagem, conforme a informação sobre o domínio e os requisitos é adquirida. Isto significa que, nem toda característica separada interage por meio de um relacionamento transversal com as demais. Por exemplo, quando a separação ocorre devido a apenas um dos três últimos casos, os relacionamentos providos pelo próprio modelo de componentes podem ser suficientes.

A seguir, na Seção 4.3, apresentamos como utilizar a estratégia definida na Seção 4.1, instanciando os pontos flexíveis de LMROA.

4.3.

Utilização da Estratégia – O V-Graph como Modelo de Componentes

Como relatamos na Seção 4.1, nossa estratégia para modelagem de requisitos é centrada na utilização de LMROA. LMROA é definida pelo modelo núcleo (apresentado na Seção 4.2), modelo de *joinpoints* e modelo de componentes. Nesta seção apresentamos como instanciar os elementos do metamodelo de integração, utilizando o V-graph como modelo de componentes de LMROA. Nas Seções 4.3.1, 4.3.2 e 4.3.3, definimos, respectivamente, as atividades de Separação, Composição e Visualização para esta instância.

4.3.1.

Separação de Características Transversais

Nesta seção, considerando o modelo de requisitos V-graph, descrevemos uma instância de LMROA, denominada AOV-graph, e algumas diretrizes para utilização desta instância. Assim, AOV-graph consiste no modelo V-graph estendido, permitindo, além da descrição de *softmetas*, metas e tarefas, o controle dos problemas de entrelaçamento e espalhamento. Esta extensão consiste em adicionar o relacionamento transversal ao V-graph, e assim, uma nova maneira de pensar na separação e representação de RFs e RNFs (de características).

4.3.1.1.

A Linguagem

Na Figura 42 e Figura 43 (em branco), apresentamos os elementos e relacionamentos do modelo *V-graph*, instância dos elementos de LMROA. Os elementos *softmetas*, *metas* e *tarefas* possuem *nomes*, *tipos* e *tópicos*. O relacionamento de decomposição, com os rótulos *And* e *Or*, e o de contribuição, com os rótulos *make*, *help*, *unknown*, *hurt*, e *break* estão representados através da agregação em *component*. Desta forma, cada componente tem um atributo denominado “*decomposition_label*” que indica qual o rótulo do relacionamento entre ele e seu componente pai (linhas 5, 7 e 9 da Figura 43). O relacionamento de correlação é definido através de *relCorrelation*.

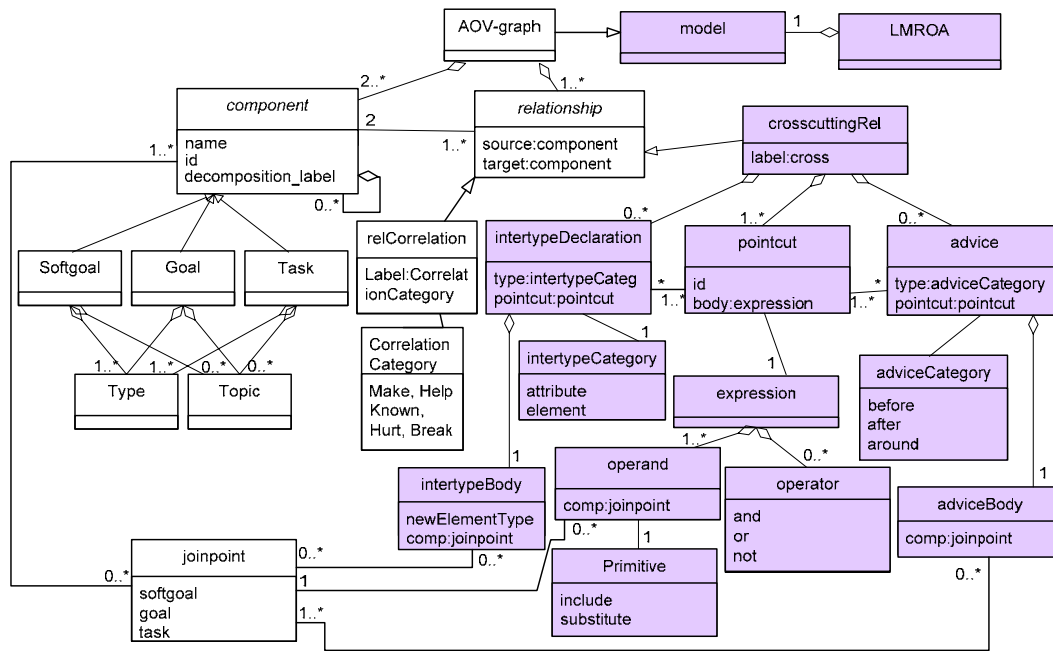


Figura 42. Instanciação de LMROA: AOV-graph

```

1. <goal_model> := goal_model (<name>; <id>){<component> <relationship>} |
2.   <goal_model> <goal_model>
3. <component> := <softgoal> | <goal> | <task> |
4.   <component> <component>
5. <softgoal>:=softgoal(<name>;<softgoal_id>;<decomposition_label>;(<type>);(<topic>))
6.   {<component><relationship>}
7. <goal> := goal (<name>; <goal_id>; <decomposition_label> ; (<type>) ; (<topic>))
8.   {<component><relationship>}
9. <task> := task (<name>; <task_id>; <decomposition_label> ; (<type>) (<topic>))
10.   {<component><relationship>}
11. <topic> := <name> | <topic>; <topic>
12. <type> := <name> | <type>; <type>
13. <relationship> := <relcorrelation> | <crosscuttingRel> |
14.   <relationship> <relationship>
15. <relcorrelation> :=
16.   correlation <correlation_label> {source=<goal_ref> target=<softgoal_ref>} |
17.   correlation <correlation_label> {source=<softgoal_ref> target=<softgoal_ref>} |
18.   correlation <correlation_label> {source=<task_ref> target=<task_ref>}
19. <correlation_label> := break | hurt | unknown | help | make
20. <decomposition_label>:= and | or | xor | break | hurt | unknown | help | make
21. <goal_ref> := <goal_id> <decomposition_label>
22. <softgoal_ref> := <softgoal_id> <decomposition_label>
23. <task_ref> := <task_id> <decomposition_label>

```

Figura 43. Sintaxe do V-graph

Exemplo

Na Figura 44, ilustramos o AOV-graph para o serviço de Persistência; em a) ilustramos sua representação gráfica e em b) sua descrição textual. Neste exemplo, vemos que “Persistence in [BD]” e “Persistence in [file]” são decomposições XOR de “Persistence”, enquanto “Make [register operation]” é uma decomposição AND.

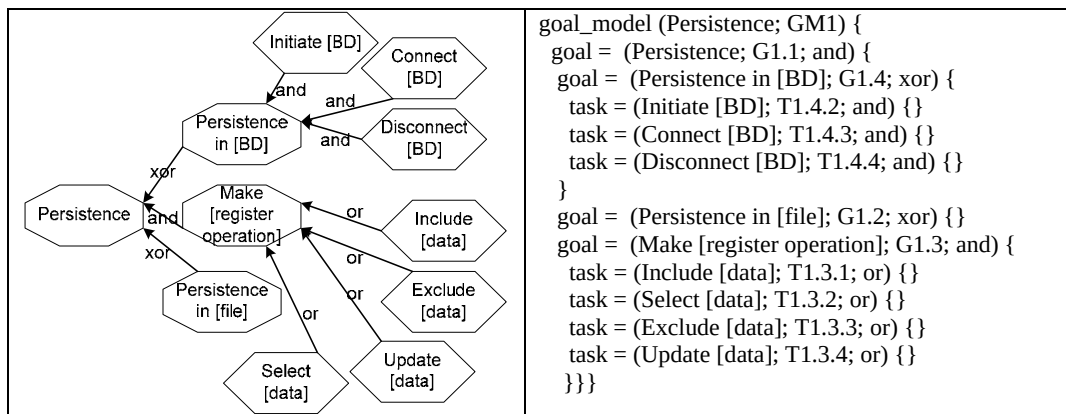


Figura 44. Exemplo do modelo V-graph a) representação gráfica e b) representação em textual

4.3.1.1.1.

Modelo de *Joinpoints*

No caso do V-graph os *joinpoints* podem ser elementos dos tipos *softmeta*, meta e tarefa, veja Figura 45. As instâncias destes tipos podem ser referenciadas como *pointcuts*. Elas podem ser identificadas: diretamente pelo nome ou identificador do elemento; ou indiretamente por expressões regulares indicando tipos e tópicos.

```
<joinpoint> := <softgoal_ref> | <goal_ref> | <task_ref>
```

Figura 45. Definição dos *joinpoints* no caso do V-graph como modelo de componentes

Instâncias destes tipos também são utilizadas na descrição do corpo do *advice* e *intertype declaration*. Como estes tipos possuem o atributo *decomposition_label* (veja linhas 21, 22 e 23 da Figura 43) então é possível descrever em *advice* e *intertype* quais os rótulos dos novos relacionamentos. Isto é equivalente ao “How much” da abordagem apresentada em (Leite, 2005).

4.3.1.1.2.

Modelo Núcleo: Relacionamento Transversal

Como apresentamos na Seção 4.2.3, os elementos *source*, *operand*, *advice_body* e *intertype_declaration_body* estão associados aos tipos definidos nos *joinpoints* (veja na Figura 38). A seguir, descrevemos a semântica destes elementos na instância AOV-graph.

Pointcuts

Pointcuts podem fazer referência a quaisquer elementos dos tipos *softmetas*, metas e tarefas. Nos *pointcuts* do AOV-graph, o operador OR indica que se os operandos referem elementos filhos do mesmo pai então apenas um dos operandos precisa ser modificado; enquanto o operador AND indica que todos devem ser atingidos; e o operador NOT exclui um determinado operando dentre os elementos especificados no *pointcut*.

Exemplo

Na Figura 46, ilustramos três exemplos de *pointcuts*: no primeiro utilizamos o operador AND para indicar que os dois *pointcuts* serão afetados; no segundo utilizamos o operador OR, indicando que caso o *pointcut* já tenham sido afetado pelo mesmo *advice* ou *intertype* então ele não precisa ser afetado novamente; e no terceiro *pointcut* utilizamos o operador NOT e expressões regulares, indicando que qualquer meta, *softmeta* ou tarefa que atende a expressão regular e que não seja a tarefa “Excluir [data]”, será afetada.

<i>Pointcut</i> exclude_data: include(Exclude [project]) and include(Exclude [scenario]) and include(Unlink [user] to [project]) <i>Pointcut</i> BD: include(include [data]) or include(delete [data]) or include(update [data]) or include (select [data]) <i>Pointcut</i> exclude: include(Exclude [*]) and not include (Exclude [data])
--

Figura 46. Exemplo de *pointcuts*

Advice

Cada *advice* define quais elementos do modelo de origem do relacionamento transversal se espalham ou se entrelaçam aos *pointcuts*. No caso do V-graph, podemos definir metas, *softmetas* e tarefas no *advice*. Como definimos na Seção 4.2.3, cada *advice* é um dos três tipos *before*, *after* ou *around*. Apesar do V-graph não realçar a sequência de ações, normalmente tenta-se escrever o modelo da esquerda para direita ou de cima para baixo, indicando uma sequência. Desta forma, utilizamos os tipos de *advice* para indicar:

- *Before* - inserir ANTES do *pointcut* como uma decomposição de seu elemento pai, veja a Figura 47(a);

- *Around* - inserir elementos como decomposições do elemento afetado, i.e., estes elementos são necessários para atingir o elemento afetado, veja Figura 47(b); e
- *After* - inserir DEPOIS do *pointcut* como uma decomposição de seu elemento pai, veja Figura 47(c).

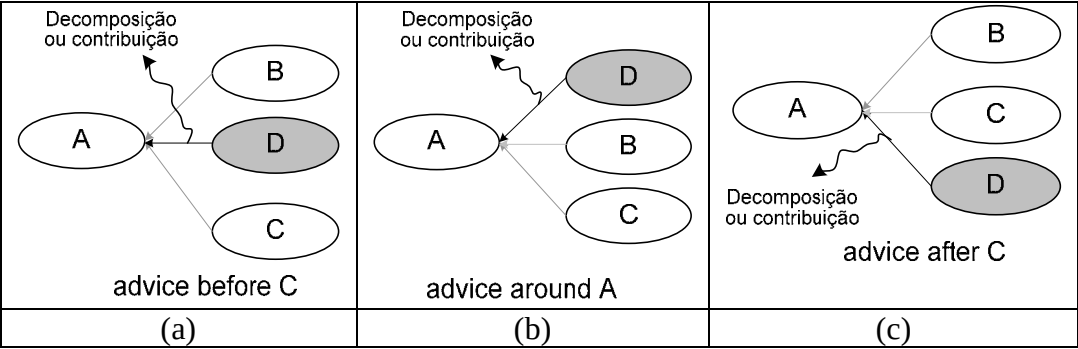


Figura 47. Semântica dos tipos de *advice* a) before, b) around e c) after

Exemplo

No exemplo da Figura 48, cada *advice* indica quais elementos do modelo *Persistence* afetam os *pointcuts* indicados pelos nomes *BD*, “*exclude_data*” e “*exclude*”. O primeiro *advice* indica a inclusão da tarefa “*Exclude [data]*” nos pontos indicados pelos *pointcuts* “*exclude*” e “*exclude_data*”. O segundo *advice* indica que as tarefas “*Initiate [BD]*” e “*Connect [BD]*” devem acontecer antes dos pontos indicados pelo *pointcut* “*BD*”. Como este *pointcut* é definido com o operador OR então estas duas operações serão incluídas apenas uma vez no mesmo nível da hierarquia. O terceiro *advice* indica que a tarefa “*Disconnect [BD]*” é incluída após os pontos definidos.

<i>Advice</i> around: <i>exclude</i> or <i>exclude_data</i> { <i>Exclude [data]</i> }
<i>Advice</i> before: <i>BD</i> { <i>Initiate [BD]</i> ; <i>Connect [BD]</i> ;}
<i>Advice</i> after: <i>BD</i> { <i>Disconnect [BD]</i> ;}

Figura 48. Exemplo de *advice*

Intertype declarations

Intertype declarations são utilizadas para modificar a estrutura dos elementos do modelo de componentes. No V-graph, utilizamos *intertype declarations* para adicionar novos tipos de elementos ao modelo de componentes. Estes elementos podem ser novos atributos, tais como, atores, exceções, custo, dentre outros, ou podem ser novas tarefas, metas ou *softmetas*.

Exemplo

Na Figura 49, ilustramos dois exemplos de *intertype declaration*. No primeiro é definido um novo tipo de atributo denominado “exception”. Este elemento não pode ser visualizado na visão V-graph, mas pode ser utilizado por outros tipos de visão, tal como Cenário. No segundo exemplo, é definida uma nova tarefa denominada “Verify if [element] exists”, que é adicionada nos *pointcuts* e no elemento origem do relacionamento transversal.

```
intertype declaration attribute: exclude and exclude_data
{ exception = “This element doesn’t exist” ;
  exception = “User doesn’t have permission” }
intertype declaration element: exclude and exclude_data
{ task = (Verify if [element] exists; T0; and) {} }
```

Figura 49. Exemplo de *intertype declarations*

4.3.1.2.

Diretrizes para Modelagem Utilizando AOV-graph

No caso do AOV-graph, os relacionamentos de contribuição (indicam maior coesão) e correlação (indicam menor coesão) explicitam como os elementos podem ser modularizados. Assim, qualquer sub-árvore pode ser considerada uma característica modelada separadamente. Por exemplo, na Figura 50, a meta “Persistence in [BD]” com as tarefas necessárias para que ela seja satisfeita pode ser considerada uma característica separada das demais.

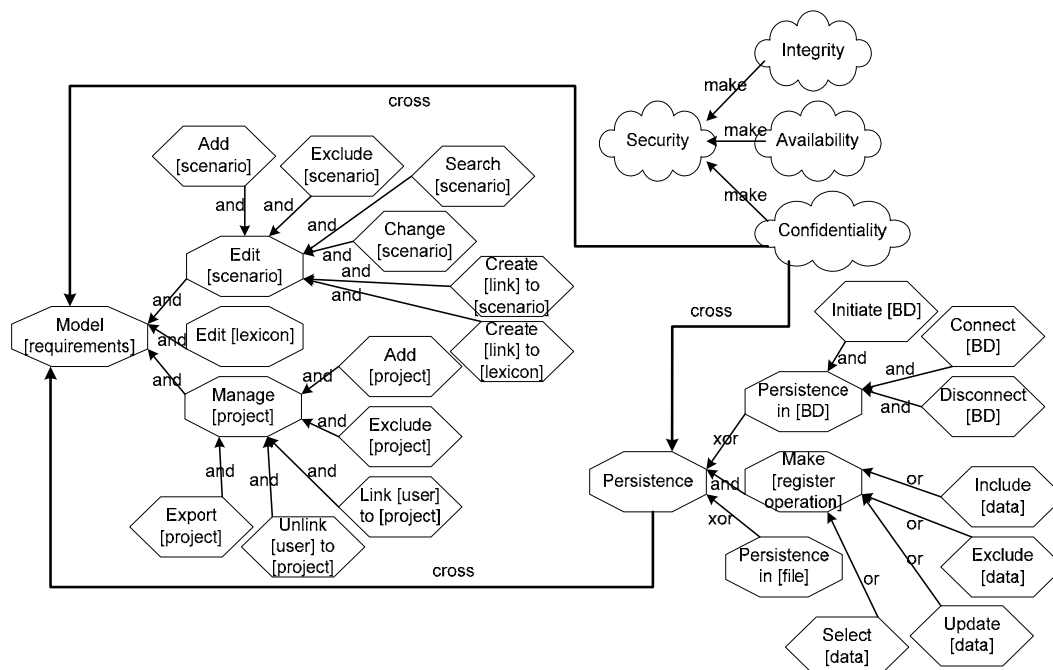


Figura 50. Exemplo da separação de modelos

Contudo, quando o fator decisivo para separação é somente um dos três últimos fatores citados na Seção 4.2.4 (alta complexidade, possibilidade de reuso, ou possuir baixo acoplamento com as demais), recomendamos que a separação seja realizada modelando cada característica como um modelo separado (um grupo de requisitos separado), por exemplo, os modelos “Persistence”, “Security” e “Model [requirements]”, ilustrados na Figura 50. Desta maneira é mais fácil delimitar o escopo de cada um.

O relacionamento transversal é fortemente indicado, no AOV-graph, quando *softmetas*, metas e tarefas:

- contribuem para satisfação de muitos elementos, indicando espalhamento. Por exemplo, na Figura 35, ilustramos que a tarefa “Include [data]” do serviço “Persistence” afeta cinco tarefas do serviço “Model [requirements]”;
- contribuem para satisfação de elementos em árvores separadas (modelos separados), indicando entrelaçamento. Por exemplo, as tarefas relacionadas a meta “Make [register operation]” contribuem para as árvores relacionadas aos serviços de persistência e de modelar requisitos.

Cada *advice* e *intertype declaration* agrupa tarefas, metas e *softmetas* que afetam juntas um ou vários *pointcuts*. O número de relacionamentos representados por cada *advice* ou *intertype* é dado pela seguinte expressão (Nr = número de relacionamentos; Np = número de pontos a serem afetados; Ne = número de metas, *softmetas* ou tarefas definidas no corpo do *advice* ou *intertype*; Na = número de atributos definidos no corpo do *advice* ou *intertype*):

- $Nr = Np \times Ne$ (no caso do *advice*);
- $Nr = Np \times Ne + 1$ (no caso de *intertype* do tipo *element*);
- $Nr = Np \times Na + 1$ (no caso de *intertype* do tipo *attribute*)

Desta forma, quanto mais elementos eles agrupam maior é o ganho obtido com o uso do relacionamento transversal porque há uma maior redução no número de relacionamentos que o engenheiro de requisitos precisa desenhar manualmente. Conseqüentemente, é mais fácil manipular os modelos e é maior a visibilidade de seus elementos.

Advice e *intertype declarations* são utilizadas nas seguintes situações:

- *Advice* é utilizado quando elementos do modelo de origem do relacionamento se espalham por outras características. Por exemplo, o relacionamento transversal de “Persistence” para “Model [requirements]”

representa relacionamentos de contribuição partindo das operacionalizações de “Persistence” para os elementos de “Model [requirements]”, veja no exemplo mostrado na Figura 35.

- *Intertypes (element)* são utilizadas para criar novos elementos dos tipos tarefa, meta ou softmeta, que são oriundos da junção de características separadas, i.e., estes elementos não existem no modelo de origem do relacionamento; e
- *Intertypes (attribute)* são utilizadas para criar novos tipos de elementos, i.e., elementos que não são dos tipos meta, softmeta e tarefa. Com *intertypes (attribute)* criamos novas formas de separar, classificar e visualizar metas, *softmetas*, e tarefas. Neste caso, os relacionamentos criados não são visualizados no grafo de metas porque eles representam outra forma de decomposição.

Por definirem os elementos que se espalham e/ou entrelaçam, *advice* e *intertype* representam candidatos naturais a serem visualizados por meio de matrizes ou grafos, focando os relacionamentos nos quais eles fazem parte. Frente às vantagens oferecidas pelo relacionamento transversal, o engenheiro de requisitos deve avaliar qual o custo/benefício de utilizar relacionamentos de contribuição/correlação ou transversais.

4.3.2. Composição de Características Transversais

A utilização do relacionamento transversal reduz a quantidade de relacionamentos entre metas, *softmetas* e tarefas porque “modulariza” muitas interações em poucos relacionamentos transversais. Esta modularização é possível porque muitas vezes um conjunto de relacionamentos é repetido para associar uma característica (origem) para muitos elementos (destino). Isto por si só diminui a complexidade do modelo ou, pelo menos, aumenta a visibilidade de seus elementos.

Visto que conseguimos agrupar relacionamentos, então é necessário um mecanismo de composição para processá-los. Além disto, *pointcuts* agrupam pontos atingidos da mesma maneira, *advice* e *intertype* agrupam estruturas que se repetem. Estes agrupamentos representam novas maneiras de modularização de elementos que estão espalhados e entrelaçados devido à decomposição dominante

do modelo de componentes, V-graph. Por exemplo, o *pointcut* “exclude_data” (Figura 46) agrupa todos os elementos afetados pela tarefa “Exclude [data]” de “Persistence in [DB]” (descrito no *advice* da Figura 48), bem como pela nova tarefa “Verify if [element] exists” e as exceções “This element doesn’t exist” e “User doesn’t have permission” (descritas no *intertype* da Figura 49).

Nesta Seção, descrevemos a atividade de composição de características transversais. Para esta etapa definimos um conjunto de regras de composição e implementamos um mecanismo para processá-las. As regras indicam como o mecanismo de composição transforma os modelos inicialmente criados em modelos integrados; elas representam a semântica dos construtos do relacionamento transversal e podem variar de acordo com o modelo de componentes utilizado.

Mecanismo de Composição

Em nossa abordagem, o mecanismo de composição é uma máquina de transformação guiada pelas regras semânticas do modelo para efetivar as mudanças que o relacionamento transversal descreve, i.e., ele é responsável por automaticamente espalhar e entrelaçar as características modeladas inicialmente de maneira separada. Ter um mecanismo automático para propagar informações num modelo de requisitos facilita a inclusão e exclusão de informações que podem estar separadas de acordo com o interesse do engenheiro de requisitos.

O mecanismo de composição recebe como entrada e gera como saída descrições estáticas de requisitos, i.e., modelos AOV-graph. A estratégia utilizada para composição não modifica o modelo AOV-graph inicial, ele cria um outro, desta maneira as versões inicial e integrada estão disponíveis. A composição ocorre sobre uma descrição estática e deve acontecer sempre que relacionamentos transversais são inseridos, modificados ou excluídos. Descrevemos a implementação do mecanismo de composição na Seção 4.3.4.2.

Regras de Composição

Na Tabela 2, apresentamos as regras de composição para AOV-graph. Há duas ações especificadas no relacionamento transversal por meio das primitivas, *include* e *substitute*. A combinação de operadores, primitivas, tipos de *advice* e *intertype declarations* determina a transformação a ser realizada em cada *pointcut*.

Tabela 2. Regras de composição

Operador	Semântica
AND	Afeta todos os operandos
OR	Afeta apenas um dos operandos se eles forem filhos do mesmo pai
NOT	Exclui o operando do conjunto de pontos a serem afetados

Primitiva	Tipo de <i>Intertype declaration</i> ou <i>advice</i>	Semântica
Include	<i>Advice before</i>	Inclui os elementos definidos no corpo do <i>advice</i> ANTES do <i>pointcut</i> , como filho de seu elemento pai
	<i>Advice after</i>	Inclui os elementos definidos no corpo do <i>advice</i> DEPOIS do <i>pointcut</i> , como filho de seu elemento pai
	<i>Advice around</i>	Inclui os elementos definidos no corpo do <i>advice</i> como filho do <i>pointcut</i>
	<i>Intertype declaration element/attribute</i>	Inclui os elementos definidos no corpo do <i>intertype</i> no elemento de origem do relacionamento transversal e no <i>pointcut</i>
Substitute	<i>Advice Before/around/after</i>	Substitui o elemento indicado no <i>pointcut</i> pelos elementos definidos no corpo do <i>advice</i>
	<i>Intertype declaration element/attribute</i>	Inclui os elementos definidos no corpo do <i>intertype</i> no elemento de origem do relacionamento transversal e no <i>pointcut</i>

Exemplo

Na Figura 51, apresentamos um exemplo de composição: (a) descrição inicial do modelo de metas, (b) relacionamento transversal e (c) resultado da composição, em negrito explicitamos os elementos adicionados por *advice* e *intertype declarations*.

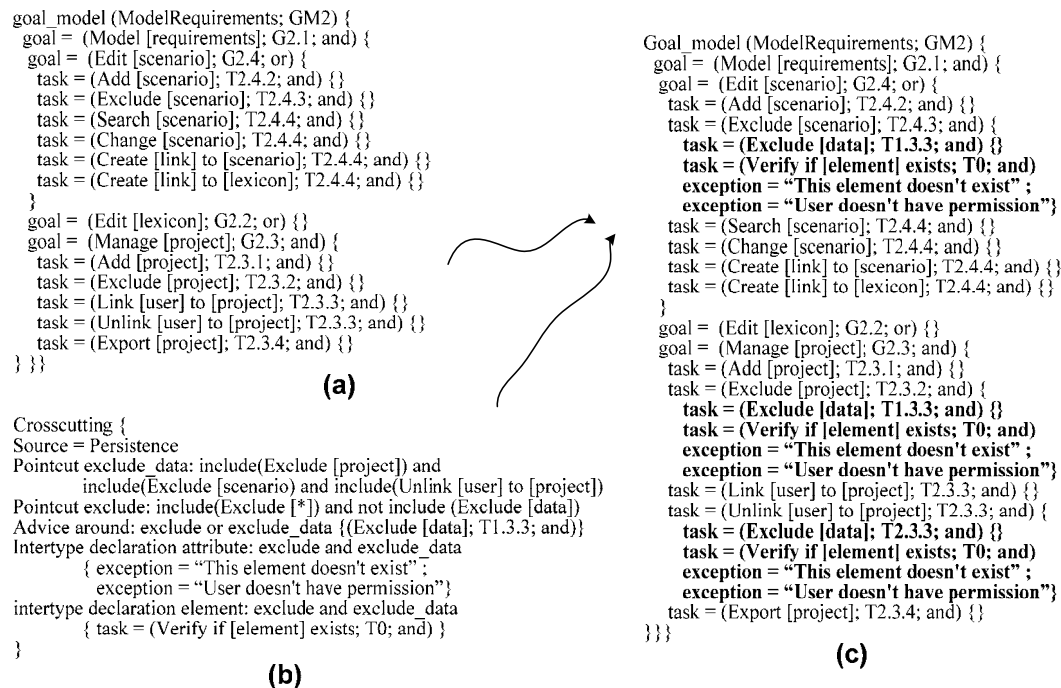


Figura 51. Exemplo de composição

Nas Seções seguintes, utilizamos o exemplo ilustrado na Figura 52 como base para demonstrar algumas visões extraídas das informações descritas no V-graph.

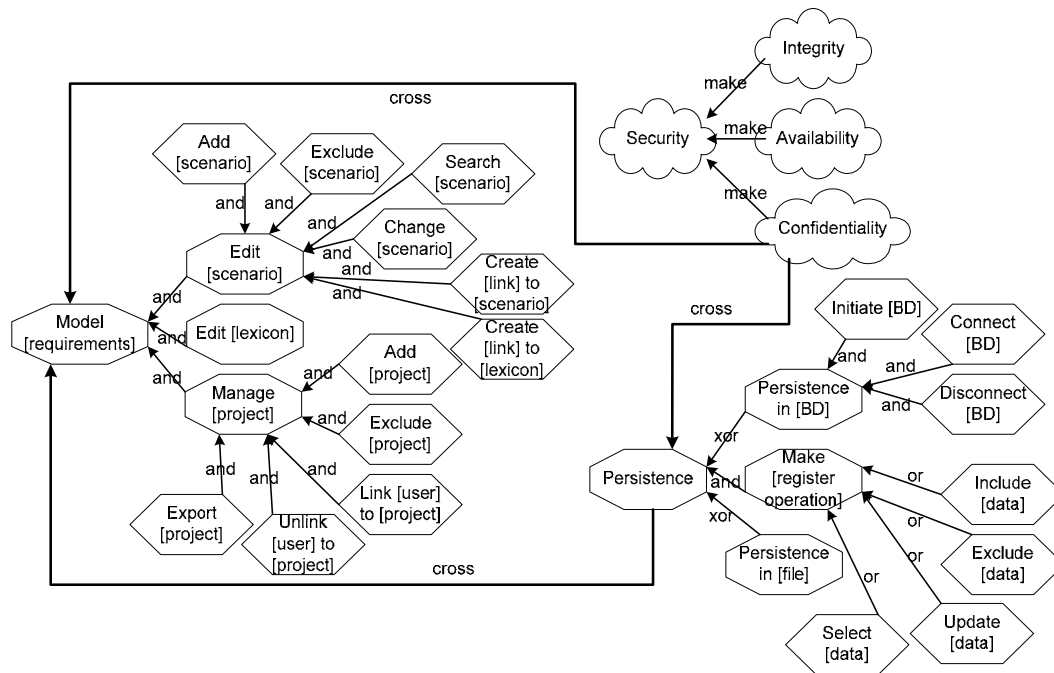


Figura 52. Exemplo base para extração de visões

4.3.3. Visualização de Características Transversais

Para a atividade de visualização definimos quais visões podem ser extraídas de informações modeladas em AOV-graph e as respectivas regras de transformação. A utilização de visões tem sido adotada em diferentes fases do desenvolvimento de software. Ela ajuda o desenvolvedor a diminuir o escopo de um problema e assim, sua complexidade, buscando a completude segundo algum interesse. Durante o processo de requisitos, bem como de projeto, i.e., durante a elaboração de soluções, é importante que o desenvolvedor possa adquirir diferentes visões do modelo base. Desta forma, ele pode analisar por diferentes ângulos a solução sendo criada.

Como definimos no Capítulo 2, uma visão é uma representação do todo ou de parte de uma arquitetura, segundo o foco em uma ou mais características, por um ou mais interessados. Desta forma, consideramos que um mecanismo de visualização é fundamental para a modelagem de requisitos. Um mecanismo

automático para gerar visões pode acelerar o processo de modelagem, porque diminui o re-trabalho e as inconsistências entre diferentes modelos.

Nosso mecanismo de visualização consiste de um componente de transformação que precisa dos seguintes conjuntos de informação: a sintaxe e semântica das linguagens de origem (AOV-graph) e de destino (representação a ser gerada), e as regras de transformação de uma linguagem para a outra, veja Figura 53.

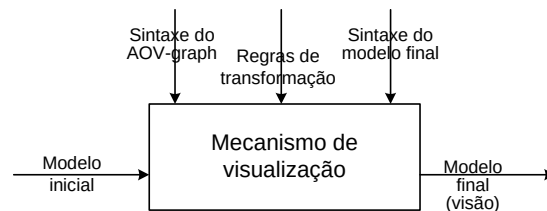


Figura 53. Mecanismo de visualização

Definimos um conjunto de visões que podem ser extraídas de AOV-graph. Estas visões compõem um conjunto mínimo de visões desejadas durante o processo de requisitos. Nas Seções 4.3.3.1 e 4.3.3.2, expomos, em duas categorias, as visões que compõem nosso mecanismo de visualização: visões com a conotação de serviço e visões no sentido de modelos, respectivamente. Estas categorias não são disjuntas, i.e. nós também podemos gerar visões no sentido de modelos apenas para alguns serviços. Na Figura 54, apresentamos, em um modelo de *features* (Czarnecki, 2000), a variabilidade de possíveis visões.

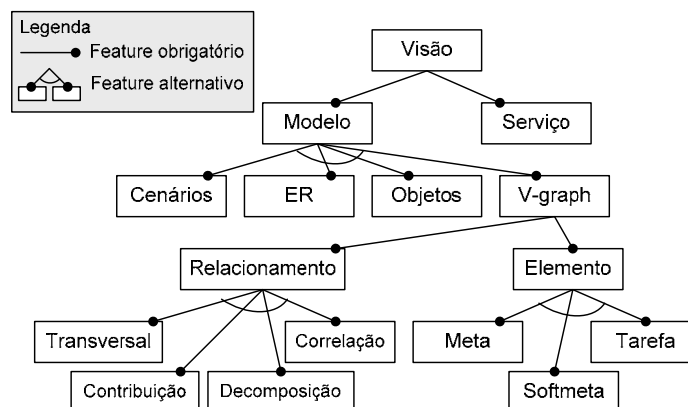


Figura 54. Modelo de *features* para configuração de visões de AOV-graph

4.3.3.1. Visões como Serviços

Como definido no Capítulo 2, visões como serviços representam partes do sistema que se complementam, tais como visão de segurança, visão de pagamento,

visão de características funcionais, dentre outras. Neste caso, temos **visões parciais** da modelagem, focando uma ou um conjunto de características por vez. No caso do AOV-graph, observamos a importância das seguintes visões:

1. Visão sem alguns tipos de relacionamento – em algumas situações o engenheiro de requisitos necessita observar, no modelo do sistema, os elementos que estão relacionados apenas por decomposição, por contribuição, por correlação ou pelo relacionamento transversal, podendo ser necessárias também visões que realcem os elementos relacionados com apenas um dos rótulos destes tipos de relacionamento. Na Figura 55, ilustramos um exemplo de AOV-graph com os relacionamentos de decomposição ocultos.

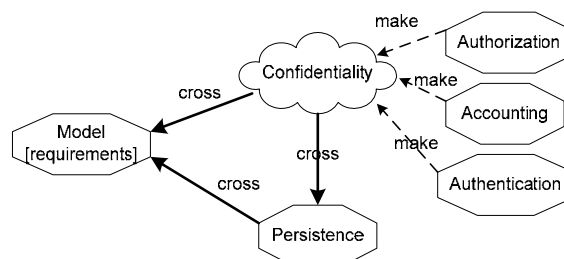


Figura 55. Exemplo de visão com os relacionamentos transversais resumidos e sem o relacionamento de decomposição

2. Visão com os relacionamentos transversais resumidos – visto que os relacionamentos transversais agrupam muitas contribuições e correlações, então quando eles estão expandidos a visibilidade da dependência entre elementos é maior, porém a visibilidade dos elementos é menor. Desta forma, é necessário ter uma visão onde os relacionamentos transversais apareçam resumidos, veja Figura 56.

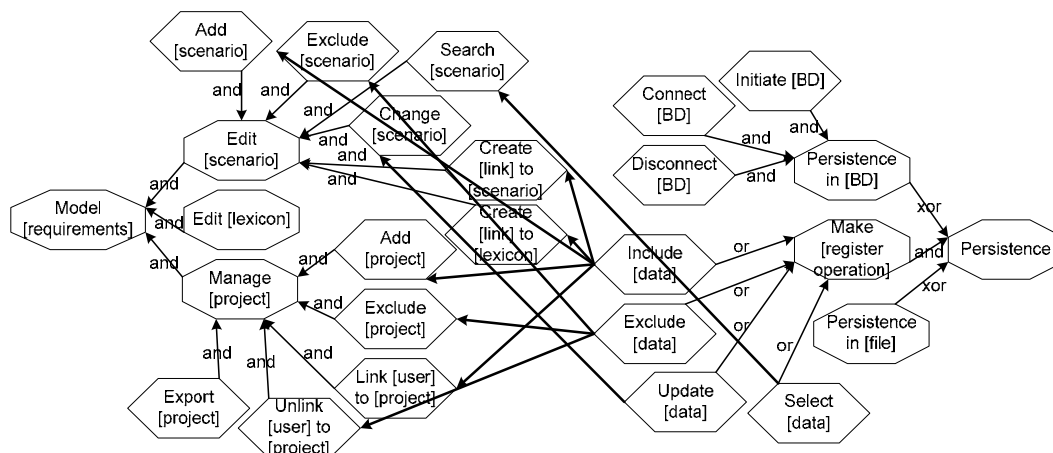


Figura 56. Exemplo de visão com o relacionamento transversal expandido

- PUC-Rio - Certificação Digital Nº 0210666/CB



PUC-Rio - Certificação Digital Nº 0210666/CB

- PUC-Rio - Certificação Digital Nº 0210666/CB

4.3.3.2.1. Cenários

A utilização de cenários durante o desenvolvimento de software é um tópico de destaque na comunidade de Engenharia de Software (Weidenhaupt, 1998; Rolland, 1998). O desenvolvimento de software baseado em cenários se apóia no conceito de que a utilização da linguagem do problema (domínio do usuário) é benéfica à interação entre usuários e desenvolvedores (Leite, 1995). Entendemos por cenários a descrição de situações comuns ao cotidiano dos usuários (Zorman, 1995). Eles devem levar em conta aspectos de usabilidade e permitir o aprofundamento do conhecimento do problema, a unificação de critérios, a obtenção do compromisso de clientes e/ou usuários, a organização de detalhes e o treinamento de pessoas (Carroll, 1994).

Cada cenário descreve, através de linguagem natural semi-estruturada, uma situação específica da aplicação, focando seu comportamento. Cenários podem ser detalhados e utilizados como desenho de maneira a auxiliar a programação. Existem várias propostas para a representação de cenários, desde a mais informal, em texto livre (Carroll, 1994) até representações formais (Hsia, 1994). Optamos por uma representação intermediária que, enquanto facilita a compreensão através da utilização de linguagem natural, força a organização da informação através de uma estrutura bem definida, proposta em (Leite, 1997).

Na Tabela 3, descrevemos a notação de cenários e as transformações que ocorrem entre AOV-graph e o modelo de cenários.

Tabela 3. Regras de transformação entre AOV-graph e Cenários

Notação			
	Objetos: cenário		
	Atributos: título, objetivo, contexto, atores, recursos, episódios, exceções		
	Notação: tabular	Title	
		Actor	
		Resource	
		Context	
		Exception	
Episode			
Relacionamento: elo			
Atributos: origem (cenário), destino (subcenário)			
Notação: representado através de elos de navegação			
Regras			
	▪ Cada cenário tem um título diferente dos demais; ▪ cada cenário deve ter apenas um título e objetivo; ▪ cada cenário deve ter pelo menos um ator, recurso, contexto, episódio; ▪ cada ator e recurso devem aparecer em pelo menos um episódio ou exceção; ▪ o texto de episódios que é igual ao título de algum cenário deve conter uma referência (elo) para este cenário.		
Transformações			
	▪ Cada meta e tarefa de AOV-graph que não é folha gera um cenário cujo título é o nome da meta ou tarefa; ▪ cada filho (and) de meta e tarefa é um episódio do cenário gerado pelo elemento pai; ▪ cada filho (or) de meta e tarefa é um episódio opcional do cenário gerado pelo elemento pai; ▪ cada episódio que se refere a elemento que não é folha gera uma referência (elo) para o cenário criado pelo elemento filho; ▪ cada elemento que possui um novo atributo (criado por <i>intertype</i>) cujo tipo é “ator” gera um ator no cenário gerado por este elemento; ▪ cada elemento que possui um novo atributo (criado por <i>intertype</i>) cujo tipo é “exceção” gera uma exceção no cenário gerado por este elemento; ▪ pré-condições são geradas de relacionamentos transversais com <i>advice</i> do tipo <i>before</i>; ▪ restrições são geradas de <i>softmetas</i> que contribuem ou se correlacionam com o elemento ou com seus pais.		

Na Figura 59, ilustramos um exemplo de cenários gerados a partir do modelo base. Neste exemplo há dois cenários, intitulados “Model [requirements]” e “Edit [scenário]”; os campos referentes a ator, recurso, contexto e exceção não foram gerados porque neste exemplo ilustrativo não há *intertypes* que incluam estes novos tipos no modelo; o campo referente a episódios é gerado com base nos elementos que decompõem “Model [requirements]” e “Edit [scenário]”. Cada episódio sublinhado indica um relacionamento para o cenário equivalente.

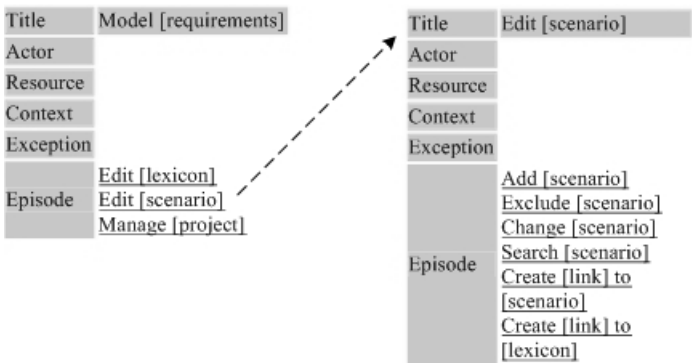


Figura 59. Exemplo da visão cenário para os grafos Model [requirements] e Edit [scenario]

4.3.3.2.2. Matriz de Rastreabilidade

Matrizes de rastreabilidade são tabelas utilizadas para registrar dependências e/ou relacionamentos entre os componentes de um sistema, sejam estes componentes, requisitos, interessados, componentes de projeto, componentes de implementação, dentre outros. Esta representação permite a fácil visualização destes relacionamentos e conseqüentemente a análise do impacto de mudanças em um dos componentes, i.e., rastreabilidade (Sommerville, 2000). Na Tabela 4, descrevemos a notação e as transformações para gerar matrizes de rastreabilidade a partir de AOV-graph.

Tabela 4. Regras de transformação entre AOV-graph e uma matriz de rastreabilidade

Notações	
	Objetos: Coluna, linha, conteúdo da célula
	Atributos: rótulo
	Notação: tabular, veja Figura 60
Regras	
	▪ Cada coluna representa um elemento do modelo, sem repetição; ▪ cada linha representa um elemento do modelo, diferente ou igual aos da coluna, sem repetição; ▪ cada célula interior da tabela representa a interseção ou relacionamento que o elemento da coluna tem com o elemento da linha segundo uma das orientações linha->coluna ou coluna->linha.
Transformações	
	▪ Cada <i>softmeta</i>, meta e tarefa do modelo de metas gera uma linha e uma coluna na matriz de rastreabilidade; ▪ as células interiores da matriz são preenchidas com o relacionamento de correlação, decomposição e contribuição existente entre os elementos das linhas e os elementos das colunas; incluindo os novos relacionamentos descritos pelo relacionamento transversal.

Na Figura 60, ilustramos um exemplo de matriz de rastreabilidade indicando relacionamentos gerados pela especificação dos relacionamentos transversais

ilustrados no modelo base (Figura 52). Nesta matriz, os rótulos das colunas e linhas indicam as tarefas, metas e *softmetas* e as células centrais indicam os rótulos dos relacionamentos criados. Considere a direção coluna→linha, por exemplo, “Include [data]” é uma tarefa necessária para “Add [cenário]”, e assim por diante.

	Select [data]	Update [data]	Exclude [data]	Include [data]	...	Initiate [BD]	Connect [BD]	...
Select [data]						And	And	
Update [data]						And	And	
Exclude [data]						And	And	
Include [data]						And	And	
...								
Add [cenário]				And				
Exclude [cenário]			And					
Search [cenário]	And							
Change [cenário]		And						
Create [link] to [cenário]				And				
Create [link] to [lexicon]				And				
Add [project]				And				
Link [user] to [Project]				And				
Unlink [user] to [Project]			And					

Figura 60. Exemplo da visão matriz de rastreabilidade

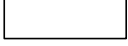
4.3.3.2.3.

Modelo Entidade-Relacionamento

O modelo entidade-relacionamento (MER) é utilizado para descrever as principais entidades do sistema e as relações entre elas. MER é normalmente utilizado para descrever estruturas de bancos de dados (Somerville, 2005). Durante a definição de requisitos o MER é utilizado para registrar as entidades e relacionamentos levando em conta o vocabulário utilizado no Universo de Informação.

Na Tabela 5, descrevemos a notação e as transformações para gerar modelos entidade-relacionamento a partir de AOV-graph.

Tabela 5. Regras de transformação entre AOV-graph e o modelo de entidade-relacionamento

Notação	
	Objetos: entidade, atributo
	Atributos: nome
	Notação:  ← Entidade  ← Atributo
	Relacionamento: relacionamento
	Atributos: origem, destino, cardinalidade
	Notação:  Relacionamento
Regras	
	Cada entidade tem um nome diferente das demais;
Transformações	
	▪ Cada tópico e novo atributo (criado por <i>intertype</i>) de <i>softmeta</i>, meta e tarefa que não é folha gera uma entidade cujo nome é o tópico ou o nome do novo atributo; ▪ cada tópico e novo atributo (criado por <i>intertype</i>) de <i>softmeta</i>, meta e tarefa que é folha gera um atributo na entidade gerada pelo elemento pai; ▪ cada relacionamento, no modelo de metas, entre elementos que geraram entidades gera relacionamento no modelo de entidade-relacionamento.

Na Figura 61, ilustramos um exemplo de MER gerado para o grafo “Model [requirement model]” do modelo base. Com as informações contidas em AOV-Graph conseguimos identificar entidades, atributos e relacionamentos, mas não conseguimos extrair a cardinalidade destes relacionamentos.

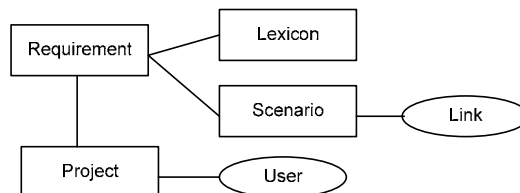


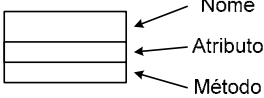
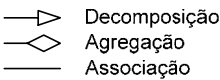
Figura 61. Exemplo da visão entidade-relacionamento para o grafo Model [requirements model]

4.3.3.2.4. Modelo de “objetos”

Objetos são entidades executáveis, instâncias de classes que definem seus atributos e serviços. Modelos de objetos ou de classes, normalmente, são utilizados quando se pretende adotar o paradigma de orientação a objetos nas atividades de desenho e implementação. Entretanto, estes modelos podem ser utilizados durante a definição de requisitos para representar os dados do sistema e as funcionalidades nas quais eles estão envolvidos (Sommerville, 2005). Neste

sentido, derivamos modelos de objetos do AOV-graph. Na Tabela 6, descrevemos a notação e as transformações para gerar modelos de objetos a partir de AOV-graph.

Tabela 6. Regras de transformação entre AOV-graph e o modelo de objetos

Notação	
Objetos: objetos	Atributos: nome, atributo, métodos
	Notação: 
	Relacionamento: agregação, decomposição, associação
Atributos: origem, destino, cardinalidade	
Notação:	
Regras	
Cada objeto tem um nome diferente dos demais;	
Transformações	
▪ Cada tópico e atributo (criado por <i>intertype</i>) de <i>softmeta</i>, meta e tarefa que não é folha gera um objeto cujo nome é o tópico ou o nome do atributo; ▪ cada tópico e atributo (criado por <i>intertype</i>) de sofmeta, meta e tarefa que é folha gera um atributo no objeto gerado pelo elemento pai; ▪ cada relacionamento de correlação, no modelo de metas, entre elementos que geraram objetos gera uma associação no modelo de objetos; ▪ cada <i>softmeta</i>, meta e tarefa gera um método no objeto gerado por seu respectivo tópico ou <i>intertype</i>.	

Na Figura 62, ilustramos um exemplo de modelo de objetos para o grafo “Model [requirements]” do modelo base. Neste exemplo temos os objetos “requirement model”, “lexicon”, “project” e “scenario”, e os atributos e operações realizadas sobre estes objetos; os relacionamentos entre eles são gerados com base no relacionamento existente entre tarefas, metas e *softmetas* cujos tópicos deram origem a estes objetos.

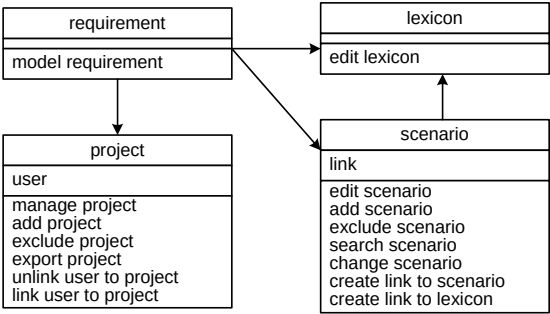


Figura 62. Exemplo da visão de objetos

4.3.4. Implementação

Implementamos a estratégia aplicada ao V-graph utilizando o padrão XML. AOV-graph foi definida em uma DTD (*Document Type Definition*), que é obedecida pelos modelos escritos na atividade de separação (modelo inicial) e os modelos gerados pela atividade de composição (modelo integrado). Os mecanismos de composição e visualização foram implementados utilizando XSLT (*eXtensible Stylesheet Language Transformation*). Na Figura 63, ilustramos como utilizamos o padrão XML para implementar nossa abordagem. O código desta implementação pode ser consultado nos Apêndices A e B.

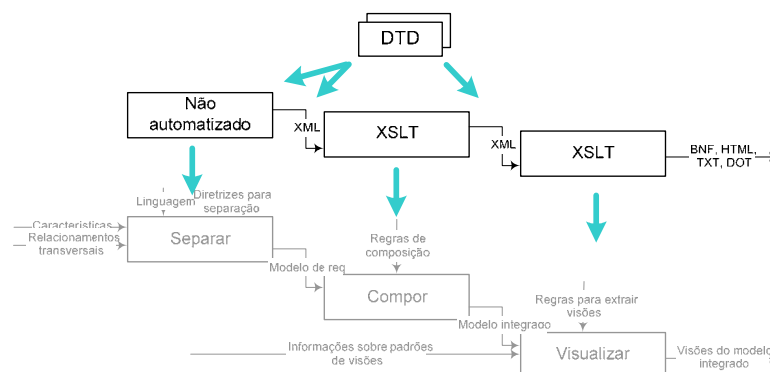


Figura 63. Implementação da estratégia para integração de características transversais

4.3.4.1. Separação

Especificamos a linguagem descrita na Seção 4.3 em uma DTD, ilustrada na Figura 64. A primeira parte da DTD é referente ao *V-graph* e a segunda parte, ao relacionamento transversal (em cinza). Como optamos por definir uma única DTD para os modelos escritos antes e depois da composição então flexibilizamos um pouco mais alguns dos elementos do modelo de componentes *V-graph*:

- Os elementos do *V-graph* – *Softmetas*, metas e tarefas possuem um identificador e nome como atributos, e os elementos tipo, tópico, referências para outros elementos do *V-graph*, novo tipo, e os relacionamentos de correlação e transversal (veja as linhas 7, 13 e 19 na Figura 64).
- Os relacionamentos do *V-graph* – o relacionamento de correlação é um elemento XML que possui o atributo *label*, podendo ser *make*, *help*, *known*, *hurt* e *break*, e os elementos *source* e *target*. Os relacionamentos de

contribuição e decomposição são representados pela hierarquia de metas, *softmetas* e tarefas. Desta forma, cada *softmeta*, meta e tarefa possui um atributo denominado rótulo do relacionamento (*decomposition_label*), podendo ser *And*, *Or*, *Make*, *Help*, *Unknown*, *Hurt* e *Break*, eles indicam o relacionamento entre filho e pai (veja as linhas 12, 18 e 24 na Figura 64).

- O relacionamento transversal – o relacionamento transversal é descrito pelos elementos origem, *pointcut*, *advice* e *intertype declaration*. Os *pointcuts* são descritos por operandos que podem definir um elemento específico do modelo *V-graph* através de seu identificador, ou um conjunto de elementos através de expressões regulares, podendo selecionar um conjunto de elementos do mesmo tipo (*softmeta*, meta e tarefa) ou qualquer dos tipos anteriores (*any*). Veja as linhas 44 a 79 na Figura 64.

```

1. <!ELEMENT aspect_oriented_model (goal_model)*>
2. <!ELEMENT goal_model ((softgoal | goal | task | softgoal_ref | goal_ref | task_ref)*,
3. (correlation_relationship | crosscutting_relationship)*)>
4. <!ATTLIST goal_model
5.   id ID #REQUIRED
6.   name CDATA #REQUIRED>
7. <!ELEMENT softgoal ( type*, topic*, (softgoal | task | softgoal_ref | task_ref | new_element_type)*,
8. (correlation_relationship | crosscutting_relationship)*)>
9. <!ATTLIST softgoal
10.   id ID #REQUIRED
11.   name CDATA #REQUIRED
12.   decomposition_label (and | or | break | hurt | unknown | help | make) "and">
13. <!ELEMENT goal ( type*, topic*, (goal | task | goal_ref | task_ref | new_element_type)*,
14. (correlation_relationship | crosscutting_relationship)*)>
15. <!ATTLIST goal
16.   id ID #REQUIRED
17.   name CDATA #REQUIRED
18.   decomposition_label (and | or | break | hurt | unknown | help | make) "and">
19. <!ELEMENT task (type*, topic*, ( task | task_ref | new_element_type)*, (correlation_relationship |
20. crosscutting_relationship)*)>
21. <!ATTLIST task
22.   id ID #REQUIRED
23.   name CDATA #REQUIRED
24.   decomposition_label (and | or | break | hurt | unknown | help | make) "and">
25. <!ELEMENT topic (#PCDATA)>
26. <!ELEMENT type (#PCDATA)>
27. <!ELEMENT correlation_relationship (source, target)>
28. <!ATTLIST correlation_relationship
29.   label (break | hurt | unknown | help | make) "help">
30. <!ELEMENT target (softgoal_ref | goal_ref | task_ref)>
31. <!ELEMENT source (softgoal_ref | goal_ref | task_ref)>
32. <!ELEMENT softgoal_ref EMPTY>
33. <!ELEMENT goal_ref EMPTY>
34. <!ELEMENT task_ref EMPTY>
35. <!ATTLIST softgoal_ref
36.   id IDREF #REQUIRED
37.   decomposition_label (and | or | break | hurt | unknown | help | make) "and">
38. <!ATTLIST goal_ref
39.   id IDREF #REQUIRED
40.   decomposition_label (and | or | break | hurt | unknown | help | make) "and">
41. <!ATTLIST task_ref
42.   id IDREF #REQUIRED
43.   decomposition_label (and | or | break | hurt | unknown | help | make) "and">

```

```

44. <!ELEMENT crosscutting_relationship (source, (pointcut | advice | intertype_declaration)*)>
45. <!ELEMENT pointcut (pointcut_expression)>
46. <!ATTLIST pointcut
47.   id ID #REQUIRED
48.   name CDATA #REQUIRED>
49. <!ELEMENT pointcut_expression (operand | ((operand | pointcut_expression), (and | or), (operand |
50. pointcut_expression)) | (not, (operand | pointcut_expression)))>
51. <!ELEMENT and EMPTY>
52. <!ELEMENT or EMPTY>
53. <!ELEMENT not EMPTY>
54. <!ELEMENT operand (softgoal_ref | goal_ref | task_ref | regular_expression)>
55. <!ATTLIST operand
56.   primitive (include | substitute) "include">
57. <!ELEMENT regular_expression (target*)>
58. <!ATTLIST regular_expression
59.   text CDATA #REQUIRED
60.   type (goal | sotgoal | task | any) "any"
61.   param (id | name) "name"
62.   path CDATA>
63. <!ELEMENT advice (a_it_expression, advice_body)>
64. <!ATTLIST advice
65.   type (after | before | around) "after">
66. <!ELEMENT a_it_expression (pointcut_ref | ((pointcut_ref | a_it_expression), (and | or), (pointcut_ref
67. | a_it_expression)) | (not, (pointcut_ref | a_it_expression)))>
68. <!ELEMENT pointcut_ref EMPTY>
69. <!ATTLIST pointcut_ref
70.   id IDREF #REQUIRED>
71. <!ELEMENT intertype_declaration (a_it_expression, intertype_declaration_body)>
72. <!ATTLIST intertype_declaration
73.   type (attribute | element) "attribute">
74. <!ELEMENT advice_body (softgoal_ref | goal_ref | task_ref)*>
75. <!ELEMENT intertype_declaration_body (new_element_type | softgoal | goal | task | softgoal_ref |
76. goal_ref | task_ref)*>
77. <!ELEMENT new_element_type (new_element_type)*>
78. <!ATTLIST new_element_type
79.   name CDATA #REQUIRED
   value CDATA #REQUIRED>

```

Figura 64. DTD referente à AOV-graph

4.3.4.2. Composição

Utilizamos XSLT para implementar o mecanismo de composição. Devido a algumas limitações da linguagem XSLT referentes ao sequenciamento na aquisição e escrita de dados, nós dividimos a composição em quatro partes: processamento de expressões regulares, identificação de transformações, composição da primitiva *substitute* e composição da primitiva *include*, veja Figura 65.

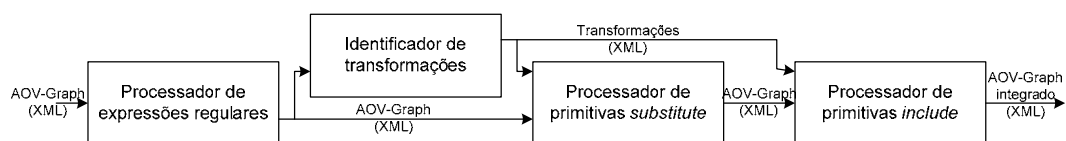


Figura 65. Componentes do mecanismo de composição

1. O processador de expressões regulares é responsável por criar os tipos e tópicos de todas as metas, *softmetas* e tarefas e por criar elementos *target* em *pointcuts* definidos por expressões regulares. Este processamento resulta em um arquivo XML que também está de acordo com a DTD definida anteriormente. Este componente exige a versão 2.0 de XSLT, pois esta inclui o tratamento de expressões regulares.
2. O identificador de transformações é responsável por ler os relacionamentos transversais e gerar um arquivo XML contendo onde e o que deve ser adicionado em cada *pointcut* segundo a ordem em que estas transformações devem ser aplicadas.
3. O processador dos elementos a serem substituídos recebe os dois arquivos XMLs anteriores (passos 1 e 2) e apaga todos os elementos (e seus filhos) indicados pelos *pointcuts* com primitiva *substitute*.
4. O processador dos elementos a serem incluídos também recebe os dois arquivos XMLs anteriores (passos 2 e 3) e inclui todos os elementos definidos em *advice* e *intertype declaration* no local indicado pelos *pointcuts* com ambas as primitivas.

4.3.4.3. Visualização

O mecanismo de visualização gera visões de AOV-graph. Cada visão é gerada por um arquivo XSLT que implementa separadamente as transformações definidas na Seção 4.3.3. Na Figura 66, ilustramos os componentes implementados para a atividade de visualização. O formato DOT é utilizado juntamente com o software GraphViz (Graphviz, 2006) para gerar figuras (JPG).

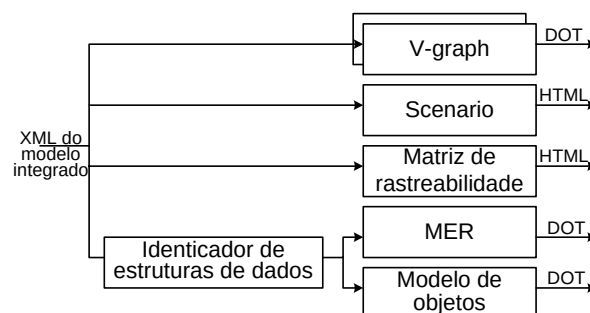


Figura 66. Componentes do mecanismo de visualização

4.4.

Aplicação da Estratégia a outros Modelos de Requisitos

A estratégia definida na Seção 4.1 pode ser aplicada a diferentes modelos de componentes, tais como lista de requisitos, casos de uso, léxico, cenários, i*, dentre outros. Para utilizá-la é necessário instanciar (Figura 31, página 65): o modelo de componentes, o modelo de *joinpoints*, os mecanismos de composição e de visualização. A seguir, descrevemos quais os benefícios e como aplicar nossa estratégia aos modelos de Cenários, Léxico e sentenças de requisitos. Definimos os modelos de componentes e de *joinpoints* para estas linguagens e apenas apontamos como os mecanismos de composição e de visualização poderiam ser definidos para estes modelos de componentes. Assim como apresentamos na Seção 4.3, seria necessário definir e implementar as regras de composição e de transformação para estes mecanismos.

4.4.1.

Cenários

Em (Leite, 1997) é definido um modelo para especificação de cenários estruturados. Cada cenário é descrito pelas informações: título, objetivo, contexto, atores, recursos, episódios. Esta linguagem é focada na sequência de cenários, ou de funções para cumprir uma funcionalidade. Como a decomposição dominante é funcional então as informações sobre o contexto, atores e recursos encontram-se espalhadas e entrelaçadas no conjunto de cenários que retrata o sistema.

A abordagem em (Leite, 1997) define um esquema de rastreabilidade entre estes elementos através de um léxico e da utilização de hiper-elos (*hyperlinks*), permitindo a navegação entre definições (no léxico) e cenários, mas que abordam apenas parcialmente estes problemas. Além disto, os relacionamentos entre cenários através da decomposição funcional muitas vezes agrupam funções (episódios) pouco coesas. Nestes casos, mudar a direção do relacionamento, tal como em AspectJ, e ter um mecanismo de composição seria interessante para incluir e excluir episódios e outras informações menos coesas de cada cenário.

Título	Adicionar cenário
Objetivo	Inserir um cenário no modelo de cenários
Ator	Usuário participante, sistema
Recurso	BD, informações do cenário
Contexto	Pre-condição: <u>Usuário participante tem uma sessão</u> , um projeto está aberto
Exceção	Se cenário já existe então enviar mensagem de erro Se usuário participante fechar janela do sistema então <u>fechar sessão</u> Se usuário participante fechar janela do sistema então <u>desconectar BD</u>
Episódio	Usuário participante preenche as informações do cenário Sistema verifica se informações do cenário estão completas Sistema verifica se sintaxe do cenário está correta <u>Se BD está desconectado então conectar BD</u> Se cenário não existe no projeto então <u>incluir cenário</u> <u>Desconectar BD</u>

Figura 67. Exemplo de espalhamento e entrelaçamento no modelo de cenários

Na Figura 67, ilustramos o cenário “Adicionar cenário”, nele as informações em cinza são referentes à infra-estrutura de armazenamento de dados (banco de dados) e controle de acesso (através de sessões). Tais informações são invasivas e não há controle sobre como elas estão espalhadas e entrelaçadas. Desta forma, ao modificar a infra-estrutura de armazenamento de banco de dados, todos os cenários têm que ser novamente inspecionados para retratar esta modificação.

```

1. <aspect_oriented_model> := <scenario_model>
2. <scenario_model>:=scenario_model(<name>;<id>){<component><relationship>} |
3.   <scenario_model> <scenario_model>
4. <component> := <scenario> | <component> <component>
5. <scenario>:=scenario (<title>;<scenario_id>)
6.   {<goal>;<context>;<actor>;<resource>;<exception>;<episode>;<relationship>}
7. <goal> := goal <name><goal_id> <relationship>
8. <actor> := actor <name><actor_id> <relationship> | <actor> <actor>
9. <resource>:= resource <name><resource_id> <relationship> | <resource> <resource>
10. <context> := context <sentence> <context_id> <relationship> | <context> <context>
11. <exception>:=exception<sentence><exception_id><relationship>|<exception><exception>
12. <episode> := episode <sentence><episode_id> <relationship> | <episode> <episode>
13. <sentence> := <name>|<scenario_ref>| <sentence><sentence>
14. <scenario_ref> := <scenario_id>
15. <goal_ref> := <goal_id>
16. <actor_ref> := <actor_id>
17. <resource_ref> := <resource_id>
18. <exception_ref> := <exception_id>
19. <episode_ref> := <episode_id>
20. <context_ref> := <context_id>
21. <title>:=”a..z” <name>:=”a..z” <scenario_id>:=”a..z” <goal_id>:=”a..z” <actor_id>:=”a..z”
22. <resource_id>:=”a..z” <exception_id>:=”a..z”
23. <episode_id>:=”a..z” <context_id>:=”a..z”
24. <relationship> := <crosscuttingRel> | <relationship> <relationship>
25.
26. <joinpoint> := <scenario_ref> | <goal_ref> | <context_ref> | <actor_ref> |
27.   <resource_ref> | <exception_ref> | <episode_ref>

```

Figura 68. Sintaxe do modelo de componentes: Cenários

Para utilizar nossa estratégia com o modelo de cenários definimos os itens da Figura 31 (página 65) como a seguir:

- Modelo de componentes – é definido pelos itens do modelo de cenários: cenário, título, objetivo, atores, recursos, contexto, episódios e exceções

(linhas 1 a 24 da Figura 68). O relacionamento transversal (em cinza na Figura 68) tem exatamente a mesma sintaxe do definido na Seção 4.2.3.

- Modelo de *joinpoints* – é definido pelos mesmos itens do modelo de componentes, com exceção do título. Eles expõem informações importantes que podem ser utilizadas para identificar os cenários que são afetados (linhas 26 e 27 da Figura 68).
- Mecanismo de composição – pode incluir, alterar ou excluir episódios em um cenário e juntamente incluir ou excluir os elementos contexto, atores, recursos e exceções intrínsecos àqueles episódios. Na Figura 69(d), apresentamos um exemplo.
- Mecanismo de visualização – a estruturação das informações descritas em cenários permite a extração de várias visões, tais como: modelo de classes; tutorial para os usuários; visão gráfica do relacionamento entre cenários, estes relacionamentos podem ser devido à utilização dos mesmos atores, recursos, exceções ou episódios ou de acordo com o tipo de interação que ocorre entre eles (Breitman, 2000); dentre outras. A Figura 69(d) ilustra uma visão parcial dos cenários após a composição, enquanto a Figura 69(a) e Figura 69(b) ilustram cenários antes da composição.

Na Figura 69, ilustramos um exemplo de descrição em cenários utilizando nossa abordagem: (a) é o cenário “Adicionar cenário” sem qualquer referência a infra-estrutura de banco de dados, (b) é o cenário “Persistência em BD”, e (c) é a descrição do relacionamento transversal de (b) para (a). A composição destes cenários resulta na descrição ilustrada em (d).

Na descrição do relacionamento transversal (Figura 69(c)), há dois *pointcuts*: em PC1 indicamos que o cenário intitulado “Adicionar cenário” é afetado e em PC2 indicamos que qualquer episódio cujo nome contenha a palavra “Incluir” é afetado. Podemos observar também a utilização de *advice* around, before e after associados a *pointcuts* com a primitiva include, indicando inclusão em, respectivamente, qualquer lugar, antes e depois do ponto definido pelo *pointcut*. Veja o resultado da composição na Figura 69(d).

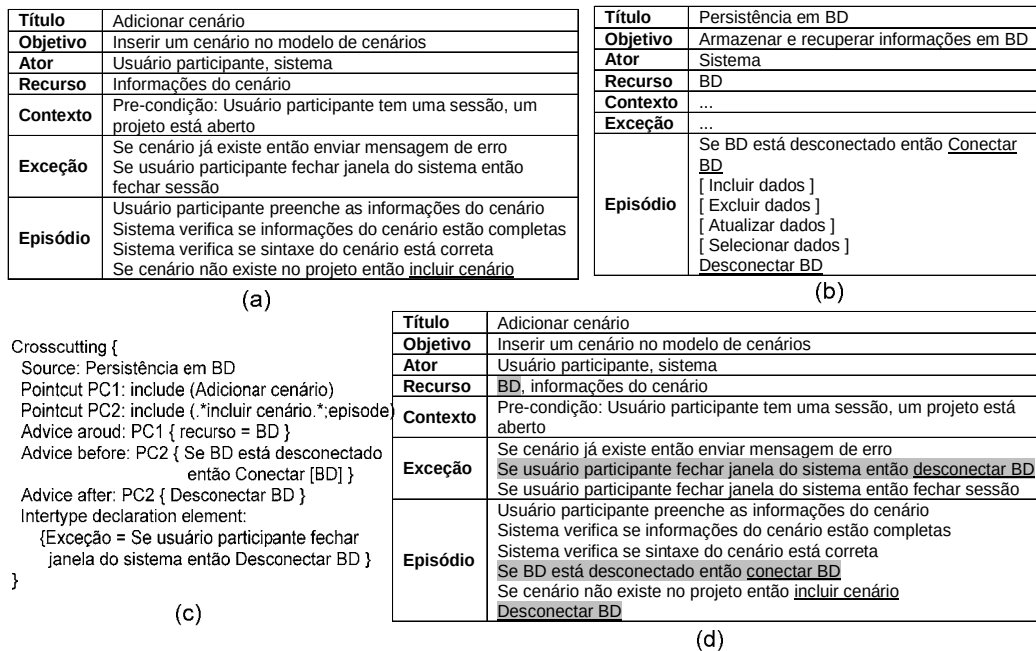


Figura 69. Exemplo do uso de nossa abordagem no modelo de cenários

4.4.2. Léxico Ampliado da Linguagem (LAL)

Em (Leite, 1990) é definido um modelo baseado em linguagem natural semi-estruturada para especificação de termos utilizados no universo de informações (UdI). Este modelo registra cada termo como um símbolo, que consiste de tipo, sinônimos, noção e impacto. O relacionamento entre símbolos ocorre quando o nome de um símbolo é citado na descrição de outro símbolo, podendo representar um relacionamento de decomposição, de abstração ou associação. Além disto, é requerido que o léxico seja circular, i.e., a maximização do uso dos símbolos definidos. Quando um símbolo é incluído ou excluído os relacionamentos para aquele símbolo são atualizados, inseridos ou excluídos. No caso de exclusão, isto significa apenas que o símbolo não é mais uma entrada do LAL, ele continua sendo referido na descrição dos outros símbolos.

O relacionamento transversal pode ser utilizado para indicar uma relação mais forte entre o nome de um símbolo e as referências a ele na descrição dos outros termos, indicando que se ele existe como entrada do LAL implica em referências a ele, e sua inexistência implica na ausência daquelas referências. Na Figura 70, ilustramos a descrição de um símbolo do léxico, nesta descrição os elementos sombreados indicam a existência do “léxico” não apenas como símbolo

do modelo, mas como documento existente no sistema modelado. Desta forma, caso o “léxico” não exista no sistema, então é necessária a inspeção de todos os símbolos para exclusão de referências a ele.

Nome	Cenário
Tipo	Objeto
Sinônimo	Cenários
Noção	Descrição em linguagem natural semi-estruturada representando uma situação que ocorre no <u>UdI</u> ou a interação entre <u>usuários</u> e <u>sistema</u> . Contém <u>título</u> , <u>objetivo</u> , <u>atores</u> , <u>recursos</u> , <u>contexto</u> , <u>episódios</u> , <u>exceções</u> . <u>Utiliza símbolos do léxico</u> .
Impacto	Adicionar cenário Excluir cenário Modificar cenário Criar referências para cenários Excluir referências para cenários <u>Criar referências para símbolos do léxico</u> <u>Excluir referências para símbolos do léxico</u> Verificar cenário

Figura 70. Exemplo de espalhamento e entrelaçamento no LAL

1.	<aspect_oriented_model> := <lexicon_model>
2.	<lexicon_model> := lexicon_model (<name>; <id>){<component> <relationship>}
3.	<lexicon_model> <lexicon_model>
4.	<component> := <symbol> <component> <component>
5.	<symbol>:= symbol (<name>;<symbol_id>)
6.	{<synonym>; type <type>;<notion>;<behavior_response><relationship>}
7.	<synonym> := synonym <name><goal_id>
8.	<type> := object state verb subject
9.	<notion> notion <sentence><notion_id> <relationship> <notion> <notion>
10.	<sentence>:= <name> <symbol_ref> <sentence><sentence>
11.	<behavior_response> :=
12.	behavior_response <sentence><behavior_response_id> <relationship>
13.	<behavior_response> <behavior_response>
14.	<symbol_ref> := <symbol_id>
15.	<synonym_ref> := <synonym_id>
16.	<notion_ref> := <notion_id>
17.	<behavior_response_ref> := <behavior_response_id>
18.	<name>:= "a..z" <symbol_id>:= "a..z" <synonym_id>:= "a..z"
19.	<notion_id>:= "a..z" <behavior_response_id>:= "a..z"
20.	<relationship> := <crosscuttingRel> <relationship> <relationship>
21.	
22.	<joinpoint>:=<symbol_ref> <notion_ref> <synonym_ref> <behavior_response_ref>

Figura 71. Sintaxe do modelo de componentes: LAL

Para utilizar nossa estratégia com o LAL, definimos os itens da Figura 31 (página 65) como a seguir:

- Modelo de componentes – é definido pelos itens do LAL: símbolo, nome do símbolo, sinônimos, tipo, noção e impacto (linhas 1 a 20 da Figura 71).
- Modelo de *joinpoints* – é definido pelos itens noção e impacto (linha 22 da Figura 71).
- Mecanismo de composição – pode incluir ou excluir sentenças na noção e no impacto de outros símbolos, registrando a referência naqueles para o “símbolo transversal”.

- Mecanismo de visualização – o LAL registra informações sobre objetos, verbos, estados e sujeitos e desta maneira permite a extração destas visões isoladamente ou em conjunto, por exemplo, quais os atores do sistema e como estão relacionados entre si, quais atores estão relacionados a quais ações, dentre outras.

A Figura 72 ilustra um exemplo de descrição de símbolos do léxico utilizando nossa abordagem: (a) representa o símbolo “Cenário”, (b) representa o termo “Símbolo”, (c) é a descrição do relacionamento transversal existente entre (b) e (a), e (d) representa o resultado da composição. Desta forma, se o sistema sendo desenvolvido não tem funcionalidades a respeito de “Símbolos do léxico”, então a exclusão do termo “Símbolo” implicaria não apenas na remoção do sublinhado em símbolos do léxico na descrição de “Cenário” (d) mas também na remoção das frases em cinza deste mesmo símbolo (d).

Nome	Cenário
Tipo	Objeto
Sinônimo	Cenários
Noção	Descrição em linguagem natural semi-estruturada representando uma situação que ocorre no <u>UdI</u> ou a interação entre <u>usuários</u> e <u>sistema</u> . Contém <u>título</u> , <u>objetivo</u> , <u>atores</u> , <u>recursos</u> , <u>contexto</u> , <u>episódios</u> , <u>exceções</u> .
Impacto	Adicionar cenário Excluir cenário Modificar cenário Criar referências para cenários Excluir referências para cenários Verificar cenário

(a)

```

Crosscutting {
Source: Símbolo
Pointcut PC1: include (Cenários)
Advice aroud: PC1 { Impacto = Criar referências para
símbolos; Impacto = Excluir referências para
símbolos }
Intertype declaration element: PC1
{ Noção = Utiliza símbolos do léxico }
}

```

(c)

Nome	Símbolo
Tipo	Objeto
Sinônimo	Símbolo do léxico, Símbolos do léxico, símbolos
Noção	Descrição em linguagem natural semi-estruturada representando os termos utilizados no <u>UdI</u> . Contém <u>nome</u> , <u>tipo</u> , <u>sinônimo</u> , <u>noção</u> , <u>impacto</u> .
Impacto	Adicionar símbolo Excluir símbolo Modificar símbolo Criar referências para símbolos Excluir referências para símbolos Verificar símbolo

(b)

Nome	Cenário
Tipo	Objeto
Sinônimo	Cenários
Noção	Descrição em linguagem natural semi-estruturada representando uma situação que ocorre no <u>UdI</u> ou a interação entre <u>usuários</u> e <u>sistema</u> . Contém <u>título</u> , <u>objetivo</u> , <u>atores</u> , <u>recursos</u> , <u>contexto</u> , <u>episódios</u> , <u>exceções</u> . <u>Utiliza símbolos do léxico.</u>
Impacto	Adicionar cenário Excluir cenário Modificar cenário Criar referências para cenários Excluir referências para cenários Verificar cenário Criar referências para <u>símbolos</u> Excluir referências para <u>símbolos</u>

(d)

Figura 72. Exemplo do uso de nossa abordagem no Léxico

4.4.3. Sentenças de Requisitos

Sentenças de requisitos é a maneira mais usual de documentar requisitos. Elas são, geralmente, escritas livremente em linguagem natural, permitindo sua leitura tanto por pessoal técnico quanto usuários e clientes. Costuma-se numerar

os requisitos para que sejam identificados e referidos. Entretanto, como apresentamos no Capítulo 2 (Figuras 6 e 7), algumas informações e características estão espalhadas e entrelaçadas em muitas sentenças de requisitos, dificultando a identificação e atualização delas.

O relacionamento transversal pode ser utilizado para mapear a interação entre sentenças, registrando onde elas se sobrepõem ou estão repetidas. Desta maneira, pode-se criar uma lista de requisitos com ou sem repetição, separando ou compondo as “sentenças transversais”. Neste caso, o relacionamento transversal pode ser entendido como uma matriz de rastreabilidade estendida, que permite o mecanismo de composição gerar as interações registradas nele.

```

1. <aspect_oriented_model> := <requirementSentence_model>
2. <requirementSentence_model> := <component> | <component> <component>
3. <component> := <requirementSentence> | <component> <component>
4. <requirementSentence> := (<name>; <requirementSentence_id>; <relationship_label>){<component>
5. <relationship>}
6. <requirementSentence_ref> := <requirementSentence_id>; <relationship_label>
7. <relationship> := <crosscuttingRel> | <relationship> <relationship>
8. <name>:="a..z" <requirementSentence_id>:="a..z" <relationship_label>:="a..z"
9. <joinpoint> := <requirementSentence_ref>

```

Figura 73. Sintaxe do modelo de componentes: sentenças de requisitos

Para utilizar nossa estratégia com sentenças de requisitos, definimos os itens da Figura 31 (página 65) como a seguir:

- Modelo de componentes – é definido por sentenças, cada sentença pode ser decomposta em outras sentenças (linhas 1 a 8 da Figura 73).
- Modelo de *joinpoints* – é definido, também, pelas sentenças de requisitos, indicando a interação entre quaisquer sentenças (linha 9 da Figura 73).
- Mecanismo de composição – pode incluir ou excluir sentenças de requisitos como parte da hierarquia de outras sentenças, espalhando e/ou entrelaçando as informações. Cada sentença a ser afetada pode ser identificada por sua numeração ou através de expressões regulares.
- Mecanismo de visualização – cada hierarquia de requisitos pode ser vista separadamente ou não; matrizes de rastreabilidade podem ser facilmente geradas; expressões regulares podem ser utilizadas para recuperar sentenças que mencionam o mesmo sujeito, verbo, ou objeto, dentre outras visões.

Na Figura 74, ilustramos: em (a) um lista de requisitos em que separamos restrições de formatação (requisito 5) e funções de edição (requisito 4) das demais porque ele se repetem no documento; em (b) a descrição de dois relacionamentos transversais cujas origens são os requisitos de número 4 e 5 respectivamente; o

primeiro espalha os requisitos 4.1, 4.2 e 4.3, e o segundo espalha uma restrição quanto ao formato de hora/data; e em (c) está a descrição de requisitos resultante da composição.

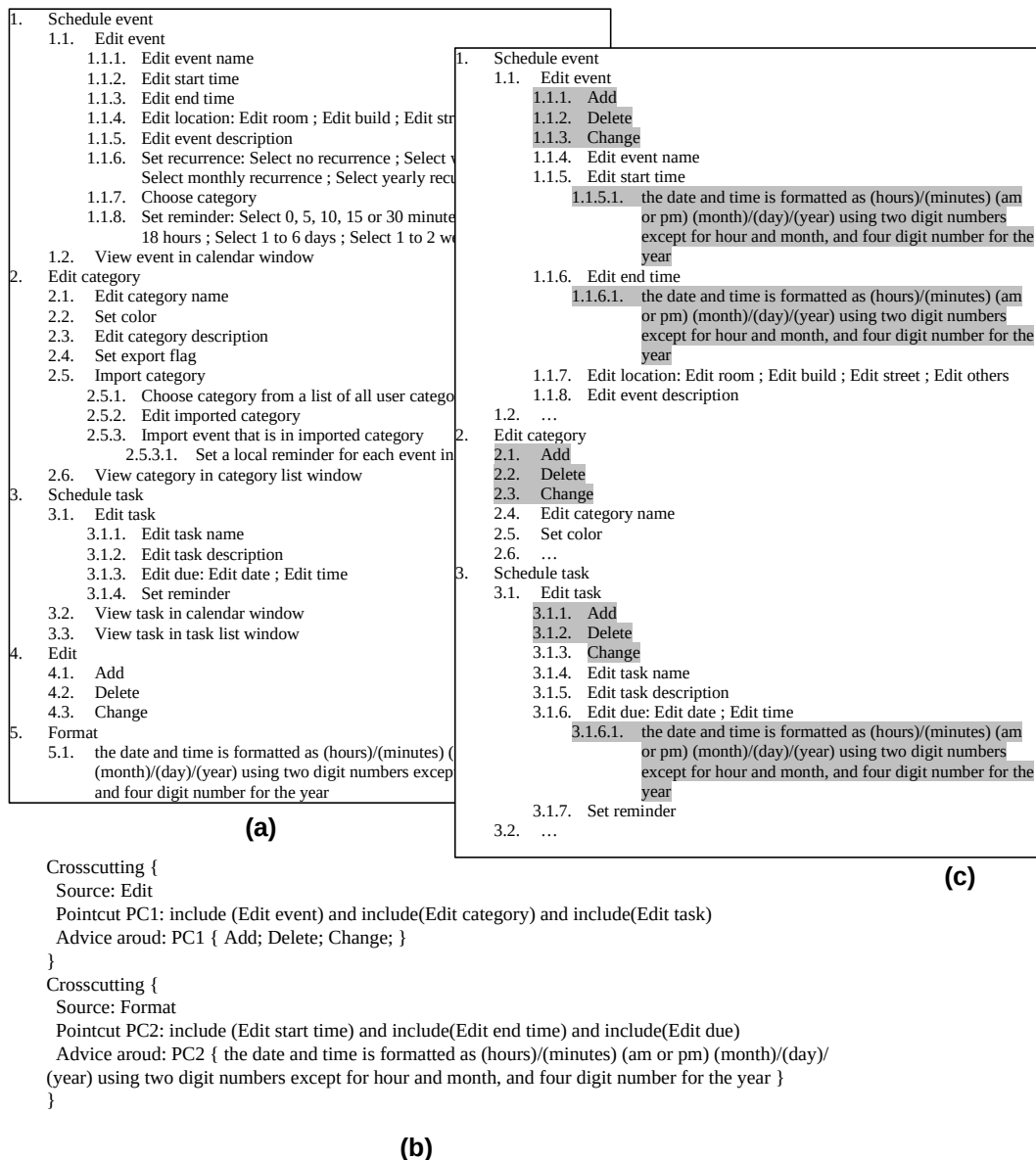


Figura 74. Exemplo do uso de nossa abordagem em sentenças de requisitos

4.5.

Processo de Engenharia de Requisitos Utilizando a Estratégia Orientada a Aspectos

Como apresentamos no Capítulo 2, o processo de engenharia de requisitos é formado pelas atividades: elicitação, modelagem e análise. O foco deste trabalho é a atividade de modelagem, mas como não há uma delimitação clara entre estas atividades, então a elicitação e análise são fortemente afetadas. Consideramos que

utilizando nossa estratégia tornamos a atividade de modelagem menos árdua e mais ágil, e desta maneira, auxiliamos a elicitação e a análise porque o engenheiro consegue obter visões mais dirigidas à atividade em questão, veja Figura 75.

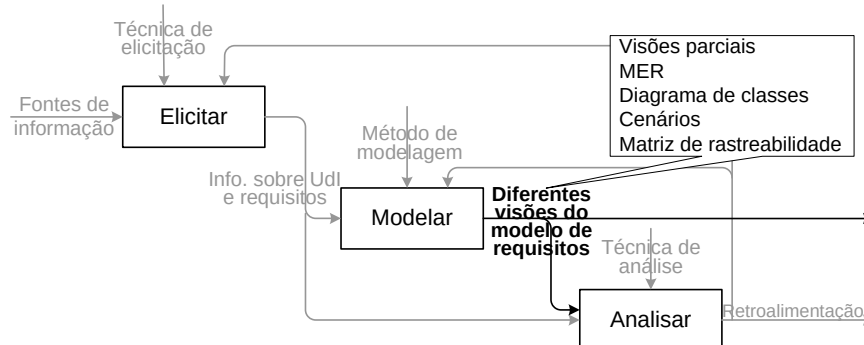


Figura 75. Processo de engenharia de requisitos modificado pela estratégia de integração de características transversais

O ciclo elicitação-modelagem-análise ocorre em muitas iterações. Propomos que a primeira iteração seja realizada da maneira convencional, i.e.:

- Elicitação – os engenheiros elicitam requisitos utilizando alguma técnica de elicitação;
- Modelagem – os engenheiros especificam os requisitos elicitados utilizando uma instância de LMROA. Inicialmente, o engenheiro deve se preocupar apenas em encontrar a melhor decomposição para os requisitos, utilizando os construtos definidos no modelo de componentes. Tendo apenas uma parte do modelo desenhado, já é possível extrair diferentes visões para serem analisadas.
- Análise – com o apoio de várias visões o engenheiro pode analisar o modelo do sistema sob diferentes ângulos, possivelmente com menor complexidade. Verificando se ele está correto e claro de acordo com o que os requerentes esperam, o engenheiro procura melhorar a especificação, tanto na decomposição e modularização dos requisitos quanto modificando ou adicionando novas informações, i.e, ele volta à atividade de modelagem e elicitação. O engenheiro deve estar atento a características que estão espalhadas e entrelaçadas, elas normalmente dificultam a compreensão do modelo de requisitos. Neste caso, as características transversais são aquelas que se repetem, que estão fortemente dependentes, mas não possuem um forte relacionamento conceitual (semântico) ou aquelas que são candidatas a muitas evoluções ou a reuso (veja na Seção 4.2.4).

Voltando à atividade de modelagem, o engenheiro deve procurar modularizar estas características e desenhar as interações delas para o restante do sistema, usando relacionamentos transversais. Neste caso, pode ser necessário, novamente, voltar aos requerentes e elicitar mais requisitos ou esclarecer pontos ambíguos.

Note que, a atividade de identificar características transversais é adiada para quando o engenheiro tem alguma parte do modelo desenhada, sendo parte da atividade de análise. Entretanto, muitas vezes a distância entre modelar e analisar se aquela é a melhor modularização e decomposição é tão pequena que, nestes casos, identificar problemas e buscar a melhor representação é parte do processo de modelagem.

Esperamos que por oferecer visões automaticamente, agilizamos a modelagem e, assim, permitimos que este ciclo ocorra em menos tempo e que ocorram mais iterações. Para a atividade de elicitação, esta estratégia de modelagem pode oferecer visões mais apropriadas para os usuários a serem abordados, mostrando os serviços que lhes interessam. Para a atividade de análise, esta estratégia oferece diferentes visões, parciais ou não, que podem ajudar o engenheiro a analisar a corretude, consistência, omissões, impacto de mudanças, custo, dentre outros.

4.6. Resumo

A estratégia apresentada neste Capítulo tem o objetivo de facilitar a modelagem de requisitos. As atividades de Separação e Composição foram baseadas nos fundamentos do paradigma de aspectos e modificadas para aplicação no processo de requisitos. A atividade de Visualização foi baseada em trabalhos sobre visões e pontos-de-vista, considerados fundamentais para o processo de requisitos (Nuseibeh, 1994; Kotonya, 1996; Leite, 1996).

Esta abordagem pode ser aplicada a diferentes modelos, sendo necessário definir os elementos mostrados na Seção 4.1. Consideramos que separação, composição e visualização constituem o conjunto mínimo de atividades necessário para tratar os problemas de espalhamento e entrelaçamento durante a definição de requisitos, quando a especificação é muito volátil devido à constante mudança na demanda e no entendimento sobre o contexto e problema em questão.

Constatamos que o trio separação-composição-visualização é importante em conjunto porque:

- Se somente separamos não temos a visão do sistema inteiro, e muito trabalho é realizado para manter a consistência entre as partes;
- Se não separamos, o modelo é demasiadamente complexo tornando-se difícil analisá-lo e mantê-lo;
- Se separamos e combinamos, diminuimos o problema de inconsistências, mas o problema em analisar o modelo complexo resultante da composição permanece;
- Se separamos e visualizamos, aumentamos o poder de análise do modelo, mas cada visão é apenas parte do modelo do sistema;
- Se não separamos e geramos várias visões, podemos obter visões parciais ou totais, i.e., o problema de complexidade pode ser controlado, entretanto esta abordagem se preocupa a posteriori com a melhor modularização e o reuso das partes, sendo necessário um trabalho de refatoração.

Assim, aplicamos nossa abordagem ao modelo de metas V-graph, instanciando LMROA e definindo os mecanismos de composição e visualização para este modelo de componentes. Além disso, apontamos como esta abordagem pode ser aplicada aos modelos: cenários, LAL e sentenças de requisitos. Finalizamos, mostrando qual o impacto de aplicar nossa abordagem orientada a aspectos no processo de definição de requisitos.

5

Estudos de Caso

Neste Capítulo, apresentamos dois estudos de caso realizados para demonstrar a abordagem descrita no Capítulo 4. O primeiro estudo de caso é um sistema colaborativo para edição de léxico e cenários (também utilizado como exemplo nos capítulos anteriores) e o segundo é um sistema para divulgação de eventos na Universidade da Califórnia.

Ambos os estudos de caso referem-se a sistemas simples (com pequeno número de funcionalidades). Ainda assim, a modelagem em AOV-graph apresenta um número excessivo de relacionamentos, dificultando a leitura, manipulação e análise dos modelos. Esta dificuldade é decorrente não apenas da decomposição dominante imposta pelo modelo de componentes (V-graph), mas principalmente do espalhamento e entrelaçamento de características. Desta forma, aplicar nossa abordagem trouxe alternativas para visualizar os modelos e representar a interação de elementos que se espalham e/ou se entrelaçam em muitas partes dos grafos (modelos de metas).

Na Seção 5.1, descrevemos o contexto e processo utilizado para elaboração destes estudos de caso. Nas Seções 5.2 e 5.3 detalhamos cada estudo de caso por meio da descrição do problema, modelagem e análise dos resultados. Na Seção 5.4, resumimos e mostramos como tais resultados confirmam as contribuições de nossa estratégia para modelagem, rastreabilidade, evolução e reuso de requisitos.

5.1.

Contextualização

A modelagem dos estudos de caso, em AOV-graph, foi realizada com base em documentos disponíveis: no primeiro caso não há um documento de requisitos, mas há uma descrição de suas funcionalidades e arquitetura em (Silva, 2005a), e o sistema, bem como seus cenários encontram-se disponível em (C&L, 2005); e no segundo caso, nos baseamos exclusivamente no documento de requisitos disponível em (Unical, 2005).

Estes documentos foram utilizados na atividade de separação de nossa abordagem. O processo de separação ocorreu da seguinte maneira:

- Leitura da documentação – a leitura ocorreu inicialmente de maneira integral para identificarmos a estruturação e organização dos requisitos, bem como conhecer qual o propósito geral do sistema. Em seguida, a leitura foi realizada de maneira pontual para reescrevermos os requisitos através de metas, *softmetas* e tarefas.
- Identificação dos principais grupos de requisitos – Com base no propósito geral, estruturação e organização dos documentos inferimos os principais requisitos (macro-requisitos ou características) do sistema. A separação mais visível entre estas características é com relação ao que pertence exclusivamente ao objetivo do sistema e ao que é decorrente da automação deste objetivo. Este segundo grupo está, normalmente associado a RNFs e suas operacionalizações, e pode muitas vezes ser modelado de maneira que possa ser reutilizado em outras aplicações. Desta forma, modelamos as características que constituem o núcleo do sistema como um modelo, e cada característica pertencente ao segundo grupo, como um modelo separado dos demais.
- Modelagem dos requisitos como tarefa, meta e *softmeta* – para cada característica, identificamos os requisitos associados e os modelamos como meta, *softmeta* ou tarefa. Sendo que a relação entre requisito (ou cenário, no caso do C&L) para elemento no AOV-graph nem sempre é de um para um. Isto acontece porque, muitas vezes, os requisitos possuem mais de uma função ou dado (entrelaçamento de informações), ou requisitos com o mesmo significado são escritos de maneiras diferentes (espalhamento de informações).
- Relacionando metas, *softmetas* e tarefas – para relacionar os requisitos utilizamos os elos de contribuição, correlação e transversal. Os elos de correlação e contribuição são escritos à medida que modelamos metas, *softmetas* e tarefas, de maneira a identificar suas decomposições. Os relacionamentos transversais são escritos quando metas, *softmetas* e tarefas afetam diferentes pontos nos modelos de maneira a tornar as informações espalhadas e entrelaçadas (veja algumas heurísticas no Capítulo 4, Seção 4.2.2).

- Modificação dos modelos criados – a modelagem das informações contidas na documentação não ocorre em uma única iteração. À medida que a decomposição de metas, *softmetas* e tarefas está sendo modelada, podemos identificar outros grupos de requisitos a serem separados, elementos a serem detalhados e relacionamentos a serem agrupados em um elo transversal.

Estes modelos foram escritos em XML e todos os diagramas foram gerados por meio de XSLT para o formato de entrada do software Visigraph (Visigraph, 2006). Este software é responsável por esquematizar e gerar figuras no formato JPG. Desta forma, a implementação não oferece suporte à edição gráfica dos modelos; para realizar qualquer alteração é necessário modificar o modelo em XML.

As atividades de composição e visualização que complementam este processo de separação ocorrem em seguida, apoiadas pela implementação descrita no Capítulo 4, Seção 4.3.4. Estes mecanismos dão apoio à atividade de separação porque geram diferentes visões do modelo original, facilitando sua análise e modificação.

Para avaliar os ganhos obtidos com o uso do relacionamento transversal, criamos uma nova visão do modelo AOV-graph que mostra todos os modelos e seus relacionamentos sem mostrar a separação entre eles. Esta visão representa como seriam os modelos em V-graph se não utilizássemos nossa abordagem. Assim, podemos mostrar as diferenças gráficas entre estas duas abordagens.

A legenda dos elementos de V-graph e AOV-graph é ilustrada na Figura 76. Esta legenda é válida para todos os modelos de metas dos estudos de caso. Os relacionamentos em **negrito** são os relacionamentos transversais ou relacionamentos gerados a partir de um relacionamento transversal.

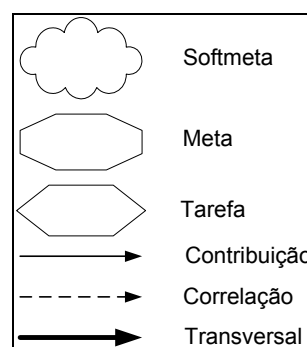


Figura 76. Legenda dos elementos de V-graph e AOV-graph

5.2.

Sistema Colaborativo para Edição de Léxico e Cenários – C&L

C&L é um sistema WEB para edição de cenários e léxico segundo a definição de Leite (1997). Este sistema provê um ambiente colaborativo para que participantes do mesmo projeto possam navegar entre léxico e cenários, e sugerir mudanças para os mesmos. Este sistema foi desenvolvido no meio acadêmico incrementalmente por diferentes equipes, a descrição de suas funcionalidades e alguns detalhes de sua arquitetura estão disponíveis em (Silva, 2005; Felicíssimo, 2004). A modelagem de C&L, apresentada nesta tese, foi realizada tendo como base, também, o sistema disponível em (C&L, 2005).

5.2.1.

Modelagem

A meta principal do sistema C&L é promover a definição de cenários e léxico por membros de equipes que trabalham colaborativamente, mesmo que estejam geograficamente distribuídos. Para realizar esta meta, opta-se por desenvolver uma ferramenta para WEB, um ambiente aberto que promove a divulgação e compartilhamento de informações. Por outro lado, este ambiente exige alguns mecanismos para prover confidencialidade às informações de cada projeto. Além dos requisitos específicos de confidencialidade e de modelagem de requisitos, características de persistência e confiabilidade também estão presentes. Denominamos a característica referente à modelagem de requisitos de CEL para facilitar a citação.

Na Figura 77, ilustramos a modelagem de C&L em V-graph; sem utilizar nossa abordagem. Os quatro grupos de requisitos (CEL, Confidencialidade, Segurança e Persistência) são modelados formando um único grafo de *softmetas*, metas e tarefas. Neste modelo não está explícito quais elementos são referentes à quais grupos de requisitos (quais os requisitos referentes à confidencialidade ou específicos do CEL?). A complexidade decorrente disto e do grande número de relacionamentos e elementos, dificulta a manipulação, análise e modelagem do modelo inteiro ou referente a cada grupo de requisitos. Isto afeta, também, a reusabilidade, porque é difícil identificar partes da modelagem que poderiam ser reutilizadas em outros projetos.

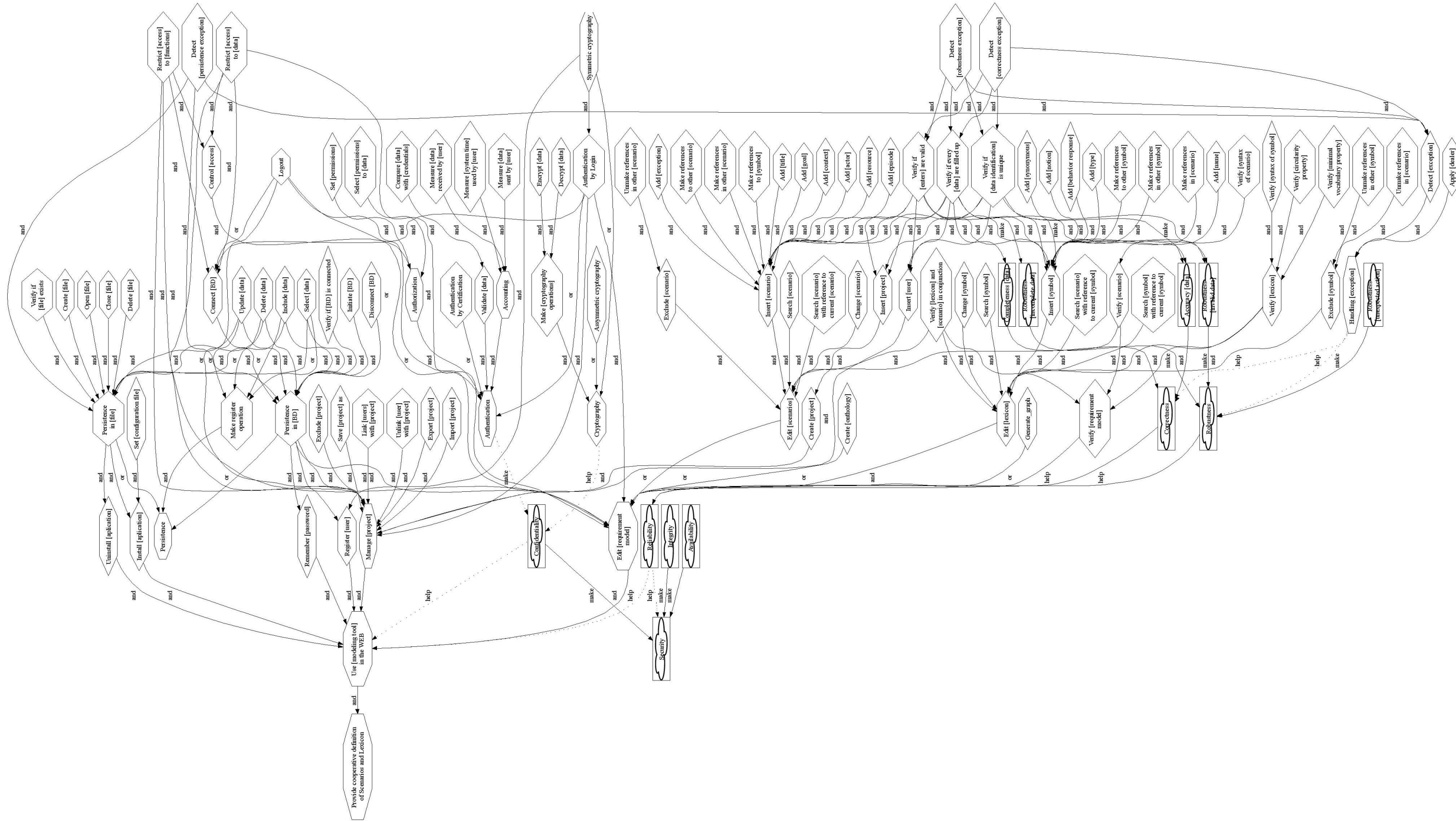


Figura 77. Modelagem de C&L em V-graph

Como estes grupos de requisitos são referentes a características distintas e não precisam coexistir no mesmo sistema, os separamos em quatro modelos. Na Figura 78, ilustramos a modelagem de C&L em AOV-graph. Neste modelo ilustramos cada grupo de requisitos com escopo delimitado e as interações entre eles. Esta nova disposição dos grupos esclarece quais *softmetas*, metas e tarefas dizem respeito a cada característica.

Utilizando o relacionamento transversal (em negrito na Figura 78), explicitamos que algumas características afetam outras de maneira que tornam os elementos mais acoplados e pouco coesos. Centralizamos a informação de como cada elemento se espalha pelos demais, e assim diminuimos a quantidade de relacionamentos no modelo. O relacionamento transversal tem um significado além de simplesmente relacionar diferentes características; com ele podemos acrescentar novos elementos que surgem da junção de modelos, i.e., a composição de características resulta em um sistema que é maior que a soma de suas partes. Utilizando, nosso mecanismo de visualização, a seguir, explicamos cada um dos modelos do sistema C&L e seus relacionamentos transversais.

Modelagem de Requisitos (CEL)

Na Figura 79, ilustramos uma visão parcial do sistema C&L, a característica Modelagem de requisitos (CEL). Neste modelo, a tarefa “Verify [requirements model]” representa uma atividade de inspeção do modelo de cenário, do léxico e de ambos os modelos em conjunto. Como esta tarefa poderia ser modelada independentemente do modelo de edição de léxico e cenários e aparece como sub-tarefa em três momentos diferentes na funcionalidade do sistema (quando editando o léxico, ou editando os cenários ou navegando pelas informações dos modelos), a consideramos uma característica que corta as tarefas “Edit [lexicon]” e “Edit [scenario]”. Isto significa que “Verify [requirements model]” modifica as tarefas de CEL, incluindo ou substituindo outras tarefas.

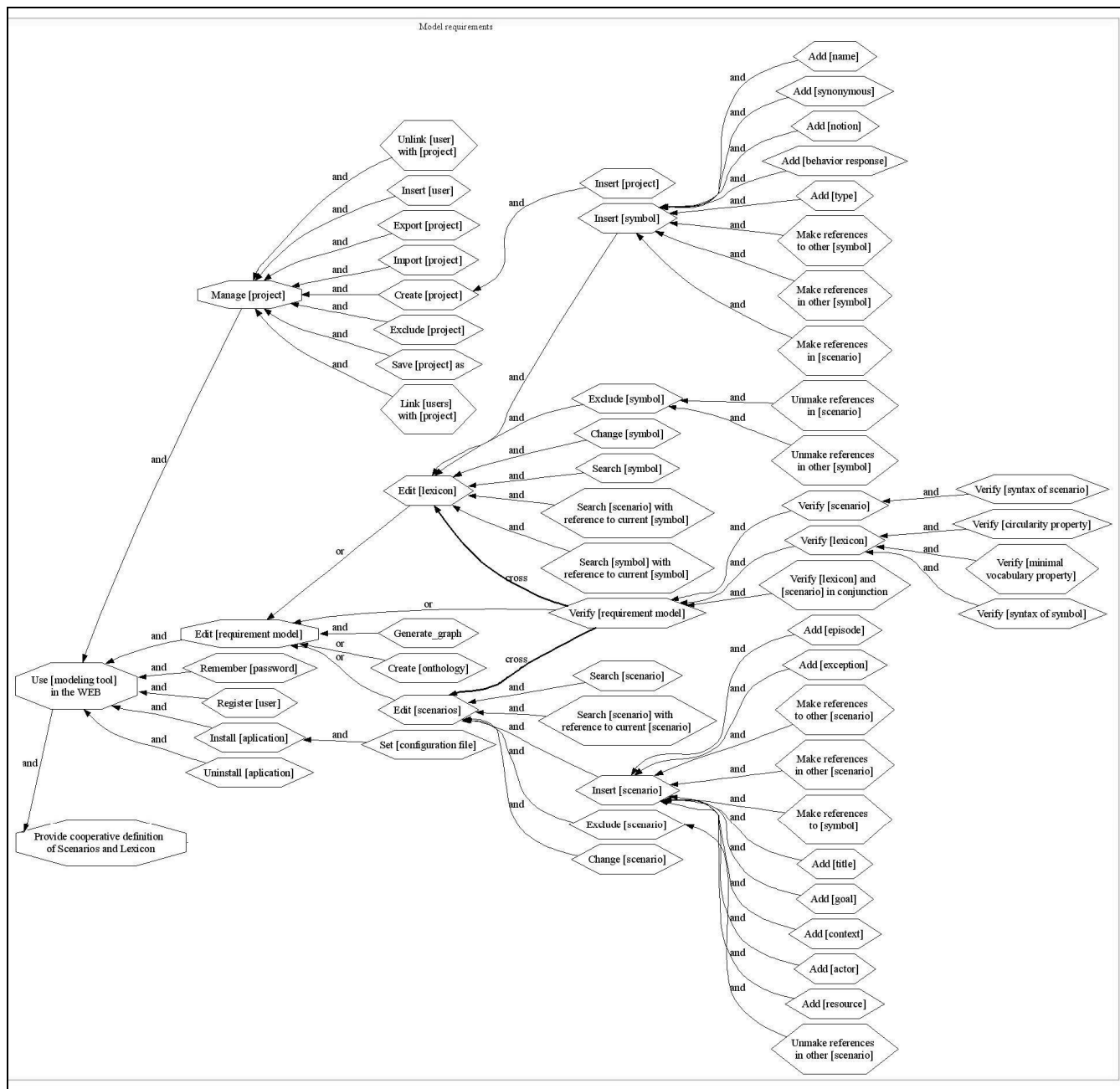


Figura 79. Modelo CEL

1. Crosscutting {
2. source: Verify [requirement model]
3. pointcut PC1: include(Edit [scenarios])
4. pointcut PC2: include(Edit [lexicon])
5. advice around: PC1 {Verify [scenario] }
6. advice around: PC2 {Verify [lexicon] }
7. advice around: PC1 and PC2 {Verify [scenario] and [lexicon] in conjunction }
8. }

Figura 80. Relacionamento transversal do modelo CEL

Na Figura 80, descrevemos o relacionamento transversal existente entre “Verify [requirements model]”, e “Edit [lexicon]” e “Edit [scenario]”. Este relacionamento indica que a atividade “Verify [lexicon]” se entrelaça à tarefa

“Edit [lexicon]”, a tarefa “Verify [scenario]” se entrelaça à tarefa “Edit [scenario]”, e a tarefa “Verify [scenario] and [lexicon] in conjunction” se entrelaça às tarefas “Edit [scenario]” e “Edit [lexicon]” – ou seja, as atividades “Edit [lexicon]” e “Edit [scenario]” incluem as atividades de verificação. Por outro lado, sob o ponto de vista da tarefa “Verify [requirements model]”, este relacionamento transversal indica que suas sub-tarefas estão espalhadas por outras sub-árvores do modelo.

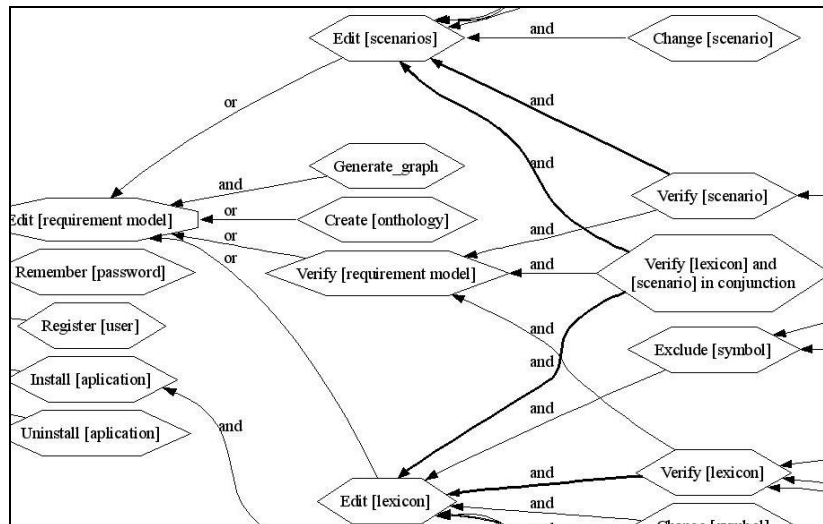


Figura 81. Modelo CEL após a composição

O mecanismo de composição é responsável por propagar estas informações, i.e., por criar os relacionamentos de contribuição entre estas tarefas, veja a Figura 81. Desta forma: 1) podemos trabalhar nestes grupos de requisitos separadamente, sendo necessário definir por meio de um relacionamento transversal como o segundo grupo (Verify [requirements model]) está relacionado ao primeiro (Edit [requirement model]), i.e., neste caso, apenas o segundo modelo afeta o primeiro; 2) quando o foco não é o relacionamento transversal (no modelo antes da composição), ele é representado graficamente por apenas dois elos, aumentando a visibilidade do modelo; 3) quando a composição é realizada o modelo pode ser visualizado com os relacionamentos transversais expandidos, i.e., no lugar de dois elos podemos ver quatro elos.

Segurança

Na Figura 82, ilustramos os requisitos de segurança e suas interações com os modelos de persistência e CEL (elementos em cinza). Neste caso apenas Confidencialidade é decomposto em tarefas. Neste modelo, as metas

“Authentication” e “Cryptography” são inícios de relacionamentos transversais. Eles afetam o elemento “Authentication by login” do próprio modelo de segurança, e afetam também os elementos “Connect [BD]”, “Manage [project]” e Edit [requirement model] dos modelos de persistência e CEL.

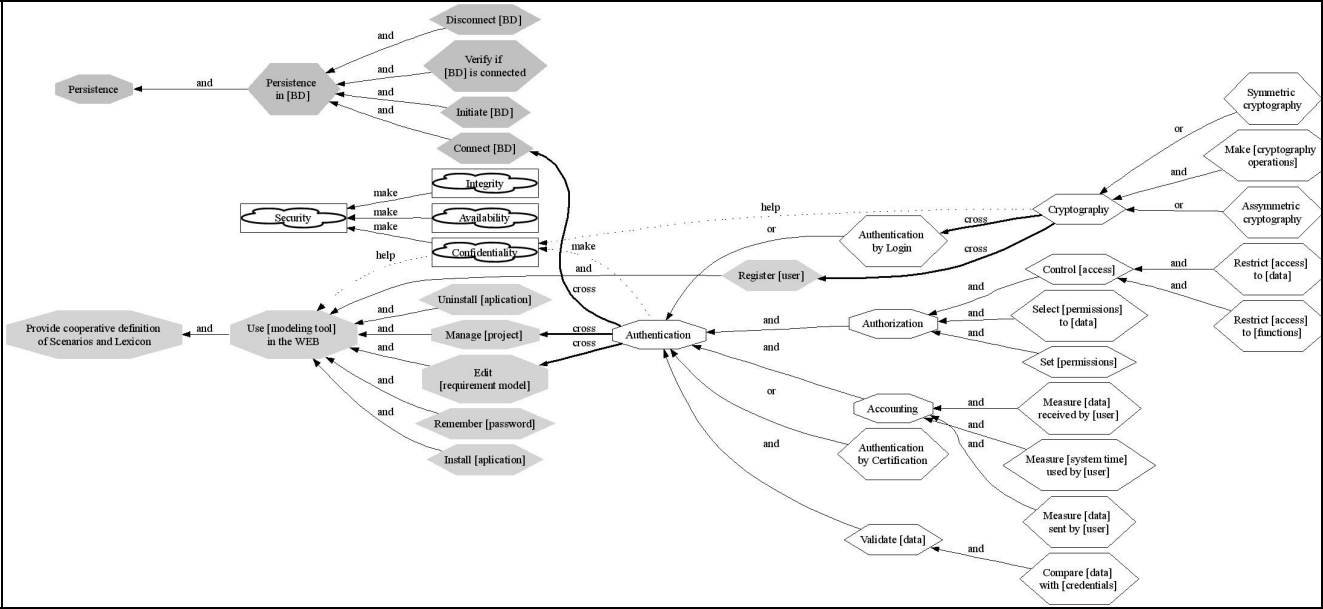


Figura 82. Modelo de segurança

Na Figura 83, apresentamos a definição dos relacionamentos transversais que têm origem no modelo de segurança. O primeiro deles indica que “Authentication” inclui as tarefas “Authentication by login”, “Restrict [access] to [data]” e “Restrict [access] to [functions]” nos elementos “Edit [requirement model]” e “Manage [project]”. Além disto, *intertype declarations* são definidas para incluir nestes mesmos elementos um atributo denominado “constraint” e valor “confidential data”, e uma tarefa denominada “Logout”.

1	1. Crosscutting { 2. source: Authentication 3. pointcut PC3: include(Edit [requirement model]) and include(Manage [project]) 4. and include(Connect [BD]) 5. advice around: PC3 { Authentication by login ; Restrict [access] to [data]; 6. Restrict [access] to [functions] } 7. intertype declaration element: PC3 { Logout } 8. intertype declaration attribute: PC3 { constraint = confidential data } 9. }
2	1. Crosscutting { 2. source: Cryptography 3. pointcut PC4: include(Authentication by login) and include(Register [user]) 4. advice around: PC4 { Symmetric cryptography } 5. }

Figura 83. Relacionamentos transversais do modelo de segurança

O segundo relacionamento transversal, descrito na Figura 83, indica que a tarefa “Crypthography” afeta as tarefas “Register [user]” e “Authentication by login” adicionando a tarefa “Symmetric crypthography”.

Na Figura 84, apresentamos estes relacionamentos transversais após a composição. Nela podemos observar a criação da nova tarefa “Logout” e os novos relacionamentos. Esta visão está focada no serviço segurança, os elementos em cinza são elementos dos modelos CEL e de persistência.

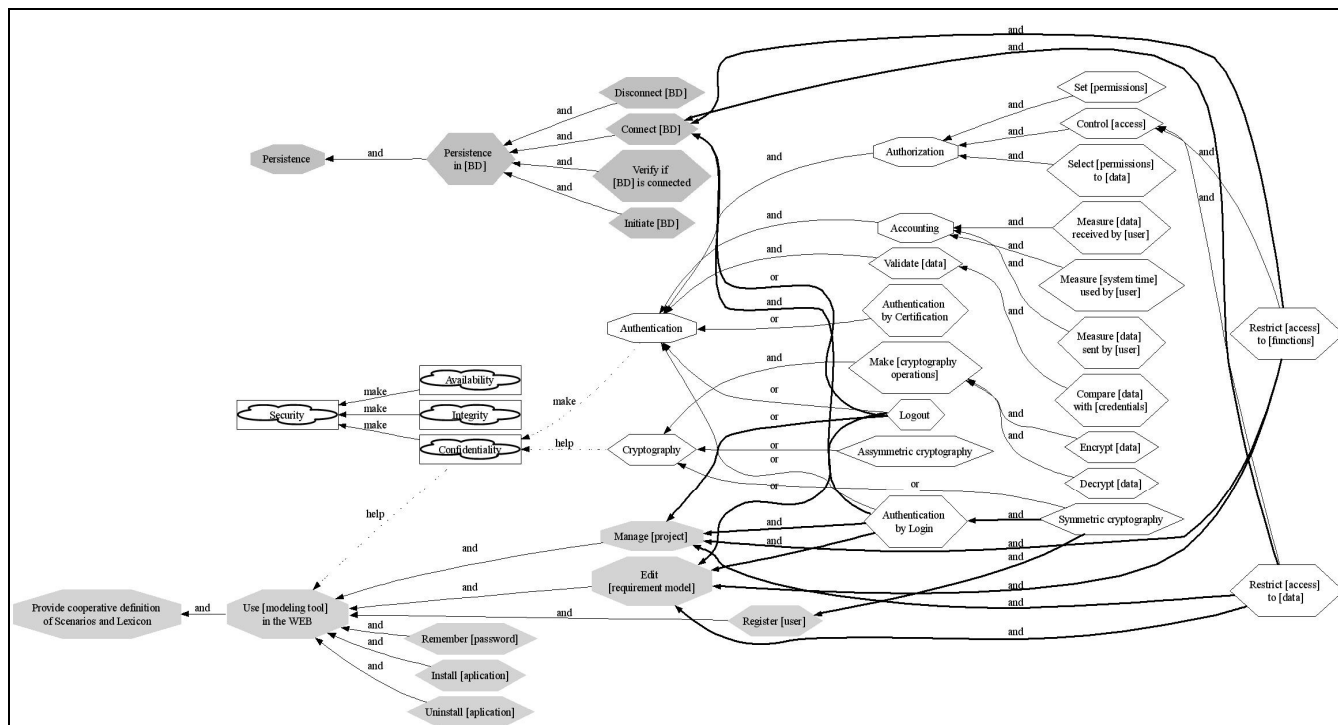


Figura 84. Modelo de segurança após a composição

Persistência

A característica Persistência é fundamental no sistema C&L, estando presente em muitas das tarefas de edição. Na Figura 85, apresentamos o modelo de persistência e os relacionamentos transversais dele para o modelo CEL.

Na Figura 86: o primeiro relacionamento indica que a tarefa “Make register operation” inclui as tarefas “Include [data]”, “Select [data]”, “Delete [data]” e “Update [data]” depois das tarefas “Connect [BD]” e “Open [file]”, i.e., estas tarefas são necessárias para realizar persistência em arquivo ou em banco de dados, mas é necessário que o arquivo tenha sido aberto e/ou o banco de dados esteja conectado; o segundo relacionamento transversal indica que as metas “Manage [project]” e “Edit [requirements model]” são afetadas pela meta

“Persistence in [BD]” e as tarefas “Install [aplication]” e “Uninstall [application]” fazem persistência em arquivo (“Persistence in [file]”). O resultado da composição é ilustrado na Figura 87.

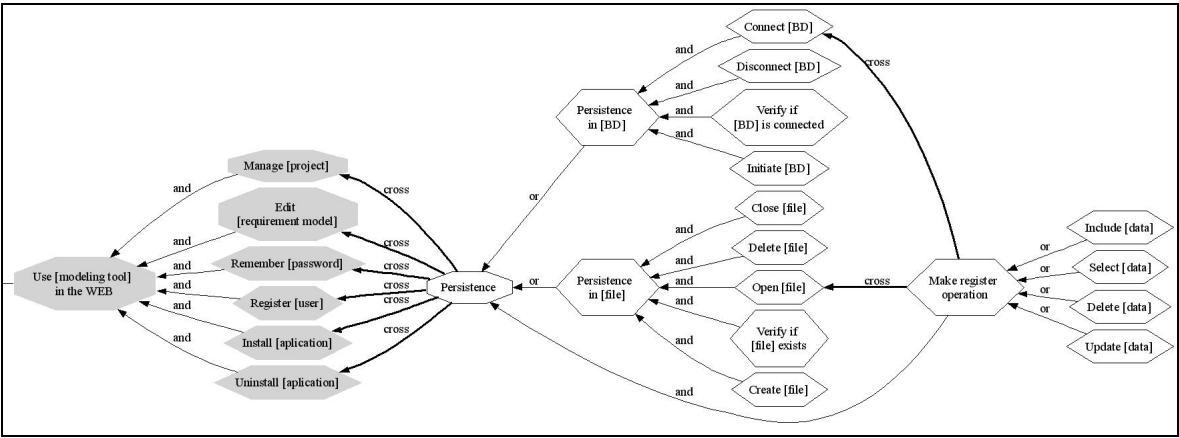


Figura 85. Modelo de persistência

1	1. Crosscutting { 2. source: Make register operation 3. pointcut PC5: include(Connect [BD]) and include(Open [file]) 4. advice after: PC5 {Include [data]; Select [data]; Delete [data]; Update [data] } 5. }
2	1. Crosscutting { 2. source: Persistence 3. pointcut PC6: include(Manage [project]) and include(Edit [requirement model]) 4. pointcut PC7: include(Install [application]) and include(Uninstall [application]) 5. advice around: PC6 {Persistence in [BD] } 6. advice around: PC7 {Persistence in [file] } 7. }

Figura 86. Relacionamentos transversais do modelo de persistência

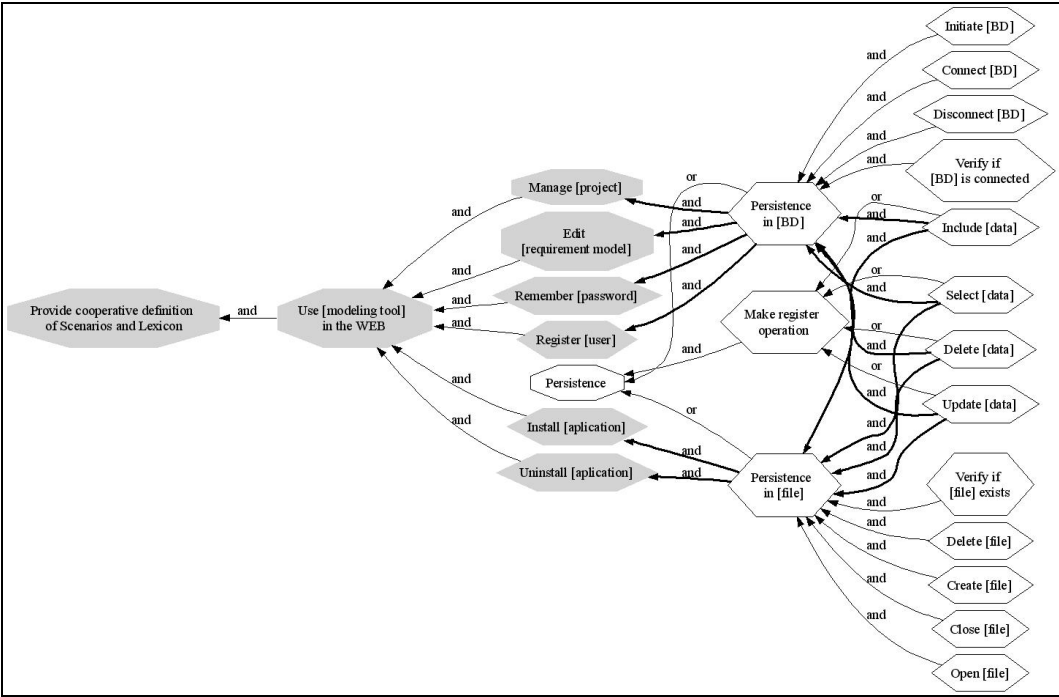


Figura 87. Modelo de persistência após a composição

Confiabilidade

No sistema C&L, apesar de não haver uma forte preocupação com confiabilidade, há algumas tarefas simples para garantir que os dados fornecidos são corretos e que o sistema não pára com o fornecimento de dados errados. Isto é realizado de maneira preventiva, realizando as seguintes verificações durante a entrada de dados: “Verify if [enters] are valid”, “Verify if every [data] are filled up” e “Verify if [data identification] is unique”, veja Figura 88.

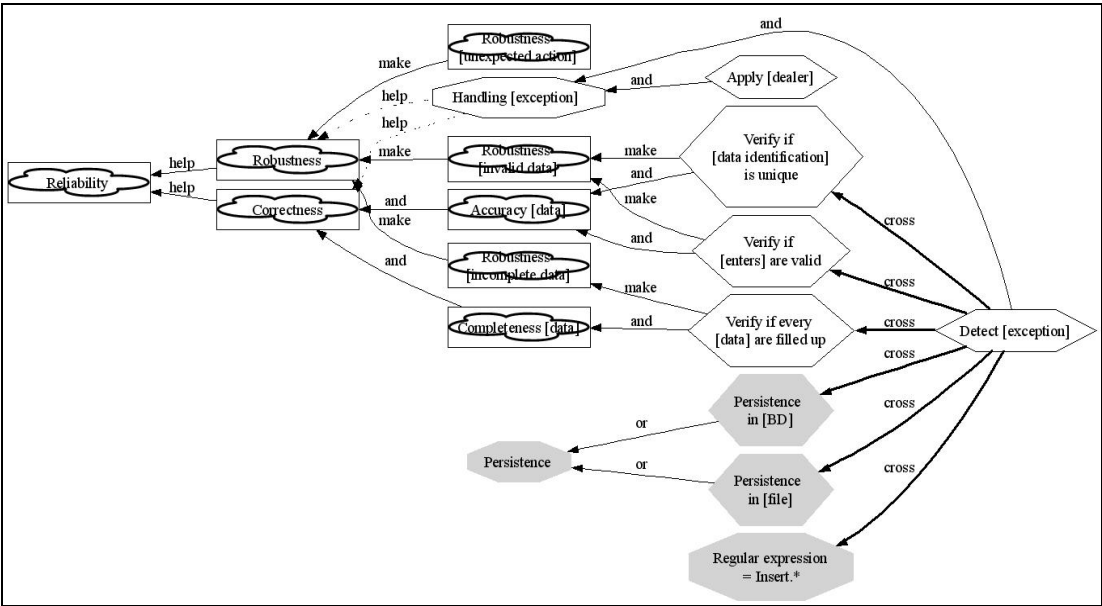


Figura 88. Modelo de confiabilidade

1	1. Crosscutting {
	2. source: Detect [exception]
	3. pointcut PC8: include(Verify if [data identification] is unique) and
	4. include(Verify if [enters] are valid)
	5. pointcut PC9: include(Verify if every [data] are filled up)
	6. pointcut PC10: include(Persistence in [BD]) and include(Persistence in [file])
	7. pointcut PC11: include(Insert.*={ (Insert [project]) (Insert [user])
	8. (Insert [scenario]) (Insert [symbol]) })
	9. advice around: PC11 { Verify if [data identification] is unique ;
	10. Verify if [enters] are valid ; Verify if every [data] are filled up }
	11. intertype declaration element: PC10 { Detect [persistence exception] }
	12. intertype declaration attribute: PC8 { exception = Invalid data }
	13. intertype declaration attribute: PC9 { exception = Incomplete data }
	14. intertype declaration element: PC8 and PC9 { Detect [robustness exception] ;
	15. Detect [correctness exception] }
	16. }

Figura 89. Relacionamentos transversais do modelo de confiabilidade

Na Figura 89, descrevemos o relacionamento transversal existente no modelo de confiabilidade. Este relacionamento transversal adiciona três novas tarefas à “Detect [exception]”, denominadas “Detect [persistence exception]”, “Detect [robustness exception]” e “Detect [correctness exception]”. A primeira

destas novas tarefas é adicionada também em “Persistence in [BD]” e “Persistence in [file]” do modelo de persistência, e as duas últimas tarefas são adicionadas nas tarefas “Verify if [data identification] is unique”, “Verify if [enters] are valid” e “Verify if every [data] are filled up” do sub-modelo de robustez. As tarefas “Verify if [data identification] is unique”, “Verify if [enters] are valid” e “Verify if every [data] are filled up” são incluídas em todas as tarefas cujo nome inicia com “Insert”. Além disto, um atributo denominado “exception” é criado nos elementos “Verify if [data identification] is unique” e “Verify if [enters] are valid” com valor “invalid data”, e em “Verify if every [data] are filled up” com valor “incomplete data”.

Na Figura 90, ilustramos o modelo de confiabilidade após a composição. Os elementos em cinza são os elementos afetados do modelo CEL e de persistência. Podemos observar três novos elementos pertencentes à junção destes modelos: “Detect [persistence exception]”, “Detect [correctness exception]” e “Detect [robustness exception]”.

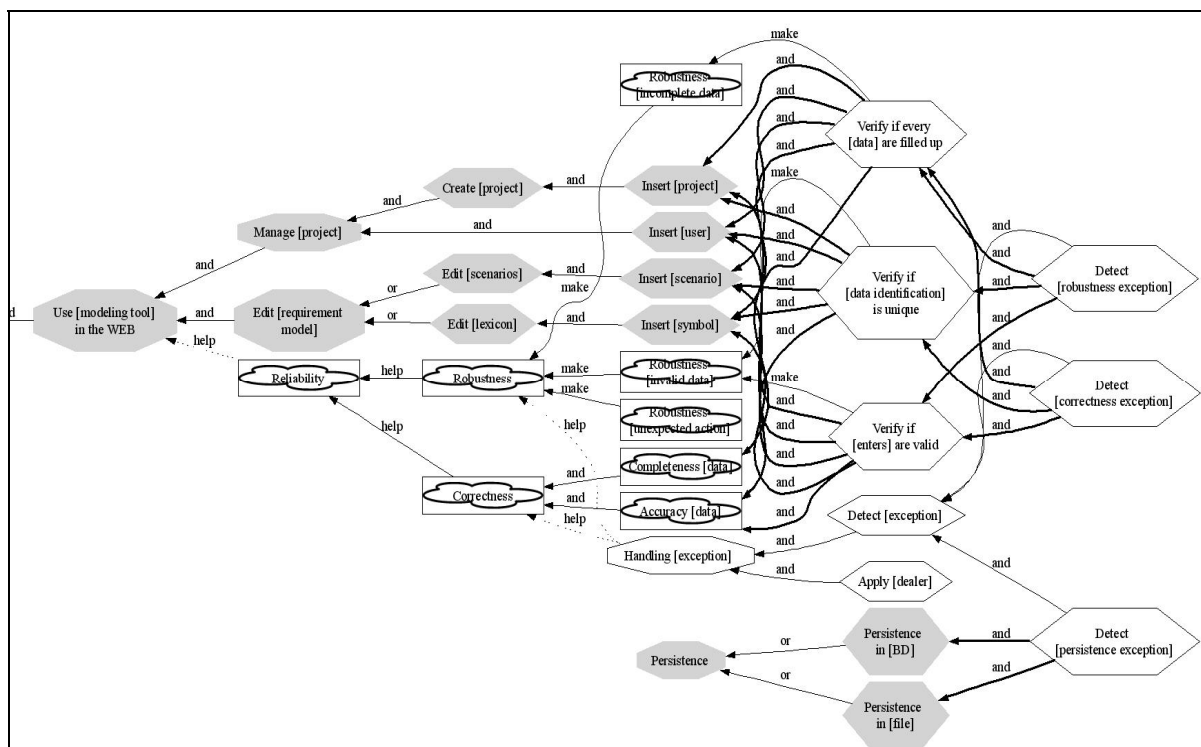
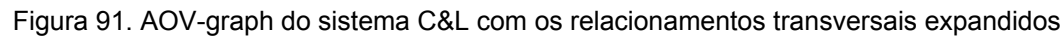


Figura 90. Modelo de confiabilidade após a composição

O mecanismo de visualização é responsável por gerar cada uma das visões anteriores, separando os grupos de requisitos ou juntando somente aqueles que afetam o grupo em foco, ou mesmo mostrando todos os grupos juntos e suas

interações. Na Figura 91, apresentamos a modelagem em AOV-graph do sistema C&L com seus relacionamentos transversais expandidos.



O V-graph é uma visão intencional do sistema, i.e., os modelos em V-graph focam a interação entre seus elementos para satisfação de metas, *softmetas* e tarefas, esta é sua decomposição dominante. Entretanto, com a estruturação de cada meta, *softmeta* e tarefa em tipos e tópicos, podemos obter também uma visão de dados (como apresentamos no Capítulo 4, Seção 4.3.3).

Na Figura 92, ilustramos as informações modeladas em AOV-graph na forma de diagrama de classes. Neste diagrama, focamos apenas os objetos relacionados ao modelo CEL. Podemos observar que a maior quantidade de funções está relacionada aos objetos “project”, “scenarios” e “symbol”. Semanticamente os objetos “modeling tool” e “aplication” dizem respeito ao mesmo elemento, mas foram modelados como objetos distintos porque esta geração se baseia em uma análise léxica dos tópicos.

Os relacionamentos são derivados com base nos relacionamentos existentes entre metas, tarefas e *softmetas* cujos tópicos geram objetos, entretanto não conseguimos identificar o tipo de relacionamento nem a cardinalidade deles. Melhorias nas regras de transformação podem ser feitas para esta identificação. Apesar de não podermos garantir que este modelo é completo e correto, ele representa uma boa visão de quais os principais objetos da modelagem, quais as ações onde eles estão envolvidos e qual a interação entre eles.

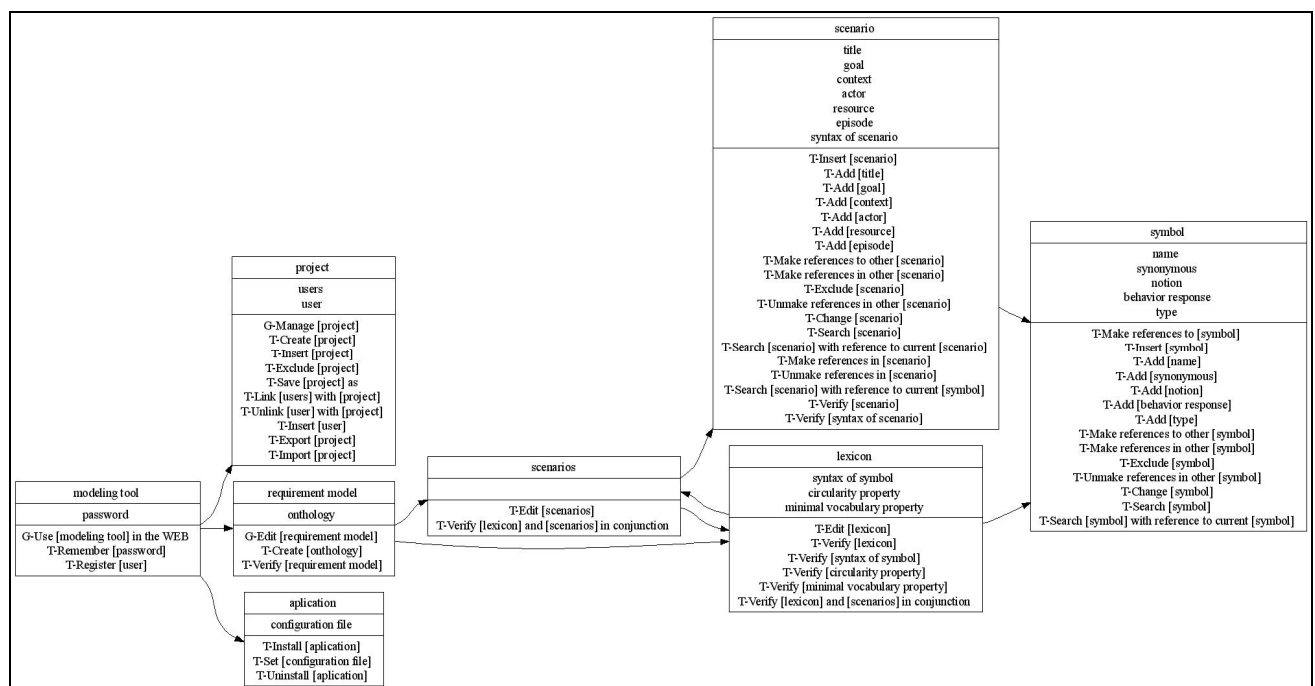


Figura 92. Diagrama de classes do modelo CEL

5.2.2. Análise dos Resultados

A modelagem de C&L inclui 12 *softmetas*, 15 metas e 110 tarefas, veja na Tabela 7. Separamos estes elementos em quatro grandes grupos de requisitos, sendo referentes ao CEL, à segurança, à persistência e à confiabilidade. Dentro de cada um deles ainda separamos em grupos menores tais como “Authentication”, “Verify [requirements model]”, “Handling [exception]”, dentre outros. Esta separação foi realizada por avaliar se cada grupo ou sub-grupo pode existir sem os demais, pode ser reutilizado ou se ele encontra-se repetido em diferentes pontos da modelagem.

Tabela 7. Estatísticas do estudo de caso – C&L

Elementos	Quantidade antes da composição	Quantidade após a composição
Metas	9	9
Tarefas	101	105
<i>Softmetas</i>	12	12
Relacionamentos transversais	6	6
Elos transversais	20+4(expressão regular)=24	56
Correlações	7	7
Contribuições	117	117+56=173
Tipos	52	55
Tópicos	44	47
Atributos	0	3

Ao realizar esta separação tornamos melhor a visibilidade de cada modelo e facilitamos a manipulação deles, possibilitando também que cada grupo de requisitos possa ser modelado por equipes diferentes ou em momentos diferentes. Os modelos de persistência, segurança e confiabilidade podem ser reutilizados em outros sistemas e podem evoluir independentemente um do outro, mesmo depois que os relacionamentos transversais tenham sido definidos.

A definição dos relacionamentos transversais é realizada quando já se tem em vista o que de cada característica é requerido no sistema. Quando os modelos evoluem ou são reutilizados, os relacionamentos transversais contidos neles devem ser redefinidos para o contexto em questão, mas muitas vezes é possível redefinir apenas os *pointcuts*, sendo as *intertype declarations* e *advice* não modificados. Isto significa que é possível que certa característica sempre interaja com outras da mesma maneira, mudando apenas o local da interação, por exemplo, para reutilizar o modelo de persistência é necessário redefinir apenas seu segundo *pointcut*, veja na Seção 5.2.

Neste estudo de caso, definimos 6 relacionamentos transversais, representados no V-graph por 24 elos. Ao processar estes relacionamentos transversais o mecanismo de composição gerou 56 elos. Houve, assim, um crescimento de mais de 130% no número destes elos.

Além de diminuir o número de relacionamentos que o engenheiro de requisitos teria que criar manualmente, a semântica dos relacionamentos transversais torna explícitas quais características têm tendência a se espalhar ou entrelaçar às demais, i.e., o relacionamento transversal não é uma simples interação entre metas, *softmetas* e tarefas. Cada relacionamento transversal representa uma matriz de interações, demonstrando parte da rastreabilidade entre os elementos. Esta parcela de rastreabilidade pode representar elementos críticos no sistema, pois inclui aqueles que afetam ou são afetados por muitas características, ou aqueles que possuem alto acoplamento.

Tabela 8. Matriz de rastreabilidade - relacionamento transversal do modelo de confiabilidade

	Source	Advice			Intertype declaration		
	Detect [exception]	Verify if [data identification] is unique	Verify if [enters] are valid	Verify if every [data] are filled up	Detect [robustness exception]	Detect [correctness exception]	Detect [persistence exception]
Detect [exception]					And	And	And
Verify if [data identification] is unique	Cross (exception = Invalid data)				And	And	
Verify if [enters] are valid	Cross (exception = Invalid data)				And	And	
Verify if every [data] are filled up	Cross (exception = Incomplete data)				And	And	
Persistence in [BD]	Cross						And
Persistence in [file]	Cross						And
Insert [project]	Cross	And	And	And			
Insert [user]	Cross	And	And	And			
Insert [scenario]	Cross	And	And	And			
Insert [symbol]	Cross	And	And	And			

Na Tabela 8, ilustramos um exemplo de matriz de rastreabilidade com as informações do relacionamento transversal do modelo de confiabilidade. Nesta matriz, podemos observar qual a interação dos elementos da coluna para os elementos das linhas. O rótulo “cross” indica que “Detect [exception]” afeta todos os *pointcuts* indicados nas linhas, e nos três primeiros adiciona um atributo denominado “exception”. Os rótulos “and” indicam relacionamentos de contribuição, ou seja, os elementos das colunas são necessários para satisfazer os elementos das linhas. A relação coluna→linha rotuladas com “cross” representam os relacionamentos transversais resumidos, enquanto o rótulo “and” representa os

relacionamentos transversais expandidos. Por exemplo: de maneira resumida, “Detect [exception]” afeta (cross) “Insert [symbol]”; de maneira expandida, “Verify if [data identification] is unique”, “Verify if [enters] are valid” e “Verify if every [data] are filled up” contribuem para “Insert [symbol]”.

5.3.

Sistema para Divulgação de Eventos – Unical

Unical é um sistema para divulgação de eventos sociais e acadêmicos na Universidade da Califórnia (UCI). Devido haver uma grande quantidade de eventos acontecendo na universidade e muitas vezes as pessoas não ficarem sabendo de um evento de interesse a tempo, foi desenvolvido o sistema Unical. Sua meta é fazer com que todos o adotem para divulgar e compartilhar eventos, registrando-os em suas agendas de maneira que sejam lembrados sobre eles no momento escolhido. O documento de requisitos deste sistema encontra-se disponível em (Unical, 2005), e é utilizado em estudos de caso desta mesma universidade. O Anexo A contém uma cópia deste documento de requisitos e foi utilizado como fonte de informação para modelagem do Unical nesta tese.

5.3.1.

Modelagem

No documento de requisitos do Unical contabilizamos 140 sentenças classificadas como requisitos funcionais e menção a 16 requisitos classificados como não funcionais. Estes últimos são apenas citados com definições abstratas do que se espera sobre eles. Porém, a maioria dos requisitos funcionais traz em sua descrição, restrições referentes a muitos destes RNFs, indicando o entrelaçamento e espalhamento destas características.

Para modelar o Unical separamos cada requisito não funcional daqueles específicos da divulgação de eventos. Na Figura 93, mostramos uma visão geral dos RNFs envolvidos na modelagem e suas correlações com o modelo do Unical. Nesta visão podemos observar contribuições e conflitos existentes entre RNFs. Por exemplo, “Profitable” e “Timeliness” são essenciais para atingir a meta principal do Unical, mas restringem todos os outros RNFs. Esta visão é essencial para a modelagem e tomada de decisões porque explicita a necessidade de avaliar

o quanto de cada requisito é suficiente para atingir a meta principal do sistema, sem o risco de torná-lo inviável.

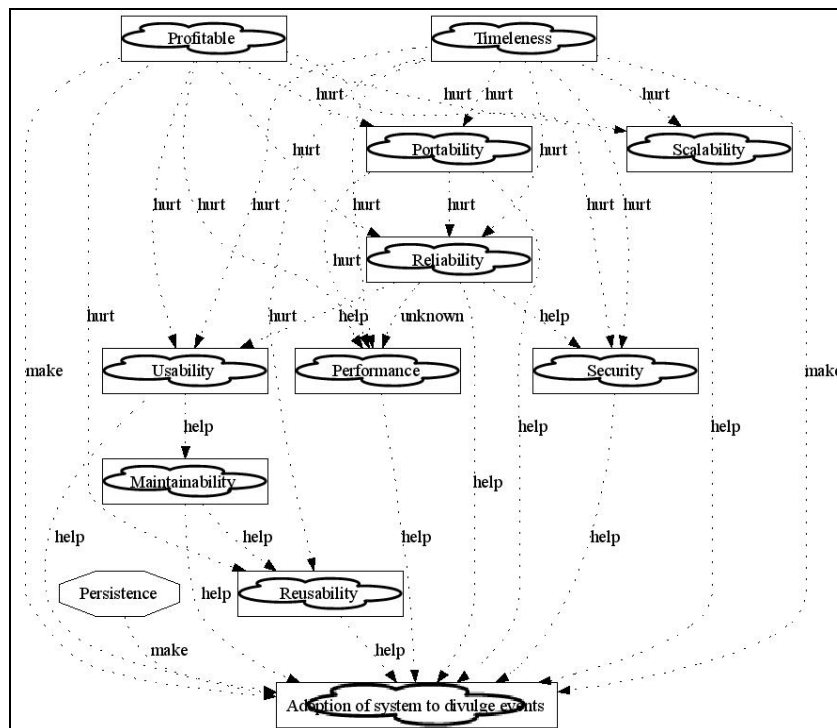
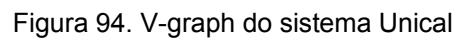


Figura 93. Visão geral do Unical e seus RNFs

Utilizando a abordagem tradicional (V-graph) estes RNFs e suas operacionalizações compõem um único grafo (veja a Figura 94). Assim como no estudo de caso anterior, este modelo apresenta uma grande quantidade de elementos e relacionamentos, tornando sua modelagem, análise, modificação e reutilização difíceis.

Em nossa modelagem utilizando AOV-graph, cada *softmeta* e meta na Figura 93 representa um modelo separado. Além disto, utilizamos relacionamentos transversais para representar a interação entre eles quando há entrelaçamento e espalhamento de seus elementos (veja a Figura 95).





Na Figura 95, ilustramos a modelagem do sistema Unical em AOV-graph. Como mencionamos anteriormente, a modelagem consiste dos modelos Unical, segurança, confiabilidade, persistência, usabilidade, performance, portabilidade, escalabilidade, manutenibilidade, reusabilidade e os modelos referentes ao tempo (timeliness) e custo (profitable) de desenvolvimento.

Os modelos referentes à portabilidade e escalabilidade não apresentam suas operacionalizações porque o documento de requisitos não descreve nenhum detalhe sobre como realizar estes RNFs. Os modelos referentes à manutenibilidade, à reusabilidade, ao tempo e custo de desenvolvimento não representam funcionalidades do sistema Unical. Eles representam características de projeto, que também são transversais e representam uma maneira de decomposição diferente dos outros RNFs. Quanto aos modelos relacionados à segurança, confiabilidade e persistência, nós reutilizamos do estudo de caso anterior (C&L) e realizamos algumas modificações para atender novos requisitos do Unical. A seguir, detalhamos os modelos referentes ao Unical, usabilidade e performance, e como realizamos a reutilização dos modelos de segurança, confiabilidade e persistência.

Divulgação de Eventos – Unical

Na Figura 96, ilustramos o AOV-graph referente aos requisitos específicos de divulgação de eventos. Neste modelo não definimos nenhum relacionamento transversal porque consideramos que os elementos que se espalham (“Stop sound” e “Set [reminder]”) não representam maior complexidade para o modelo.

Na Figura 97, ilustramos o MER gerado com base nesta parte do modelo AOV-graph do sistema Unical. Podemos observar as principais entidades (retângulos) e seus atributos (elipses), bem como os relacionamentos entre entidades (elos direcionados), gerados a partir dos relacionamentos de contribuição de AOV-graph.

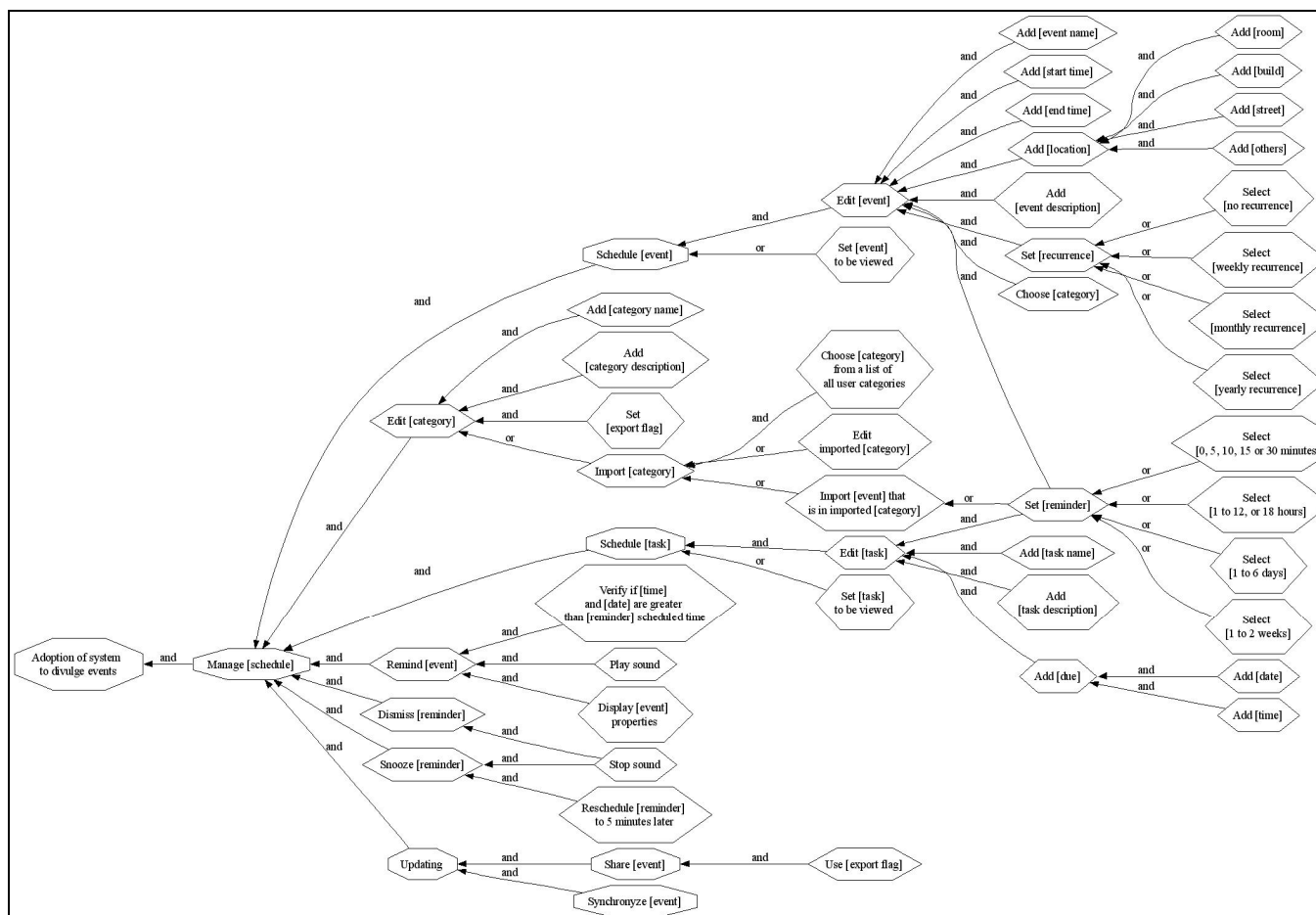


Figura 96. Modelo Unical – divulgação de eventos

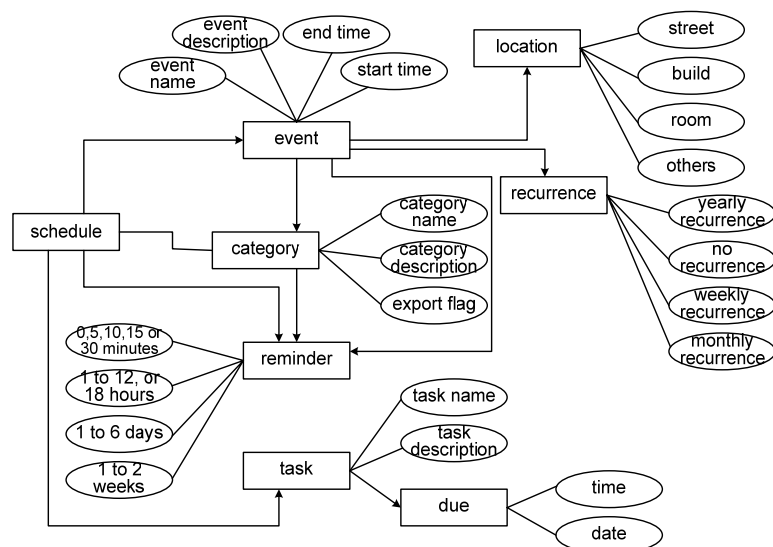


Figura 97. MER para o modelo Unical

A seguir apresentamos como os modelos de performance, usabilidade e segurança afetam o modelo Unical.

Performance

O documento de requisitos do Unical apresenta explicitamente a preocupação com a característica performance no sentido tempo de resposta. Esta característica é descrita somente pela sentença a seguir:

“Performance is crucial to UniCal – if the system is too slow, customers may revert to sending e-mails and using word of mouth. Synchronizing events with UniCal should take no more than one second for every fifty events. All other operations must be performed nearly instantaneously.”

Entretanto, esta sentença restringe cada um dos requisitos funcionais, i.e., todas as tarefas de Unical estão relacionadas à restrição de performance. Mesmo não havendo tarefas que indiquem como este tempo de resposta é atingido, decidimos modelá-lo como um novo tipo de atributo denominado “performance” que tem os valores “instantaneously” e “one second for 50 events”. Na Figura 98a, ilustramos o modelo de performance e na Figura 98b apresentamos a descrição de seu relacionamento transversal.

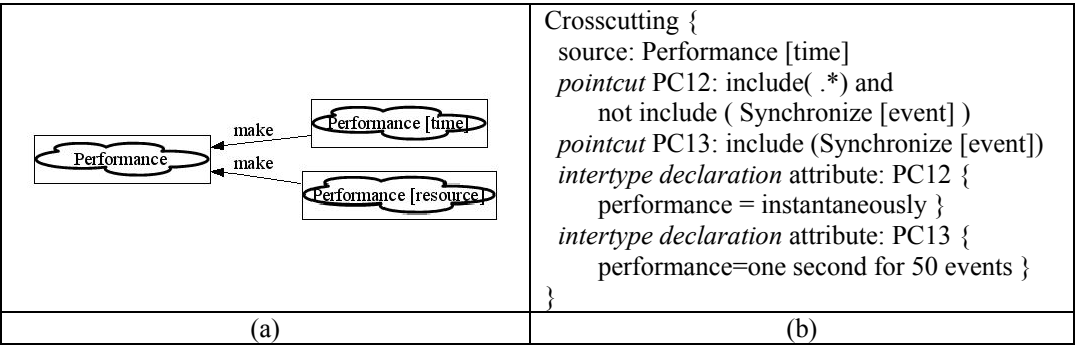


Figura 98. a) Modelo de performance e b) relacionamento transversal do modelo de performance

O atributo adicionado por este relacionamento transversal não é ilustrado no AOV-graph, mas pode ser analisado quando outras visões deste modelo são geradas. Por exemplo, na Figura 99, ilustramos cenários do Unical em que esta restrição de performance é representada.

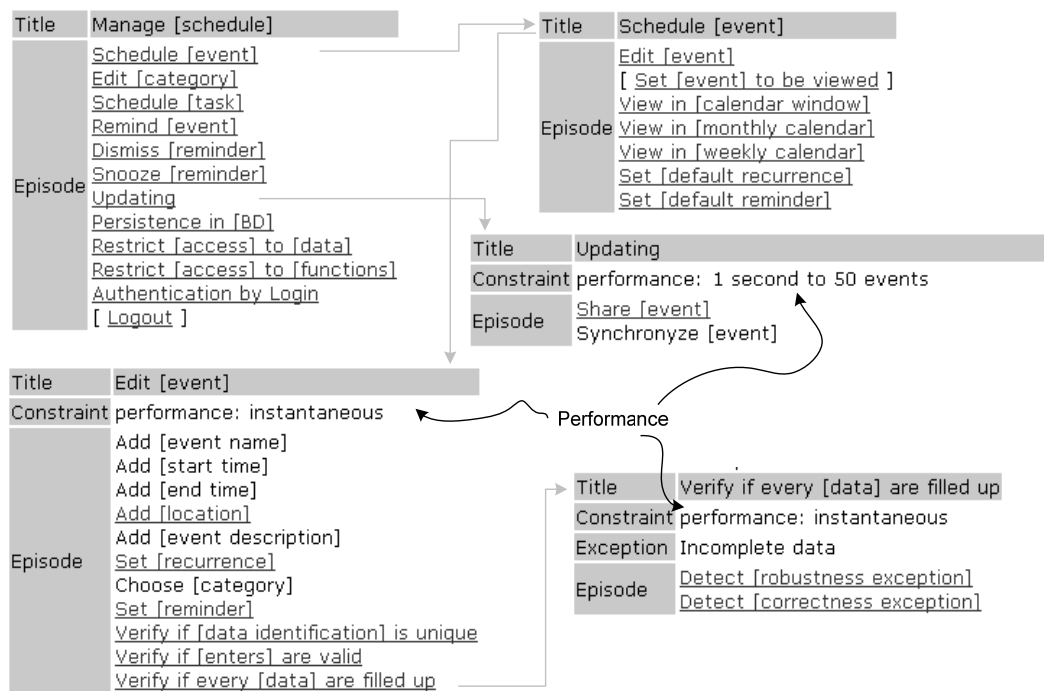


Figura 99. Visão de cenários do modelo Unical

Usabilidade

Na descrição de RFs do documento de requisitos do Unical são mencionados vários detalhes da interface com o usuário. Com base nestes detalhes, modelamos algumas características genéricas relacionadas à usabilidade. Na Figura 100, apresentamos o modelo de usabilidade utilizado neste estudo de caso, e na Figura 101 apresentamos sua interação com o modelo Unical (os elementos em cinza pertencem ao modelo Unical).

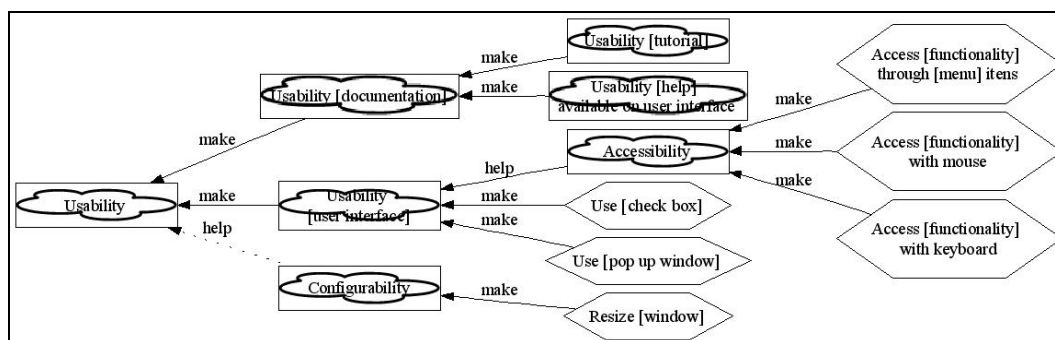


Figura 100. Modelo de usabilidade

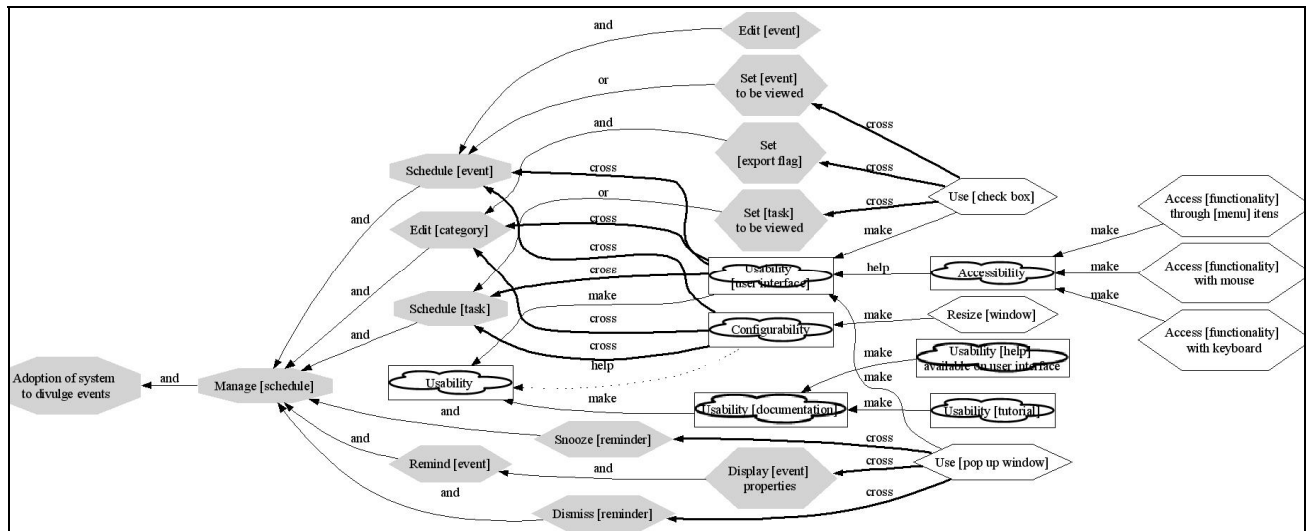


Figura 101. Interação do modelo de usabilidade com o modelo Unical

Novas tarefas e metas surgem com a integração destes dois modelos. Estes novos elementos nem fazem parte do modelo Unical, nem do modelo de usabilidade isoladamente. Na Figura 102, apresentamos a descrição dos relacionamentos transversais existentes entre o modelo de usabilidade e o Unical.

Com o primeiro relacionamento transversal é incluída a nova tarefa “Use [reminder pop up window]” em “Use [pop up window]”, “Display [event] properties”, “Snooze [reminder]” e “Dismiss [reminder]”. Da mesma maneira, no segundo e terceiro relacionamento transversal, as tarefas indicadas nos *intertype declarations* são incluídas no ponto de origem do relacionamento (“Use [checkbox]” e Usability [user interface]) e nos seus respectivos *pointcuts*. No quarto relacionamento transversal, temos um exemplo de *pointcut* definido por expressão regular. Neste caso todas as tarefas que têm a palavra “window” é um *pointcut*, porém o operador “not” é utilizado para excluir a tarefa “Resize [window]” deste *pointcut*, i.e., o *advice* que atinge o *pointcut* PC11 inclui a tarefa “Resize [window]” em todas as tarefas que contenham a palavra “window” com exceção dela mesma.

1	1. Crosscutting { 2. source: Use [pop up window] 3. pointcut PC1: include(Display [event] properties) and 4. include(Snooze [reminder]) and include(Dismiss [reminder]) 5. intertype declaration element: PC1 { Use [reminder pop up window] } 6. }
2	1. Crosscutting { 2. source: Use [check box] 3. pointcut PC2: include(Set [event] to be viewed) 4. pointcut PC3: include(Set [task] to be viewed) 5. pointcut PC4: include(Set [export flag]) 6. intertype declaration element: PC3 { Mark the [category name] to be exported ; 7. Unmark the [category name] to be not exported } 8. intertype declaration element: PC2 { Mark the [category name] are visible in the 9. [calendar view window] ; Unmark the [category name] are invisible in the 10. [calendar view window] } 11. intertype declaration element: PC4 { Mark the [task] as completed in the [task 12. list window]; Unmark the [task] as not completed in the [task list window] } 13. }
3	1. Crosscutting { 2. source: Usability [user interface] 3. pointcut PC5: include(Schedule [event]) 4. pointcut PC6: include(Edit [category]) 5. pointcut PC7: include(Schedule [task]) 6. intertype declaration element: PC5 and PC7 { View in [calendar window] { 7. View in [monthly calendar] ; View in [weekly calendar] } } 8. intertype declaration element: PC6 and PC7 { View in [list window] } 9. }
4	1. Crosscutting { 2. source: Configurability 3. pointcut PC8: include(Schedule [event]) 4. pointcut PC9: include(Edit [category]) 5. pointcut PC10: include(Schedule [task]) 6. pointcut PC11: include(.*window={ (Use [pop up window]) 7. (Use [reminder pop up window]) (Mark the [category name] are visible in the 8. [calendar view window]) (Unmark the [category name] are invisible in the 9. [calendar view window]) (Mark the [task] as completed in the [task list 10. window)) (Unmark the [task] as not completed in the [task list window]) 11. (View in [calendar window]) (View in [list window]) (Resize [window]) }) 12. and not include (Resize [window]) 13. advice around: PC11 { Resize [window] } 14. intertype declaration element: PC9 { Set [color] ; Set [default sorting] } 15. intertype declaration element: PC8 and PC10 { Set [default recurrence] ; 16. Set [default reminder] } }

Figura 102. Relacionamentos transversais do modelo de Usabilidade

Na Figura 103, apresentamos o resultado da composição, com foco no modelo Unical. Os elementos em cinza são aqueles que surgem com o interesse em trazer usabilidade para o sistema de divulgação de eventos. Por outro lado, na Figura 104, ilustramos as mudanças no modelo de usabilidade, decorrentes de sua instanciação para o contexto do Unical.

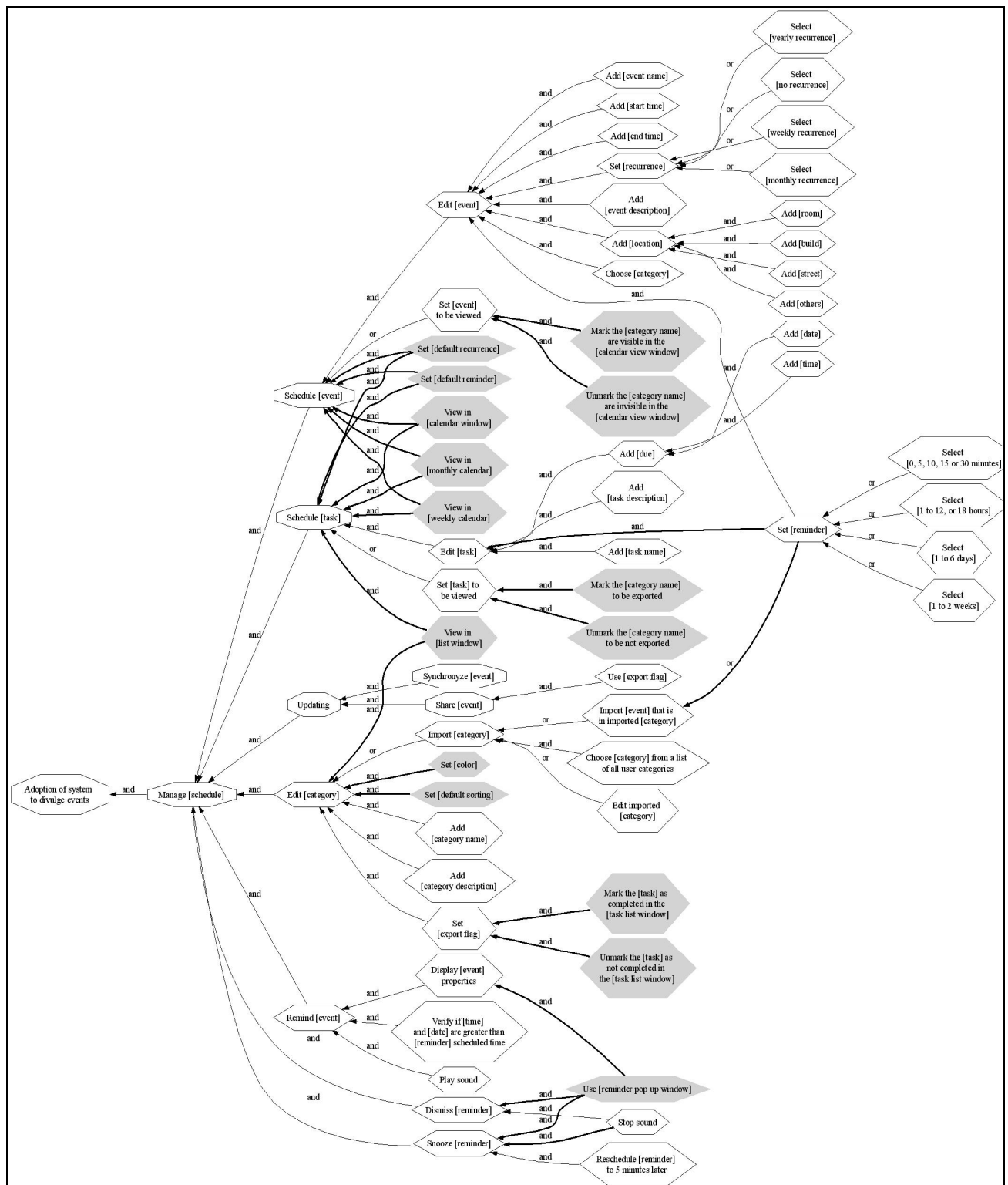


Figura 103. Modelo Unical após a composição

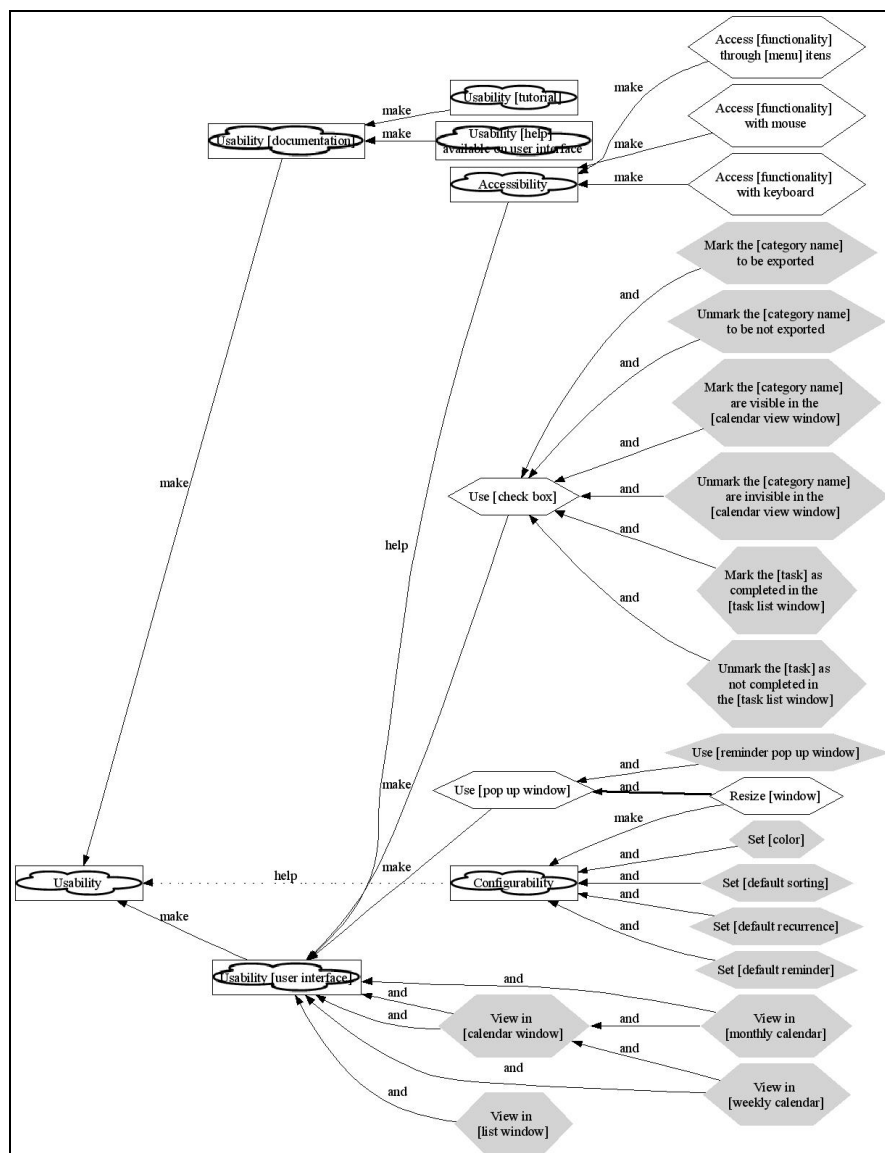


Figura 104. Modelo de usabilidade após a composição

Segurança, Persistência e Confiabilidade

Na Figura 105, ilustramos o modelo de segurança para o sistema Unical. Em cinza realçamos as modificações realizadas no modelo de segurança reutilizado do estudo de caso anterior. Neste modelo incluímos tarefas referentes à gerência das contas dos usuários “Manage [account]” (em cinza).

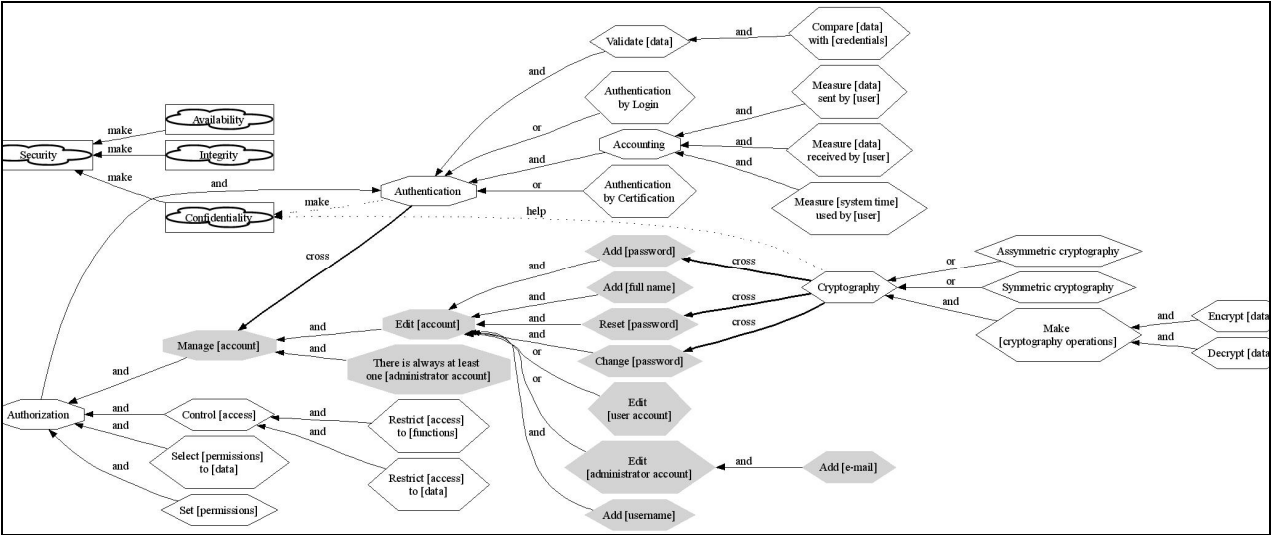


Figura 105. Modelo de segurança

Além disto, modificamos os dois relacionamentos transversais existentes neste modelo para refletir os elementos afetados do Unical. Na Figura 106, ilustramos a descrição destes relacionamentos instanciados para o Unical. As modificações em relação ao estudo de caso anterior estão em cinza.

1	1. Crosscutting { 2. source: Authentication 3. <i>pointcut</i> PC3: include(Manage [account]) and include(Manage [schedule]) 4. and include(Connect [BD]) 5. <i>advice</i> around: PC3 { Authentication by login ; Restrict [access] to [data]; 6. Restrict [access] to [functions] } 7. <i>intertype declaration</i> element: PC3 { Logout } 8. <i>intertype declaration</i> attribute: PC3 { constraint = confidential data } 9. }
2	1. Crosscutting { 2. source: Cryptography 3. <i>pointcut</i> PC4: include(. *password={ Add [password]; Reset [password]; Change [password] }) 4. <i>advice</i> around: PC4 { Symmetric cryptography } 5. }

Figura 106. Relacionamentos transversais do modelo de segurança para o Unical

A separação do modelo de segurança permitiu-nos a fácil identificação de seus elementos, evolução e reutilização em dois estudos de caso. Como mostrado na Figura 106, parte dos relacionamentos transversais, também, é reaproveitada; modificamos apenas seus *pointcuts*. Na Figura 107, ilustramos as modificações realizadas nos relacionamentos transversais dos modelos de persistência e confiabilidade (em cinza).

Persistência	
1	1. Crosscutting { 2. source: Make register operation 3. pointcut PC5: include(Connect [BD]) and include(Open [file]) 4. advice after: PC5 {Include [data]; Select [data]; Delete [data]; Update [data] } 5. }
2	1. Crosscutting { 2. source: Persistence 3. pointcut PC6: include(Manage [schedule]) 4. pointcut PC7: include(Configurability) 5. advice around: PC6 {Persistence in [BD] } 6. advice around: PC7 {Persistence in [file] } 7. }
Confiabilidade	
3	1. Crosscutting { 2. source: Detect [exception] 3. pointcut PC8: include(Verify if [data identification] is unique) and 4. include(Verify if [enters] are valid) 5. pointcut PC9: include(Verify if every [data] are filled up) 6. pointcut PC10: include(Persistence in [BD]) and include(Persistence in [file]) 7. pointcut PC11: include(Edit.*={ (Edit [event]) (Edit [category]) (Edit imported 8. [category]) (Edit [task]) (Edit [user account]) (Edit [administrator account]) }) 9. advice around: PC11 { Verify if [data identification] is unique ; 10. Verify if [enters] are valid ; Verify if every [data] are filled up } 11. intertype declaration element: PC10 { Detect [persistence exception] } 12. intertype declaration attribute: PC8 { exception = Invalid data } 13. intertype declaration attribute: PC9 { exception = Incomplete data } 14. intertype declaration element: PC8 and PC9 { Detect [robustness exception] ; 15. Detect [correctness exception] } 16. }

Figura 107. Relacionamentos transversais do modelo de persistência e confiabilidade

Com a definição de novos tipos (atributos) utilizando *intertype declarations* é possível modelar características transversais além de metas, *softmetas* e tarefas, tais como, interessados por cada parte da modelagem ou custo e tempo de desenvolvimento. A seguir, apresentamos como modelar e visualizar elementos para os quais o modelo de componentes V-graph não dá suporte (não fazem parte da decomposição dominante do V-graph).

Modelagem de interessados

Adicionamos este novo modelo à modelagem do sistema Unical, que consiste em apenas uma tarefa denominada “Model [stakeholders]”. Esta tarefa está relacionada aos elementos dos outros modelos por acrescentar o atributo “actor”. Desta forma modelamos os interessados em cada grupo de requisitos. Na Figura 108, ilustramos seu relacionamento transversal.

1. Crosscutting {
2. Source: Model [stakeholder]
3. Pointcut PC12: include (Manage [schedule]) and include(Usability) and
4. include (Reliability) and include (Confidentiality) and include (Portability)
5. and include(Scalability) and include(Performance)
6. Pointcut PC13: include(Manage [account])
7. Pointcut PC14: include(Profitable) and include(Timeliness)
8. Pointcut PC15: include (Maintanability) and include(Reusability)
9. Intertype declaration attribute: PC12 and PC13 {actor = user}
10. Intertype declaration attribute: PC13 {actor = administrator user}
11. Intertype declaration attribute: PC14 and PC15 {actor = project manager }
12. Intertype declaration attribute: PC15 {actor = development team} }

Figura 108. Relacionamento transversal do modelo de cenários

Com este relacionamento transversal inserimos os atores interessados em cada grupo de requisitos. A composição espalha estes atributos na modelagem em AOV-graph. Como o AOV-graph não possui elementos gráficos para sua visualização então geramos uma outra visão para mostrá-los, veja a Figura 109.

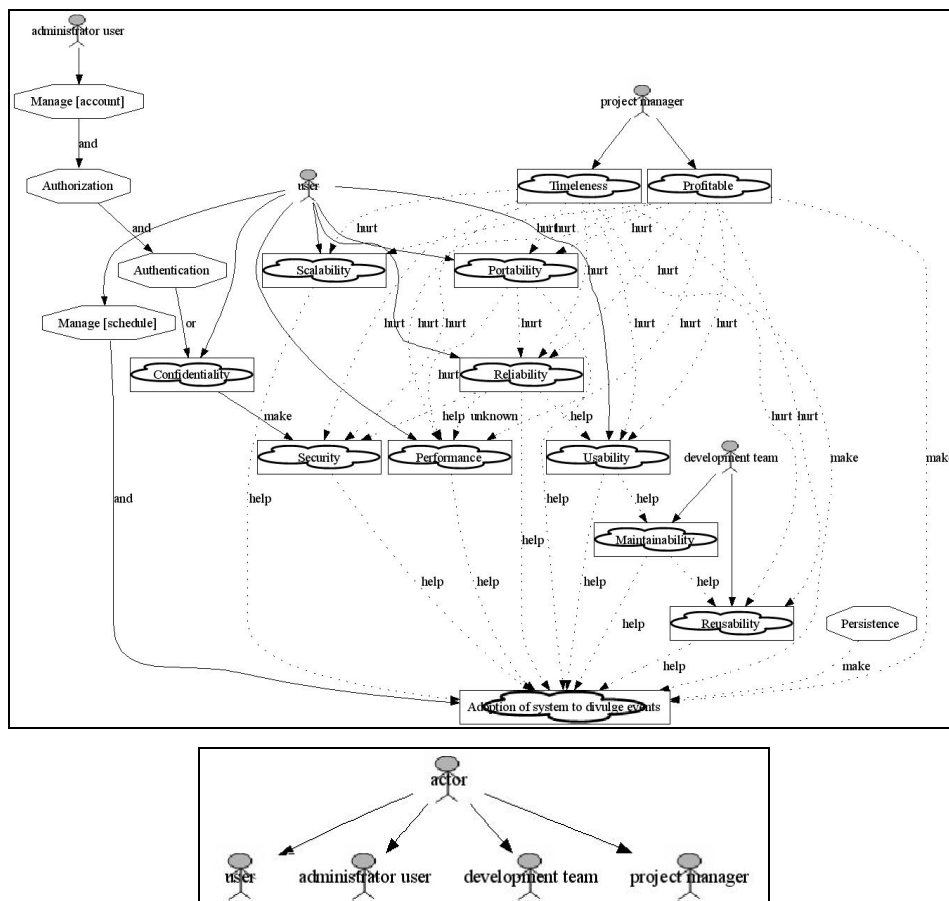


Figura 109. Visão dos interessados do sistema

Assim, a utilização de *intertype declarations* associada ao mecanismo de visualização permite a separação de características transversais, tanto utilizando os elementos da decomposição dominante, quanto outros elementos.

5.3.2. Análise dos Resultados

A modelagem do Unical inclui 41 *softmetas*, 15 metas e 123 tarefas, veja Tabela 9. Separamos estes elementos em doze grupos de requisitos, sendo referentes à divulgação de eventos, usabilidade, performance, persistência, segurança, confiabilidade, manutenibilidade, reusabilidade, portabilidade, escalabilidade e referentes ao custo e tempo de desenvolvimento.

Tabela 9. Estatísticas do estudo de caso – Unical

Elementos	Quantidade antes da composição	Quantidade após a composição
Metas	15	15
Tarefas	123	138
<i>Softmetas</i>	41	41
Relacionamentos transversais	10	10
Elos transversais	40	91
Correlações	35	35
Contribuições	157	157+91=248
Tipos	57	60
Tópicos	79	89
Atributos	0	5

Com a composição 15 novas tarefas foram adicionadas. Além disto, tivemos a adição de 51 elos transversais, representando um crescimento de 127% em relação à quantidade de elos transversais antes da composição. Este crescimento de elos indica:

- Espalhamento de alguns elementos, pois eles estão envolvidos na realização de muitas, metas, *softmetas* e tarefas. Por exemplo, as tarefas “Verify if [data identification] is unique”, “Verify if [enters] are valid” e “Verify if every [data] are filled up” contribuem para “Edit [event]”, “Edit [category]”, “Edit imported [category]”, “Edit [task]”, “Edit [user account]” e “Edit [administrator account]”, com o intuito de prover robustez ao sistema Unical.
- Entrelaçamento de algumas características, pois para realização de cada uma delas é necessária a contribuição de metas, *softmetas* e tarefas específicas de outras características. Por exemplo, para dar suporte a configurabilidade, a persistência em arquivo é necessária para registrar localmente as opções de configuração de cada usuário.

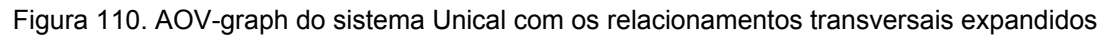
Se considerarmos uma única visão com os doze modelos juntos (mostrado na Figura 110), este espalhamento e entrelaçamento causam dificuldades à

visualização, modelagem e análise dos modelos. Além disto, se nenhum mecanismo de visualização é utilizado, é árduo o retrabalho em escrever e atualizar estes elementos que se repetem em várias visões parciais. Assim, o mecanismo de visualização é essencial para:

- Oferecer visões mais apropriadas para cada tarefa do processo de definição de requisitos, ou para cada interessado no sistema. Por exemplo a visão ilustrada na Figura 96 concentra boa parte das funcionalidades de interesse do usuário final do sistema Unical;
- Oferecer visões parciais e, desta forma, diminuir a complexidade do modelo e permitir melhor definição de cada parte do modelo, tais como cada uma das visões ilustradas na Figura 100 e Figura 105, o modelo de usabilidade e o modelo de segurança, respectivamente;
- Diminuir o retrabalho em escrever e atualizar elementos que estão repetidos em várias visões parciais. Por exemplo, a Figura 103 e a Figura 104 contém os mesmos novos elementos criados pelo relacionamento transversal do modelo de usabilidade;
- Oferecer “visões transversais”, visões que mostram as informações do modelo seguindo uma outra forma de decomposição. Por exemplo, os diagramas de classes, MER e cenários criados com as informações modeladas em AOV-graph.

Na Figura 110, ilustramos o resultado da composição dos doze modelos que compõem a modelagem do sistema Unical. Neste modelo tanto quanto no modelo ilustrado na Figura 94 (página 133), é difícil analisar o emaranhado de relacionamentos existentes. Entretanto, na Figura 110 está explícito a que sub-modelo cada elemento pertence (ou está mais associado) e há maior potencial para extrair visões.

Neste estudo de caso, ilustramos exemplos de reuso quando utilizamos o mesmo modelo de segurança, persistência e confiabilidade desenvolvido para o C&L. Neste caso modificamos apenas os *pointcuts* de seus relacionamentos transversais, como mostrado na Figura 106 e Figura 107 (páginas 143 e 144, respectivamente). Desta forma, a separação e o uso do relacionamento transversal facilitam o reuso, pois a composição dos dois modelos fica centralizada no relacionamento transversal.



5.4. Resumo

Neste Capítulo apresentamos dois estudos de caso referentes aos sistemas C&L e Unical. Apesar de serem sistemas simples e razoavelmente pequenos, a aplicação de nossa estratégia trouxe os seguintes benefícios quanto à modelagem, ao reuso, à rastreabilidade e à evolução:

- Facilidade de modelagem – diminuição no número de relacionamentos que o engenheiro de requisitos deve escrever; centralização das interações partindo do mesmo elemento em um único relacionamento transversal; geração de visões com o relacionamento transversal resumido (não processado) ou expandido (processado pelo mecanismo de composição); geração das visões, matriz de rastreabilidade (Tabela 8, página 130), cenários (Figura 99, página 138), MER (Figura 97, página 136) e diagrama de classes (Figura 92, página 128). Além disso, com o relacionamento transversal resumido explicitamos a natureza transversal de algumas características.
- Reuso – conseguimos reutilizar os modelos de segurança, confiabilidade, persistência e usabilidade em contextos diferentes. Toda a interação entre estes modelos está centralizada nos relacionamentos transversais, eles podem ser reescritos ou, simplesmente, apagados. Os modelos de performance, manutenibilidade e portabilidade podem ser utilizados como ponto de partida para o detalhamento destas características.
- Rastreabilidade – cada relacionamento transversal é uma matriz parcial de rastreabilidade, que pode ser consultada quando necessário (veja o exemplo ilustrado na Tabela 8, página 130). Além disto, a definição dos relacionamentos transversais induz o engenheiro de requisitos a pensar sobre o impacto que as características exercem umas sobre as outras e o induz a analisar tais características enquanto realiza a modelagem. Desta maneira, a Rastreabilidade deixa de ser postergada ou esquecida, como geralmente ocorre durante a definição de requisitos.
- Evolução – mudanças na definição de relacionamentos transversais são propagadas automaticamente. Cada modelo também pode sofrer modificações independentemente dos demais e se os relacionamentos

transversais também precisam ser modificados, é fácil identificar os pontos afetados por meio da matriz de rastreabilidade fornecida pelos próprios relacionamentos transversais. As facilidades providas pela separação dos modelos, geração de visões e reuso também podem contribuir para análise dos modelos sendo criados e sua melhoria, i.e., evolução.

As facilidades de modelagem providas pela separação e pelo relacionamento transversal facilitam a manipulação de modelos grandes. Estimamos que quanto mais elementos a modelagem contém, maior a necessidade de separação, e assim, se tornam essenciais mecanismos que ajudem na composição e visualização das partes separadas. O mecanismo de composição, mesmo utilizando XSLT (tecnologia muito simples de implementação), foi realizado em apenas 2,08 e 3,16 segundos para o primeiro e segundo estudo de caso, respectivamente.

6 Conclusão

Neste Capítulo apresentamos um resumo de nossa abordagem (Seção 6.1), bem como suas contribuições (Seção 6.2) e diferenças das abordagens relacionadas (Seção 6.3). Em seguida, apresentamos suas limitações e os trabalhos futuros (Seção 6.4).

6.1. Resumo

Esta tese aborda os problemas de entrelaçamento e espalhamento de características transversais durante a modelagem de requisitos.

Fomos influenciados por abordagens de programação orientadas a aspectos na definição de uma estratégia de modelagem que considera atividades e mecanismos de separação e composição de características transversais. Por outro lado, considerando abordagens utilizadas em engenharia de requisitos para gerenciar a complexidade de modelos, adotamos como essencial um mecanismo de visualização.

Assim, nossa estratégia consiste de mecanismos para separação, composição e visualização de características. Estes mecanismos funcionam com base em uma linguagem de modelagem (LMROA) que permite a descrição antecipada de como as características estão entrelaçadas e espalhadas. LMROA é um modelo conceitual que define um novo tipo de relacionamento, denominado relacionamento transversal, para ser adicionado a linguagens tradicionais de modelagem de requisitos.

Para validar nossa estratégia, instanciamos LMROA por utilizar o V-graph como modelo de componentes e definimos (e implementamos) os mecanismos de separação, composição e visualização para esta instância. Além disto, dois estudos de caso foram realizados, ilustrando os ganhos obtidos por aplicar uma abordagem orientada a aspectos durante a definição de requisitos.

6.2. Contribuições

Esta tese mostra os problemas de espalhamento e entrelaçamento em modelos de requisitos. Estes problemas aumentam a complexidade dos modelos, dificultando a modelagem, rastreabilidade, evolução e reuso de requisitos. Assim, nós exploramos as vantagens de explicitar e tratar características transversais durante a modelagem. Como citamos no Capítulo 1, as contribuições desta tese são:

1) **Metamodelo de integração de características transversais** – O metamodelo define as atividades de separação, composição e visualização como sendo essenciais para abordar os problemas de espalhamento e entrelaçamento de características durante a modelagem de requisitos. Este metamodelo pode ser utilizado como base em outras fases do desenvolvimento para definição de mecanismos práticos de redução dos danos causados pelos problemas de espalhamento e entrelaçamento. O metamodelo é apresentado no Capítulo 4, Seção 4.1.

2) **Linguagem de modelagem de requisitos orientada a aspectos** – Em LMROA definimos o “relacionamento transversal”, como um novo construto para estender linguagens tradicionais de modelagem de requisitos. Diferentemente das abordagens orientadas a aspectos, nossa abordagem define um relacionamento como o elemento responsável por descrever como as características se entrelaçam e se espalham. Desta forma, como relatamos no Capítulo 4, Seção 4.2.3, todas as características, transversais ou não, são descritas utilizando apenas os construtos do modelo de componentes escolhidos. Assim, a extensão realizada é menos intrusiva à linguagem de modelagem de requisitos escolhida; retardamos a decisão e descrição das informações relativas à transversalidade de requisitos; facilitamos o reuso de características que em alguns projetos têm esta propriedade de transversalidade e em outros não a têm; separamos as informações referentes a “o quê” é transversal, representada pelos elementos do modelo de componentes, das referentes a “como” é transversal, representadas no relacionamento transversal. LMROA é descrita no Capítulo 4, Seção 4.2

3) **Criação de AOV-graph** – AOV-graph é uma extensão de V-graph que permite explicitar como as características estão transversais. Definimos diretrizes

para separação de características, semântica para os construtos de LMROA, e visões essenciais para a análise e evolução destes modelos de metas. Esta nova linguagem permite, além da descrição e análise de RNFs e RFs, a descrição e análise de características transversais. Por meio do relacionamento transversal é possível reduzir o número de relacionamentos entre metas, *softmetas* e tarefas e assim, diminuir o acoplamento entre estes elementos, ou pelo menos, facilitar a visualização e manipulação do modelo. AOV-graph é apresentado no Capítulo 4, Seção 4.3.

4) **Geração das visões Cenários, MER e diagrama de classes a partir de informações descritas no modelo AOV-graph** – O modelo de metas AOV-Graph é uma visão intencional do sistema. Porém, as informações retratadas em seus construtos (*softmetas*, metas, tarefas, tipo, tópico e relacionamentos) podem ser decompostas e representadas de outras formas, tais como: cenários, MER e diagrama de classes. O mecanismo de visualização que definimos para AOV-graph inclui estas formas de organizar as informações do sistema. Para isto, elaboramos regras que transformam as informações em AOV-graph no modelo de cenários, MER e diagrama de classes. As regras para gerar estas visões são descritas no Capítulo 4, Seção 4.3.3.

5) **Conjunto de características transversais reutilizáveis** – Realizamos dois estudos de casos utilizando AOV-graph. Nestes estudos de caso, modelamos separadamente as características Segurança, Persistência, Usabilidade e Confiabilidade. Estes modelos podem ser reutilizados em outros projetos, ou pelo menos, usados como ponto de partida para melhor elaboração destas características. Os relacionamentos transversais contidos neles precisam ser modificados para que se relacionem aos novos pontos afetados. Contudo, algumas informações contidas em cada *intertype* e *advice* podem ser reaproveitadas.

Estas contribuições estão correlacionadas ao reuso, rastreabilidade, evolução e ao desenvolvimento orientado a aspectos da seguinte maneira:

1) **Reuso** – é melhorado porque cada característica pode ser modelada separadamente, muitas vezes, de maneira mais abstrata, com o intuito antecipado de utilizá-la em diferentes projetos;

2) **Rastreabilidade** – parte do rastreamento é realizada antecipadamente, enquanto o engenheiro modela as características usando o relacionamento transversal, e não após elas terem sido modeladas, como acontece usualmente;

3) **Evolução** – a evolução é afetada por três razões: i) devido a separação, cada modelo é menos complexo, então é mais fácil modificar cada um separadamente; ii) devido ao uso do relacionamento transversal, com a centralização de informações sobre o entrelaçamento e espalhamento é mais fácil controlar e modificar as interações que envolvem características transversais, e iii) com os mecanismos de composição e visualização qualquer mudança nos relacionamentos transversais é automaticamente propagada pelos modelos (visões) do sistema;

4) **Desenvolvimento de software orientado a aspectos** – com a utilização de instâncias de LMROA em projetos reais, tal como fizemos com AOV-graph, é possível analisar se as características modeladas como transversais (aquelas que são origem de relacionamentos transversais) durante a definição de requisitos são também transversais nas fases posteriores de desenvolvimento e, assim, ter maior conhecimento sobre “o que são aspectos” nesta fase. Desta forma, consideramos que nossa abordagem pode ajudar a abordagens de identificação, mapeamento e análise de características transversais.

6.3.

Comparação com Trabalhos Relacionados

Na Tabela 11, listamos os principais trabalhos relacionados ao nosso (descritos nos Capítulos 2 e 3). Estes trabalhos têm em comum ao nosso a preocupação explícita com a separação e composição de características transversais durante a definição de requisitos.

Tabela 10. Trabalhos relacionados

Referência	Título	Foco
Rashid, 2002 (Capítulo 3)	Early aspects: a model for aspect-oriented requirements engineering	Processo
Moreira, 2002 (Capítulo 3)	Crosscutting quality attributes for requirements engineering	Processo
Rashid, 2003 (Capítulo 3)	Modularization and composition of aspectual requirements	Processo - identificação, modelagem, e mapeamento
Sousa, 2003 (Capítulo 3)	Adapting the NFR framework to aspect-oriented requirements engineering	Processo – modelagem
Sousa, 2004 (Capítulo 3)	Separation of crosscutting concerns from requirements to design: Adapting the use case driven approach	Processo – modelagem
Brito, 2004 (Capítulo 3)	Integrating the NFR framework in a RE model	Processo – identificação
Moreira, 2005 (Capítulo 3)	Multi-Dimensional Separation of Concerns in Requirements Engineering	Modelagem
Leite, 2005 (Capítulo 2)	Quality-Based Software Reuse	Reuso e modelagem

Em resumo, nossa abordagem se distingue destes trabalhos tanto na motivação quanto na maneira de realizar a integração de características transversais:

Quanto à motivação

Estes trabalhos tentam, durante a definição de requisitos, identificar características que **mais tarde** possam ser aspectos, i.e., **mais tarde** elas estarão espalhadas e entrelaçadas; enquanto nossa abordagem tenta diminuir o entrelaçamento e espalhamento **durante** a modelagem de requisitos. Isto significa que estas abordagens buscam favorecer primeiramente ao desenvolvimento orientado a aspectos, enquanto nós buscamos primeiramente ajudar a Engenharia de Requisitos, considerando que todas as atividades (elicitação, modelagem, análise, reuso, evolução, rastreamento) durante esta etapa se beneficiam da abordagem orientada a aspectos.

Quanto ao modelo de integração

Nenhum dos trabalhos relacionados se preocupa em oferecer diferentes visões do modelo integrado. Desta forma, após a composição, os modelos são mais complexos e, assim, é mais difícil analisá-los e modificá-los. Por outro lado, nossa abordagem é restrita a modelagem, não se preocupando com a identificação e mapeamento de características transversais, estas atividades estão incluídas em nossos trabalhos futuros.

Quanto à linguagem de modelagem

Na Tabela 11, resumimos a comparação entre nossa linguagem de modelagem e as demais, utilizando as propriedades definidas em (Chavez, 2004). As colunas denominadas Q7, Sousa, Rashid e Moreira se referem às linguagens definidas, respectivamente, em (Leite, 2004; Sousa, 2004; Rashid, 2003; e Moreira, 2005). Nos concentramos nestes trabalhos porque eles abordam a modelagem de maneira mais concreta.

1) Modelo de componentes – LMROA pode ser aplicada a diferentes modelos de componentes. AOV-graph é uma instância de LMROA que utiliza um modelo de metas como modelo de componentes, denominado V-graph, seus componentes são: metas, *softmetas* e tarefas. Em Q7, Sousa, Rashid e Moreira, as estratégias para separação e composição são específicas, respectivamente, para os modelos de componentes: V-graph, casos de uso, sentenças de requisitos em pontos de vista e sentenças de requisitos. Além disto, Sousa e Rashid incluem extensões para

representar de maneira diferenciada os elementos transversais, denominadas casos de uso transversais e *concerns*, respectivamente.

Tabela 11. Comparação de linguagens para integração de características transversais

Características	LMROA	AOV-graph	Q7	Sousa	Rashid	Moreira
Modelo de componentes	Qualquer	Goal model	Goal model	Casos de uso e casos de usos transversais (RNFs)	Baseado na decomposição de pontos de vista e <i>concerns</i> (RNFs) em requisitos	Baseado na decomposição de <i>concerns</i> em requisitos
Componente	Qualquer	Tarefa, meta, <i>softmeta</i>	Tarefa, meta, <i>softmeta</i>	Casos de uso	Pontos de vista, <i>concerns</i> e requisitos	<i>Concerns</i> e requisitos
Linguagem de componentes	Qualquer	V-graph	V-graph	Diagrama de Casos de uso	Sentenças de requisitos	Sentenças de requisitos
Modelo de <i>joinpoint</i>						
<i>Joinpoints</i> dinâmicos	-	-	-	-	-	-
<i>Joinpoints</i> estáticos	Depende do modelo de componentes	Tarefas, metas, <i>softmetas</i> , tipos, tópicos	Tarefas, metas e <i>softmetas</i>	Casos de uso	Sentenças de requisitos nos pontos de vista	Sentenças de requisitos
Modelo núcleo						
Aspecto	Relacionamento transversal	Relacionamento transversal	Template 5w2h	Template para composição	Template para composição	Template para composição
Interface transversal	Origem e <i>pointcuts</i>	Tarefas, metas, <i>softmetas</i> Expressões regulares	Where: meta, <i>softmeta</i> e tarefa	caso de uso e ação afetada	requisito no ponto de vista afetado	requisito que é afetado
Feature transversal	<i>Advice</i> , <i>intertype declaration</i>	Tarefas, metas, <i>softmetas</i> e novos atributos ou elementos	What: meta, <i>softmeta</i> e tarefa	Ações do caso de uso transversal	Requirements no <i>concern</i>	Requirements no <i>concern</i>
Modelo de composição	Estático	Estático	Estático	Estático	Estático	Estático
Dicotomia	Não	Não	Não	Não	Não	Não
Quantificação	Sim	Sim	Não	Não	Não	Não
Transparência	Sim	Sim	Sim	Sim	Sim	Sim

2) Modelo de *joinpoint* – em todas as abordagens apenas *joinpoints* estáticos são definidos. LMROA define explicitamente quais são os pontos a serem

afetados, através do elemento *joinpoint*. No caso de AOV-graph, *softmetas*, metas e tarefas podem afetar *softmetas*, metas, tarefas, tipos e tópicos, permitindo a separação e composição de RNFs, RFs, dados e ações. Em Q7, apenas *softmetas*, metas e tarefas podem ser afetadas. Em Sousa e Rashid, apenas RNFs podem afetar RFs, sendo que em Rashid, RFs estão relacionados a pontos de vista. Assim como em nossa abordagem, em Moreira, tanto RFs quanto RNFs podem afetar e/ou ser afetados.

3) Modelo núcleo – em LMROA definimos o relacionamento transversal como o relacionamento responsável por registrar a transversalidade dos componentes. Este relacionamento pode ser aplicado a diferentes modelos de componentes. Sua interface transversal é descrita pelos *pointcuts* e as *features* pelas *intertype declarations* e *advice*. Em AOV-graph, os *pointcuts* podem ser qualquer elemento dos tipos meta, *softmeta*, tarefas, tipos e tópicos, e as *features* podem ser elementos destes mesmos tipos ou novos tipos descritos nas *intertype declarations*. Em Q7, as interfaces e *features* transversais podem ser elementos dos tipos, meta, *softmeta* e tarefa. Em Sousa, as interfaces são os identificadores dos casos de uso mais a ação afetada nele, e as *features* são definidas por ações do caso de uso transversal. Em Rashid e Moreira, as interfaces e *features* transversais são definidas apenas por elementos do tipo sentenças de requisitos.

4) Modelo de composição – Em todas as abordagens o modelo de composição: gera um novo modelo (seguindo o mesmo formato do modelo de componentes) com as informações integradas; a composição ocorre diretamente na descrição “estática” de requisitos. Em AOV-graph, definimos um modelo de composição automatizado, que permite o uso de expressões regulares para identificação dos pontos a serem atingidos. Em Moreira e Rashid o modelo de composição também é automático. Enquanto Q7 e Sousa não disponibilizam um modelo de composição automatizado. Em Sousa as informações sobre a composição são descritas informalmente, servindo apenas como guia para composição.

5) Dicotomia – Nenhuma das abordagens modularizam características transversais em um único elemento. Elas utilizam elementos do modelo de componentes para descrever as características e elementos de composição para descrever o entrelaçamento. Em AOV-graph e Q7 as características são definidas pelos próprios elementos do modelo de componentes (metas, *softmetas* e tarefas) e a transversalidade pelo relacionamento transversal e 5W2H, respectivamente. Em

Sousa as características transversais são definidas por casos de uso de RNFs transversais (casos de uso transversais) e a transversalidade por um template de composição. Em Rashid e Moreira, as características são representadas pelo template de *concerns* e a transversalidade por templates de composição.

6) Quantificação – Nossa abordagem permite a quantificação através de expressões regulares. Em Rashid e Moreira é possível especificar que um ou todos os requisitos de uma característica corta todos ou um requisito de uma outra característica, mas para especificar mais de um requisito (que não sejam todos) é necessário definir um a um. Em Sousa e Q7, a quantificação também não é abordada.

7) Transparência – Todas as abordagens assumem que não é necessário realizar nenhuma mudança na descrição dos elementos afetados.

Como podemos observar na Tabela, LMROA é um **modelo genérico** que pode ser utilizado para estender linguagens de modelagem de requisitos, provendo o tratamento de características transversais. Apesar de AOV-graph e Q7 usarem o mesmo modelo de componentes, AOV-graph permite a criação de novos elementos e a modificação de tipos e tópicos além dos permitidos em Q7. Com a nossa abordagem o modelo de componentes é estendido apenas pelo relacionamento transversal, enquanto as abordagens definidas em (Sousa, 2004; Rashid, 2003) criam extensões para representar tanto as características transversais em si (RNFs), quanto para representar a transversalidade destas características. A abordagem descrita em (Moreira, 2005) utiliza um template de composição para relacionar quaisquer requisitos, enquanto em AOV-graph podemos utilizar, também, as correlações e contribuições de seu modelo de componentes. Além disto, em AOV-graph os elementos metas, *softmetas*, tarefas, tipos e tópicos possibilitam mais formas de separação e composição do que apenas sentenças não estruturadas de requisitos, como utilizado em (Moreira, 2005).

6.4.

Limitações e Trabalhos Futuros

A seguir, citamos as limitações de nossa abordagem e perspectivas para trabalhos futuros:

Identificação e mapeamento de características transversais

O escopo desta tese é a modelagem de requisitos. Desta forma, as atividades de identificação e mapeamento de características transversais não são abordadas. Consideramos que estas atividades são independentes do meta-modelo de integração e do relacionamento transversal de LMROA. Elas são influenciadas pelo modelo de componentes utilizado. Desta forma, para cada instância de LMROA, heurísticas podem ser definidas para a identificação, modelagem e mapeamento ou adaptadas de outras abordagens, tais como (Sousa, 2004; Rashid, 2003).

Análise

Consideramos que nossa abordagem ajuda a atividade de análise de requisitos porque oferece visões, parciais ou não, mostrando diferentes propriedades do sistema modelado. Entretanto, por meio dos estudos de caso realizados, nos limitamos à análise de corretude e identificação de requisitos omissos. Entretanto, seria interessante agregar outros tipos de análise à nossa abordagem, tais como, análise de obstáculos (Lamsweerde, 2000); análise quantitativa de metas (Giorgini, 2002); análise de variabilidade (Gonzáles, 2004; Park, 2004), dentre outras.

Apoio de ferramentas

Uma das principais limitações de abordagens de engenharia de requisitos orientada a aspectos é quanto ao apoio de ferramentas. Como relatamos no Capítulo 3, Seção 3.4, muitas das abordagens não apresentam suporte automatizado para realizar a composição. Nossa abordagem baseia-se em uma linguagem que permite a automação desta atividade, bem como a geração de visões. Entretanto, como apresentamos no Capítulo 4, Seção 4.3.4, nossa implementação não inclui ferramentas para edição gráfica de modelos em AOV-graph, nem interatividade com as visões geradas, de maneira que o engenheiro conseguisse minimizar, ocultar, realçar ou mesmo modificar os elementos de interesse a cada momento do processo de definição de requisitos.

Regras de transformação

Quanto às visões geradas com base em AOV-graph, as regras de transformação definidas no Capítulo 4, Seção 4.3.3.2, podem ser refinadas para obtenção de modelos mais precisos, tal como a inclusão de cardinalidade no MER e diagrama de classes, inferência do tipo de relacionamento no diagrama de classes, dentre outros.

Integração de métodos

Se as visões geradas pudessem ser modificadas diretamente, então um metamodelo definindo duplo fluxo de transformações é necessário. Pesquisas sobre a adequação ou adaptação do relacionamento transversal quando trabalhando com múltiplos modelos pode ser inspirada nas idéias de Nuseibeh (1994) ou Cysneiros (2001). Isto ampliaria nossa abordagem por incluir, também, a integração de métodos.

Armazenamento e recuperação de características

Como mostrado anteriormente, as características (ou serviços) podem ser separadas e reutilizadas em diferentes projetos. Entretanto, não discutimos como elas podem ser armazenadas e recuperadas. Em (Leite, 2004), define-se uma abordagem interessante para o reuso de características transversais baseada na linguagem Q7. Podemos unir esta abordagem à nossa, realizando uma transformação dos elementos descritos nos relacionamentos transversais para o formato Q7. Isto tornaria as informações em Q7 mais precisas e proveria nossa abordagem de um mecanismo de organização e recuperação para reuso.

Extensão da linguagem de modelagem

Definimos LMROA de forma genérica para que ela possa ser adaptada a modelos de componentes diferentes, tal como apresentamos no Capítulo 4, Seção 4.4. Acreditamos que esta descrição possa ser facilmente estendida caso haja necessidade. Por exemplo, em descrever qual a seqüência das composições, ou outras primitivas além de *include* e *substitute*.

Experimentação

Demonstramos através dos estudos de caso que nossa abordagem contribui positivamente para o reuso, rastreabilidade e evolução. Porém, mais estudos precisam ser realizados para obtermos mais conhecimento sobre o uso desta estratégia. A realização de experimentos utilizando nossa abordagem e abordagens mais tradicionais em paralelo, com métricas bem definidas, é importante para avaliarmos os ganhos de produtividade com o uso de nossa estratégia.

AKSIT, M.; BERGMANS, L.; VURAL, S. An object-oriented language-database integration model: The composition-Filters approach. In: PROC. OF THE 7th EUROPEAN CONF. OBJECT-ORIENTED PROGRAMMING, Springer-Verlag Lecture Notes in Computer Science, 1992. p. 372-395.

ALEXANDER, I.; STEVENS, R. **Writing Better Requirements** Addison-Wesley, 2002. 176 p. ISBN: 0-932633-55-2.

AspectJ Project. Disponível em: <http://www.eclipse.org/aspectj/>. Acesso em: 7 março 2006.

BAKKER J.; TEKINERDOGAN, B.; AKSIT, M. Characterization of early aspects approaches. In: THE EARLY ASPECTS WORKSHOP, 2005.

BANIASSAD, E.; CLARKE, S. Theme: An approach for aspect-oriented analysis and design. In: PROC. OF THE 7th INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING (ICSE'04), Scotland, 2004. p. 158-167.

BREITMAN, K. K. **Evolução de Software**. Rio de Janeiro, 2000. 133p. Tese de Doutorado em Engenharia de Software - PUC-Rio.

BRICHAU, J.; HAUPT, M. Survey of aspect-oriented languages and execution models. Technical Report AOSD-Europe-VUB-01, 2005. 260 p. Disponível em: <http://www.aosd-europe.net/documents/index.htm>. Acesso em: 7 março 2006.

BRITO, I.; MOREIRA, A. Integrating the NFR framework in a RE model. In: THE EARLY ASPECTS WORKSHOP AT AOSD, England, 2004.

BOOCH, G.; RUMBAUGH, J.; JAVOBSON, I. **The Unified Modeling Language User Guide**. Addison-Wesley, 1999.

CARROLL, J. et al. d'etre: capturing design history and rationale in multimedia narratives. In: HUMAN FACTORS IN COMPUTING SYSTEMS (CHI94), Boston-USA, ACM Press, 1994, p. 192-197.

C&L. Disponível em: <http://sl.les.inf.puc-rio.br/cel>. Acesso em: 7 março 2006.

CHAVEZ, C. **A Model-Driven approach to aspect-oriented design**. Rio de Janeiro, 2004. 305 p. Tese de Doutorado em Engenharia de Software - PUC-Rio.

CHAVEZ, C.; LUCENA, C. A Theory of Aspects for Aspect-Oriented Software Development. In: PROC. OF THE XVII BRAZILIAN SIMPOSIUM ON SOFTWARE ENGINEERING, 2003. p. 130-145.

CHEN, P. P. The Entity Relationship Model - Toward a Unified View of Data. **ACM Transactions Database Systems**, Vol.1, No. 1, p. 9-36, 1976.

CHITCHYAN, R. et al. Survey of aspect-oriented analysis and design approaches. Technical Report AOSD-Europe-ULANC-9, AOSDEurope, 2005. Disponível em: <http://www.aosd-europe.net/documents/index.htm>. Acesso em: 7 março 2006.

CHITCHYAN, R.; RASHID, A.; SAWYER, P. Comparing Requirement Engineering Approaches for Handling Crosscutting Concerns. In: PROC. OF THE WORKSHOP EM ENGENHARIA DE REQUISITOS (WER2005), Porto-Portugal, 2005. p 1-12.

CHUNG, L. et al. (2000). **Non-Functional Requirements in Software Engineering**, Boston: Kluwer Academic Publishers. ISBN 0-7923-8666-3.

CZARNECKI, K; EISENECKER, U. **Generative Programming: Methods, Tools, and Applications**, Addison-Wesley, 2000.

CLARKE, S. et al. Subject-oriented design: Towards improved alignment of requirements, design and code. In: PROC. OBJECT-ORIENTED PROGRAMMING, SYSTEMS, LANGUAGES AND APPLICATIONS (OOPSLA), 1999. p. 325–339.

CYSNEIROS, L. M.; LEITE, J. C. S. P.; NETO, J. M. S. A Framework for Integrating Non-Functional Requirements into Conceptual Models. **Requirements Engineering Journal**, Vol. 6, No. 2, p. 97-115, 2001, Springer-Verlag London Limited.

CYSNEIROS, L. M.; YU, E.; LEITE, J. C. S. P. Cataloguing Non-Functional Requirements as Softgoal Networks. In: PROC. OF REQUIREMENTS ENGINEERING FOR ADAPTABLE ARCHITECTURES at 11th International Requirements Engineering Conference, 2003. p.13-20.

DAVIS, A. M. **Software Requirements: Analysis and Specification**, Prentice-Hall Inc, 1990.

DIJKSTRA, E. **A Discipline of Programming**. Prentice-Hall, 1976.

DUTOIT, A. H.; PAECH, B. "Rationale Management in Software Engineering. In: S.K. Chang (Ed.), **Handbook of Software Engineering and Knowledge Engineering**. World Scientific, 2001.

EASTERBROOK, S.; NUSEIBEH, B. (1996) Using Viewpoints for Inconsistency Management. **BCS/IEE Software Engineering Journal**, Vol. 11, No.1. p. 31-43.

FELICÍSSIMO, C. H. et al. C&L: Um Ambiente para Edicao e Visualizao de Cenarios e Léxicos. In: WORKSHOP DE FERRAMENTAS DO XVIII SIMPOSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE, Publicantions in CD, Brasilia, 2004. p. 43-48. ISBN 85-7669-004-7.

FERREIRA, A. B. H. **Mini Aurélio: o dicionário da língua portuguesa**, ed. 6, Editora Positivo, 2004. ISBN 85-7472-416-5.

Filman, R. E. et al. **Aspect-Oriented Software Development**, Addison-Wesley, 2005.

FINKELSTEIN, A. et al. Viewpoints: A framework for integrating multiple perspectives in system development. **International Journal of Software Engineering and Knowledge Engineering**, No. 2, 1992. p. 31-58.

FINKELSTEIN, A.; SOMMERVILLE, I. The Viewpoints FAQ. **BCS/IEE Software Engineering Journal**, Vol. 11, No. 1, 1996.

GIORGINI P. et al. Reasoning with goal models. In: PROC. OF THE 21ST INTERNATIONAL CONFERENCE ON CONCEPTUAL MODELING, 2002. p. 167-181.

GOGUEN, J. A.; LINDE, C. Techniques for Requirements Elicitation. In: PROC. OF THE FIRST IEEE INTERNATIONAL SYMPOSIUM ON REQUIREMENTS ENGINEERING, San Diego-Ca, IEEE Computer Society Press, 1994. p 152-164.

GONZÁLES, B.; LAGUNA, M.; LEITE, J. C. S. P. Visual variability analysis with goal models. IN: PROC. OF IEEE INTERNATIONAL SYMPOSIUM ON REQUIREMENTS ENGINEERING (RE'04), Japan, 2004. p. 38-47.

GOTEL, O. C.; FINKELSTEIN, A. C. An analysis of the requirements traceability problem. In: PROC. OF THE FIRST INTERNATIONAL CONFERENCE ON REQUIREMENTS ENGINEERING (ICRE'94), IEEE Computer Society Press., 1994. p. 94-101.

GRAPHVIZ. Disponível em: <http://www.graphviz.org/>. Acesso em: 7 março 2006

HARRISON, W.; OSSHER, H. Subject-Oriented Programming: A Critique of Pure Objects. In: PROC. OF THE 7th CONF. ON OBJECT- ORIENTED PROGRAMMING, SYSTEMS, LANGUAGES AND APPLICATIONS (OOPSLA93), 1993. p. 411-428.

HARRISON, W.; OSSHER, H; TARR, P. Asymmetrically vs. Symmetrically Organized Paradigms for Software Composition, Technical Report RC22685, IBM Research, 2002.

HSIA, P. et al. Formal Approach to Scenario Analysis. **IEEE Software**, vol. 11, No. 2, 1994. p. 33-41.

HYPER/J Web Page. Disponível em: <http://www.research.ibm.com/hyperspace/HyperJ/HyperJ.htm>. Acesso em: 7 março 2006.

IEEE Std. 830-1998; IEEE Recommended Practice for Software Requirements Specifications. Institute of Electrical and Electronics Engineers, 1998, 38p. ISBN 0-7381-0332-2.

IEEE Std 1471-2000; IEEE Recommended Practice for Architectural Description of Software-Intensive Systems; Software Engineering Standards Committee of the IEEE Computer Society; Approved 21 September 2000.

KAPLAN, M. et al. Subject-Oriented Design and the Watson Subject Compiler. In: Subjectivity Workshop at OOPSLA'96, 1996. Disponível em: <http://www.research.ibm.com/sop/>. Acesso em: 7 março 2006.

KICZALES, G. et al. Aspect-Oriented Programming. In: PROC. OF THE 11th EUROPEAN CONFERENCE ON OBJECT-ORIENTED PROGRAMMING, Vol. 1241 de LNCS, Springer-Verlag, 1997. p. 220-242.

KICZALES, G. et al. An Overview of AspectJ. In: EUROPEAN CONF. ON OBJECT-ORIENTED PROGRAMMING, Budapest, Hungary, 2001. p. 18-22.

KOTONYA, G.; SOMMERVILLE, I. Requirements Engineering with Viewpoints. **BCS/IEE Software Engineering Journal**, Vol. 11, No.1, 1996. p. 5-18.

KOTONYA, G.; SOMMERVILLE, I. **Requirements Engineering: Process and Techniques**. Chichester, UK: John Wiley & Sons Inc., 1998.

JACOBSON, I. et al. **Object-Oriented Software Development- A Use Case Driven Approach**, Addison Wesley, 1992.

JACOBSON, I.; BOOCH, G.; RUMBAUGH, J. **The Unified Software Development Process**, Addison-Wesley, 1999. ISBN 0-201-57169-2.

LAMSWEERDE, A.; LETIER, E. Handling obstacles in goal-oriented requirements engineering. **IEEE Transaction Software Engineering**, Vol. 26, No.10, 2000. p.978–1005.

LAMSWEERDE, A. Goal-Oriented Requirements Engineering: A Guided Tour. In: PROC. OF THE 5TH IEEE INTERNATIONAL SYMPOSIUM ON REQUIREMENTS ENGINEERING (RE'01), Toronto, 2001. p. 249-263.

LEHMAN, M. M. Laws of software evolution revisited. **Lecture Notes in Computer Science**, Vol. 1149, 1996. p.108-120.

LEITE, J. C. S. P. Viewpoint Resolution in Requirements Elicitation. Irvine, 1988. PhD Thesis, Dept. of Computer Science, University of California.

LEITE, J. C. S. P. Viewpoints analysis: a case study. **ACM Software Engineering Notes**, Vol. 14, No. 3, 1989. p. 111-1.

LEITE, J. C. S. P.; FRANCO, A. P. M. O Uso de Hipertexto na Elicitação de Linguagens da Aplicação. In: ANAIS DE IV SIMPÓSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE, 1990. p. 134–149.

LEITE, J. C. S. P.; FREEMAN, P. Requirements Validation Through Viewpoint Resolution. **IEEE Transactions on Software Engineering**, Vol. 17, No 12, 1991.

LEITE, J. C. S. P.; OLIVEIRA, A. P. A Client Oriented Requirements Baseline. In: PROC. OF THE SECOND IEEE INTERNATIONAL SYMPOSIUM ON REQUIREMENTS ENGINEERING (RE'95), 1995. Los Alamitos: IEEE Computer Society Press, 1995. p.108-115.

LEITE, J. C. S. P. Viewpoints on Viewpoints. In: ACM JOINT PROCEEDINGS OF THE SIGSOFT'96 WORKSHOPS, ACM Press, 1996. p. 285-288.

LEITE, J. C. S. P. et al. Enhancing a requirements baseline with scenarios. In: PROC. OF THE THIRD IEEE INTERNATIONAL SYMPOSIUM ON REQUIREMENTS ENGINEERING (RE'97), IEEE Computer Society Press, 1997. p. 44-53.

LEITE, J. C. S. P. et al. Scenario Construction Process. **Requirements Engineering Journal**, Vol. 5, No. 1, 2000, Springer-Verlag London Limited. p. 38-61.

LEITE, J. C. S. P. et al. Quality-Based Software Reuse. In: PROC. OF THE CAISE 2005-LNCS 3520, 2005. p. 535-550.

LIEBERHERR, K. J.; SILVA-LEPE, I.; XIAO, C. Adaptive object-oriented programming using graph-based customization. **Communications of the ACM**, Vol. 37, No. 5, 1994. p. 94-101.

LIEBERHERR, K. J. 1996. Adaptive Object-Oriented Software: The Demeter Method with Propagation Patterns. **PWS Publishing Company**, Boston, 1996.

LOPES, C. V. AOP: A Historical Perspective. In: R. Filman et al. (eds.) **Aspect-Oriented Software Development**, Addison-Wesley, 2004. ISBN 0321219767.

MARCO, T. **Structured Analysis and Structured Specifications**. Prentice Hall, 1979.

MOREIRA, A.; ARAÚJO, J.; BRITO, I. Crosscutting quality attributes for requirements engineering. In: PROC. OF THE 14TH INTERNATIONAL CONFERENCE ON SOFTWARE ENGINEERING AND KNOWLEDGE ENGINEERING (SEKE 2002), ACM Press, 2002.

MOREIRA, A.; ARAÚJO, J.; RASHID, A. A Model for Multi-Dimensional Separation of Concerns in Requirements Engineering. In: THE 17TH CONFERENCE ON ADVANCED INFORMATION SYSTEMS ENGINEERING (CAISE'05), Lecture Notes in Computer Science, Vol. 3520, Porto-Portugal, 2005.

MOREIRA, A.; RASHID, A.; ARAÚJO, J. Multi-Dimensional Separation of Concerns in Requirements Engineering. In: THE 13TH INTERNATIONAL CONFERENCE ON REQUIREMENTS ENGINEERING (RE'05), IEEE Computer Society, Paris-France, 2005. p.285-296

MYLOPOULOS, J.; CHUNG, L.; NIXON, B. Representing and using nonfunctional requirements: A process-oriented approach. **IEEE Transactions on Software Engineering**, Vol. 18, No. 6, 1992, p. 483–497.

MYLOPOULOS, J. et al. Exploring Alternatives during Requirements Analysis. **IEEE Software**, Jan/Feb 2001. p. 2-6.

NUSEIBEH, B.; KRAMER, J.; FINKELSTEIN, A. A Framework for Expressing the Relationships Between Multiple Views in Requirements Specification. **IEEE Transactions on Software Engineering**, Vol. 20, No. 10, 1994. p. 760-773.

NUSEIBEH, B. Crosscutting requirements. In: PROC. OF THE 3RD INTERNATIONAL CONF. ON ASPECT-ORIENTED SOFTWARE DEVELOPMENT (AOSD 2004), Lancaster-UK, 2004. p. 3-4. ISBN:1-58113-842-3.

OSSHER, H.; TARR, P. Using Multi-dimensional Separation of Concerns to (Re)Shape Evolving Software. **Communications of the ACM**, Vol. 44, No.10, 2001. p. 43-50.

PARK, S.; KIM, M.; SUGUMARAN, V. A scenario, goal, and feature oriented domain analysis approach for developing software product lines. **Journal on Industrial Management & Data Systems**, Vol. 104, No. 4, 2004. p. 296-308.

PARNAS, D. On the Criteria To Be Used in Decomposing Systems into Modules. **Communications of the ACM**, Vol. 15, No. 12, 1972. p. 1053-1058.

PIVETA E. K. et al. Termos em português para Desenvolvimento de Software Orientado a Aspectos. In: 1º WORKSHOP BRASILEIRO DE DESENVOLVIMENTO DE SOFTWARE ORIENTADO A ASPECTOS (WASP'04), 2004.

RASHID, A. et al. Early aspects: a model for aspect-oriented requirements engineering. In: PROC. OF IEEE JOINT CONFERENCE ON REQUIREMENTS ENGINEERING, Germany, 2002. p. 199-202.

RASHID, A.; MOREIRA, A.; ARAÚJO, J. Modularization and composition of aspectual requirements. In: PROC. OF THE 2ND INTERNATIONAL

CONFERENCE ON ASPECT-ORIENTED SOFTWARE DEVELOPMENT, ACM, 2003. p. 11-20.

ROBINSON, W. N.; PAWLOWSKI, S. D.; VOLKOV, V. Requirements Interaction Management. **ACM Computing Surveys**, Vol. 35, No. 2, 2003. p. 132-190.

ROLLAND, C. et al. A proposal for a scenario classification framework. **Journal of Requirements Engineering**, Vol. 3, Springer Verlag, 1998. p. 23-47.

ROSS, D.; SCHOMAN, A. Structured analysis for requirements definition. **IEEE Transactions on Software Engineering**, Vol. 3, No. 1, 1977. p. 6-15.

SAYÃO, M.; LEITE, J. C. S. P. Rastreabilidade de Requisitos. Relatório Técnico 20/05, série Monografias em Ciência da Computação, DI/PUC-Rio, 2005.

SILVA, L. F.; LEITE, J. C. S. P.; BREITMAN, K. K. C&L uma Ferramenta de Apoio à Engenharia de Requisitos. **Revista de Informática Teórica e Aplicada (RITA)**, Vol. 12, Número 1, 2005. p. 23-46. ISSN 0103-4308.

SILVA, L. F.; LEITE, J. C. S. P. Uma linguagem de modelagem de requisitos orientada a aspectos. In: PROC. OF THE REQUIREMENT ENGINEERING WORKSHOP, junto ao CAISE 2005, Porto-Portugal, 2005. p. 13-25.

SILVA, L. F.; LEITE, J. C. S. P. Integração de Características Transversais Durante a Modelagem de Requisitos. In: ANAIS DO SIMPÓSIO BRASILEIRO DE ENGENHARIA DE SOFTWARE (SBES-2005), Uberlândia-MG, 2005.

SILVA, L. F.; LEITE, J. C. S. P. An Aspect-Oriented Approach to Model Requirements. In: RE'05 DOCTORAL CONSORTIUM, junto ao 13th IEEE International Requirements Engineering Conference, Paris-France, 2005.

SOMMERVILLE, I. **Software Engineering**, Ed. 6th, Addison- Wesley, 2000.

SOUSA, G.; SILVA, G.; CASTRO, J. Adapting the NFR framework to aspect-oriented requirements engineering, In: PROC. OF THE 17TH BRAZILIAN SYMPOSIUM ON SOFTWARE ENGINEERING (SBES'2003), Brazil, 2003.

SOUSA, G. et al. Separation of crosscutting concerns from requirements to design: Adapting the use case driven approach. In: PROC. OF THE EARLY ASPECTS WORKSHOP AT AOSD, England, 2004.

SOUSA, G. **Uma Abordagem Direcionada a Casos de Uso para o Desenvolvimento de Software Orientado a Aspectos**. Recife, 2004, 193 p. Dissertação de Mestrado - Centro de Informática, Universidade Federal de Pernambuco.

STAA, A. V. **Programação Modular: Desenvolvendo programas complexos de forma organizada e segura**. Publicação: 05/2000, Campus. p. 128-133. ISBN: 8535206086

TARR, P. et al. N Degrees of Separation: Multi-Dimensional Separation of Concerns. In: PROC. OF THE 21ST INT'L CONF. ON SOFTWARE ENGINEERING (ICSE'99), 1999. p. 107-119.

TARR, P.; OSSHER, H. Hyper/J User and Installation Manual; 2000. Disponível em: <http://www.alphaworks.ibm.com/tech/hyperj>. Acesso em: 7 março 2006.

TEKINERDOĞAN, B. et al. Early aspects: aspect-oriented requirements engineering and architecture design. Report. In: EARLY ASPECTS WORKSHOP AT AOSD, England, 2004.

UNICAL, Requirements Document of Unical. University of California at Irvine. Disponível em: http://www.ics.uci.edu/~taylor/ICS_52_FQ04/unical-req.html. Acesso em: 7 março 2006.

VOLERE. Disponível em: <http://www.volere.co.uk/>. Acesso em: 7 março 2006.

WEIDENHAUPT, K. et al. Scenario Usage in system development: current practice. **IEEE Software**, Vol. 15, No. 2, 1998. p. 34-45.

YOU Z. **Using meta-model-driven views to address scalability in istar models**. Toronto, 2004, 225 p. Master Thesis, Graduate Department of Computer Science, University of Toronto.

YU, E. **Modelling Strategic Relationships for Process Reengineering**. Toronto, 1995, 124 p. PhD Thesis, Graduate Department of Computer Science, University of Toronto.

YU, E. Towards Modelling and Reasoning Support for Early-Phase Requirements Engineering. In: PROC. OF THE 3RD IEEE INT. SYMP. ON REQUIREMENTS ENGINEERING (RE'97), Washington D.C., USA, 1997. p. 226-235.

YU, Y.; LEITE, J.; MYLOPOULOS, J. From goals to aspects: discovering aspects from requirements goal models. In: PROC. OF IEEE INTERNATIONAL SYMPOSIUM ON REQUIREMENTS ENGINEERING (RE'04), Japan, 2004. pp. 38-47.

ZAVE, P.; JACKSON, M. Four dark corners of requirements engineering. **ACM Transaction Software Engineering Methodologies**, Vol. 6, No.1, 1997. p.1–30.

ZORMAN, L. **Requirements Envisaging through utilizing scenarios – REBUS**. 1995. Ph.D. Dissertation, University of Southern California.

Apêndice A

Gramática da Linguagem Orientada a Aspectos para Modelagem de Requisitos

Neste Apêndice apresentamos a especificação da linguagem de modelagem de requisitos orientada a aspectos, em BNF e DTD.

BNF de AOV-graph

```

<aspect_oriented_model> := <goal_model>
<goal_model> := goal_model (<name>; <id>){<component> <relationship>} | <goal_model> <goal_model>
<component> := <softgoal> | <goal> | <task> | <component> <component>
<softgoal>:=softgoal(<name>;<softgoal_id>;<decomposition_label>;(<type>);(<topic>))
    {<component><relationship>}
<goal> := goal (<name>; <goal_id>; <decomposition_label> ; (<type>) ; (<topic>))
    {<component><relationship>}
<task> := task (<name>; <task_id>; <decomposition_label> ; (<type>) (<topic>))
    {<component><relationship>}
<topic> := <name> | <topic>; <topic>
<type> := <name> | <type>; <type>
<relationship> := <correlation_relationship> | <crosscuttingRel> | <relationship> <relationship>
<correlation_relationship> :=
    correlation <correlation_label> {source=<goal_ref> target=<softgoal_ref>} |
    correlation <correlation_label> {source=<softgoal_ref> target=<softgoal_ref>} |
    correlation <correlation_label> {source=<task_ref> target=<task_ref>}
<correlation_label> := break | hurt | unknown | help | make
<decomposition_label>:=and | or | xor | break | hurt | unknown | help | make
<goal_ref> := <goal_id> <decomposition_label>
<softgoal_ref> := <softgoal_id> <decomposition_label>
<task_ref> := <task_id> <decomposition_label>

<joinpoint> := <softgoal_ref> | <goal_ref> | <task_ref>

<crosscuttingRel> := crosscutting <crosscutting_label>{source=<joinpoint>
    <pointcut><advice><intertype_declaration>}
<crosscutting_label>:= cross
<pointcut> := pointcut (<name>; <pointcut_id>) : <pointcut_expression> | <pointcut> <pointcut>
<pointcut_expression> := <operand> |
    not <operand> |
    <pointcut_expression> and <pointcut_expression> |
    <pointcut_expression> or <pointcut_expression>
<operand> := <primitive> (<joinpoint>) | <primitive> (<regular_expression>)
<primitive> := include | substitute
<regular_expression> := <value>; "<joinpoint>"; <atribute_type>; <path>
<atribute_type> := id | name
<advice> := advice <advice_type>; <a_it_expression> {<advice_body>} | <advice> <advice>
<advice_type> := after | before | around
<a_it_expression> := <pointcut_id> |
    not <pointcut_id> |
    <a_it_expression> and <a_it_expression> |
    <a_it_expression> or <a_it_expression>
<intertype_declaration> := intertype <intertype_type>; <a_it_expression>
    {<intertype_declaration_body>} |
    <intertype_declaration> <intertype_declaration>
<intertype_type> := attribute | element
<advice_body> := <joinpoint> | <advice_body>; <advice_body>
<intertype_declaration_body> := new_element_type {<new_element_type>} |
    <joinpoint> |
    <intertype_declaration_body>; <intertype_declaration_body>
  
```

```

<new_element_type> := <name>=<value> | <new_element_type>; <new_element_type>
<text> := <letter> | <digit>
<name> := <text>
<path> :=<text>
<task_id> := <text>
<softgoal_id> := <text>
<goal_id> := <text>
<pointcut_id> := <text>

```

DTD de AOV-graph

```

<!ELEMENT aspect_oriented_model (goal_model)*>
<!ELEMENT goal_model ((softgoal | goal | task | softgoal_ref | goal_ref | task_ref)*, (correlation_relationship |
crosscutting_relationship)*)>
<!ATTLIST goal_model
  id ID #REQUIRED
  name CDATA #REQUIRED
>

<!ELEMENT softgoal ( type*, topic*, (softgoal | task | softgoal_ref | task_ref | new_element_type)*,
(correlation_relationship | crosscutting_relationship)*)>
<!ATTLIST softgoal
  id ID #REQUIRED
  name CDATA #REQUIRED
  decomposition_label (and | or | break | hurt | unknown | help | make) "and"
>

<!ELEMENT goal ( type*, topic*, (goal | task | goal_ref | task_ref | new_element_type)*, (correlation_relationship |
crosscutting_relationship)*)>
<!ATTLIST goal
  id ID #REQUIRED
  name CDATA #REQUIRED
  decomposition_label (and | or | break | hurt | unknown | help | make) "and"
>

<!ELEMENT task (type*, topic*, ( task | task_ref | new_element_type)*, (correlation_relationship |
crosscutting_relationship)*)>
<!ATTLIST task
  id ID #REQUIRED
  name CDATA #REQUIRED
  decomposition_label (and | or | break | hurt | unknown | help | make) "and"
>

<!ELEMENT topic (#PCDATA)>
<!ELEMENT type (#PCDATA)>
<!ELEMENT correlation_relationship (source, target)>
<!ATTLIST correlation_relationship
  label (break | hurt | unknown | help | make) "help"
>

<!ELEMENT target (softgoal_ref | goal_ref | task_ref)>
<!ELEMENT source (softgoal_ref | goal_ref | task_ref)>
<!ELEMENT softgoal_ref EMPTY>
<!ELEMENT goal_ref EMPTY>
<!ELEMENT task_ref EMPTY>
<!ATTLIST softgoal_ref
  id IDREF #REQUIRED
  decomposition_label (and | or | break | hurt | unknown | help | make) "and"
>
<!ATTLIST goal_ref
  id IDREF #REQUIRED
  decomposition_label (and | or | break | hurt | unknown | help | make) "and"
>
<!ATTLIST task_ref
  id IDREF #REQUIRED
  decomposition_label (and | or | break | hurt | unknown | help | make) "and"
>

<!ELEMENT crosscutting_relationship (source, (pointcut | advice | intertype_declaration)*)>
<!ATTLIST crosscutting_relationship
  label (cross) "cross"
>

<!ELEMENT pointcut (pointcut_expression)>
<!ATTLIST pointcut
  id ID #REQUIRED

```

```

    name CDATA #REQUIRED
  >
  <![ELEMENT pointcut_expression (operand | ((operand | pointcut_expression), (and | or), (operand | pointcut_expression)) |
(not, (operand | pointcut_expression)))>
  <![ELEMENT and EMPTY>
  <![ELEMENT or EMPTY>
  <![ELEMENT not EMPTY>
  <![ELEMENT operand (softgoal_ref | goal_ref | task_ref | regular_expression)>
  <![ATTLIST operand
    primitive (include | substitute) "include"
  >

  <![ELEMENT regular_expression (target*)>
  <![ATTLIST regular_expression
    text CDATA #REQUIRED
    type (goal | sotgoal | task | any) "any"
    param (id | name) "name"
    path CDATA
  >

  <![ELEMENT advice (a_it_expression, advice_body)>
  <![ATTLIST advice
    type (after | before | around) "after"
  >

  <![ELEMENT a_it_expression (pointcut_ref | ((pointcut_ref | a_it_expression), (and | or), (pointcut_ref | a_it_expression)) |
(not, (pointcut_ref | a_it_expression)))>
  <![ELEMENT pointcut_ref EMPTY>
  <![ATTLIST pointcut_ref
    id IDREF #REQUIRED
  >

  <![ELEMENT intertype_declaration (a_it_expression, intertype_declaration_body)>
  <![ATTLIST intertype_declaration
    type (attribute | element) "attribute"
  >

  <![ELEMENT advice_body (softgoal_ref | goal_ref | task_ref)*>
  <![ELEMENT intertype_declaration_body (new_element_type | softgoal | goal | task | softgoal_ref | goal_ref | task_ref)*>
  <![ELEMENT new_element_type (new_element_type)*>
  <![ATTLIST new_element_type
    name CDATA #REQUIRED
    value CDATA #REQUIRED
  >

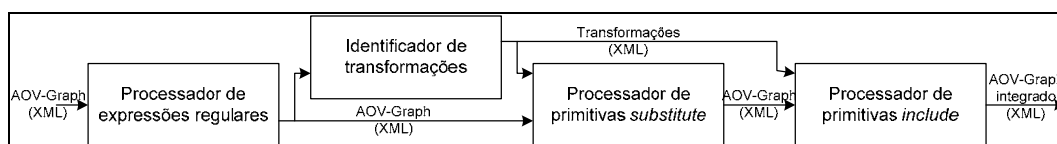
```

Apêndice B

Implementação dos Mecanismos de Composição e Visualização

Mecanismo de Composição

Como apresentado no Capítulo 4, o mecanismo de composição é composto por quatro componentes: processamento de expressões regulares, identificação de transformações, composição da primitiva *substitute* e composição da primitiva *include*, veja a seguir.



Componentes do mecanismo de composição

O processador de expressões regulares é responsável por criar os tipos e tópicos de todas as metas, *softmetas* e tarefas e por criar elementos target em *pointcuts* definidos por expressões regulares. Este processamento resulta em um arquivo XML que também está de acordo com a DTD definida anteriormente. Este componente exige a versão 2.0 de XSLT, pois esta inclui o tratamento de expressões regulares.

O identificador de transformações é responsável por ler os relacionamentos transversais e gerar um arquivo XML contendo onde e o que deve ser adicionado em cada *pointcut* segundo a ordem em que estas transformações devem ser aplicadas.

O processador dos elementos a serem substituídos recebe os dois arquivos XMLs anteriores e apaga todos os elementos (e seus filhos) indicados pelos *pointcuts* com a primitiva *substitute*.

O processador dos elementos a serem incluídos também recebe os dois arquivos XMLs anteriores e inclui todos os elementos definidos em *advice* e *intertype declarations* no local indicado pelos *pointcuts* com ambas as primitivas. A implementação destes componentes está a seguir.

Composição parte 1

```

<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="2.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="xml" indent="yes"/>
  <xsl:template match="/">
    <xsl:apply-templates select="aspect_oriented_model"/>
  </xsl:template>

  <xsl:template match="aspect_oriented_model">
    <aspect_oriented_model>
      <xsl:apply-templates select="goal_model"/>
    </aspect_oriented_model>
  </xsl:template>

  <xsl:template match="goal_model">
    <xsl:element name="{name()}">
      <xsl:for-each select="@*">
        <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
      </xsl:for-each>
      <xsl:apply-templates select="goal | softgoal | task | correlation_relationship | crosscutting_relationship"/>
    </xsl:element>
  </xsl:template>

  <xsl:template match="goal | task | softgoal">
    <xsl:element name="{name()}">
      <xsl:for-each select="@*">
        <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
      </xsl:for-each>
      <xsl:call-template name="make_type"><xsl:with-param name="element_name" select="@name"/></xsl:call-template>
      <xsl:call-template name="make_topic"><xsl:with-param name="element_name"
select="@name"/></xsl:call-template>
      <xsl:apply-templates select="goal | softgoal | task | goal_ref | softgoal_ref | task_ref"/>
      <xsl:apply-templates select="correlation_relationship | crosscutting_relationship | new_element_type"/>
    </xsl:element>
  </xsl:template>

  <xsl:template name="make_topic">
    <xsl:param name="element_name"/>
    <xsl:variable name="regular_exp" select="$element_name"/>
    <xsl:analyze-string select="$regular_exp" regex=".*\{([\^\\[])*\}.*">
      <xsl:matching-substring>
        <xsl:element name="topic"><xsl:value-of select="regex-group(1)"/></xsl:element>
      </xsl:matching-substring>
    </xsl:analyze-string>
  </xsl:template>

  <xsl:template name="make_type">
    <xsl:param name="element_name"/>
    <xsl:if test="substring-before($element_name, '[')!="">
      <xsl:element name="type"><xsl:value-of select="substring-before($element_name, '[')"/></xsl:element>
    </xsl:if>
  </xsl:template>

  <xsl:template match="name | or | and | not | new_element_type | correlation_relationship | advice |
intertype_declaration | source | goal_ref | softgoal_ref | task_ref">
    <xsl:copy-of select="."/>
  </xsl:template>

  <xsl:template match="name" mode="regular_expression">
    <xsl:value-of select="text()"/>
  </xsl:template>

  <xsl:template match="crosscutting_relationship">
    <xsl:element name="{name()}">
      <xsl:for-each select="@*">
        <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
      </xsl:for-each>
      <xsl:apply-templates select="source | pointcut | advice | intertype_declaration"/>
    </xsl:element>
  </xsl:template>

```



```

<xsl:template name="make_target">
  <xsl:variable name="operands" select="//operand"/>
  <xsl:for-each select="$operands">
    <xsl:choose>
      <xsl:when test="count(goal_ref | softgoal_ref | task_ref)>0">
        <xsl:element name="target"><xsl:copy-of select="goal_ref | softgoal_ref | task_ref"/></xsl:element>
      </xsl:when>
      <xsl:otherwise>
        <xsl:for-each select="//target"><xsl:element name="target"><xsl:copy-of
select="."/></xsl:element></xsl:for-each>
      </xsl:otherwise>
    </xsl:choose>
  </xsl:for-each>
</xsl:template>

<xsl:template match="pointcut">
  <xsl:element name="{name()}">
    <xsl:for-each select="@*">
      <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
    </xsl:for-each>
    <xsl:apply-templates select="pointcut_expression"/>
  </xsl:element>
</xsl:template>

<xsl:template match="pointcut_expression">
  <xsl:element name="{name()}">
    <xsl:for-each select="@*">
      <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
    </xsl:for-each>
    <xsl:apply-templates select="operand | pointcut_expression | or | and | not"/>
  </xsl:element>
</xsl:template>

<xsl:template match="operand">
  <xsl:element name="{name()}">
    <xsl:for-each select="@*">
      <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
    </xsl:for-each>
    <xsl:apply-templates select="goal_ref | softgoal_ref | task_ref | regular_expression"/>
  </xsl:element>
</xsl:template>

<xsl:template match="regular_expression">
  <xsl:element name="{name()}">
    <xsl:for-each select="@*">
      <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
    </xsl:for-each>
    <xsl:call-template name="choose_search_in"/>
  </xsl:element>
</xsl:template>

<xsl:template name="make_target_from_regular_expression">
  <xsl:param name="search_in"/>
  <xsl:variable name="regular_exp" select="@text"/>
  <xsl:for-each select="$search_in">
    <xsl:variable name="father" select=".."/>
    <xsl:variable name="id_father" select="$father/@id"/>
    <xsl:variable name="type" select="concat($father/name(),'_ref')"/>
    <xsl:choose>
      <xsl:when test="matches(.,$regular_exp)">
        <xsl:element name="target"><xsl:element name="{ $type }">
          <xsl:attribute name="id"><xsl:value-of select="$id_father"/></xsl:attribute>
        </xsl:element></xsl:element>
      </xsl:when>
    </xsl:choose>
  </xsl:for-each>
</xsl:template>

<xsl:template name="choose_search_in">
  <xsl:variable name="search_type" select="@type"/>
  <xsl:variable name="search_param" select="@param"/>
  <xsl:choose>
    <xsl:when test="$search_type='goal' and $search_param='id'"><xsl:call-template
name="make_target_from_regular_expression">
      <xsl:with-param name="search_in" select="//goal/@id"/></xsl:call-template></xsl:when>

```

```

        <xsl:when test="$search_type='goal' and $search_param='name'"><xsl:call-template
name="make_target_from_regular_expression">
    <xsl:with-param name="search_in" select="//goal/@name"/></xsl:call-template></xsl:when>
    <xsl:when test="$search_type='softgoal' and $search_param='id'"><xsl:call-template
name="make_target_from_regular_expression">
    <xsl:with-param name="search_in" select="//softgoal/@id"/></xsl:call-template></xsl:when>
    <xsl:when test="$search_type='softgoal' and $search_param='name'"><xsl:call-template
name="make_target_from_regular_expression">
    <xsl:with-param name="search_in" select="//softgoal/@name"/></xsl:call-template></xsl:when>
    <xsl:when test="$search_type='task' and $search_param='id'"><xsl:call-template
name="make_target_from_regular_expression">
    <xsl:with-param name="search_in" select="//task/@id"/></xsl:call-template></xsl:when>
    <xsl:when test="$search_type='task' and $search_param='name'"><xsl:call-template
name="make_target_from_regular_expression">
    <xsl:with-param name="search_in" select="//task/@name"/></xsl:call-template></xsl:when>
    <xsl:when test="$search_type='any' and $search_param='id'"><xsl:call-template
name="make_target_from_regular_expression">
    <xsl:with-param name="search_in" select="//goal/@id | //task/@id | //softgoal/@id"/></xsl:call-
template></xsl:when>
    <xsl:otherwise><xsl:call-template name="make_target_from_regular_expression">
    <xsl:with-param name="search_in" select="//goal/@name | //task/@name |
//softgoal/@name"/></xsl:call-template></xsl:otherwise>
    <xsl:choose>
    <xsl:template>
</xsl:stylesheet>

```

Composição parte 2

```

<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="xml" indent="yes" />
    <xsl:template match="/">
        <xsl:apply-templates select="aspect_oriented_model"/>
    </xsl:template>

    <!-- Não estou considerando os operadores AND, OR e NOT -->
    <xsl:template match="aspect_oriented_model">
        <xsl:element name="transformations">
            <xsl:apply-templates select="goal_model"/>
        </xsl:element>
    </xsl:template>

    <xsl:template match="goal_model">
        <xsl:apply-templates select="goal | softgoal | task | crosscutting_relationship"/>
    </xsl:template>

    <xsl:template match="goal | softgoal | task">
        <xsl:apply-templates select="goal | softgoal | task | crosscutting_relationship"/>
    </xsl:template>

    <xsl:template match="crosscutting_relationship">
        <xsl:apply-templates select="advice | intertype_declaration"/>
    </xsl:template>

    <xsl:template match="advice">
        <xsl:apply-templates select="a_it_expression" mode="advice"/>
    </xsl:template>

    <xsl:template match="a_it_expression" mode="advice">
        <xsl:variable name="pointcut_ref" select="//pointcut_ref"/>
        <xsl:variable name="father" select=".." />
        <xsl:variable name="grandfather" select="..." />
        <xsl:variable name="source" select="$grandfather/source/goal_ref | $grandfather/source/softgoal_ref |
$grandfather/source/task_ref"/>

        <xsl:for-each select="$pointcut_ref">
            <xsl:variable name="pointcut_ref_actual" select="."/>
            <xsl:variable name="pointcut" select="$grandfather/pointcut[@id=$pointcut_ref_actual/@id]"/>
            <xsl:variable name="operand" select="$pointcut/operand"/>
            <xsl:for-each select="$operand">
                <xsl:variable name="operand_primitive" select="./@primitive"/>
                <xsl:variable name="where" select="//goal_ref | //task_ref | //softgoal_ref"/>

```

```

        <xsl:choose>
          <xsl:when test="$father/@type='after' or $father/@type='before'">
            <xsl:for-each select="$where">
              <xsl:call-template name="make_transformation_table">
                <xsl:with-param name="type" select="advice"/>
                <xsl:with-param name="source" select="$source"/>
                <xsl:with-param name="how" select="$operand_primitive"/>
                <xsl:with-param name="when" select="$father/@type"/>
                <xsl:with-param name="where_inicial" select="."/>
                <xsl:with-param name="where_final" select="."/>
                <xsl:with-param name="what" select="$father/goal_ref | $father/task_ref |
$father/softgoal_ref"/>
              </xsl:call-template>
            </xsl:for-each>
          </xsl:when>
          <xsl:otherwise>
            <xsl:call-template name="make_transformation_table">
              <xsl:with-param name="type" select="advice"/>
              <xsl:with-param name="source" select="$source"/>
              <xsl:with-param name="how" select="$operand_primitive"/>
              <xsl:with-param name="when" select="$father/@type"/>
              <xsl:with-param name="where_inicial" select="$where"/>
              <xsl:with-param name="where_final" select="$where"/>
              <xsl:with-param name="what" select="$father/goal_ref | $father/task_ref |
$father/softgoal_ref"/>
            </xsl:call-template>
          </xsl:otherwise>
        </xsl:choose>
      </xsl:for-each>
    </xsl:for-each>
  </xsl:template>

  <xsl:template match="a_it_expression" mode="intertype_declaration">
    <xsl:variable name="father" select="."/>
    <xsl:variable name="pointcut_ref" select="//pointcut_ref"/>
    <xsl:variable name="grandfather" select="."/>
    <xsl:variable name="source" select="$grandfather/source/goal_ref | $grandfather/source/softgoal_ref |
$grandfather/source/task_ref"/>

    <xsl:for-each select="$pointcut_ref">
      <xsl:variable name="pointcut_ref_actual" select="."/>
      <xsl:variable name="pointcut" select="$grandfather/pointcut[@id=$pointcut_ref_actual/@id]"/>
      <xsl:variable name="operand" select="$pointcut/operand"/>
      <xsl:for-each select="$operand">
        <xsl:variable name="operand_primitive" select="./@primitive"/>
        <xsl:call-template name="make_transformation_table">
          <xsl:with-param name="type" select="intertype_declaration"/>
          <xsl:with-param name="source" select="$source"/>
          <xsl:with-param name="how" select="$operand_primitive"/>
          <xsl:with-param name="when" select="$father/@type"/>
          <xsl:with-param name="where_inicial" select="//goal_ref | ./task_ref | ./softgoal_ref"/>
          <xsl:with-param name="where_final" select="//goal_ref | ./task_ref | ./softgoal_ref"/>
          <xsl:with-param name="what" select="$father/goal | $father/task | $father/softgoal |
$father/new_element_type"/>
        </xsl:call-template>
      </xsl:for-each>
    </xsl:for-each>
  </xsl:template>

  <xsl:template match="intertype_declaration">
    <xsl:apply-templates select="a_it_expression" mode="intertype_declaration"/>
  </xsl:template>

  <xsl:template name="make_transformation_table">
    <xsl:param name="type"/>
    <xsl:param name="source"/>
    <xsl:param name="how"/>
    <xsl:param name="when"/>
    <xsl:param name="where_inicial"/>
    <xsl:param name="where_final"/>
    <xsl:param name="what"/>

    <xsl:choose>
      <xsl:when test="$when='element' and $how='include'">
        <xsl:element name="transformation">

```

```

        <xsl:attribute name="type"><xsl:value-of select="$type"/></xsl:attribute>
        <xsl:element name="how"><xsl:value-of select="$show"/></xsl:element>
        <xsl:element name="when"><xsl:value-of select="$when"/></xsl:element>
        <xsl:element name="where_inicial"><xsl:copy-of select="$where_inicial"/></xsl:element>
        <xsl:element name="where_final"><xsl:copy-of select="$source"/></xsl:element>
        <xsl:element name="what"><xsl:copy-of select="$what"/></xsl:element>
      </xsl:element>
    </xsl:when>
    <xsl:when test="$when='element' and $show='substitute'">
      <xsl:element name="transformation">
        <xsl:attribute name="type"><xsl:value-of select="$type"/></xsl:attribute>
        <xsl:element name="how"><xsl:value-of select="include"/></xsl:element>
        <xsl:element name="when"><xsl:value-of select="$when"/></xsl:element>
        <xsl:element name="where_inicial"><xsl:copy-of select="$where_inicial"/></xsl:element>
        <xsl:element name="where_final"><xsl:copy-of select="$source"/></xsl:element>
        <xsl:element name="what"><xsl:copy-of select="$what"/></xsl:element>
      </xsl:element>
    </xsl:when>
  </xsl:choose>
  <xsl:if test="$show='substitute'"> <!-- este elemento servirá para composition_part3.xml apagar todos os elementos
substituídos -->
    <xsl:element name="transformation">
      <xsl:attribute name="type"><xsl:value-of select="$type"/></xsl:attribute>
      <xsl:element name="how"><xsl:value-of select="delete"/></xsl:element>
      <xsl:element name="when"><xsl:value-of select="$when"/></xsl:element>
      <xsl:element name="where_inicial"><xsl:copy-of select="$where_inicial"/></xsl:element>
      <xsl:element name="where_final"><xsl:copy-of select="$where_final"/></xsl:element>
      <xsl:element name="what"><xsl:copy-of select="$what"/></xsl:element>
    </xsl:element>
  </xsl:if>

  <xsl:element name="transformation">
    <xsl:attribute name="type"><xsl:value-of select="$type"/></xsl:attribute>
    <xsl:element name="how"><xsl:value-of select="$show"/></xsl:element>
    <xsl:element name="when"><xsl:value-of select="$when"/></xsl:element>
    <xsl:element name="where_inicial"><xsl:copy-of select="$where_inicial"/></xsl:element>

    <xsl:element name="where_final">
      <xsl:choose>

        <xsl:when test="($when='before' or $when='after') and $show='include'">
          <xsl:variable name="Y" select="//goal[@id=$where_inicial/@id] |
//softgoal[@id=$where_inicial/@id] | //task[@id=$where_inicial/@id]"/>
          <xsl:for-each select="$Y">
            <xsl:variable name="X" select=".." />
            <xsl:for-each select="$X">
              <xsl:variable name="element_name" select="concat(name(), '_ref')"/>
              <xsl:element name="{ $element_name }">
                <xsl:attribute name="id"><xsl:value-of select="@id"/></xsl:attribute>
                <xsl:attribute name="decomposition_label"><xsl:value-of
select="@decomposition_label"/></xsl:attribute>
              </xsl:element>
            </xsl:for-each></xsl:for-each>
          </xsl:when>

          <xsl:when test="$when='around' and $show='include'">
            <xsl:copy-of select="$where_final"/>
          </xsl:when>

          <xsl:when test="$type='advice' and $show='substitute'">
            <xsl:variable name="Y" select="//goal[@id=$where_inicial/@id] |
//softgoal[@id=$where_inicial/@id] | //task[@id=$where_inicial/@id]"/>
            <xsl:for-each select="$Y">
              <xsl:variable name="X" select=".." />
              <xsl:for-each select="$X">
                <xsl:variable name="element_name" select="concat(name(), '_ref')"/>
                <xsl:element name="{ $element_name }">
                  <xsl:attribute name="id"><xsl:value-of select="@id"/></xsl:attribute>
                  <xsl:attribute name="decomposition_label"><xsl:value-of
select="@decomposition_label"/></xsl:attribute>
                </xsl:element>
              </xsl:for-each></xsl:for-each>
            </xsl:when>

            <xsl:when test="$when='attribute' and $show='include'">
              <xsl:copy-of select="$where_final"/>
            </xsl:when>
          </xsl:choose>
        </xsl:element>
      </xsl:element>
    </xsl:element>
  </xsl:if>

```

```

</xsl:when>

<xsl:when test="$when='element' and $show='include'">
  <xsl:copy-of select="$where_final"/>
</xsl:when>

<xsl:when test="$when='attribute' and $show='substitute'">
  <xsl:copy-of select="$where_final"/>
</xsl:when>

<xsl:when test="$when='element' and $show='substitute'">
  <!-- estes for-eachs servem apenas para eu conseguir levar o ponteiro para o elemento que eu quero-->
  <xsl:variable name="Y" select="//goal[@id=$where_inicial/@id] |
//softgoal[@id=$where_inicial/@id] | //task[@id=$where_inicial/@id]"/>
  <xsl:for-each select="$Y">
    <xsl:variable name="X" select=".."/>
    <xsl:for-each select="$X">
      <xsl:variable name="element_name" select="concat(name(), '_ref')"/>
      <xsl:element name="{ $element_name }">
        <xsl:attribute name="id"><xsl:value-of select="@id"/></xsl:attribute>
        <xsl:attribute name="decomposition_label"><xsl:value-of
select="@decomposition_label"/></xsl:attribute>
      </xsl:element>
    </xsl:for-each></xsl:for-each>
  </xsl:when>

</xsl:choose>
</xsl:element>

<xsl:choose>
  <xsl:when test="$when='element'">
    <xsl:element name="what">
      <xsl:for-each select="$what">
        <xsl:variable name="element_name" select="concat(name(), '_ref')"/>
        <xsl:element name="{ $element_name }">
          <xsl:attribute name="id"><xsl:value-of select="@id"/></xsl:attribute>
          <xsl:attribute name="decomposition_label"><xsl:value-of
select="@decomposition_label"/></xsl:attribute>
        </xsl:element>
      </xsl:for-each>
    </xsl:element>
  </xsl:when>
  <xsl:otherwise><xsl:element name="what"><xsl:copy-of
select="$what"/></xsl:element></xsl:otherwise>
  </xsl:choose>
</xsl:element>
</xsl:template>

</xsl:stylesheet>

```

Composição parte 3

```

<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="xml" indent="yes"/>
  <xsl:template match="/">
    <xsl:apply-templates select="aspect_oriented_model"/>
  </xsl:template>

  <xsl:template match="aspect_oriented_model">
    <aspect_oriented_model>
      <xsl:apply-templates select="goal_model"/>
    </aspect_oriented_model>
  </xsl:template>

  <xsl:template match="goal_model">
    <xsl:element name="{name()}">
      <xsl:for-each select="@*">
        <xsl:attribute name="{name()}"><xsl:value-of select="."/></xsl:attribute>
      </xsl:for-each>
      <xsl:apply-templates select="goal | softgoal | task | softgoal_ref | goal_ref | task_ref | correlation_relationship |
crosscutting_relationship"/>
    </xsl:element>
  </xsl:template>

```

```

</xsl:template>

<xsl:template match="goal | task | softgoal | softgoal_ref | goal_ref | task_ref">

    <xsl:variable name="actual_element" select="./@id"/>
    <xsl:variable name="filename" select="output_part2.xml"/>
    <xsl:variable name="all_transformation" select="document($filename)//transformation"/>
    <xsl:variable name="transformations" select="$all_transformation[where_final/goal_ref/@id=$actual_element] |
$all_transformation[where_final/softgoal_ref/@id=$actual_element] |
$all_transformation[where_final/task_ref/@id=$actual_element]"/>

    <xsl:variable name="transformations_delete" select="$transformations/how[text()='delete']"/>

    <xsl:if test="count($transformations_delete)=0 or
count($transformations_delete)=count($transformations_delete/when[text()='attribute'])">
        <xsl:element name="{name()}">
            <xsl:for-each select="@*">
                <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
            </xsl:for-each>
            <xsl:apply-templates select="topic | type | goal | softgoal | task | softgoal_ref | goal_ref | task_ref |
correlation_relationship | crosscutting_relationship | new_element_type"/>
        </xsl:element>
    </xsl:if>
</xsl:template>

<xsl:template match="topic | type | crosscutting_relationship">
    <xsl:copy-of select="."/>
</xsl:template>

<xsl:template match="correlation_relationship">
    <xsl:variable name="source" select="./@id"/>
    <xsl:variable name="target" select="./@id"/>
    <xsl:variable name="filename" select="output_part2.xml"/>
    <xsl:variable name="all_transformation" select="document($filename)//transformation"/>
    <xsl:variable name="transformations" select="$all_transformation[where_final/@id=$source] |
$all_transformation[where_final/@id=$target]"/>

    <xsl:variable name="transformations_delete" select="$transformations/how[text()='delete']"/>
    <xsl:if test="count($transformations_delete)=0">
        <xsl:copy-of select="."/>
    </xsl:if>
</xsl:template>

<xsl:template match="new_element_type">
    <xsl:variable name="name" select="./@name"/>
    <xsl:variable name="value" select="./@value"/>
    <xsl:variable name="filename" select="output_part2.xml"/>
    <xsl:variable name="all_transformation" select="document($filename)//transformation"/>
    <xsl:variable name="transformations_delete" select="$all_transformation/how[text()='delete']"/>

    <xsl:variable name="transformations" select="$transformations_delete/what[@name=$name and
@value=$value]"/>
    <xsl:if test="count($transformations)=0">
        <xsl:copy-of select="."/>
    </xsl:if>
</xsl:template>
</xsl:stylesheet>

```

Composição parte 4

```

<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="xml" indent="yes" />
    <xsl:template match="/">
        <xsl:apply-templates select="aspect_oriented_model"/>
    </xsl:template>

    <xsl:template match="aspect_oriented_model">
        <aspect_oriented_model>
            <xsl:apply-templates select="goal_model"/>
        </aspect_oriented_model>
    </xsl:template>

    <xsl:template match="goal_model">

```

```

        <xsl:element name="{name()}">
            <xsl:for-each select="@*">
                <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
            </xsl:for-each>
            <xsl:apply-templates select="goal | softgoal | task | softgoal_ref | goal_ref | task_ref | correlation_relationship |
crosscutting_relationship"/>
        </xsl:element>
    </xsl:template>

    <xsl:template match="goal | task | softgoal">
        <xsl:variable name="actual_element" select="./@id"/>
        <xsl:variable name="filename" select="'output_part2.xml'"/>
        <xsl:variable name="all_transformation" select="document($filename)//transformation"/>
        <xsl:variable name="transformations_inicial"
select="$all_transformation[where_inicial//@id=$actual_element]"/>
        <xsl:variable name="transformations_final" select="$all_transformation[where_final//@id=$actual_element]"/>
        <xsl:variable name="transformations_include_inicial" select="$transformations_inicial[how/text()='include']"/>

        <xsl:variable name="transformations_advice" select="$transformations_include_inicial[@type='advice']"/>
        <xsl:variable name="transformations_before" select="$transformations_advice[when/text()='before']"/>
        <xsl:variable name="transformations_after" select="$transformations_advice[when/text()='after']"/>

        <xsl:for-each select="$transformations_before"> <!-- include advice before -->
            <xsl:copy-of select="./what/goal_ref | ./what/softgoal_ref | ./what/task_ref"/>
        </xsl:for-each>

        <xsl:element name="{name()}">
            <xsl:for-each select="@*">
                <xsl:attribute name="{name()}"> <xsl:value-of select="."/></xsl:attribute>
            </xsl:for-each>
            <xsl:apply-templates select="topic | type"/>
            <xsl:apply-templates select="goal | softgoal | task | softgoal_ref | goal_ref | task_ref"/>
            <xsl:for-each select="$transformations_final">
                <xsl:copy-of select="./what/goal_ref | ./what/softgoal_ref | ./what/task_ref | ./what/goal | ./what/softgoal |
./what/task | ./what/new_element_type"/>
            </xsl:for-each>
            <xsl:apply-templates select="correlation_relationship | crosscutting_relationship | new_element_type"/>
        </xsl:element>

        <xsl:for-each select="$transformations_after">
            <xsl:copy-of select="./what/goal_ref | ./what/softgoal_ref | ./what/task_ref"/>
        </xsl:for-each>

    </xsl:template>

<!-- esta função não está sendo utilizada pq eu troquei pelo FOR anterior ao FOR do Include_advice_after-->
    <xsl:template name="process_transformation_table">
        <xsl:variable name="actual_element" select="./@id"/>
        <xsl:variable name="filename" select="./xml/Output_part2.xml"/>
        <xsl:for-each select="document($filename)//transformation/where_final/goal_ref[@id=$actual_element] |
document($filename)//transformation/where_final/softgoal_ref[@id=$actual_element] |
document($filename)//transformation/where_final/task_ref[@id=$actual_element]">
            <xsl:variable name="grandfather" select="../.."/>
            <xsl:variable name="element_type" select="$grandfather/@type"/>
            <xsl:variable name="primitive" select="$grandfather/how"/>
            <xsl:variable name="type" select="$grandfather/when"/>
            <xsl:variable name="body" select="$grandfather/what/goal_ref | $grandfather/what/softgoal_ref
|$grandfather/what/task_ref | $grandfather/what/new_element_type | $grandfather/what/goal | $grandfather/what/softgoal
|$grandfather/what/task"/>
            <xsl:choose> <!-- Deixei separado pq pode surgir alguma diferenciação entre as ações a seguir -->
                <xsl:when test="$element_type='intertype_declaration' and $primitive='include'"> <!-- OK -->
                    <xsl:copy-of select="$body"/>
                </xsl:when>
                <xsl:when test="$type='element' and $primitive='substitute'"> <!-- o Delete é feito em
Composition_part3.xml -->
                    <xsl:copy-of select="$body"/>
                </xsl:when>
                <xsl:when test="$type='attribute' and $primitive='substitute'"> <!-- o Delete é feito em
Composition_part3.xml -->
                    <xsl:copy-of select="$body"/>
                </xsl:when>
                <xsl:when test="$type='around' and $primitive='include'"> <!-- OK -->
                    <xsl:copy-of select="$body"/>
                </xsl:when>
                <xsl:when test="$type='lili' and $primitive='include'"> <!-- OK -->
                    <xsl:copy-of select="$body"/>
            </xsl:choose>
        </xsl:for-each>
    </xsl:template>

```

```

</xsl:when>
<xsl:when test="$type='hili' and $primitive='include'"> <!-- OK-->
  <xsl:copy-of select="$body"/>
</xsl:when>
<xsl:when test="$element_type='advice' and $primitive='substitute'"> <!-- o Delete é feito em
Composition_part3.xml -->
  <xsl:copy-of select="$body"/>
</xsl:when>
</xsl:choose>
</xsl:for-each>
</xsl:template>
<xsl:template match="topic | type | softgoal_ref | goal_ref | task_ref | crosscutting_relationship | new_element_type |
correlation_relationship">
  <xsl:copy-of select="."/>
</xsl:template>
</xsl:stylesheet>

```

Mecanismo de Visualização

O mecanismo de visualização gera visões de AOV-graph. Cada visão é gerada por um arquivo XSLT que implementa separadamente as transformações definidas no Capítulo 4. O formato DOT é utilizado juntamente com o software Visigraph (Visigraph, 2006) para gerar figuras (JPG).

AOV-Graph com relacionamentos transversais resumidos

```

<?xml version="1.0" encoding="UTF-8"?> <!-- este arquivo deve ser aplicado ao XML inicial (antes da composiÃ§Ã£o)
para representar o diagrama com os crosscutting relationships resumidos -->
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="text" indent="yes"/>
  <xsl:template match="/">
    <xsl:apply-templates select="aspect_oriented_model"/>
  </xsl:template>

  <xsl:template match="aspect_oriented_model">
    digraph aspect_oriented_goal_model{
      rankdir=RL;
      fontsize=12;
      ranksep=.5;
      nodesep=.10;

      <xsl:apply-templates select="goal_model"/>
      <xsl:call-template name="decomposition_relationship_references"/>
      <xsl:call-template name="correlation_relationship"/>
      <xsl:call-template name="crosscutting_relationship"/>
    }
  </xsl:template>

  <xsl:template match="goal_model">
    subgraph <xsl:value-of select="concat('cluster_',./@id)"/> {
      label = "<xsl:value-of select="."/ @name"/>";
      color = tomato;

      <xsl:apply-templates select="goal | softgoal | task "/>
    }
  </xsl:template>

  <xsl:template match="goal">
    "<xsl:value-of select="@id"/>" [height=.3, label="<xsl:value-of select="@name"/>", shape=octagon];
    <xsl:apply-templates select="goal | softgoal | task"/>
    <xsl:call-template name="decomposition_relationship"/>
  </xsl:template>

  <xsl:template match="task">
    "<xsl:value-of select="@id"/>" [height=.3, label="<xsl:value-of select="@name"/>", shape=hexagon];
    <xsl:apply-templates select="goal | softgoal | task "/>
    <xsl:call-template name="decomposition_relationship"/>
  </xsl:template>

  <xsl:template match="softgoal">

```



```

" <xsl:value-of select="@id"/>" [height=.3, label="<xsl:value-of select="@name"/>",
shapefile="NFRSoftgoal.gif"];
<xsl:apply-templates select="goal | softgoal | task"/>
<xsl:call-template name="decomposition_relationship"/>
</xsl:template>

<xsl:template name="crosscutting_relationship">
<xsl:for-each select="//crosscutting_relationship">
<xsl:variable name="target" select="//operand/goal_ref/@id | //operand/softgoal_ref/@id |
//operand/task_ref/@id"/>
<xsl:variable name="target2" select="//operand/regular_expression/@id"/>
<xsl:variable name="source" select="//source/@id"/>
<xsl:for-each select="$target">
<xsl:variable name="x" select="."/>
"<xsl:value-of select="$source"/>" -&gt; "<xsl:value-of select="$x"/>" [arrowhead=normal, label="cross",
style=bold];
</xsl:for-each>

<!-- <xsl:for-each select="$target2"> Ã© necessÃ¡rio retirar apenas os que se referem a inter-types
<xsl:variable name="x" select="."/>
"<xsl:value-of select="$source"/>" -&gt; "<xsl:value-of select="$x"/>" [arrowhead=normal, label="cross",
color=lightgray];
</xsl:for-each> -->
</xsl:for-each>
</xsl:template>

<xsl:template name="correlation_relationship">
<xsl:for-each select="//correlation_relationship">
"<xsl:value-of select="//source/@id"/>" -&gt; "<xsl:value-of select="//target/@id"/>" [arrowhead=normal,
label="<xsl:value-of select="@label"/>", style=dotted];
</xsl:for-each>
</xsl:template>

<xsl:template name="decomposition_relationship">
<xsl:for-each select="//goal | //task | //softgoal">
"<xsl:value-of select="@id"/>" -&gt; "<xsl:value-of select="//@id"/>" [arrowhead=normal,
label="<xsl:value-of select="@decomposition_label"/>"];
</xsl:for-each>
</xsl:template>

<xsl:template name="decomposition_relationship_references">
<xsl:for-each select="//goal/goal_ref | //goal/task_ref | //goal/softgoal_ref | //task/goal_ref | //task/task_ref |
//task/softgoal_ref | //softgoal/goal_ref | //softgoal/task_ref | //softgoal/softgoal_ref">
"<xsl:value-of select="@id"/>" -&gt; "<xsl:value-of select="//@id"/>" [arrowhead=normal,
label="<xsl:value-of select="@decomposition_label"/>"];
</xsl:for-each>
</xsl:template>
</xsl:stylesheet>

```

AOV-Graph com relacionamentos transversais expandidos

```

<?xml version="1.0" encoding="UTF-8"?> <!-- este arquivo deve ser aplicado ao XML final (depois da composicao) para
representar o diagrama com os crosscutting relationships expandidos -->
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<xsl:output method="text" indent="yes"/>
<xsl:template match="/">
<xsl:apply-templates select="aspect_oriented_model"/>
</xsl:template>

<xsl:template match="aspect_oriented_model">
digraph aspect_oriented_goal_model{
rankdir=RL;
ranksep=.5;
nodesep=.10;
fontsize=12;

<xsl:apply-templates select="goal_model"/>

<xsl:call-template name="decomposition_relationship_references"/>
<xsl:call-template name="correlation_relationship"/>
}
</xsl:template>

<xsl:template match="goal_model">

```

```

        subgraph <xsl:value-of select="concat('cluster_',./@id)"/> {
            label = "<xsl:value-of select='./@name'"/>";
            color = tomato;
            <xsl:for-each select="//softgoal"><xsl:value-of select="@id"/>" [height=.3, label="<xsl:value-of
select="@name"/>", shapefile="NFRSoftgoal.gif"];
            </xsl:for-each>
            <xsl:for-each select="//goal"><xsl:value-of select="@id"/>" [height=.3, label="<xsl:value-of
select="@name"/>", shape=octagon];
            </xsl:for-each>
            <xsl:for-each select="//task"><xsl:value-of select="@id"/>" [height=.3, label="<xsl:value-of
select="@name"/>", shape=hexagon];
            </xsl:for-each>
            <xsl:apply-templates select="goal | softgoal | task "/>
        }
    </xsl:template>

    <xsl:template match="goal">
        <xsl:apply-templates select="goal | softgoal | task"/>
        <xsl:call-template name="decomposition_relationship"/>
    </xsl:template>
    <xsl:template match="task">
        <xsl:apply-templates select="goal | softgoal | task "/>
        <xsl:call-template name="decomposition_relationship"/>
    </xsl:template>
    <xsl:template match="softgoal">
        <xsl:apply-templates select="goal | softgoal | task "/>
        <xsl:call-template name="decomposition_relationship"/>
    </xsl:template>

    <xsl:template name="crosscutting_relationship">
        <xsl:for-each select="//crosscutting_relationship">
            <xsl:variable name="target" select="//operand/goal_ref/@id | //operand/softgoal_ref/@id |
//operand/task_ref/@id"/>
            <xsl:variable name="target2" select="//operand/regular_expression//@id"/>
            <xsl:variable name="source" select="//source//@id"/>
            <xsl:for-each select="$target">
                <xsl:variable name="x" select="."/>
                "<xsl:value-of select="$source"/>" -&gt; "<xsl:value-of select="$x"/>" [arrowhead=normal, label="cross",
style=bold];
            </xsl:for-each>

            <xsl:for-each select="$target2">
                <xsl:variable name="x" select="."/>
                "<xsl:value-of select="$source"/>" -&gt; "<xsl:value-of select="$x"/>" [arrowhead=normal, label="cross",
color=lightgray];
            </xsl:for-each>
        </xsl:for-each>
    </xsl:template>

    <xsl:template name="correlation_relationship">
        <xsl:for-each select="//correlation_relationship">
            "<xsl:value-of select="//source//@id"/>" -&gt; "<xsl:value-of select="//target//@id"/>" [arrowhead=normal,
label="<xsl:value-of select="@label"/>", style=dotted];
        </xsl:for-each>
    </xsl:template>

    <xsl:template name="decomposition_relationship">
        <xsl:for-each select="//goal | //task | //softgoal">
            "<xsl:value-of select="@id"/>" -&gt; "<xsl:value-of select="//@id"/>" [arrowhead=normal,
label="<xsl:value-of select="@decomposition_label"/>"];
        </xsl:for-each>
    </xsl:template>

    <xsl:template name="decomposition_relationship_references">
        <xsl:for-each select="//goal/goal_ref | //goal/task_ref | //goal/softgoal_ref | //task/goal_ref | //task/task_ref |
//task/softgoal_ref | //softgoal/goal_ref | //softgoal/task_ref | //softgoal/softgoal_ref">
            "<xsl:value-of select="@id"/>" -&gt; "<xsl:value-of select="//@id"/>" [arrowhead=normal, style=bold,
label="<xsl:value-of select="@decomposition_label"/>"];
        </xsl:for-each>
    </xsl:template>
</xsl:stylesheet>

```

Diagrama de classes

```

<?xml version="1.0" encoding="UTF-8"?>
<stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="xml" indent="yes" />
  <xsl:template match="/">
    <xsl:apply-templates select="aspect_oriented_model"/>
  </xsl:template>

  <xsl:template match="aspect_oriented_model">
    <xsl:element name="entities">
      <xsl:apply-templates select="goal_model"/>
    </xsl:element>
  </xsl:template>

  <xsl:key name="topics" match="topic" use="text()"/>

  <xsl:template name="is_class">
    <xsl:param name="topic"/>
    <xsl:variable name="son" select="//goal[topic=$topic]/goal | //softgoal[topic=$topic]/goal |
//task[topic=$topic]/goal | //goal[topic=$topic]/softgoal | //softgoal[topic=$topic]/softgoal | //task[topic=$topic]/softgoal |
//goal[topic=$topic]/task | //softgoal[topic=$topic]/task | //task[topic=$topic]/task | //task[topic=$topic]/new_element_type |
//goal[topic=$topic]/new_element_type | //softgoal[topic=$topic]/new_element_type"/>
    <xsl:choose>
      <xsl:when test="count($son)>0"><xsl:value-of select="true"/></xsl:when>
      <xsl:otherwise><xsl:value-of select="false"/></xsl:otherwise>
    </xsl:choose>
  </xsl:template>

  <xsl:template name="is_class_ref">
    <xsl:param name="topic"/>
    <xsl:variable name="son" select="//goal[topic=$topic]/goal | //softgoal[topic=$topic]/goal |
//task[topic=$topic]/goal | //goal[topic=$topic]/softgoal | //softgoal[topic=$topic]/softgoal | //task[topic=$topic]/softgoal |
//goal[topic=$topic]/task | //softgoal[topic=$topic]/task | //task[topic=$topic]/task | //goal[topic=$topic]/goal_ref |
//softgoal[topic=$topic]/goal_ref | //task[topic=$topic]/goal_ref | //goal[topic=$topic]/softgoal_ref |
//softgoal[topic=$topic]/softgoal_ref | //task[topic=$topic]/softgoal_ref | //goal[topic=$topic]/task_ref |
//softgoal[topic=$topic]/task_ref | //task[topic=$topic]/task_ref | //task[topic=$topic]/new_element_type |
//goal[topic=$topic]/new_element_type | //softgoal[topic=$topic]/new_element_type"/>
    <xsl:choose>
      <xsl:when test="count($son)>0"><xsl:value-of select="true"/></xsl:when>
      <xsl:otherwise><xsl:value-of select="false"/></xsl:otherwise>
    </xsl:choose>
  </xsl:template>

  <xsl:template match="goal_model">
    <xsl:element name="goal_model">
      <xsl:attribute name="name"><xsl:value-of select="@name"/></xsl:attribute>
      <xsl:apply-templates select="goal | softgoal | task | goal_ref | softgoal_ref | task_ref | crosscutting_relationship"/>
    </xsl:element>
  </xsl:template>

  <xsl:template match="goal | softgoal | task">
    <xsl:variable name="id_element_actual" select="./@id"/>
    <xsl:for-each select="./topic">
      <xsl:variable name="is_class"><xsl:call-template name="is_class"><xsl:with-param name="topic"
select="."/></xsl:call-template></xsl:variable>
      <xsl:if test="$is_class='true'">
        <xsl:element name="entity">
          <xsl:attribute name="name"><xsl:value-of select="."/></xsl:attribute>
          <xsl:element name="method"><xsl:call-template name="make_element"><xsl:with-param
name="element" select="."/></xsl:call-template></xsl:element>
          <xsl:apply-templates select="./goal | ./softgoal | ./task | ./goal_ref | ./softgoal_ref | ./task_ref |
../crosscutting_relationship | ../new_element_type"/>
          <xsl:call-template name="correlation_relationship"><xsl:with-param name="element"
select="$id_element_actual"/></xsl:call-template>
        </xsl:element>
      </xsl:if>
      <xsl:if test="$is_class!='true'">
        <xsl:element name="attribute">
          <xsl:attribute name="name"><xsl:value-of select="."/></xsl:attribute>
          <xsl:call-template name="correlation_relationship"><xsl:with-param name="element"
select="$id_element_actual"/></xsl:call-template>
        </xsl:element>
        <xsl:element name="method"><xsl:call-template name="make_element"><xsl:with-param name="element"
select="."/></xsl:call-template></xsl:element>
        <xsl:apply-templates select="./goal | ./softgoal | ./task | ./goal_ref | ./softgoal_ref | ./task_ref |
../crosscutting_relationship | ../new_element_type"/>
      </xsl:if>
    </xsl:for-each>
  </xsl:template>

```

```

</xsl:for-each>
<xsl:if test="count(/.topic)=0"><xsl:apply-templates select="goal | softgoal | task | goal_ref | softgoal_ref | task_ref |
crosscutting_relationship"/></xsl:if>
</xsl:template>

<xsl:template match="new_element_type">
  <xsl:variable name="name" select="./@name"/>
  <xsl:variable name="value" select="./@value"/>
  <xsl:element name="attribute">
    <xsl:attribute name="name"><xsl:value-of select="concat($name, ' ', $value)"/></xsl:attribute>
  </xsl:element>
</xsl:template>

<xsl:template match="goal_ref | softgoal_ref | task_ref">
  <xsl:variable name="id_reference_actual" select="./@id"/>
  <xsl:variable name="element_actual" select="//goal[@id=$id_reference_actual]"/>
  <xsl:for-each select="$element_actual">
    <xsl:for-each select="./topic">
      <xsl:variable name="is_class1"><xsl:call-template name="is_class_ref"><xsl:with-param name="topic"
select="."/></xsl:call-template></xsl:variable>
      <xsl:if test="$is_class1=true">
        <xsl:element name="entity">
          <xsl:attribute name="name"><xsl:value-of select="."/></xsl:attribute>
        </xsl:element>
      </xsl:if>
      <xsl:if test="$is_class1!=true">
        <xsl:element name="attribute">
          <xsl:attribute name="name"><xsl:value-of select="."/></xsl:attribute></xsl:element>
        </xsl:if>
      </xsl:if>
    </xsl:for-each>
  </xsl:for-each>
</xsl:template>

<xsl:template name="make_element">
  <xsl:param name="element"/>
  <xsl:for-each select="$element">
    <xsl:element name="{name()}">
      <xsl:for-each select="@*">
        <xsl:attribute name="{name()}"><xsl:value-of select="."/></xsl:attribute>
      </xsl:for-each>
    </xsl:element>
  </xsl:for-each>
</xsl:template>

<xsl:template name="correlation_relationship">
  <xsl:param name="element"/>
  <xsl:variable name="correlations" select="//correlation_relationship[source/goal_ref/@id=$element] |
//correlation_relationship[source/softgoal_ref/@id=$element] |
//correlation_relationship[source/task_ref/@id=$element]"/>

  <xsl:for-each select="$correlations">
    <xsl:variable name="label" select="./@label"/>
    <xsl:variable name="target" select="./target/@id"/>
    <xsl:for-each select="$target">
      <xsl:variable name="target_actual" select="."/>
      <xsl:variable name="element_target" select="//goal[@id=$target_actual] | //softgoal[@id=$target_actual]
| //task[@id=$target_actual]"/>
      <xsl:variable name="parent_element_target" select="//goal[goal/@id=$target_actual] |
//softgoal[goal/@id=$target_actual] | //task[goal/@id=$target_actual] | //goal[task/@id=$target_actual] |
//softgoal[task/@id=$target_actual] | //task[task/@id=$target_actual] | //goal[softgoal/@id=$target_actual] |
//softgoal[softgoal/@id=$target_actual] | //task[softgoal/@id=$target_actual]"/>
      <xsl:for-each select="$element_target">
        <xsl:variable name="element_target_topic" select="./topic"/>
        <xsl:for-each select="$element_target_topic">
          <xsl:variable name="is_class"><xsl:call-template name="is_class"><xsl:with-param
name="topic" select="."/></xsl:call-template></xsl:variable>
          <xsl:choose>
            <xsl:when test="$is_class='true'"><xsl:call-template name="association"><xsl:with-param
name="target" select="."/><xsl:with-param name="label" select="$label"/>
            </xsl:call-template></xsl:when>
            <xsl:otherwise>
              <xsl:variable name="parent_element_target_topic"
select="$parent_element_target/topic"/>
              <xsl:for-each select="$parent_element_target/topic">
                <xsl:variable name="is_class1"><xsl:call-template name="is_class"><xsl:with-
param name="topic" select="."/></xsl:call-template></xsl:variable>

```

```

        <xsl:if test="is_class1='true'"><xsl:call-template name="association"><xsl:with-
param name="target" select="."/><xsl:with-param name="label" select="$label"/></xsl:call-template></xsl:if></xsl:for-
each>

        </xsl:otherwise>
        </xsl:choose>
        </xsl:for-each>
        </xsl:for-each>
        </xsl:for-each>

    </xsl:template>

    <xsl:template match="correlation_relationship1"> <!--esta não é usada-->
        <xsl:variable name="source" select="./source/@id"/>
        <xsl:variable name="target" select="./target/@id"/>
        <xsl:variable name="element_source" select="//goal[@id=$source] | //softgoal[@id=$source] |
//task[@id=$source]"/>
        <xsl:variable name="element_target" select="//goal[@id=$target] | //softgoal[@id=$target] |
//task[@id=$target]"/>
        <xsl:variable name="topic_source" select="$element_source/topic"/>
        <xsl:variable name="topic_target" select="$element_target/topic"/>

        <xsl:variable name="father_source" select="//goal/goal[@id=$source] | //softgoal/goal[@id=$source] |
//task/goal[@id=$source] | //goal/task[@id=$source] | //softgoal/task[@id=$source] | //task/task[@id=$source] |
//goal/softgoal[@id=$source] | //softgoal/softgoal[@id=$source] | //task/softgoal[@id=$source]"/>
        <xsl:variable name="father_target" select="//goal/goal[@id=$target] | //softgoal/goal[@id=$target] |
//task/goal[@id=$target] | //goal/task[@id=$target] | //softgoal/task[@id=$target] | //task/task[@id=$target] |
//goal/softgoal[@id=$target] | //softgoal/softgoal[@id=$target] | //task/softgoal[@id=$target]"/>
        <xsl:variable name="is_class1"><xsl:call-template name="is_class"><xsl:with-param name="topic"
select="$topic_source"/></xsl:call-template></xsl:variable>
        <xsl:variable name="is_class2"><xsl:call-template name="is_class"><xsl:with-param name="topic"
select="$topic_target"/></xsl:call-template></xsl:variable>

        <xsl:choose>
            <xsl:when test="$is_class1='true' and $is_class2='true'"><xsl:call-template name="association"><xsl:with-
param name="source" select="$topic_source"/><xsl:with-param name="target" select="$topic_target"/></xsl:call-
template></xsl:when>
            <xsl:when test="$is_class1='true' and $is_class2!='true'"><xsl:call-template name="association"><xsl:with-
param name="source" select="$topic_source"/><xsl:with-param name="target" select="$father_target/topic"/></xsl:call-
template></xsl:when>
            <xsl:when test="$is_class1!='true' and $is_class2='true'"><xsl:call-template name="association"><xsl:with-
param name="source" select="$father_source/topic"/><xsl:with-param name="target" select="$topic_target"/></xsl:call-
template></xsl:when>
        </xsl:choose>
    </xsl:template>

    <xsl:template name="association"> <!-- Não precisa ver se ambos são classes-->
        <xsl:param name="target"/>
        <xsl:param name="label"/>
        <xsl:element name="association">
            <xsl:attribute name="target"><xsl:value-of select="$target"/></xsl:attribute>
            <xsl:attribute name="label"><xsl:value-of select="$label"/></xsl:attribute>
        </xsl:element>
    </xsl:template>

    <xsl:template match="crosscutting_relationship">
    </xsl:template>
</xsl:stylesheet>

<?xml version="1.0" encoding="UTF-8"?><!-- este arquivo tem que ser aplicado ao XML de saída de de
dataIntermediate.xml-->
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
    <xsl:output method="text" indent="yes"/>

    <xsl:template match="/">
        <xsl:apply-templates select="entities"/>
    </xsl:template>

    <xsl:template match="entities">
        <xsl:text>digraph class_diagram{
rankdir=LR;
node [shape = record,height=1.];
ranksep=.5;

</xsl:text>
        <xsl:apply-templates select="goal_model"/>
    </xsl:template>

```

```

</xsl:template>

<xsl:key name="topics" match="topic" use="text()"/>
<xsl:key name="all_entities" match="entity" use="@name"/>
<xsl:key name="all_attribute" match="attribute" use="@name"/>
<xsl:key name="all_method" match="method/goal | method/softgoal | method/task" use="@name"/>

<xsl:template match="goal_model">
<!--
  <xsl:text>
    subgraph </xsl:text><xsl:value-of select="concat('cluster_',./@name)"/><xsl:text> {
      label = "</xsl:text><xsl:value-of select="."/@name"/>"<xsl:text>;
      color = tomato;</xsl:text>-->

      <xsl:variable name="all_entities_teste" select="."/entity[generate-id(.)=generate-
id(key("all_entities",@name)[1])]/>
      <xsl:variable name="all_entities" select="."/entity[generate-id(.)=generate-id(key("all_entities",@name)[1])]/>
      <xsl:for-each select="$all_entities">
        <xsl:call-template name="create_class"><xsl:with-param name="topic" select="."/></xsl:call-template>
      </xsl:for-each>
      <xsl:apply-templates select="entity"/>
      <xsl:text>
<!--
    }-->
  </xsl:text>
</xsl:template>

<xsl:template match="entity">
  <xsl:variable name="entity_actual" select="."/>
  <xsl:variable name="children" select="."/entity"/>
  <xsl:variable name="children_teste" select="."/entity[generate-id(.)=generate-id(key("all_entities",@name)[1])]/>
  <xsl:variable name="children_teste2" select="$entity_actual[generate-id(.)=generate-
id(key("all_entities",@name)[1])]/>
  <xsl:variable name="children1" select="."/entity[generate-id(.)=generate-id(key("all_entities",@name)[1])]/> <!--
  Não consigo fazer com que ele não repita relacionamentos-->
  <xsl:for-each select="$children1">
    <xsl:call-template name="decomposition"><xsl:with-param name="source"
select="$entity_actual"/><xsl:with-param name="target" select="."/></xsl:call-template>
  </xsl:for-each>
  <xsl:apply-templates select="association | entity"/>

</xsl:template>

<xsl:template match="association">
  <xsl:variable name="entity_actual" select="."/>
  <xsl:variable name="parent" select="."/parent"/>
  <xsl:for-each select="$parent">
    <xsl:text>
      "</xsl:text><xsl:value-of select="."/@name"/><xsl:text>" -&gt; "</xsl:text><xsl:value-of
select="$entity_actual/@target"/>
      <xsl:text>" [arrowhead=normal, style=normal, label="</xsl:text><xsl:value-of
select="$entity_actual/@label"/><xsl:text>"];</xsl:text>
    </xsl:for-each>
  </xsl:template>

<xsl:template name="decomposition">
  <xsl:param name="source"/>
  <xsl:param name="target"/>
  <xsl:text>
    "</xsl:text><xsl:value-of select="$source/@name"/><xsl:text>" -&gt; "</xsl:text><xsl:value-of
select="$target/@name"/>
    <xsl:text>" [arrowhead=odot, style=normal];</xsl:text>
  </xsl:template>

<xsl:template name="create_class">
  <xsl:param name="topic"/>
  <xsl:text>
    "</xsl:text>
  <xsl:value-of select="$topic/@name"/>
  <xsl:text>" [label="</xsl:text>
  <xsl:value-of select="$topic/@name"/><xsl:text> | </xsl:text>
  <xsl:call-template name="create_attribute"><xsl:with-param name="topic" select="$topic"/></xsl:call-
template><xsl:text> | </xsl:text>
  <xsl:call-template name="create_method"><xsl:with-param name="topic" select="$topic"/></xsl:call-
template><xsl:text> ";</xsl:text>
  </xsl:template>

```

```

<xsl:template name="create_attribute">
  <xsl:param name="topic"/>
  <xsl:variable name="son" select="//entity[@name=$topic/@name]/attribute"/>
  <xsl:variable name="all_attributes" select="$son[generate-id(.)=generate-id(key("all_attribute",@name)[1])]/>

  <xsl:for-each select="$all_attributes">
    <xsl:value-of select="."/><xsl:text> \n </xsl:text>
  </xsl:for-each>
</xsl:template>

<xsl:template name="create_method">
  <xsl:param name="topic"/>
  <xsl:variable name="son" select="//entity[@name=$topic/@name]/method/goal |
//entity[@name=$topic/@name]/method/task | //entity[@name=$topic/@name]/method/softgoal"/>
  <xsl:variable name="all_methods" select="$son[generate-id(.)=generate-id(key("all_method",@name)[1])]/>
  <xsl:for-each select="$all_methods">
    <xsl:choose>
      <xsl:when test="name()='goal'">G-<xsl:value-of select="."/> \n </xsl:when>
      <xsl:when test="name()='softgoal'">SG-<xsl:value-of select="."/> \n </xsl:when>
      <xsl:when test="name()='task'">T-<xsl:value-of select="."/> \n </xsl:when>
    </xsl:choose>
  </xsl:for-each>
</xsl:template>
</xsl:stylesheet>

```

Modelo entidade-relacionamento

```

<?xml version="1.0" encoding="UTF-8"?><!-- este arquivo tem que ser aplicado ao XML de saÃ-de de
dataIntermediate.xml-->
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="text" indent="yes"/>

  <xsl:template match="/">
    <xsl:apply-templates select="entities"/>
  </xsl:template>

  <xsl:template match="entities">
    <xsl:text>digraph class_diagram{
rankdir=LR;
node [shape = record,height=.1];

</xsl:text>
    <xsl:apply-templates select="goal_model"/>
  </xsl:template>

  <xsl:key name="topics" match="topic" use="text()"/>
  <xsl:key name="all_entities" match="entity" use="@name"/>
  <xsl:key name="all_attribute" match="attribute" use="@name"/>

  <xsl:template match="goal_model">
    <xsl:text>
      subgraph </xsl:text><xsl:value-of select="concat('cluster_',./@name)"/><xsl:text> {
        label = "</xsl:text><xsl:value-of select="."/><xsl:text>;
        color = tomato;</xsl:text>

        <xsl:variable name="all_entities" select='./entity[generate-id(.)=generate-id(key("all_entities",@name)[1])]/>
        <xsl:for-each select="$all_entities">
          <xsl:call-template name="create_class"><xsl:with-param name="topic" select="."/></xsl:call-template>
        </xsl:for-each>
        <xsl:apply-templates select="entity"/>
        <xsl:text>
        }
      </xsl:text>
    </xsl:template>

    <xsl:template match="entity">
      <xsl:variable name="entity_actual" select="."/>
      <xsl:variable name="children" select="./entity"/>
      <xsl:variable name="children1" select='./entity[generate-id(.)=generate-id(key("all_entities",@name)[1])]/> <!--
nÃo consigo fazer com que ele nÃo repita relaionamentos-->
      <xsl:for-each select="$children1">

```

```

        <xsl:call-template name="decomposition"><xsl:with-param name="source"
select="$entity_actual"/><xsl:with-param name="target" select="."/></xsl:call-template>
    </xsl:for-each>
    <xsl:apply-templates select="association"/>
    <xsl:apply-templates select="entity"/>
</xsl:template>

<xsl:template match="association">
    <xsl:variable name="entity_actual" select="."/>
    <xsl:variable name="parent" select="."/>
    <xsl:for-each select="$parent">
        <xsl:text>
            "</xsl:text><xsl:value-of select="."/><xsl:text>" -&gt; "</xsl:text><xsl:value-of
select="$entity_actual/@target"/>
            <xsl:text>" [arrowhead=normal, style=normal, label="</xsl:text><xsl:value-of
select="$entity_actual/@label"/><xsl:text>"];</xsl:text>
        </xsl:for-each>
    </xsl:template>

<xsl:template name="decomposition">
    <xsl:param name="source"/>
    <xsl:param name="target"/>
    <xsl:text>
        "</xsl:text><xsl:value-of select="$source/@name"/><xsl:text>" -&gt; "</xsl:text><xsl:value-of
select="$target/@name"/>
        <xsl:text>" [arrowhead=odot, style=normal];</xsl:text>
    </xsl:template>

<xsl:template name="create_class">
    <xsl:param name="topic"/>
    <xsl:text>
        "</xsl:text>
    <xsl:value-of select="$topic/@name"/>
    <xsl:text>" [label="</xsl:text>
    <xsl:value-of select="$topic/@name"/><xsl:text> | </xsl:text>
    <xsl:call-template name="create_attribute"><xsl:with-param name="topic" select="$topic"/></xsl:call-
template><xsl:text> "]" </xsl:text>
    </xsl:template>

<xsl:template name="create_attribute">
    <xsl:param name="topic"/>
    <xsl:variable name="son" select="//entity[@name=$topic/@name]/attribute"/>
    <xsl:variable name="all_attributes" select="$son[generate-id(.)=generate-id(key("all_attribute",@name)[1])]/>

    <xsl:for-each select="$all_attributes">
        <xsl:value-of select="."/><xsl:text> \n </xsl:text>
    </xsl:for-each>
</xsl:template>
</xsl:stylesheet>

```

Cenários

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet xmlns:xsl="http://www.w3.org/1999/XSL/Transform" version="1.0">
    <xsl:output method="html" indent="yes"/>
    <xsl:template match="/">
        <html>
            <head>
                <title>Aspect-oriented goal model - documentation</title>
            </head>
            <body>
                <font face="verdana">
                    <h1><p align="center">Scenarios</p> </h1>
                    <xsl:apply-templates select="aspect_oriented_model"/>
                </font>
            </body>
        </html>
    </xsl:template>

    <xsl:template match="aspect_oriented_model">
        <xsl:apply-templates select="goal_model"/>
    </xsl:template>

```



```

<xsl:template match="goal_model">
  <xsl:apply-templates select="goal | task | softgoal"/>
</xsl:template>

<xsl:template match="goal | task">

  <xsl:choose>
    <xsl:when test="count(/goal | /task | /softgoal | /goal_ref | /softgoal_ref | /task_ref)=0"></xsl:when>
    <xsl:otherwise>
      <xsl:call-template name="make_table">
        <xsl:with-param name="title" select="."/>
        <xsl:with-param name="actor" select="/new_element_type[@name='actor']"/>
        <xsl:with-param name="exception" select="/new_element_type[@name='exception']"/>
        <xsl:with-param name="constraint" select="/new_element_type[@name='constraint']"/>
        <xsl:with-param name="context" select="/new_element_type[@name='context']"/>
        <xsl:with-param name="resource" select="/new_element_type[@name='resource']"/>
        <xsl:with-param name="episode" select="/goal | /task | /softgoal | /goal_ref | /softgoal_ref |
./task_ref"/>
      </xsl:call-template>
      <br/>
    </xsl:otherwise>
  </xsl:choose>
  <xsl:apply-templates select="goal | task"/>

</xsl:template>

<xsl:template name="make_table">
  <xsl:param name="title"/>
  <xsl:param name="actor"/>
  <xsl:param name="exception"/>
  <xsl:param name="constraint"/>
  <xsl:param name="context"/>
  <xsl:param name="resource"/>
  <xsl:param name="episode"/>
  <xsl:variable name="title_id" select="$title/@id"/>
  <xsl:variable name="context_soft" select="//correlation_relationship[source/@id=$title_id]"/>

  <table border="0">
    <tr>
      <td bgcolor="lightgreen">Title</td>
      <td bgcolor="lightgreen">
        <a>
          <xsl:attribute name="name"><xsl:value-of select="$title/@id"/></xsl:attribute>
          <xsl:value-of select="$title/@name"/>
        </a>
      </td>
    </tr>
    <xsl:if test="count($actor)>0">
      <tr>
        <td bgcolor="lightgreen">Actor</td>
        <td><xsl:for-each select="$actor"><xsl:value-of select="@value"/>; </xsl:for-each></td>
      </tr>
    </xsl:if>
    <xsl:if test="count($resource)>0">
      <tr>
        <td bgcolor="lightgreen">Resource</td>
        <td><xsl:for-each select="$resource"><xsl:value-of select="@value"/>; </xsl:for-each></td>
      </tr>
    </xsl:if>
    <xsl:if test="count($context)>0 or count($context_soft)>0">
      <tr>
        <td bgcolor="lightgreen">Context</td>
        <td><xsl:if test="count($context)>0"><xsl:for-each select="$context"><xsl:value-of select="@value"/>;
</xsl:for-each></xsl:if>
          <xsl:if test="count($context_soft)>0"><xsl:for-each select="$context_soft"><xsl:call-template
name="get_name"><xsl:with-param name="id" select="/target/@id"/></xsl:call-template>; </xsl:for-each></xsl:if>
        </td>
      </tr>
    </xsl:if>
    <xsl:if test="count($constraint)>0">
      <tr>
        <td bgcolor="lightgreen">Constraint</td>
        <td><xsl:for-each select="$constraint"><xsl:value-of select="@value"/><br/> </xsl:for-each></td>
      </tr>
    </xsl:if>
    <xsl:if test="count($exception)>0">

```

```

<tr>
  <td bgcolor="lightgreen">Exception</td>
  <td><xsl:for-each select="$exception"><xsl:value-of select="@value"/><br/> </xsl:for-each></td>
</tr>
</xsl:if>

<tr>
  <td bgcolor="lightgreen">Episode</td>
  <td>
    <xsl:for-each select="$episode">
      <xsl:choose>
        <xsl:when test="name()='goal_ref' or name()='softgoal_ref' or name()='task_ref' or count(./goal
| ./softgoal | ./task | ./goal_ref | ./softgoal_ref | ./task_ref)>0">
          <xsl:choose>
            <xsl:when test="./@decomposition_label='or'">
              <xsl:text>[ </xsl:text><a><xsl:attribute name="href"><xsl:value-of
select="concat('#',@id)"/></xsl:attribute>
              <xsl:call-template name="get_name"><xsl:with-param name="id"
select="./@id"/></xsl:call-template>
              </a>
              <xsl:text> ]</xsl:text>
            </xsl:when>
            <xsl:otherwise>
              <a><xsl:attribute name="href"><xsl:value-of
select="concat('#',@id)"/></xsl:attribute>
              <xsl:call-template name="get_name"><xsl:with-param name="id"
select="./@id"/></xsl:call-template>
              </a>
            </xsl:otherwise>
          </xsl:choose>
        </xsl:when>
        <xsl:otherwise>
          <xsl:choose>
            <xsl:when test="./@decomposition_label='or'">
              <xsl:text>[ </xsl:text><xsl:value-of select="./@name"/><xsl:text> ]</xsl:text>
            </xsl:when>
            <xsl:otherwise><xsl:value-of select="./@name"/></xsl:otherwise>
          </xsl:choose>
        </xsl:otherwise>
      </xsl:choose>
    <br/></xsl:for-each></td>
</tr>
</table>
</xsl:template>

<xsl:template name="get_name">
  <xsl:param name="id"/>
  <xsl:value-of select="//goal[@id=$id]/@name | //softgoal[@id=$id]/@name | //task[@id=$id]/@name"/>
</xsl:template>
</xsl:stylesheet>

```

Apêndice C

Especificação do Estudo de Caso – C&L

Neste apêndice incluimos a modelagem realizada par o sistema C&L. Os documentos gerados são: lista de softmetas, metas e tarefas, cenários; diagrama de classes; e modelo entidade relacionamento. Estes documentos foram gerados a partir da descrição em XML com o apoio da implementação desenvolvida. No final deste apêndice incluímos a descrição do sistema C&L formatada de acordo com a BNF descrita no Apêndice A. Todos estes documentos retratam as informações contidas nos AOV-graph após a composição.

Lista de softmetas, metas e tarefas

1 Model: Model requirements (93)
1.1 Provide cooperative definition of Scenarios and Lexicon
1.1.1 Use [modeling tool] in the WEB
1.1.1.1 Manage [project]
1.1.1.1.1 Create [project]
1.1.1.1.1.1 Insert [project]
1.1.1.1.1.1.1 **Verify if [data identification] is unique**
1.1.1.1.1.1.1.2 **Verify if [enters] are valid**
1.1.1.1.1.1.3 **Verify if every [data] are filled up**
1.1.1.1.2 Exclude [project]
1.1.1.1.3 Save [project] as
1.1.1.1.4 Link [users] with [project]
1.1.1.1.5 Unlink [user] with [project]
1.1.1.1.6 Insert [user]
1.1.1.1.6.1 **Verify if [data identification] is unique**
1.1.1.1.6.2 **Verify if [enters] are valid**
1.1.1.1.6.3 **Verify if every [data] are filled up**
1.1.1.1.7 Export [project]
1.1.1.1.8 Import [project]
1.1.1.1.9 Persistence in [BD]
1.1.1.1.10 Restrict [access] to [data]
1.1.1.1.11 Restrict [access] to [functions]
1.1.1.1.12 Authentication by Login
1.1.1.1.13 Logout
1.1.1.2 Edit [requirement model]
1.1.1.2.1 Edit [scenarios]
1.1.1.2.1.1 Insert [scenario]
1.1.1.2.1.1.1 Add [title]
1.1.1.2.1.1.2 Add [goal]
1.1.1.2.1.1.3 Add [context]
1.1.1.2.1.1.4 Add [actor]
1.1.1.2.1.1.5 Add [resource]
1.1.1.2.1.1.6 Add [episode]
1.1.1.2.1.1.7 Add [exception]
1.1.1.2.1.1.8 Make references to other [scenario]
1.1.1.2.1.1.9 Make references in other [scenario]
1.1.1.2.1.1.10 Make references to [symbol]
1.1.1.2.1.1.11 **Verify if [data identification] is unique**
1.1.1.2.1.1.12 **Verify if [enters] are valid**
1.1.1.2.1.1.13 **Verify if every [data] are filled up**

1.1.1.2.1.2 Exclude [scenario]
1.1.1.2.1.2.1 Unmake references in other [scenario]
1.1.1.2.1.3 Change [scenario]
1.1.1.2.1.4 Search [scenario]
1.1.1.2.1.5 Search [scenario] with reference to current [scenario]
1.1.1.2.1.6 **Verify [scenario]**
1.1.1.2.1.7 **Verify [lexicon] and [scenario] in conjunction**
1.1.1.2.2 Edit [lexicon]
1.1.1.2.2.1 Insert [symbol]
1.1.1.2.2.1.1 Add [name]
1.1.1.2.2.1.2 Add [synonymous]
1.1.1.2.2.1.3 Add [notion]
1.1.1.2.2.1.4 Add [behavior response]
1.1.1.2.2.1.5 Add [type]
1.1.1.2.2.1.6 Make references to other [symbol]
1.1.1.2.2.1.7 Make references in other [symbol]
1.1.1.2.2.1.8 Make references in [scenario]
1.1.1.2.2.1.9 **Verify if [data identification] is unique**
1.1.1.2.2.1.10 **Verify if [enters] are valid**
1.1.1.2.2.1.11 **Verify if every [data] are filled up**
1.1.1.2.2.2 Exclude [symbol]
1.1.1.2.2.2.1 Unmake references in [scenario]
1.1.1.2.2.2.2 Unmake references in other [symbol]
1.1.1.2.2.3 Change [symbol]
1.1.1.2.2.4 Search [symbol]
1.1.1.2.2.5 Search [scenario] with reference to current [symbol]
1.1.1.2.2.6 Search [symbol] with reference to current [symbol]
1.1.1.2.2.7 **Verify [lexicon]**
1.1.1.2.2.8 **Verify [lexicon] and [scenario] in conjunction**
1.1.1.2.3 Create [ontology]
1.1.1.2.4 Verify [requirement model]
1.1.1.2.4.1 Verify [scenario]
1.1.1.2.4.1.1 Verify [syntax of scenario]
1.1.1.2.4.2 Verify [lexicon]
1.1.1.2.4.2.1 Verify [syntax of symbol]
1.1.1.2.4.2.2 Verify [circularity property]

1.1.1.2.4.2.3. Verify [minimal vocabulary property]
 1.1.1.2.4.3. Verify [lexicon] and [scenario] in conjunction
 1.1.1.2.5. Generate_graph
 1.1.1.2.6. **Persistence in [BD]**
 1.1.1.2.7. **Restrict [access] to [data]**
 1.1.1.2.8. **Restrict [access] to [functions]**
 1.1.1.2.9. **Authentication by Login**
 1.1.1.2.10. **Logout**
 1.1.1.3. Remember [password]
 1.1.1.3.1. **Persistence in [BD]**
 1.1.1.4. Register [user]
 1.1.1.4.1. **Persistence in [BD]**
 1.1.1.4.2. **Symmetric cryptography**
 1.1.1.5. Install [application]
 1.1.1.5.1. Set [configuration file]
 1.1.1.5.2. **Persistence in [file]**
 1.1.1.6. Uninstall [application]
 1.1.1.6.1. **Persistence in [file]**

2 Model: Persistence (39)

2.1 Persistence
 2.1.1 Persistence in [BD]
 2.1.1.1. Verify if [BD] is connected
 2.1.1.2. Initiate [BD]
 2.1.1.3. Connect [BD]
 2.1.1.3.1. **Restrict [access] to [data]**
 2.1.1.3.2. **Restrict [access] to [functions]**
 2.1.1.3.3. **Authentication by Login**
 2.1.1.3.4. **Logout**
 2.1.1.4. **Include [data]**
 2.1.1.5. **Select [data]**
 2.1.1.6. **Delete [data]**
 2.1.1.7. **Update [data]**
 2.1.1.8. Disconnect [BD]
 2.1.1.9. **Detect [persistence exception]**
 2.1.2 Persistence in [file]
 2.1.2.1. Verify if [file] exists
 2.1.2.2. Create [file]
 2.1.2.3. Open [file]
 2.1.2.4. **Include [data]**
 2.1.2.5. **Select [data]**
 2.1.2.6. **Delete [data]**
 2.1.2.7. **Update [data]**
 2.1.2.8. Close [file]
 2.1.2.9. Delete [file]
 2.1.2.10. **Detect [persistence exception]**
 2.1.3 Make register operation
 2.1.3.1. Include [data]
 2.1.3.2. Select [data]
 2.1.3.3. Delete [data]
 2.1.3.4. Update [data]

3 Model: Reliability (34)

3.1 Reliability

3.1.1 Robustness
 3.1.1.1 Robustness [incomplete data]
 3.1.1.1.1. Verify if every [data] are filled up
 3.1.1.1.1.1. **Detect [robustness exception]**
 3.1.1.1.1.2. **Detect [correctness exception]**
 3.1.1.2 Robustness [invalid data]
 3.1.1.2.1. Verify if [data identification] is unique
 3.1.1.2.1.1. **Detect [robustness exception]**
 3.1.1.2.1.2. **Detect [correctness exception]**
 3.1.1.2.2. Verify if [enters] are valid
 3.1.1.2.2.1. **Detect [robustness exception]**
 3.1.1.2.2.2. **Detect [correctness exception]**
 3.1.1.3 Robustness [unexpected action]
 3.1.2 Correctness
 3.1.2.1 Accuracy [data]
 3.1.2.1.1. **Verify if [data identification] is unique**
 3.1.2.1.2. **Verify if [enters] are valid**
 3.1.2.2 Completeness [data]
 3.1.2.2.1. **Verify if every [data] are filled up**
 3.2 Handling [exception]
 3.2.1 Detect [exception]
 3.2.1.1 Detect [persistence exception]
 3.2.1.2 Detect [robustness exception]
 3.2.1.3 Detect [correctness exception]
 3.2.2 Apply [dealer]

4 Model: Security (30)

4.1 Security
 4.1.1 Confidentiality
 4.1.2 Integrity
 4.1.3 Availability
 4.2 Authentication
 4.2.1 Authentication by Certification
 4.2.2 Authentication by Login
 4.2.2.1. **Symmetric cryptography**
 4.2.3 Validate [data]
 4.2.3.1 Compare [data] with [credentials]
 4.2.4 Authorization
 4.2.4.1 Select [permissions] to [data]
 4.2.4.2 Set [permissions]
 4.2.4.3 Control [access]
 4.2.4.3.1 Restrict [access] to [functions]
 4.2.4.3.2 Restrict [access] to [data]
 4.2.5 Accounting
 4.2.5.1 Measure [system time] used by [user]
 4.2.5.2 Measure [data] sent by [user]
 4.2.5.3 Measure [data] received by [user]
 4.2.6 Logout
 4.3 Cryptography
 4.3.1 Asymmetric cryptography
 4.3.2 Symmetric cryptography
 4.3.3 Make [cryptography operations]
 4.3.3.1 Encrypt [data]
 4.3.3.2 Decrypt [data]

Cenários

Title	Provide cooperative definition of Scenarios and Lexicon
-------	---

Episode	Use [modeling tool] in the WEB
---------	--

Title	Use [modeling tool] in the WEB
-------	--------------------------------

Episode	Manage [project]
	Edit [requirement model]
	Remember [password]
	Register [user]
	Uninstall [application]

Title	Create [project]
-------	------------------

Episode	Insert [project]
---------	----------------------------------

Title	Insert [project]
-------	------------------

Episode	Verify if [data identification] is unique
	Verify if [enters] are valid
	Verify if every [data] are filled up

Episode	Verify if every [data] are filled up
---------	--

Title	Manage [project]
-------	------------------

Episode	Create [project]
	Exclude [project]
	Save [project] as
	Link [users] with [project]

Unlink [user] with [project]
[Insert \[user\]](#)
 Export [project]
 Import [project]
[Persistence in \[BD\]](#)
[Restrict \[access\] to \[data\]](#)
[Restrict \[access\] to \[functions\]](#)
[Authentication by Login](#)
[\[Logout \]](#)

Title Insert [user]

Episode [Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Edit [requirement model]

[[Edit \[scenarios\]](#)]
 [[Edit \[lexicon\]](#)]
 [Create [onthology]]
 [[Verify \[requirement model\]](#)]
 Episode Generate_graph
[Persistence in \[BD\]](#)
[Restrict \[access\] to \[data\]](#)
[Restrict \[access\] to \[functions\]](#)
[Authentication by Login](#)
[\[Logout \]](#)

Title Edit [scenarios]

Episode [Insert \[scenario\]](#)
[Exclude \[scenario\]](#)
 Change [scenario]
 Search [scenario]
 Search [scenario] with reference to current [scenario]
[Verify \[scenario\]](#)
[Verify \[lexicon\] and \[scenario\] in conjunction](#)

Title Insert [scenario]

Add [title]
 Add [goal]
 Add [context]
 Add [actor]
 Add [resource]
 Add [episode]
 Episode Add [exception]
 Make references to other [scenario]
 Make references in other [scenario]
 Make references to [symbol]
[Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Exclude [scenario]

Episode Unmake references in other [scenario]

Title Edit [lexicon]

Episode [Insert \[symbol\]](#)
[Exclude \[symbol\]](#)
 Change [symbol]
 Search [symbol]
 Search [scenario] with reference to current [symbol]
 Search [symbol] with reference to current [symbol]
[Verify \[lexicon\]](#)
[Verify \[lexicon\] and \[scenario\] in conjunction](#)

Title Insert [symbol]

Episode Add [name]
 Add [synonymous]

Add [notion]
 Add [behavior response]
 Add [type]
 Make references to other [symbol]
 Make references in other [symbol]
 Make references in [scenario]
[Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Exclude [symbol]

Episode Unmake references in [scenario]
 Unmake references in other [symbol]

Title Verify [requirement model]

Episode [Verify \[scenario\]](#)
[Verify \[lexicon\]](#)
 Verify [lexicon] and [scenario] in conjunction

Title Verify [scenario]

Episode Verify [syntax of scenario]

Title Verify [lexicon]

Episode Verify [syntax of symbol]
 Verify [circularity property]
 Verify [minimal vocabulary property]

Title Remember [password]

Episode [Persistence in \[BD\]](#)

Title Register [user]

Episode [Persistence in \[BD\]](#)
[Symmetric cryptography](#)

Title Install [aplication]

Episode Set [configuration file]
[Persistence in \[file\]](#)

Title Uninstall [aplication]

Episode [Persistence in \[file\]](#)

Title Persistence

Episode [[Persistence in \[BD\]](#)]
 [[Persistence in \[file\]](#)]
[Make register operation](#)

Title Persistence in [BD]

Episode Verify if [BD] is connected
 Initiate [BD]
[Connect \[BD\]](#)
[Include \[data\]](#)
[Select \[data\]](#)
[Delete \[data\]](#)
[Update \[data\]](#)
 Disconnect [BD]
[Detect \[persistence exception\]](#)

Title Connect [BD]

Episode [Restrict \[access\] to \[data\]](#)
[Restrict \[access\] to \[functions\]](#)
[Authentication by Login](#)
[\[Logout \]](#)

Title Persistence in [file]

Episode	Verify if [file] exists
	Create [file]
	Open [file]
	Include [data]
	Select [data]
	Delete [data]
	Update [data]
	Close [file]
	Delete [file]
Episode	Detect [persistence exception]

Title Make register operation

Episode	[Include [data]]
	[Select [data]]
	[Delete [data]]
	[Update [data]]

Title Verify if every [data] are filled up

Exception Incomplete data

Episode	Detect [robustness exception]
	Detect [correctness exception]

Title Verify if [data identification] is unique

Exception Invalid data

Episode	Detect [robustness exception]
	Detect [correctness exception]

Title Verify if [enters] are valid

Exception Invalid data

Episode	Detect [robustness exception]
	Detect [correctness exception]

Title Handling [exception]

Context Robustness; Correctness;

Episode	Detect [exception]
	Apply [dealer]

Title Detect [exception]

Episode	Detect [persistence exception]
	Detect [robustness exception]
	Detect [correctness exception]

Title	Authentication
Context	Confidentiality;
Constraint	confidential data
Episode	[Authentication by Certification]
	[Authentication by Login]
	Validate [data]
	Authorization
	Accounting
	[Logout]

Title Authentication by Login

Episode [Symmetric cryptography](#)

Title Validate [data]

Episode Compare [data] with [credentials]

Title Authorization

Episode	Select [permissions] to [data]
	Set [permissions]
	Control [access]

Title Control [access]

Episode	Restrict [access] to [functions]
	Restrict [access] to [data]

Title Accounting

Episode	Measure [system time] used by [user]
	Measure [data] sent by [user]
	Measure [data] received by [user]

Title Cryptography

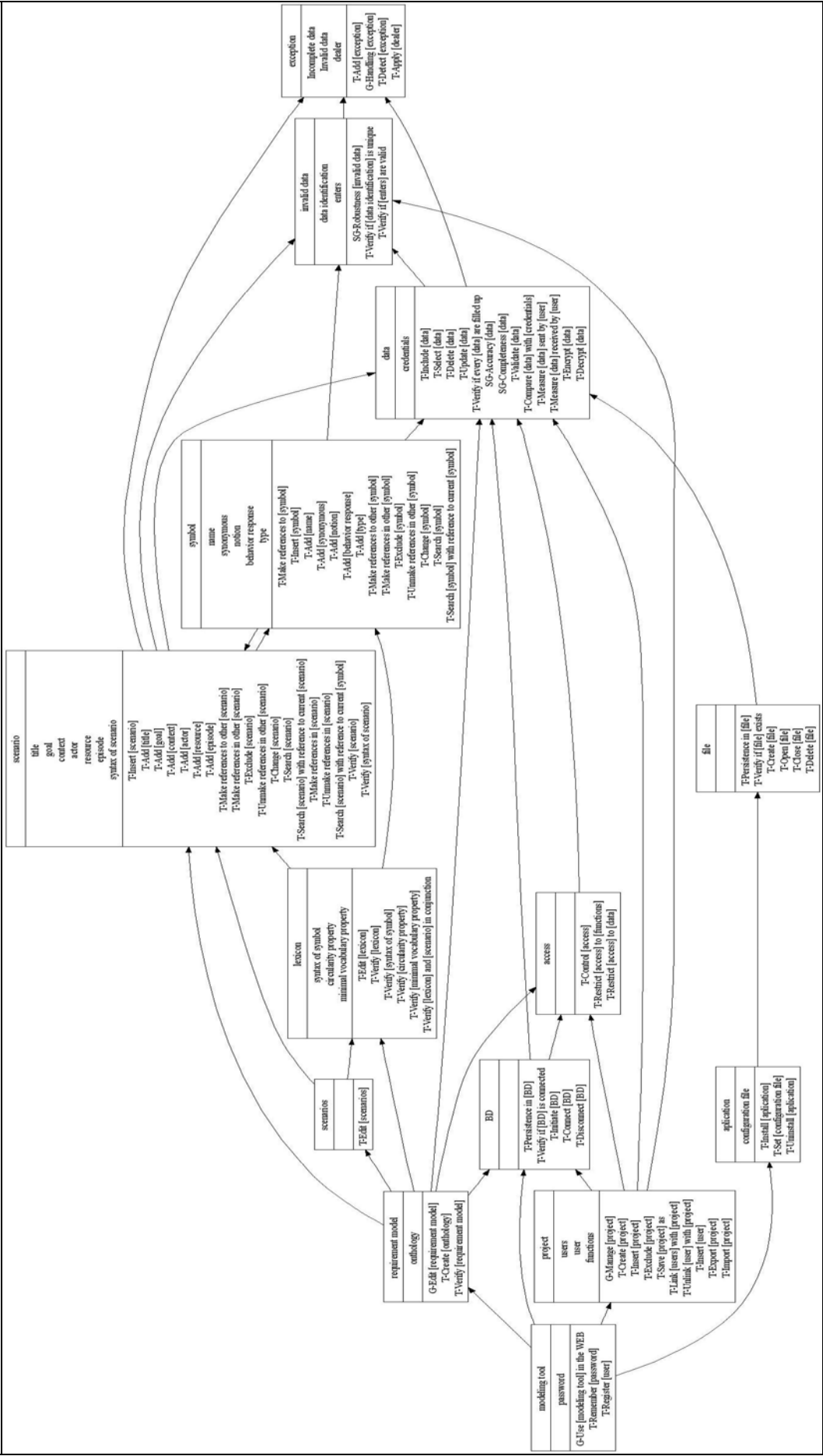
Context Confidentiality;

Episode	[Asymmetric cryptography]
	[Symmetric cryptography]
	Make [cryptography operations]

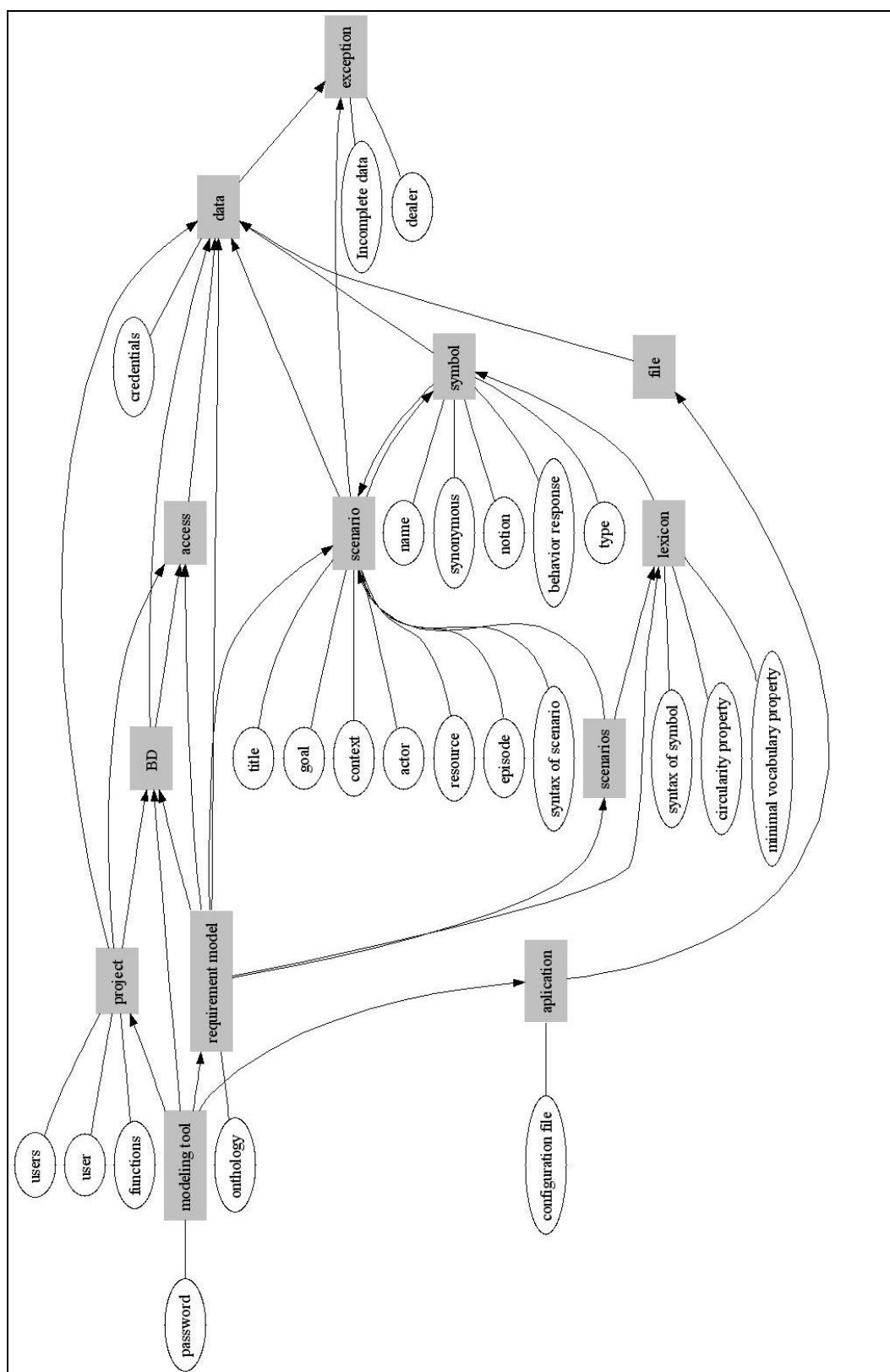
Title Make [cryptography operations]

Episode	Encrypt [data]
	Decrypt [data]

Diagrama de classes do C&L após a composição



Modelo de entidade relacionamento do C&L após a composição



BNF

```

goal_model (Model requirements; GM1) {
  goal = (Provide cooperative definition of Scenarios and Lexicon; G1; ) {
  goal = (Use [modeling tool] in the WEB; G2; ) {
  goal = (Manage [project]; G2.1; and) {
  task = (Create [project]; T2.1.1; and) {
  task = (Insert [project]; T2.1.1.2; ) {
  task_ref = (Verify if [data identification] is unique; T5.4; )
  task_ref = (Verify if [enters] are valid; T5.5; )
  task_ref = (Verify if every [data] are filled up; T5.6; )) }
  task = (Exclude [project]; T2.1.2; and) {}
  task = (Save [project] as; T2.1.3; and) {}
  task = (Link [users] with [project]; T2.1.4; and) {}
  task = (Unlink [user] with [project]; T2.1.5; and) {}
  task = (Insert [user]; T2.1.6; and) {
  task_ref = (Verify if [data identification] is unique; T5.4; )
  task_ref = (Verify if [enters] are valid; T5.5; )
  task_ref = (Verify if every [data] are filled up; T5.6; ))
  task = (Export [project]; T2.1.7; and) {}
  task = (Import [project]; T2.1.8; and) {}
  task_ref = (Persistence in [BD]; T15.1.1; )
  task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
  task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
  task_ref = (Authentication by Login; T9.3.3; )
  task_ref = (Logout Logout Logout Logout; T9.3.3.1; or)}
  goal = (Edit [requirement model]; G2.2; and) {
  task = (Edit [scenarios]; T2.2.1; or) {
  task = (Insert [scenario]; T2.2.1.2; ) {
  task = (Add [title]; T2.2.1.2.1; ) {}
  task = (Add [goal]; T2.2.1.2.2; ) {}
  task = (Add [context]; T2.2.1.2.3; ) {}
  task = (Add [actor]; T2.2.1.2.4; ) {}
  task = (Add [resource]; T2.2.1.2.5; ) {}
  task = (Add [episode]; T2.2.1.2.6; ) {}
  task = (Add [exception]; T2.2.1.2.7; ) {}
  task = (Make references to other [scenario]; T2.2.1.2.8; ) {}
  task = (Make references in other [scenario]; T2.2.1.2.9; ) {}
  task = (Make references to [symbol]; T2.2.1.2.10; ) {}
  task_ref = (Verify if [data identification] is unique; T5.4; )
  task_ref = (Verify if [enters] are valid; T5.5; )
  task_ref = (Verify if every [data] are filled up; T5.6; ))
  task = (Exclude [scenario]; T2.2.1.3; ) {
  task = (Unmake references in other [scenario]; T2.2.1.3.1; ) {} }
  task = (Change [scenario]; T2.2.1.4; ) {}
  task = (Search [scenario]; T2.2.1.5; ) {}
  task = (Search [scenario] with reference to current [scenario]; T2.2.1.6; ) {} }
  task = (Edit [lexicon]; T2.2.2; or) {
  task = (Insert [symbol]; T2.2.2.2; ) {
  task = (Add [name]; T2.2.2.2.1; ) {}
  task = (Add [synonymous]; T2.2.2.2.2; ) {}
  task = (Add [notion]; T2.2.2.2.3; ) {}
  task = (Add [behavior response]; T2.2.2.2.4; ) {}
  task = (Add [type]; T2.2.2.2.5; ) {}
  task = (Make references to other [symbol]; T2.2.2.2.6; ) {}
  task = (Make references in other [symbol]; T2.2.2.2.7; ) {}
  task = (Make references in [scenario]; T2.2.2.2.8; ) {}
  task_ref = (Verify if [data identification] is unique; T5.4; )
  task_ref = (Verify if [enters] are valid; T5.5; )
  task_ref = (Verify if every [data] are filled up; T5.6; ))
  task = (Exclude [symbol]; T2.2.2.3; ) {
  task = (Unmake references in [scenario]; T2.2.2.3.1; ) {}
  task = (Unmake references in other [symbol]; T2.2.2.3.2; ) {} }
  task = (Change [symbol]; T2.2.2.4; ) {}
  task = (Search [symbol]; T2.2.2.5; ) {}
  task = (Search [scenario] with reference to current [symbol]; T2.2.2.6; ) {}
  task = (Search [symbol] with reference to current [symbol]; T2.2.2.7; ) {} }
  task = (Create [onthology]; T2.2.3; or) {}
  task = (Verify [requirement model]; T2.2.4; or) {
  task = (Verify [scenario]; T2.2.4.1; ) {
  task = (Verify [syntax of scenario]; T2.2.4.1.1; ) {} }

```

```

task = (Verify [lexicon]; T2.2.4.2; ) {
task = (Verify [syntax of symbol]; T2.2.4.2.1; ) {}
task = (Verify [circularity property]; T2.2.4.2.2; ) {}
task = (Verify [minimal vocabulary property]; T2.2.4.2.3; ) {} }
task = (Verify [lexicon] and [scenario] in conjunction; T2.2.1.1; ) {}
crosscutting {source = Verify [requirement model](T2.2.4)
  pointcut (verify_scenario; PC1.2.2.4.1): (Edit [scenarios]; T2.2.1)
  pointcut (verify_lexicon; PC1.2.2.4.2): (Edit [lexicon]; T2.2.2)
  advice (around): PC1.2.2.4.1 {
    task_ref = (Verify [scenario]; T2.2.4.1; )
    task_ref = (Verify [lexicon] and [scenario] in conjunction; T2.2.1.1; )}
  advice (around): PC1.2.2.4.2 {
    task_ref = (Verify [lexicon]; T2.2.4.2; )
    task_ref = (Verify [lexicon] and [scenario] in conjunction; T2.2.1.1; )} } }
task = (Generate_graph; T2.2.5; and) {}
task_ref = (Persistence in [BD]; T15.1.1; )
task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
task_ref = (Authentication by Login; T9.3.3; )
task_ref = (Logout Logout Logout Logout; T9.3.3.1; or) }
task = (Remember [password]; T2.3; and) {
task_ref = (Persistence in [BD]; T15.1.1; )}
task = (Register [user]; T2.4; ) {
task_ref = (Persistence in [BD]; T15.1.1; )
task_ref = (Symmetric cryptography; T9.2.4; )}
task = (Install [aplication]; T2.5; ) {
task = (Set [configuration file]; T2.5.1; and) {}
task_ref = (Persistence in [file]; T15.1.2; )}
task = (Uninstall [aplication]; T2.6; ) {
task_ref = (Persistence in [file]; T15.1.2; )
} } } }

goal_model (Persistence; GM15) {
  goal = (Persistence; G15.1; ) {
    task = (Persistence in [BD]; T15.1.1; or) {
      task = (Verify if [BD] is connected; T15.1.1.1; ) {}
      task = (Initiate [BD]; T15.1.1.2; ) {}
      task = (Connect [BD]; T15.1.1.3; ) {
        task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
        task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
        task_ref = (Authentication by Login; T9.3.3; )
        task_ref = (Logout; T9.3.3.1; or) }
        task_ref = (Include [data]; T15.1.3.1; )
        task_ref = (Select [data]; T15.1.3.2; )
        task_ref = (Delete [data]; T15.1.3.3; )
        task_ref = (Update [data]; T15.1.3.4; )
        task = (Disconnect [BD]; T15.1.1.4; ) {}
        task_ref = (Detect [persistence exception]; T5.2.1.3; and)}
      task = (Persistence in [file]; T15.1.2; or) {
        task = (Verify if [file] exists; T15.1.2.1; ) {}
        task = (Create [file]; T15.1.2.2; ) {}
        task = (Open [file]; T15.1.2.3; ) {}
        task_ref = (Include [data]; T15.1.3.1; )
        task_ref = (Select [data]; T15.1.3.2; )
        task_ref = (Delete [data]; T15.1.3.3; )
        task_ref = (Update [data]; T15.1.3.4; )
        task = (Close [file]; T15.1.2.4; ) {}
        task = (Delete [file]; T15.1.2.5; ) {}
        task_ref = (Detect [persistence exception]; T5.2.1.3; and)}
      task = (Make register operation; T15.1.3; and) {
        task = (Include [data]; T15.1.3.1; or) {}
        task = (Select [data]; T15.1.3.2; or) {}
        task = (Delete [data]; T15.1.3.3; or) {}
        task = (Update [data]; T15.1.3.4; or) {} }
    crosscutting {source = Persistence(G15.1)
      pointcut (persistence_BD; PC15.2): include(Manage [project]; G2.1) and
      pointcut (persistence_file; PC15.3): include(Install [aplication]; T2.5) and include(Uninstall [aplication]; T2.6)
      advice (around): PC15.2 {
        task_ref = (Persistence in [BD]; T15.1.1; )}
      advice (around): PC15.3 {
        task_ref = (Persistence in [file]; T15.1.2; )} }
    crosscutting {source = Make register operation(T15.1.3)
      pointcut (register_operation; PC15.1): include(Connect [BD]; T15.1.1.3) and include(Open [file]; T15.1.2.3)
      advice (after): PC15.1 {
        task_ref = (Include [data]; T15.1.3.1; )
        task_ref = (Select [data]; T15.1.3.2; )

```

```

task_ref = (Delete [data]; T15.1.3.3; )
task_ref = (Update [data]; T15.1.3.4; ) } } }

goal_model (Reliability; GM5) {
  softgoal = (Reliability; S5.1; ) {
    softgoal = (Robustness; S5.1.1; help) {
      softgoal = (Robustness [incomplete data]; S5.1.1.1; make) {
        task = (Verify if every [data] are filled up; T5.6; make) {
          task_ref = (Detect [robustness exception]; T5.2.1.1; and)
          task_ref = (Detect [correctness exception]; T5.2.1.2; and)} }
        softgoal = (Robustness [invalid data]; S5.1.1.2; make) {
          task = (Verify if [data identification] is unique; T5.4; make) {
            task_ref = (Detect [robustness exception]; T5.2.1.1; and)
            task_ref = (Detect [correctness exception]; T5.2.1.2; and)} }
          task = (Verify if [enters] are valid; T5.5; make) {
            task_ref = (Detect [robustness exception]; T5.2.1.1; and)
            task_ref = (Detect [correctness exception]; T5.2.1.2; and)} }
          softgoal = (Robustness [unexpected action]; S5.1.1.3; make) {} }
          softgoal = (Correctness; S5.3; help) {
            softgoal = (Accuracy [data]; S5.3.1; ) {
              task_ref = (Verify if [data identification] is unique; T5.4; )
              task_ref = (Verify if [enters] are valid; T5.5; )
              softgoal = (Completeness [data]; S5.3.2; ) {
                task_ref = (Verify if every [data] are filled up; T5.6; ) } }
              goal = (Handling [exception] ; G5.2; ) {
                task = (Detect [exception]; T5.2.1; and) {
                  task = (Detect [persistence exception]; T5.2.1.3; and) {}
                  task = (Detect [persistence exception]; T5.2.1.3; and) {}
                  task = (Detect [robustness exception]; T5.2.1.1; and) {}
                  task = (Detect [correctness exception]; T5.2.1.2; and) {}
                  task = (Detect [robustness exception]; T5.2.1.1; and) {}
                  task = (Detect [correctness exception]; T5.2.1.2; and) {}
                  task = (Detect [robustness exception]; T5.2.1.1; and) {}
                  task = (Detect [correctness exception]; T5.2.1.2; and) {} } }
                task = (Apply [dealer]; T5.2.4; and) {} }
              crosscutting {source = Detect [exception](T5.2.1)
                pointcut (invalid_data; PC5.1): include(Verify if [data identification] is unique; T5.4) and include(Verify if
[enters] are valid; T5.5)
                pointcut (incomplete_data; PC5.2): include(Verify if every [data] are filled up; T5.6)
                pointcut (persistence; PC5.3): include(Persistence in [BD]; T15.1.1) and include(Persistence in [file]; T15.1.2)
                pointcut (insert_data; PC5.4): include(Insert.*; task; name)
                advice (around): PC5.4 {
                  task_ref = (Verify if [data identification] is unique; T5.4; )
                  task_ref = (Verify if [enters] are valid; T5.5; )
                  task_ref = (Verify if every [data] are filled up; T5.6; )
                }
                intertype declaration (element): PC5.3 {
                  task = (Detect [persistence exception]; T5.2.1.3; and) {} }
                intertype declaration (attribute): PC5.1 {exception = Invalid data; }
                intertype declaration (attribute): PC5.2 {exception = Incomplete data; }
                intertype declaration (element): PC5.1 and PC5.2 {
                  task = (Detect [robustness exception]; T5.2.1.1; and) {}
                  task = (Detect [correctness exception]; T5.2.1.2; and) {} } }
              correlation (help) {
                source = goal_ref = (Handling [exception] ; G5.2; )
                target = softgoal_ref = (Robustness; S5.1.1; )
              }
            }
          }
        }
      }
    }
  }
}

goal_model (Security; GM9) {
  softgoal = (Security; S9.0; ) {
    softgoal = (Confidentiality; S9.1; make) {}
    softgoal = (Integrity; S9.2; make) {}
    softgoal = (Availability; S9.3; make) {} }
  goal = (Authentication; G9.3; ) {
    task = (Authentication by Certification; T9.3.2; or) {}
    task = (Authentication by Login; T9.3.3; or) {
      task_ref = (Symmetric cryptography; T9.2.4; )
      task = (Validate [data]; T9.3.4; ) {
        task = (Compare [data] with [credentials]; T9.3.4.2; ) {} }
      goal = (Authorization; G9.3.5; ) {

```

```

task = (Select [permissions] to [data]; T9.3.5.1; ) {}
task = (Set [permissions]; T9.3.5.2; ) {}
task = (Control [access]; T9.3.5.3; ) {}
task = (Restrict [access] to [functions]; T9.3.5.3.1; ) {}
task = (Restrict [access] to [data]; T9.3.5.3.2; ) {}
crosscutting {source = Authentication(G9.3)
  pointcut (Authentication_login; PC9.3.1): include(Manage [project]; G2.1) and
  advice (around): PC9.3.1 {
    task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
    task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
    task_ref = (Authentication by Login; T9.3.3; )
  }
  intertype declaration (element): PC9.3.1 {
    task = (Logout; T9.3.3.1; or) {} }
  intertype declaration (element): PC9.3.1 {constraint = confidential data; } } }
goal = (Accounting; G9.3.6; ) {}
task = (Measure [system time] used by [user]; T9.3.6.1; ) {}
task = (Measure [data] sent by [user]; T9.3.6.2; ) {}
task = (Measure [data] received by [user]; T9.3.6.3; ) {} }
task = (Logout; T9.3.3.1; or) {}
task = (Logout; T9.3.3.1; or) {}
task = (Logout; T9.3.3.1; or) {}
correlation (make) {
  source = softgoal_ref = (Authentication; G9.3; )
  target = softgoal_ref = (Confidentiality; S9.1; ) }
task = (Cryptography; T9.2; ) {}
task = (Asymmetric cryptography; T9.2.3; or) {}
task = (Symmetric cryptography; T9.2.4; or) {}
task = (Make [cryptography operations]; T9.2.5; ) {
  task = (Encrypt [data]; T9.2.1; ) {}
  task = (Decrypt [data]; T9.2.2; ) {} }
correlation (help) {
  source = task_ref = (Cryptography; T9.2; )
  target = softgoal_ref = (Confidentiality; S9.1; ) }
crosscutting {source = Cryptography(T9.2)
  pointcut (Symmetric; PC9.3.5.1): include(Authentication by Login; T9.3.3) and include(Register [user]; T2.4)
  advice (around): PC9.3.5.1 {
    task_ref = (Symmetric cryptography; T9.2.4; ) } }
correlation (help) {
  source = softgoal_ref = (Confidentiality; S9.1; )
  target = goal_ref = (Use [modeling tool] in the WEB; G2; ) }

```

Apêndice D

Especificação do Estudo de Caso – Unical

Neste apêndice incluímos a modelagem realizada par o sistema Unical. Os documentos gerados são: lista de softmetas, metas e tarefas, cenários; diagrama de classes; e modelo entidade relacionamento. Estes documentos também foram gerados a partir da descrição em XML com o apoio da implementação desenvolvida. No final deste apêndice incluímos a descrição do sistema Unical formatada de acordo com a BNF descrita no Apêndice A. Todos estes documentos retratam as informações contidas nos AOV-graph após a composição.

Lista de softmetas, metas e tarefas

- | | |
|---|--|
| <ul style="list-style-type: none"> 1 Model: Unical (96) 1.1Adoption of system to divulge events 1.1.1.Manage [schedule] 1.1.1.1.Schedule [event] 1.1.1.1.1.Edit [event] 1.1.1.1.1.1.Add [event name] 1.1.1.1.1.2.Add [start time] 1.1.1.1.1.3.Add [end time] 1.1.1.1.1.4.Add [location] 1.1.1.1.1.4.1.Add [room] 1.1.1.1.1.4.2.Add [build] 1.1.1.1.1.4.3.Add [street] 1.1.1.1.1.4.4.Add [others] 1.1.1.1.1.5.Add [event description] 1.1.1.1.1.6.Set [recurrence] 1.1.1.1.1.6.1.Select [no recurrence] 1.1.1.1.1.6.2.Select [weekly recurrence] 1.1.1.1.1.6.3.Select [monthly recurrence] 1.1.1.1.1.6.4.Select [yearly recurrence] 1.1.1.1.1.7.Choose [category] 1.1.1.1.1.8.Set [reminder] 1.1.1.1.1.8.1.Select [0, 5, 10, 15 or 30 minutes] 1.1.1.1.1.8.2.Select [1 to 12, or 18 hours] 1.1.1.1.1.8.3.Select [1 to 6 days] 1.1.1.1.1.8.4.Select [1 to 2 weeks] 1.1.1.1.1.9.Verify if [data identification] is unique 1.1.1.1.1.10.Verify if [enters] are valid 1.1.1.1.1.11.Verify if every [data] are filled up 1.1.1.1.2.Set [event] to be viewed 1.1.1.1.2.1.Mark the [category name] are visible in the [calendar view window] 1.1.1.1.2.2.Unmark the [category name] are invisible in the [calendar view window] 1.1.1.1.3.View in [calendar window] 1.1.1.1.4.View in [monthly calendar] 1.1.1.1.5.View in [weekly calendar] 1.1.1.1.6.Set [default recurrence] 1.1.1.1.7.Set [default reminder] 1.1.1.2.Edit [category] 1.1.1.2.1.Add [category name] 1.1.1.2.2.Add [category description] | <ul style="list-style-type: none"> 1.1.1.2.3.Set [export flag] 1.1.1.2.3.1.Mark the [task] as completed in the [task list window] 1.1.1.2.3.2.Unmark the [task] as not completed in the [task list window] 1.1.1.2.4.Import [category] 1.1.1.2.4.1.Choose [category] from a list of all user categories 1.1.1.2.4.2.Edit imported [category] 1.1.1.2.4.2.1.Verify if [data identification] is unique 1.1.1.2.4.2.2.Verify if [enters] are valid 1.1.1.2.4.2.3.Verify if every [data] are filled up 1.1.1.2.4.3.Import [event] that is in imported [category] 1.1.1.2.4.3.1.Set [reminder] 1.1.1.2.5.View in [list window] 1.1.1.2.6.Set [color] 1.1.1.2.7.Set [default sorting] 1.1.1.2.8.Verify if [data identification] is unique 1.1.1.2.9.Verify if [enters] are valid 1.1.1.2.10.Verify if every [data] are filled up 1.1.1.3.Schedule [task] 1.1.1.3.1.Edit [task] 1.1.1.3.1.1.Add [task name] 1.1.1.3.1.2.Add [task description] 1.1.1.3.1.3.Add [due] 1.1.1.3.1.3.1.Add [date] 1.1.1.3.1.3.2.Add [time] 1.1.1.3.1.4.Set [reminder] 1.1.1.3.1.5.Verify if [data identification] is unique 1.1.1.3.1.6.Verify if [enters] are valid 1.1.1.3.1.7.Verify if every [data] are filled up 1.1.1.3.2.Set [task] to be viewed 1.1.1.3.2.1.Mark the [category name] to be exported 1.1.1.3.2.2.Unmark the [category name] to be not exported 1.1.1.3.3.View in [calendar window] 1.1.1.3.4.View in [monthly calendar] 1.1.1.3.5.View in [weekly calendar] 1.1.1.3.6.View in [list window] 1.1.1.3.7.Set [default recurrence] 1.1.1.3.8.Set [default reminder] |
|---|--|

- 1.1.1.4.Remind [event]
- 1.1.1.4.1.Verify if [time] and [date] are greater than [reminder] scheduled time
- 1.1.1.4.2.Play sound
- 1.1.1.4.3.Display [event] properties
- 1.1.1.4.3.1.**Use [reminder pop up window]**
- 1.1.1.5.Dismiss [reminder]
- 1.1.1.5.1.Stop sound
- 1.1.1.5.2.**Use [reminder pop up window]**
- 1.1.1.6.Snooze [reminder]
- 1.1.1.6.1.**Stop sound**
- 1.1.1.6.2.Reschedule [reminder] to 5 minutes later
- 1.1.1.6.3.**Use [reminder pop up window]**
- 1.1.1.7.Updating
- 1.1.1.7.1.Share [event]
- 1.1.1.7.1.1.Use [export flag]
- 1.1.1.7.2.Synchronyze [event]
- 1.1.1.8.**Persistence in [BD]**
- 1.1.1.9.**Restrict [access] to [data]**
- 1.1.1.10.**Restrict [access] to [functions]**
- 1.1.1.11.**Authentication by Login**
- 1.1.1.12.**Logout**

2 Model: Usability (58)

2.1Usability

- 2.1.1.Usability [documentation]
- 2.1.1.1.Usability [tutorial]
- 2.1.1.2.Usability [help] available on user interface
- 2.1.2.Usability [user interface]
- 2.1.2.1.Accessibility
- 2.1.2.1.1.Access [functionality] through [menu] itens
- 2.1.2.1.2.Access [functionality] with mouse
- 2.1.2.1.3.Access [functionality] with keyboard
- 2.1.2.2.Use [check box]
- 2.1.2.2.1.Mark the [category name] to be exported
- 2.1.2.2.2.Unmark the [category name] to be not exported
- 2.1.2.2.3.Mark the [category name] are visible in the [calendar view window]
- 2.1.2.2.4.Unmark the [category name] are invisible in the [calendar view window]
- 2.1.2.2.5.Mark the [task] as completed in the [task list window]
- 2.1.2.2.6.Unmark the [task] as not completed in the [task list window]
- 2.1.2.3.Use [pop up window]
- 2.1.2.3.1.Use [reminder pop up window]
- 2.1.2.3.2.Use [reminder pop up window]
- 2.1.2.3.3.Use [reminder pop up window]
- 2.1.2.3.4.**Resize [window]**
- 2.1.2.4.View in [calendar window]
- 2.1.2.4.1.View in [monthly calendar]
- 2.1.2.4.2.View in [weekly calendar]
- 2.1.2.5.View in [monthly calendar]
- 2.1.2.6.View in [weekly calendar]
- 2.1.2.7.View in [calendar window]
- 2.1.2.7.1.View in [monthly calendar]
- 2.1.2.7.2.View in [weekly calendar]
- 2.1.2.8.View in [monthly calendar]
- 2.1.2.9.View in [weekly calendar]
- 2.1.2.10.View in [list window]
- 2.1.2.11.View in [list window]
- 2.2Configurability
- 2.2.1.Resize [window]
- 2.2.1.1.**Resize [window]**
- 2.2.2.Set [color]
- 2.2.3.Set [default sorting]
- 2.2.4.Set [default recurrence]
- 2.2.5.Set [default reminder]
- 2.2.6.Set [default recurrence]
- 2.2.7.Set [default reminder]
- 2.2.8.**Persistence in [file]**

3 Model: Persistence (39)

3.1Persistence

- 3.1.1.Persistence in [BD]

- 3.1.1.1.Verify if [BD] is connected
- 3.1.1.2.Initiate [BD]
- 3.1.1.3.Connect [BD]
- 3.1.1.3.1.**Restrict [access] to [data]**
- 3.1.1.3.2.**Restrict [access] to [functions]**
- 3.1.1.3.3.**Authentication by Login**
- 3.1.1.3.4.**Logout**
- 3.1.1.4.**Include [data]**
- 3.1.1.5.**Select [data]**
- 3.1.1.6.**Delete [data]**
- 3.1.1.7.**Update [data]**
- 3.1.1.8.Disconnect [BD]
- 3.1.1.9.**Include [data]**
- 3.1.1.10.**Select [data]**
- 3.1.1.11.**Delete [data]**
- 3.1.1.12.**Update [data]**
- 3.1.1.13.**Detect [persistence exception]**
- 3.1.2.Persistence in [file]
- 3.1.2.1.Verify if [file] exists
- 3.1.2.2.Create [file]
- 3.1.2.3.Open [file]
- 3.1.2.4.**Include [data]**
- 3.1.2.5.**Select [data]**
- 3.1.2.6.**Delete [data]**
- 3.1.2.7.**Update [data]**
- 3.1.2.8.Close [file]
- 3.1.2.9.Delete [file]
- 3.1.2.10.**Include [data]**
- 3.1.2.11.**Select [data]**
- 3.1.2.12.**Delete [data]**
- 3.1.2.13.**Update [data]**
- 3.1.2.14.**Detect [persistence exception]**
- 3.1.3.Make register operation
- 3.1.3.1.Include [data]
- 3.1.3.2.Select [data]
- 3.1.3.3.Delete [data]
- 3.1.3.4.Update [data]

4 Model: Reliability (34)

4.1Reliability

- 4.1.1.Robustness
- 4.1.1.1.Robustness [incomplete data]
- 4.1.1.1.1.Verify if every [data] are filled up
- 4.1.1.1.1.1.**Detect [robustness exception]**
- 4.1.1.1.1.2.**Detect [correctness exception]**
- 4.1.1.2.Robustness [invalid data]
- 4.1.1.2.1.Verify if [data identification] is unique
- 4.1.1.2.1.1.**Detect [robustness exception]**
- 4.1.1.2.1.2.**Detect [correctness exception]**
- 4.1.1.2.2.Verify if [enters] are valid
- 4.1.1.2.2.1.**Detect [robustness exception]**
- 4.1.1.2.2.2.**Detect [correctness exception]**
- 4.1.1.3.Robustness [unexpected action]
- 4.1.2.Correctness
- 4.1.2.1.Accuracy [data]
- 4.1.2.1.1.**Verify if [data identification] is unique**
- 4.1.2.1.2.**Verify if [enters] are valid**
- 4.1.2.2.Completeness [data]
- 4.1.2.2.1.**Verify if every [data] are filled up**
- 4.2Handling [exception]
- 4.2.1.Detect [exception]
- 4.2.1.1.Detect [persistence exception]
- 4.2.1.2.Detect [persistence exception]
- 4.2.1.3.Detect [robustness exception]
- 4.2.1.4.Detect [correctness exception]
- 4.2.1.5.Detect [robustness exception]
- 4.2.1.6.Detect [correctness exception]
- 4.2.1.7.Detect [robustness exception]
- 4.2.1.8.Detect [correctness exception]
- 4.2.2.Apply [dealer]

5 Model: Security (53)

5.1Security

- 5.1.1.Confidentiality
- 5.1.2.Integrity
- 5.1.3.Availability

5.2 Authentication
 5.2.1 Authentication by Certification
 5.2.2 Authentication by Login
 5.2.3 Validate [data]
 5.2.3.1 Compare [data] with [credentials]
 5.2.4 Authorization
 5.2.4.1 Select [permissions] to [data]
 5.2.4.2 Set [permissions]
 5.2.4.3 Control [access]
 5.2.4.3.1 Restrict [access] to [functions]
 5.2.4.3.2 Restrict [access] to [data]
 5.2.5 Accounting
 5.2.5.1 Measure [system time] used by [user]
 5.2.5.2 Measure [data] sent by [user]
 5.2.5.3 Measure [data] received by [user]
 5.2.6 Manage [account]
 5.2.6.1 Edit [account]
 5.2.6.1.1 Edit [user account]
 5.2.6.1.1.1 **Verify if [data identification] is unique**
 5.2.6.1.1.2 **Verify if [enters] are valid**
 5.2.6.1.1.3 **Verify if every [data] are filled up**
 5.2.6.1.2 Edit [administrator account]
 5.2.6.1.2.1 Add [e-mail]
 5.2.6.1.2.2 **Verify if [data identification] is unique**
 5.2.6.1.2.3 **Verify if [enters] are valid**
 5.2.6.1.2.4 **Verify if every [data] are filled up**
 5.2.6.1.3 Add [username]
 5.2.6.1.4 Add [password]
 5.2.6.1.4.1 **Symmetric cryptography**
 5.2.6.1.5 Add [full name]
 5.2.6.1.6 Reset [password]
 5.2.6.1.6.1 **Symmetric cryptography**
 5.2.6.1.7 Change [password]
 5.2.6.1.7.1 **Symmetric cryptography**
 5.2.6.2 There is always at least one [administrator account]
 5.2.6.3 **Restrict [access] to [data]**
 5.2.6.4 **Restrict [access] to [functions]**
 5.2.6.5 **Authentication by Login**
 5.2.6.6 **Logout**
 5.2.7 Logout
 5.2.8 Logout
 5.2.9 Logout
 5.3 Cryptography
 5.3.1 Asymmetric cryptography

5.3.2 Symmetric cryptography
 5.3.3 Make [cryptography operations]
 5.3.3.1 Encrypt [data]
 5.3.3.2 Decrypt [data]

6 Model: Portability (7)
 6.1 Portability
 6.1.1 Portability [platform]
 6.1.1.1 Use [Microsoft Windows]
 6.1.1.2 Use [Macintosh OS]
 6.1.1.3 Use [PDA]
 6.1.1.4 Use [Web]
 6.1.1.5 Program with [Java TM (jdk 1.4)]

7 Model: Performance (3)
 7.1 Performance
 7.1.1 Performance [time]
 7.1.2 Performance [resource]

8 Model: Reusability (2)
 8.1 Reusability
 8.1.1 Reusability [system]

9 Model: Maintainability (6)
 9.1 Maintainability
 9.1.1 Reparability
 9.1.2 Evolvability
 9.2 Understandability
 9.2.1 Understandability [code]
 9.2.2 Understandability [documentation]

10 Model: Scalability (2)
 10.1 Scalability
 10.1.1 Scalability [quantity of users]

11 Model: Timeliness (5)
 11.1 Timeliness
 11.1.1 Timeliness [development time]
 11.1.1.1 Timeliness [design]
 11.1.1.2 Timeliness [implementation]
 11.1.1.3 Timeliness [test]

12 Model: Profitable (2)
 12.1 Profitable
 12.1.1 Profitable [development cost]

Cenários

Title	Adoption of system to divulge events
Episode	Manage [schedule]

Title	Manage [schedule]
Actor	User
Episode	Schedule [event] Edit [category] Schedule [task] Remind [event] Dismiss [reminder] Snooze [reminder] Updating Persistence in [BD] Restrict [access] to [data] Restrict [access] to [functions] Authentication by Login [Logout]

Title	Schedule [event]
Episode	Edit [event] [Set [event] to be viewed] View in [calendar window]

	View in [monthly calendar] View in [weekly calendar] Set [default recurrence] Set [default reminder]
--	---

Title	Add [location]
Constraint	performance: instantaneous
Episode	Add [room] Add [build] Add [street] Add [others]
Title	Edit [event]
Constraint	performance: instantaneous
Episode	Add [event name] Add [start time] Add [end time] Add [location] Add [event description] Set [recurrence] Choose [category] Set [reminder] Verify if [data identification] is unique

[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Set [recurrence]
Constraint performance: instantaneous
Episode [Select [no recurrence]]
 [Select [weekly recurrence]]
 [Select [monthly recurrence]]
 [Select [yearly recurrence]]

Title Set [reminder]
Constraint performance: instantaneous
Episode [Select [0, 5, 10, 15 or 30 minutes]]
 [Select [1 to 12, or 18 hours]]
 [Select [1 to 6 days]]
 [Select [1 to 2 weeks]]

Title Set [event] to be viewed
Constraint performance: instantaneous
Episode [Mark the \[category name\] are visible in the \[calendar view window\]](#)
[Unmark the \[category name\] are invisible in the \[calendar view window\]](#)

Title Edit [category]
Constraint performance: instantaneous
Episode Add [category name]
 Add [category description]
[Set \[export flag\]](#)
 [[Import \[category\]](#)]
[View in \[list window\]](#)
[Set \[color\]](#)
[Set \[default sorting\]](#)
[Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Set [export flag]
Constraint performance: instantaneous
Episode [Mark the \[task\] as completed in the \[task list window\]](#)
[Unmark the \[task\] as not completed in the \[task list window\]](#)

Title Import [category]
Constraint performance: instantaneous
Episode Choose [category] from a list of all user categories
 [[Edit imported \[category\]](#)]
 [[Import \[event\] that is in imported \[category\]](#)]

Title Edit imported [category]
Constraint performance: instantaneous
Episode [Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Import [event] that is in imported [category]
Constraint performance: instantaneous
Episode [[Set \[reminder\]](#)]

Title Schedule [task]
Episode [Edit \[task\]](#)
 [[Set \[task\] to be viewed](#)]

[View in \[calendar window\]](#)
[View in \[monthly calendar\]](#)
[View in \[weekly calendar\]](#)
[View in \[list window\]](#)
[Set \[default recurrence\]](#)
[Set \[default reminder\]](#)

Title Edit [task]
Constraint performance: instantaneous
Episode Add [task name]
 Add [task description]
[Add \[due\]](#)
[Set \[reminder\]](#)
[Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Add [due]
Constraint performance: instantaneous
Episode Add [date]
 Add [time]

Title Set [task] to be viewed
Constraint performance: instantaneous
Episode [Mark the \[category name\] to be exported](#)
[Unmark the \[category name\] to be not exported](#)

Title Remind [event]
Constraint performance: instantaneous
Episode Verify if [time] and [date] are greater than [reminder] scheduled time
 Play sound
[Display \[event\] properties](#)

Title Display [event] properties
Constraint performance: instantaneous
Episode [Use \[reminder pop up window\]](#)

Title Dismiss [reminder]
Constraint performance: instantaneous
Episode Stop sound
[Use \[reminder pop up window\]](#)

Title Snooze [reminder]
Constraint performance: instantaneous
Episode [Stop sound](#)
 Reschedule [reminder] to 5 minutes later
[Use \[reminder pop up window\]](#)

Title Updating
Constraint performance: 1 second to 50 events
Episode [Share \[event\]](#)
 Synchronyze [event]

Title Share [event]
Episode Use [export flag]

Title Use [check box]
Constraint performance: instantaneous
Episode Mark the [category name] to be exported
 Unmark the [category name] to be not exported
 Mark the [category name] are visible in the

	[calendar view window] Unmark the [category name] are invisible in the [calendar view window] Mark the [task] as completed in the [task list window] Unmark the [task] as not completed in the [task list window]
--	--

Title	Use [pop up window]
Constraint	performance: instantaneous
Episode	Use [reminder pop up window] Resize [window]

Title	View in [calendar window]
Episode	View in [monthly calendar] View in [weekly calendar]

Title	Resize [window]
Constraint	performance: instantaneous
Episode	Resize [window]

Title	Persistence
Episode	[Persistence in [BD]] [Persistence in [file]] Make register operation

Title	Persistence in [BD]
Constraint	performance: instantaneous
Episode	Verify if [BD] is connected Initiate [BD] Connect [BD] Include [data] Select [data] Delete [data] Update [data] Disconnect [BD] Detect [persistence exception]

Title	Connect [BD]
Constraint	performance: instantaneous
Episode	Restrict [access] to [data] Restrict [access] to [functions] Authentication by Login [Logout]

Title	Persistence in [file]
Constraint	performance: instantaneous
Episode	Verify if [file] exists Create [file] Open [file] Include [data] Select [data] Delete [data] Update [data] Close [file] Delete [file] Detect [persistence exception]

Title	Make register operation
Constraint	performance: instantaneous
Episode	[Include [data]] [Select [data]] [Delete [data]] [Update [data]]

Title	Verify if every [data] are filled up
-------	--------------------------------------

Constraint	performance: instantaneous
Exception	Incomplete data
Episode	Detect [robustness exception] Detect [correctness exception]

Title	Verify if [data identification] is unique
Constraint	performance: instantaneous
Exception	Invalid data
Episode	Detect [robustness exception] Detect [correctness exception]

Title	Verify if [enters] are valid
Constraint	performance: instantaneous
Exception	Invalid data
Episode	Detect [robustness exception] Detect [correctness exception]

Title	Handling [exception]
Context	Robustness; Correctness;
Episode	Detect [exception] Apply [dealer]

Title	Detect [exception]
Constraint	performance: instantaneous
Episode	Detect [persistence exception] Detect [robustness exception] Detect [correctness exception]

Title	Authentication
Context	Confidentiality;
Constraint	confidential data
Episode	[Authentication by Certification] [Authentication by Login] Validate [data] Authorization Accounting Manage [account] [Logout]

Title	Validate [data]
Constraint	performance: instantaneous
Episode	Compare [data] with [credentials]

Title	Authorization
Episode	Select [permissions] to [data] Set [permissions] Control [access]

Title	Control [access]
Constraint	performance: instantaneous
Episode	Restrict [access] to [functions] Restrict [access] to [data]

Title	Accounting
Episode	Measure [system time] used by [user] Measure [data] sent by [user] Measure [data] received by [user]

Title	Manage [account]
Actor	Administrator user
Episode	Edit [account] There is always at least one [administrator account]

[Restrict \[access\] to \[data\]](#)
[Restrict \[access\] to \[functions\]](#)
[Authentication by Login](#)
[\[Logout \]](#)

Title Edit [account]
 [[Edit \[user account\]](#)]
 [[Edit \[administrator account\]](#)]
 Add [username]
 Episode [Add \[password\]](#)
 Add [full name]
[Reset \[password\]](#)
[Change \[password\]](#)

Title Edit [user account]
 Constraint performance: instantaneous
 Episode [Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Edit [administrator account]
 Constraint performance: instantaneous
 Add [e-mail]
 Episode [Verify if \[data identification\] is unique](#)
[Verify if \[enters\] are valid](#)
[Verify if every \[data\] are filled up](#)

Title Add [password]

Constraint performance: instantaneous
 Episode [Symmetric cryptography](#)

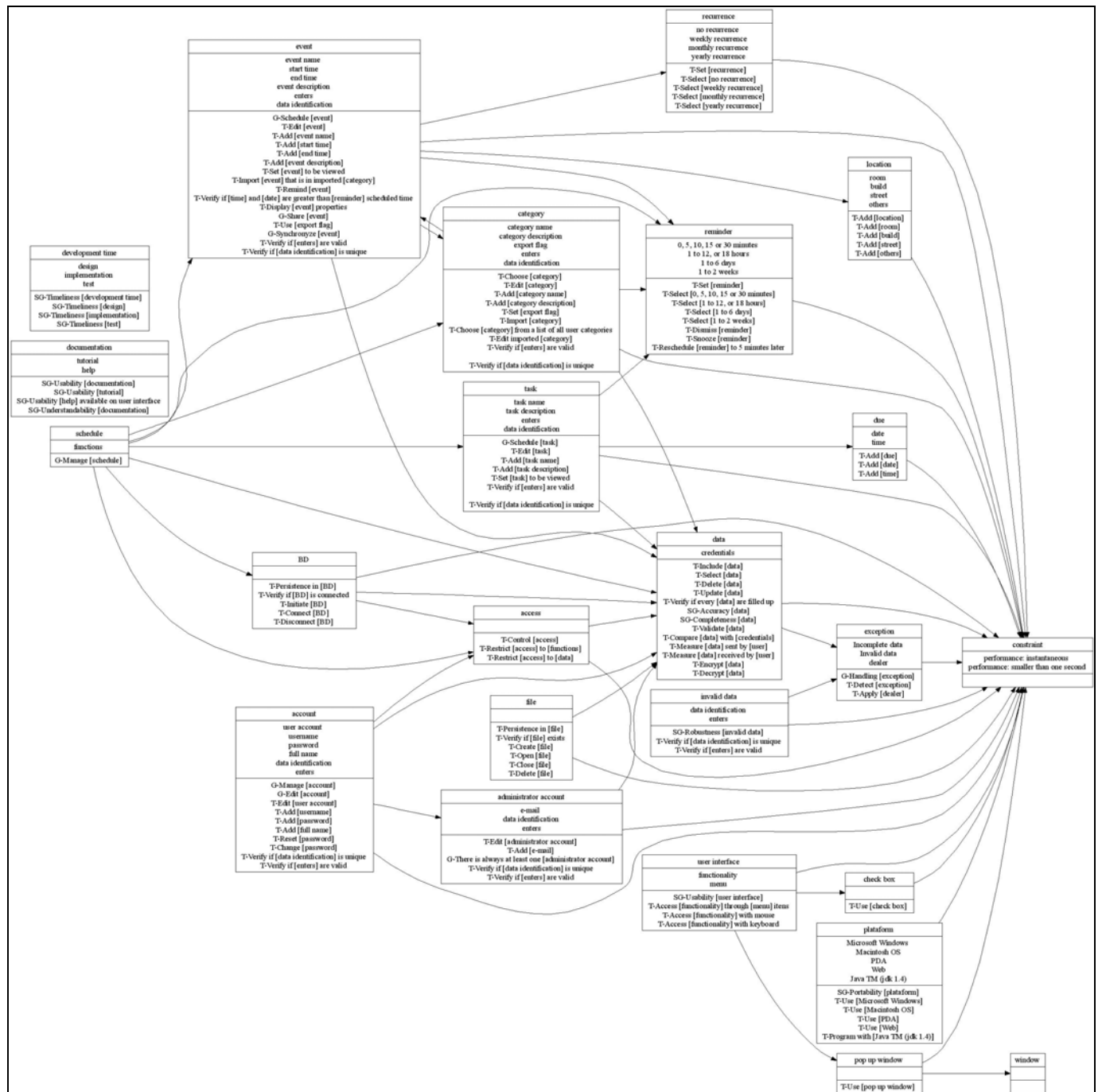
Title Reset [password]
 Constraint performance: instantaneous
 Episode [Symmetric cryptography](#)

Title Change [password]
 Constraint performance: instantaneous
 Episode [Symmetric cryptography](#)

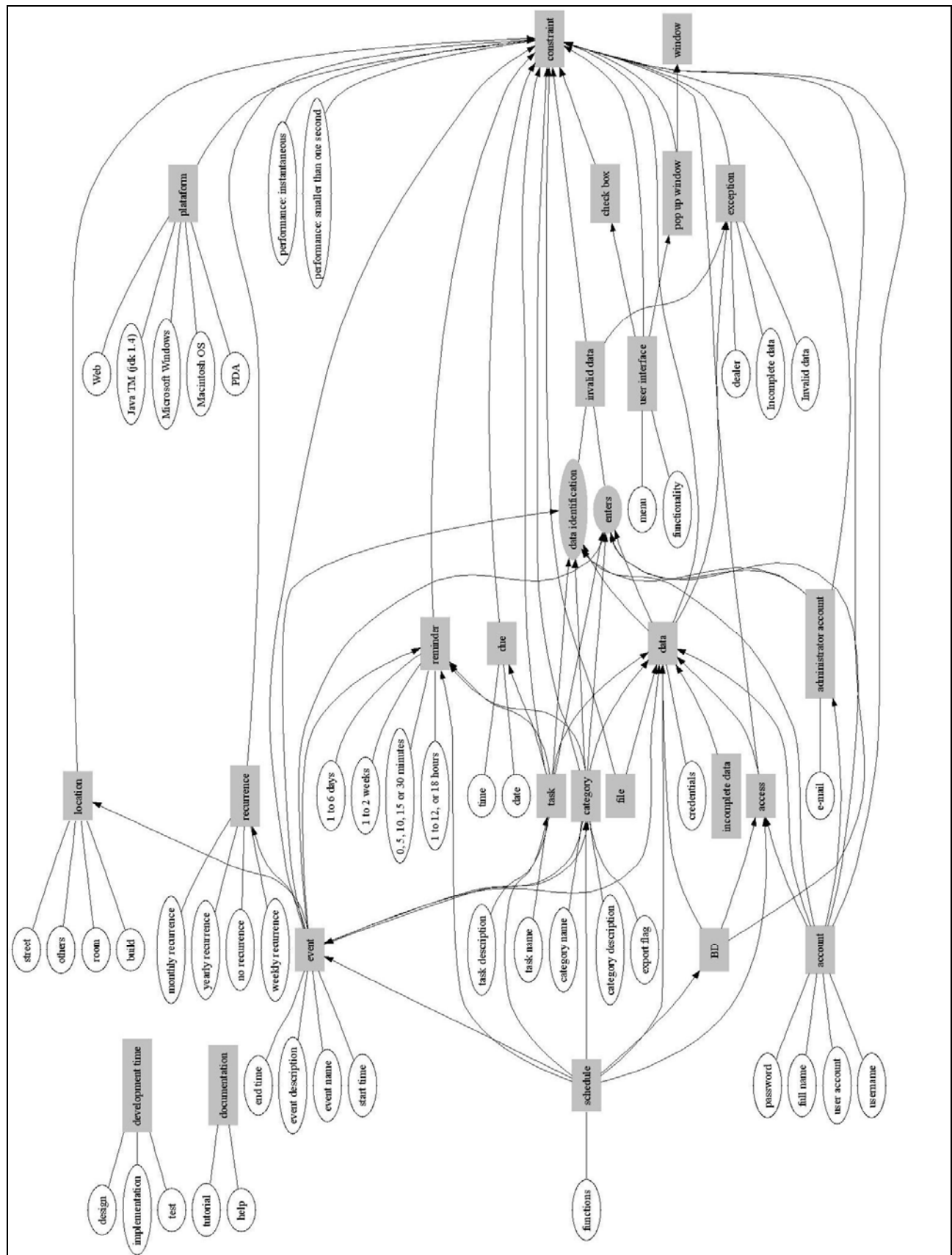
Title Cryptography
 Context Confidentiality;
 Constraint performance: instantaneous
 Episode [Asymmetric cryptography]
 [Symmetric cryptography]
[Make \[cryptography operations\]](#)

Title Make [cryptography operations]
 Constraint performance: instantaneous
 Episode Encrypt [data]
 Decrypt [data]

Diagrama de classes do Unical após a composição



Modelo de entidade-relacionamento do Unical após a composição



BNF

```
goal_model (Unical; GM1) {
  goal = (Adoption of system to divulge events; S1; ) {
    goal = (Manage [schedule]; G1.10; and) {
      goal = (Schedule [event]; T1.1.1; ) {
        task = (Edit [event]; T1.1.1.1; ) {
          task = (Add [event name]; T1.1.1.1.1; ) {}
          task = (Add [start time]; T1.1.1.1.2; ) {}
          task = (Add [end time]; T1.1.1.1.3; ) {}
          task = (Add [location]; T1.1.1.1.4; ) {
            task = (Add [room]; T1.1.1.1.4.1; ) {}
            task = (Add [build]; T1.1.1.1.4.2; ) {}
            task = (Add [street]; T1.1.1.1.4.3; ) {}
            task = (Add [others]; T1.1.1.1.4.4; ) {} }
          task = (Add [event description]; T1.1.1.1.5; ) {}
          task = (Set [recurrence]; T1.1.1.1.6; ) {
            task = (Select [no recurrence]; T1.1.1.1.6.1; or) {}
            task = (Select [weekly recurrence]; T1.1.1.1.6.2; or) {}
            task = (Select [monthly recurrence]; T1.1.1.1.6.3; or) {}
            task = (Select [yearly recurrence]; T1.1.1.1.6.4; or) {} }
          task = (Choose [category]; T1.1.1.1.7; and) {}
          task = (Set [reminder]; T1.1.1.1.8; and) {
            task = (Select [0, 5, 10, 15 or 30 minutes]; T1.1.1.1.8.1; or) {}
            task = (Select [1 to 12, or 18 hours]; T1.1.1.1.8.2; or) {}
            task = (Select [1 to 6 days]; T1.1.1.1.8.3; or) {}
            task = (Select [1 to 2 weeks]; T1.1.1.1.8.4; or) {} }
          task_ref = (Verify if [data identification] is unique; T5.4; )
          task_ref = (Verify if [enters] are valid; T5.5; )
          task_ref = (Verify if every [data] are filled up; T5.6; ))
          task = (Set [event] to be viewed; T1.1.1.1.9; or) {
            task_ref = (Mark the [category name] are visible in the [calendar view window]; T3.1.2.5.3; )
            task_ref = (Unmark the [category name] are invisible in the [calendar view window]; T3.1.2.5.4; ))
            task_ref = (View in [calendar window]; T3.1.2.5.7; )
            task_ref = (View in [monthly calendar]; T3.1.2.5.7.1; )
            task_ref = (View in [weekly calendar]; T3.1.2.5.7.2; )
            task_ref = (Set [default recurrence]; T3.1.2.5.11; )
            task_ref = (Set [default reminder]; T3.1.2.5.12; ))
            task = (Edit [category]; T1.1.5; ) {
              task = (Add [category name]; T1.1.5.1; and) {}
              task = (Add [category description]; T1.1.5.4; and) {}
              task = (Set [export flag]; T1.1.5.5; and) {
                task_ref = (Mark the [task] as completed in the [task list window]; T3.1.2.5.5; )
                task_ref = (Unmark the [task] as not completed in the [task list window]; T3.1.2.5.6; ))
                task = (Import [category]; T1.1.5.2; or) {
                  task = (Choose [category] from a list of all user categories; T1.1.1.5.2.1; ) {}
                  task = (Edit imported [category]; T1.1.1.5.2.2; or) {
                    task_ref = (Verify if [data identification] is unique; T5.4; )
                    task_ref = (Verify if [enters] are valid; T5.5; )
                    task_ref = (Verify if every [data] are filled up; T5.6; ))
                    task = (Import [event] that is in imported [category]; T1.1.1.5.2.3; or) {
                      task_ref = (Set [reminder]; T1.1.1.1.8; or) {} }
                      task_ref = (View in [list window] View in [list window] View in [list window]; T3.1.2.5.8; )
                      task_ref = (Set [color] Set [color]; T3.1.2.5.9; )
                      task_ref = (Set [default sorting] Set [default sorting]; T3.1.2.5.10; )
                      task_ref = (Verify if [data identification] is unique; T5.4; )
                      task_ref = (Verify if [enters] are valid; T5.5; )
                      task_ref = (Verify if every [data] are filled up; T5.6; ))
                    goal = (Schedule [task]; G1.2; ) {
                      task = (Edit [task]; T1.2.1; ) {
                        task = (Add [task name]; T1.2.1.1; ) {}
                        task = (Add [task description]; T1.2.1.2; ) {}
                        task = (Add [due]; T1.2.1.3; ) {
                          task = (Add [date]; T1.2.1.3.1; ) {}
                          task = (Add [time]; T1.2.1.3.2; ) {} }
                          task_ref = (Set [reminder]; T1.1.1.1.8; )
                          task_ref = (Verify if [data identification] is unique; T5.4; )
                          task_ref = (Verify if [enters] are valid; T5.5; )
                          task_ref = (Verify if every [data] are filled up; T5.6; ))
                        task = (Set [task] to be viewed; T1.2.2; or) {
                          task_ref = (Mark the [category name] to be exported; T3.1.2.5.1; )
```

```

task_ref = (Unmark the [category name] to be not exported; T3.1.2.5.2; )}
task_ref = (View in [calendar window]; T3.1.2.5.7; )
task_ref = (View in [monthly calendar]; T3.1.2.5.7.1; )
task_ref = (View in [weekly calendar]; T3.1.2.5.7.2; )
task_ref = (View in [list window]; T3.1.2.5.8; )
task_ref = (Set [default recurrence]; T3.1.2.5.11; )
task_ref = (Set [default reminder]; T3.1.2.5.12; )}
task = (Remind [event]; T1.3; ) {
task = (Verify if [time] and [date] are greater than [reminder] scheduled time; T1.3.3; ) {}
task = (Play sound; T1.3.1; ) {}
task = (Display [event] properties; T1.3.2; ) {
task_ref = (Use [reminder pop up window]; T3.1.2.6.1; )} }
task = (Dismiss [reminder]; T1.7; ) {
task = (Stop sound; T1.7.1; ) {}
task_ref = (Use [reminder pop up window]; T3.1.2.6.1; )}
task = (Snooze [reminder]; T1.8; ) {
task_ref = (Stop sound; T1.7.1; )
task = (Reschedule [reminder] to 5 minutes later; T1.8.1; ) {}
task_ref = (Use [reminder pop up window]; T3.1.2.6.1; )}
goal = (Updating; G1.9; ) {
goal = (Share [event]; G1.9.1; ) {
task = (Use [export flag]; T1.9.1.1; and) {} }
goal = (Synchronyze [event]; G1.9.2; ) {} }
task_ref = (Persistence in [BD]; T15.1.1; )
task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
task_ref = (Authentication by Login; T9.3.3; )
task_ref = (Logout Logout Logout; T9.3.3.1; or) } }

```

```

goal_model (Usability; GM3) {
softgoal = (Usability; S3.1; ) {
softgoal = (Usability [documentation]; S3.1.1; make) {
softgoal = (Usability [tutorial]; S3.1.1.1; make) {}
softgoal = (Usability [help] available on user interface; S3.1.1.2; make) {} }
softgoal = (Usability [user interface]; S3.1.2; make) {
softgoal = (Accessibility; S3.1.2.7; help) {
task = (Access [functionality] through [menu] itens; T3.1.2.1; make) {}
task = (Access [functionality] with mouse; T3.1.2.2; make) {}
task = (Access [functionality] with keyboard; T3.1.2.3; make) {} }
task = (Use [check box]; T3.1.2.5; make) {
task = (Mark the [category name] to be exported; T3.1.2.5.1; ) {}
task = (Unmark the [category name] to be not exported; T3.1.2.5.2; ) {}
task = (Mark the [category name] are visible in the [calendar view window]; T3.1.2.5.3; ) {}
task = (Unmark the [category name] are invisible in the [calendar view window]; T3.1.2.5.4; ) {}
task = (Mark the [task] as completed in the [task list window]; T3.1.2.5.5; ) {}
task = (Unmark the [task] as not completed in the [task list window]; T3.1.2.5.6; ) {} }
task = (Use [pop up window]; T3.1.2.6; make) {
task = (Use [reminder pop up window]; T3.1.2.6.1; ) {}
task = (Use [reminder pop up window]; T3.1.2.6.1; ) {}
task = (Use [reminder pop up window]; T3.1.2.6.1; ) {}
task_ref = (Resize [window]; T3.1.2.4; )}
task = (View in [calendar window]; T3.1.2.5.7; ) {
task = (View in [monthly calendar]; T3.1.2.5.7.1; ) {}
task = (View in [weekly calendar]; T3.1.2.5.7.2; ) {} }
task = (View in [list window]; T3.1.2.5.8; ) {}
task = (View in [list window]; T3.1.2.5.8; ) {}
crosscutting {source = Use [pop up window](T3.1.2.6)
pointcut (context_menu; PC3.3): include(Display [event] properties; T1.3.2) and
intertype declaration (element): PC3.3 {
task = (Use [reminder pop up window]; T3.1.2.6.1; ) {} } }
crosscutting {source = Use [check box](T3.1.2.5)
pointcut (check_box; PC3.2): include(Set [event] to be viewed; T1.1.1.1.9)
pointcut (check_box; PC3.5): include(Set [task] to be viewed; T1.2.2)
pointcut (check_box; PC3.4): include(Set [export flag]; T1.1.5.5)
intertype declaration (element): PC3.5 {
task = (Mark the [category name] to be exported; T3.1.2.5.1; ) {}
task = (Unmark the [category name] to be not exported; T3.1.2.5.2; ) {} }
intertype declaration (element): PC3.2 {
task = (Mark the [category name] are visible in the [calendar view window]; T3.1.2.5.3; ) {}
task = (Unmark the [category name] are invisible in the [calendar view window]; T3.1.2.5.4; ) {} }
intertype declaration (element): PC3.4 {
task = (Mark the [task] as completed in the [task list window]; T3.1.2.5.5; ) {}
task = (Unmark the [task] as not completed in the [task list window]; T3.1.2.5.6; ) {} } }
crosscutting {source = Usability [user interface](S3.1.2)
pointcut (view; PC3.6): include(Schedule [event]; T1.1.1)
pointcut (view; PC3.7): include(Edit [category]; T1.1.5)

```

```

pointcut (view; PC3.8): include(Schedule [task]; G1.2)
intertype declaration (element): PC3.6 and PC3.8 {
    task = (View in [calendar window]; T3.1.2.5.7; ) {
        task = (View in [monthly calendar]; T3.1.2.5.7.1; ) {}
        task = (View in [weekly calendar]; T3.1.2.5.7.2; ) {} } }
intertype declaration (element): PC3.7 and PC3.8 {
    task = (View in [list window]; T3.1.2.5.8; ) {} } }
crosscutting {source = Configurability(S3.2)
    pointcut (view; PC3.9): include(Schedule [event]; T1.1.1)
    pointcut (view; PC3.10): include(Edit [category]; T1.1.5)
    pointcut (view; PC3.11): include(Schedule [task]; G1.2)
    pointcut (window; PC3.12): include(*.window; task; name)
    advice (around): PC3.12 {
        task_ref = (Resize [window]; T3.1.2.4; )
    }
    intertype declaration (element): PC3.10 {
        task = (Set [color]; T3.1.2.5.9; ) {}
        task = (Set [default sorting]; T3.1.2.5.10; ) {} }
    intertype declaration (element): PC3.9 and PC3.11 {
        task = (Set [default recurrence]; T3.1.2.5.11; ) {}
        task = (Set [default reminder]; T3.1.2.5.12; ) {} } } }
softgoal = (Configurability; S3.2; make) {
    task = (Resize [window]; T3.1.2.4; make) {
        task_ref = (Resize [window]; T3.1.2.4; )
        task = (Set [color]; T3.1.2.5.9; ) {}
        task = (Set [default sorting]; T3.1.2.5.10; ) {}
        task = (Set [default recurrence]; T3.1.2.5.11; ) {}
        task = (Set [default reminder]; T3.1.2.5.12; ) {}
        task = (Set [default recurrence]; T3.1.2.5.11; ) {}
        task = (Set [default reminder]; T3.1.2.5.12; ) {}
        task_ref = (Persistence in [file]; T15.1.2; )
    }
    correlation (help) {
        source = softgoal_ref = (Configurability; S3.2; )
        target = softgoal_ref = (Usability; S3.1; ) } } }

goal_model (Persistence; GM15) {
    goal = (Persistence; G15.1; ) {
        task = (Persistence in [BD]; T15.1.1; or) {
            task = (Verify if [BD] is connected; T15.1.1.1; ) {}
            task = (Initiate [BD]; T15.1.1.2; ) {}
            task = (Connect [BD]; T15.1.1.3; ) {
                task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
                task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
                task_ref = (Authentication by Login; T9.3.3; )
                task_ref = (Logout Logout Logout Logout; T9.3.3.1; or)}
            task_ref = (Include [data]; T15.1.3.1; )
            task_ref = (Select [data]; T15.1.3.2; )
            task_ref = (Delete [data]; T15.1.3.3; )
            task_ref = (Update [data]; T15.1.3.4; )
            task = (Disconnect [BD]; T15.1.1.4; ) {}
            task_ref = (Detect [persistence exception]; T5.2.1.3; and)}
        task = (Persistence in [file]; T15.1.2; or) {
            task = (Verify if [file] exists; T15.1.2.1; ) {}
            task = (Create [file]; T15.1.2.2; ) {}
            task = (Open [file]; T15.1.2.3; ) {}
            task_ref = (Include [data]; T15.1.3.1; )
            task_ref = (Select [data]; T15.1.3.2; )
            task_ref = (Delete [data]; T15.1.3.3; )
            task_ref = (Update [data]; T15.1.3.4; )
            task = (Close [file]; T15.1.2.4; ) {}
            task = (Delete [file]; T15.1.2.5; ) {}
            task_ref = (Detect [persistence exception]; T5.2.1.3; and)}
        task = (Make register operation; T15.1.3; and) {
            task = (Include [data]; T15.1.3.1; or) {}
            task = (Select [data]; T15.1.3.2; or) {}
            task = (Delete [data]; T15.1.3.3; or) {}
            task = (Update [data]; T15.1.3.4; or) {} }
    crosscutting {source = Persistence(G15.1)
        pointcut (persistence_BD; PC15.2): include(Manage [schedule]; G1.10)
        pointcut (persistence_file; PC15.3): include(Configurability; S3.2)
        advice (around): PC15.2 {
            task_ref = (Persistence in [BD]; T15.1.1; )
        }
        advice (around): PC15.3 {
            task_ref = (Persistence in [file]; T15.1.2; ) }
    crosscutting {source = Make register operation(T15.1.3)
        pointcut (register_operation; PC15.1): include(Connect [BD]; T15.1.1.3) and include(Open [file]; T15.1.2.3)
        advice (after): PC15.1 {

```

```

    task_ref = (Include [data]; T15.1.3.1; )
    task_ref = (Select [data]; T15.1.3.2; )
    task_ref = (Delete [data]; T15.1.3.3; )
    task_ref = (Update [data]; T15.1.3.4; ) } } }

goal_model (Reliability; GM5) {
  softgoal = (Reliability; S5.1; ) {
    softgoal = (Robustness; S5.1.1; help) {
      softgoal = (Robustness [incomplete data]; S5.1.1.1; make) {
        task = (Verify if every [data] are filled up; T5.6; make) {
          task_ref = (Detect [robustness exception]; T5.2.1.1; and)
          task_ref = (Detect [correctness exception]; T5.2.1.2; and)} }
        softgoal = (Robustness [invalid data]; S5.1.1.2; make) {
          task = (Verify if [data identification] is unique; T5.4; make) {
            task_ref = (Detect [robustness exception]; T5.2.1.1; and)
            task_ref = (Detect [correctness exception]; T5.2.1.2; and)} }
          task = (Verify if [enters] are valid; T5.5; make) {
            task_ref = (Detect [robustness exception]; T5.2.1.1; and)
            task_ref = (Detect [correctness exception]; T5.2.1.2; and)} }
          softgoal = (Robustness [unexpected action]; S5.1.1.3; make) {} }
        softgoal = (Correctness; S5.3; help) {
          softgoal = (Accuracy [data]; S5.3.1; ) {
            task_ref = (Verify if [data identification] is unique; T5.4; )
            task_ref = (Verify if [enters] are valid; T5.5; )
            softgoal = (Completeness [data]; S5.3.2; ) {
              task_ref = (Verify if every [data] are filled up; T5.6; ) } }
          goal = (Handling [exception]; G5.2; ) {
            task = (Detect [exception]; T5.2.1; and) {
              task = (Detect [persistence exception]; T5.2.1.3; and) {}
              task = (Detect [persistence exception]; T5.2.1.3; and) {}
              task = (Detect [robustness exception]; T5.2.1.1; and) {}
              task = (Detect [correctness exception]; T5.2.1.2; and) {}
              task = (Detect [robustness exception]; T5.2.1.1; and) {}
              task = (Detect [correctness exception]; T5.2.1.2; and) {}
              task = (Detect [robustness exception]; T5.2.1.1; and) {}
              task = (Detect [correctness exception]; T5.2.1.2; and) {} }
            task = (Apply [dealer]; T5.2.4; and) {} }
          crosscutting {source = Detect [exception](T5.2.1)
            pointcut (invalid_data; PC5.1): include(Verify if [data identification] is unique; T5.4) and include(Verify if
[enters] are valid; T5.5)
            pointcut (incomplete_data; PC5.2): include(Verify if every [data] are filled up; T5.6)
            pointcut (persistence; PC5.3): include(Persistence in [BD]; T15.1.1) and include(Persistence in [file]; T15.1.2)
            pointcut (insert_data; PC5.4): include(Edit.*; task; name)
            advice (around): PC5.4 {
              task_ref = (Verify if [data identification] is unique; T5.4; )
              task_ref = (Verify if [enters] are valid; T5.5; )
              task_ref = (Verify if every [data] are filled up; T5.6; )
            }
            intertype declaration (element): PC5.3 {
              task = (Detect [persistence exception]; T5.2.1.3; and) {} }
            intertype declaration (attribute): PC5.1 {exception = Invalid data; }
            intertype declaration (attribute): PC5.2 {exception = Incomplete data; }
            intertype declaration (element): PC5.1 and PC5.2 {
              task = (Detect [robustness exception]; T5.2.1.1; and) {}
              task = (Detect [correctness exception]; T5.2.1.2; and) {} } }
          correlation (help) {
            source = goal_ref = (Handling [exception]; G5.2; )
            target = softgoal_ref = (Robustness; S5.1.1; )
          }
          correlation (help) {
            source = goal_ref = (Handling [exception]; G5.2; )
            target = softgoal_ref = (Correctness; S5.3; )
          }
          correlation (help) {
            source = softgoal_ref = (Reliability; S5.1; )
            target = goal_ref = (Adoption of system to divulge events; S1; )
          }
          correlation (help) {
            source = softgoal_ref = (Reliability; S5.1; )
            target = softgoal_ref = (Security; S9.0; ) } }
        }
      }
    }
  }
}

goal_model (Security; GM9) {
  softgoal = (Security; S9.0; ) {
    softgoal = (Confidentiality; S9.1; make) {}
    softgoal = (Integrity; S9.2; make) {}
    softgoal = (Availability; S9.3; make) {} }
  goal = (Authentication; G9.3; ) {
    task = (Authentication by Certification; T9.3.2; or) {}
    task = (Authentication by Login; T9.3.3; or) {}
    task = (Validate [data]; T9.3.4; ) {

```



```

task = (Compare [data] with [credentials]; T9.3.4.2; ) {} }
goal = (Authorization; G9.3.5; ) {
task = (Select [permissions] to [data]; T9.3.5.1; ) {}
task = (Set [permissions]; T9.3.5.2; ) {}
task = (Control [access]; T9.3.5.3; ) {
task = (Restrict [access] to [functions]; T9.3.5.3.1; ) {}
task = (Restrict [access] to [data]; T9.3.5.3.2; ) {}
crosscutting {source = Authentication(G9.3)
  pointcut (Authetication_login; PC9.3.1): include(Manage [account]; G9.4) and
  advice (around): PC9.3.1 {
    task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
    task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
    task_ref = (Authentication by Login; T9.3.3; )}
  intertype declaration (element): PC9.3.1 {
    task = (Logout; T9.3.3.1; or) {} }
  intertype declaration (element): PC9.3.1 {constraint = confidential data; } } }
goal = (Accounting; G9.3.6; ) {
task = (Measure [system time] used by [user]; T9.3.6.1; ) {}
task = (Measure [data] sent by [user]; T9.3.6.2; ) {}
task = (Measure [data] received by [user]; T9.3.6.3; ) {} }
goal = (Manage [account]; G9.4; ) {
goal = (Edit [account]; G9.4.1; ) {
task = (Edit [user account]; T9.4.1.1; or) {
task_ref = (Verify if [data identification] is unique; T5.4; )
task_ref = (Verify if [enters] are valid; T5.5; )
task_ref = (Verify if every [data] are filled up; T5.6; )}
task = (Edit [administrator account]; T9.4.1.2; or) {
task = (Add [e-mail]; T9.4.1.2.1; and) {}
task_ref = (Verify if [data identification] is unique; T5.4; )
task_ref = (Verify if [enters] are valid; T5.5; )
task_ref = (Verify if every [data] are filled up; T5.6; )}
task = (Add [username]; T9.4.1.3; and) {}
task = (Add [password]; T9.4.1.4; and) {
task_ref = (Symmetric cryptography; T9.2.4; )}
task = (Add [full name]; T9.4.1.5; and) {}
task = (Reset [password]; T9.4.1.6; and) {
task_ref = (Symmetric cryptography; T9.2.4; )}
task = (Change [password]; T9.4.1.7; and) {
task_ref = (Symmetric cryptography; T9.2.4; )} }
goal = (There is always at least one [administrator account]; G9.4.2; ) {}
task_ref = (Restrict [access] to [data]; T9.3.5.3.2; )
task_ref = (Restrict [access] to [functions]; T9.3.5.3.1; )
task_ref = (Authentication by Login; T9.3.3; )
task_ref = (Logout Logout Logout Logout; T9.3.3.1; or)}
task = (Logout; T9.3.3.1; or) {}
correlation (make) {
  source = softgoal_ref = (Authentication; G9.3; )
  target = softgoal_ref = (Confidentiality; S9.1; ) }
task = (Cryptography; T9.2; ) {
task = (Asymmetric cryptography; T9.2.3; or) {}
task = (Symmetric cryptography; T9.2.4; or) {}
task = (Make [cryptography operations]; T9.2.5; ) {
task = (Encrypt [data]; T9.2.1; ) {}
task = (Decrypt [data]; T9.2.2; ) {} }
correlation (help) {
  source = task_ref = (Cryptography; T9.2; )
  target = softgoal_ref = (Confidentiality; S9.1; )}
crosscutting {source = Cryptography(T9.2)
  pointcut (Symetric; PC9.3.5.1): include(. *password; task; name)
  advice (around): PC9.3.5.1 {
    task_ref = (Symmetric cryptography; T9.2.4; ) } }
correlation (help) {
  source = softgoal_ref = (Confidentiality; S9.1; )
  target = softgoal_ref = (Adoption of system to divulge events; S1; ) }

goal_model (Portability; GM2) {
  softgoal = (Portability; S2.1; ) {
  softgoal = (Portability [plataform]; S2.1.1; make) {
  task = (Use [Microsoft Windows]; T2.1.2; make) {}
  task = (Use [Macintosh OS]; T2.1.3; make) {}
  task = (Use [PDA]; T2.1.4; make) {}
  task = (Use [Web]; T2.1.5; make) {}
  task = (Program with [Java TM (jdk 1.4)]; T2.1.6; help) {} } }
  correlation (help) {
    source = softgoal_ref = (Portability [plataform]; S2.1.1; )
    target = softgoal_ref = (Adoption of system to divulge events; S1; )}

```

```

correlation (hurt) {
  source = softgoal_ref = (Portability [plataform]; S2.1.1; )
  target = softgoal_ref = (Reliability; S5.1; )}
correlation (hurt) {
  source = softgoal_ref = (Portability [plataform]; S2.1.1; )
  target = softgoal_ref = (Performance [time]; S6.1.1; )}
correlation (hurt) {
  source = softgoal_ref = (Portability [plataform]; S2.1.1; )
  target = softgoal_ref = (Security; S9.0; )} }

goal_model (Performance; GM6) {
  softgoal = (Performance; S6.1; ) {
    softgoal = (Performance [time]; S6.1.1; make) {}
    softgoal = (Performance [resource]; S6.1.2; make) {}
    crosscutting {source = Performance [time](S6.1.1)
      pointcut (performance_smallerthanone; PC6.1): include(Synchronyze [event]; G1.9.2)
      pointcut (performance_instantaneous; PC6.2): include(*; task; name)
      intertype declaration (attribute): PC6.1 {constraint = performance: smaller than one second; }
      intertype declaration (attribute): PC6.2 {constraint = performance: instantaneous; } } }
  correlation (help) {
    source = softgoal_ref = (Performance [time]; S6.1.1; )
    target = softgoal_ref = (Adoption of system to divulge events; S1; )} }

goal_model (Reusability; GM7) {
  softgoal = (Reusability; S7.1; ) {
    softgoal = (Reusability [system]; S7.1.1; make) {} }
  correlation (help) {
    source = softgoal_ref = (Reusability [system]; S7.1.1; )
    target = softgoal_ref = (Adoption of system to divulge events; S1; )} }

goal_model (Maintainability; GM11) {
  softgoal = (Maintainability; S11.1; ) {
    softgoal = (Reparability; S11.2; make) {}
    softgoal = (Evolvability; S11.3; make) {} }
  softgoal = (Understandability; S11.4; ) {
    softgoal = (Understandability [code]; S11.4.1; make) {}
    softgoal = (Understandability [documentation]; S11.4.2; make) {} }
  correlation (help) {
    source = softgoal_ref = (Understandability; S11.4; )
    target = softgoal_ref = (Reparability; S11.2; )}
  correlation (help) {
    source = softgoal_ref = (Understandability; S11.4; )
    target = softgoal_ref = (Evolvability; S11.3; )}
  correlation (help) {
    source = softgoal_ref = (Maintainability; S11.1; )
    target = softgoal_ref = (Reusability; S7.1; )}
  correlation (help) {
    source = softgoal_ref = (Maintainability; S11.1; )
    target = softgoal_ref = (Adoption of system to divulge events; S1; )} }

goal_model (Scalability; GM12) {
  softgoal = (Scalability; S12.1; ) {
    softgoal = (Scalability [quantity of users]; S12.1.1; make) {} }
  correlation (help) {
    source = softgoal_ref = (Scalability [quantity of users]; S12.1.1; )
    target = softgoal_ref = (Adoption of system to divulge events; S1; )} }

goal_model (Timeliness; GM14) {
  softgoal = (Timeliness; S14.1; ) {
    softgoal = (Timeliness [development time]; S14.1.1; make) {}
    softgoal = (Timeliness [design]; S14.1.1.1; make) {}
    softgoal = (Timeliness [implementation]; S14.1.1.2; make) {}
    softgoal = (Timeliness [test]; S14.1.1.3; make) {}
    correlation (help) {
      source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
      target = softgoal_ref = (Adoption of system to divulge events; S1; )}
    correlation (hurt) {
      source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
      target = softgoal_ref = (Reliability; S5.1; )}
    correlation (hurt) {
      source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
      target = softgoal_ref = (Security; S9.0; )}
    correlation (hurt) {
      source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
      target = softgoal_ref = (Usability; S3.1; )}
    correlation (hurt) {

```

```

        source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
        target = softgoal_ref = (Performance; S6.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
        target = softgoal_ref = (Reusability; S7.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
        target = softgoal_ref = (Maintainability; S11.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Timeliness [development time]; S14.1.1; )
        target = softgoal_ref = (Timeliness; S14.1; )} } }

goal_model (Profitable; GM4) {
    softgoal = (Profitable; S4.1; ) {
        softgoal = (Profitable [development cost]; S4.1.1; ) {}
    correlation (help) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Adoption of system to divulge events; S1; )}
    correlation (hurt) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Reliability; S5.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Security; S9.0; )}
    correlation (hurt) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Usability; S3.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Performance; S6.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Reusability; S7.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Maintainability; S11.1; )}
    correlation (hurt) {
        source = softgoal_ref = (Profitable [development cost]; S4.1.1; )
        target = softgoal_ref = (Timeliness; S14.1; )} } }

```

Anexo A

Documento de Requisitos do Sistema Unical

Requirements Document for UniCal

1 Introduction

Since the opening of the campus, the University of California, Irvine (UCI) has held many academic and non-academic events for its many students, faculty, staff, and visitors. Until now, UCI has informed people of these events using a haphazard collection of e-mails, web pages, flyers, and word of mouth. However, as the campus grows, it becomes increasingly more difficult to inform people of the many different UCI events. UCI now wishes to supplement this process by creating a university calendar system, called UniCal, to inform interested parties of campus events.

Because UCI is a research oriented institution, it does not want to burden its professors or students with production oriented software engineering projects. Consequently, it has hired Retro Software Inc. to develop this requirements specification for UniCal, as well as its subsequent design and implementation. The information in this requirements specification is based on interviews conducted with the relevant parties at UCI, by the requirements engineers at Retro Software Inc. All information in this document was determined to be accurate at the time of this writing.

This document contains the detailed requirements specification for UniCal that Retro Software Inc. is developing for UCI. The document should serve as the official basis for any further development of UniCal.

This document contains the following sections:

- Executive Summary
- Application Context
- Functional Requirements
- Environmental Requirements
- Software Qualities
- Other Requirements
- Time Schedule
- Potential Risks
- Future Changes
- Glossary
- References

2. Executive Summary

UniCal aims to provide a solution for scheduling campus events and publicizing their information to UCI faculty, staff, students, and visitors. The application allows customers to schedule, share, and view events via any computer running UniCal. UniCal facilitates communication by unifying and providing easy access to event information. Changes in events will be updated to users' schedules so that everybody is well informed. By maintaining consistent and reliable event information, UniCal enables effective event arrangement and collaboration among users. Retro Software anticipates that the overall effect of adopting UniCal will result in more accurate, efficient and convenient schedule coordinating among UCI personnel. UniCal provides the following key features:

- **Scheduling and sharing events** - A user can schedule events and group them into specific categories using UniCal. These events can be seen by other users of the system.
- **Viewing events** - A user can view all of the events in their calendar in either a weekly or monthly view.
- **Event reminders** - UniCal will provide reminders for a user by playing a sound and displaying a pop-up window with details of the event at a customizable time interval before each event.
- **Task list** - Users can also create a task list with deadlines and reminders to help them meet goals.

Some of the most important risks posed by the development of UniCal are the following:

- **Lack of adoption** - Because there are many other calendaring systems currently on the market, it is possible that students, faculty, and staff will not adopt UniCal into their normal routine. This would effectively render the system as no more useful than current approaches to informing people of events.

- **Usability** – To help encourage adoption of the system, UniCal must be extremely intuitive and easy to use.

- **Rapid development** – The schedule according to which UniCal will be developed is extremely aggressive to ensure that another company will not develop a similar product first and claim the market. Other qualities cannot be sacrificed as a result of this rapid development.

Although the contents of this document were thoroughly verified, it may still occur that some requirements are inconclusive or ambiguous. If it is so determined, it is requested that Retro Software Inc. be contacted via e-mail at shendric@uci.edu to resolve the issue.

3. Application Context

UniCal will be used in the institutional environment, UC, Irvine, where scheduling and coordinating events can be difficult because of the large numbers of people involved. The introduction of UniCal should reduce the effort required to coordinate schedules among multiple people by automating much of the effort of keeping everyone's schedule up to date. Adopting UniCal will result in substantial change for UCI. It will require that the application be installed on each person's computer, and that the application be used as the primary means of scheduling and coordinating events on campus.

Users of UniCal fall into two different roles: administrators, and users: Administrators of UniCal have all of the same functionality as a user, but in addition are responsible for managing the different accounts of the system. They may create, delete, and edit user accounts, as well as other administrator accounts. Should users forget their password, it is the responsibility of an administrator to reset their password so that they may once again gain access to the system. Should an administrator forget their password, they may have the password e-mailed to the e-mail address associated with their account.

Users of the UniCal system use the system to schedule and share events. Users may be faculty, staff, students, or even offices who wish to publicize office events. Users may create events, schedule reminders for events, specify recurrences for events, and group events together into categories. The categories that users create may be shared with other users of the system and included on their calendars. Users may also maintain a private task list of tasks that need to be completed. It is the hope of UCI that UniCal will eventually be used by other campuses. Such a change would require the application to be highly scalable and likely require instances of the UniCal system to interoperate since, due to political issues, it is unlikely that all campuses will share one instance.

4. Functional Requirements

4.1. ADMINISTRATOR FUNCTIONALITY

4.1.1. Administrators have the following functionality in addition to regular users (see 4.2) of the system

4.1.2. Login

4.1.2.1. The administrator should log in to UniCal using their username and password

4.1.2.2. The administrator password must be changed the first time the administrator logs in to the system

4.1.2.3. There should be one default administrator account with the username and password of 'admin' and 'admin' respectively. This account cannot be deleted

4.1.3. Adding/deleting/editing administrator accounts

4.1.3.1. An administrator should be able to add, delete, or edit other administrator accounts through the user interface

4.1.3.2. An administrator can enter the following information for each administrator. All information must be entered for an administrator account to be added to the system

4.1.3.2.1. **Username:** must be between 5 and 20 characters and should consist only of upper case, lower case, and numeric characters

4.1.3.2.2. **Password:** must be between 5 and 20 characters and should consist only of upper case, lower case, numeric, and punctuation characters

4.1.3.2.3. **Full Name:** must be less than 100 characters

4.1.3.2.4. **E-mail:** must conform to section 3.4 of RFC 2822 (<http://www.ietf.org/rfc/rfc2822.txt?number=2822>).

4.1.3.3. The administrator should be able to edit and save their password, full name, and e-mail address

4.1.3.4. An administrator should be able to reset the password of administrator accounts that they created or that were directly or indirectly created by accounts that they created

4.1.3.5. An administrator should be able to delete only administrator accounts that they created or that were directly or indirectly created by accounts that they created

4.1.4. Adding/deleting/editing user accounts

4.1.4.1. An administrator should be able to add, delete, or edit all user accounts through the user interface

4.1.4.2. An administrator can enter the following information for each user. All information must be entered for a user account to be added to the system

4.1.4.2.1. **Username:** see 4.1.3.2.1

4.1.4.2.2. **Password:** see 4.1.3.2.2

4.1.4.2.3. **Full Name:** must be less than 100 characters

4.1.4.3. All usernames in the system must be unique at all times

4.1.4.4. An administrator should be able to reset a user's password when requested by a user. However, an administrator should not be able to see any users current password

4.1.5. User Interface

4.1.5.1. UCI has few restrictions on the user interface for administrator functionality except that it should be functional and user friendly. The layout of the user interface for administrator functionality is left to the designers

4.2. USER FUNCTIONALITY

4.2.1. Login

4.2.1.1. A user logs in to UniCal using a username and password (see 4.1.4.2.1, 4.1.4.2.2)

4.2.1.2. The user password must be changed the first time a user logs in to the system

4.2.2. Adding/deleting/editing categories

4.2.2.1. A user should be able to add, delete, or edit categories through the user interface

4.2.2.2. A user can enter the following information for each category. All information must be entered for a category to be added to the users account

4.2.2.2.1. **Name:** a single line description used to identify the category (50 characters max)

4.2.2.2.2. **Color:** the color in which all events in the category are displayed

4.2.2.2.3. **Description:** a multi-line description of the category

4.2.2.3. All category names for a given user must be unique at all times.

4.2.2.3.1. If a change in the system will cause a category name to be identical to another category name, then the user should be notified of this and given an opportunity to change the name before continuing with the function

4.2.2.4. Once a category is created, events (see 4.2.5) may be added to the category

4.2.3. Importing categories

4.2.3.1. A user may include another users category (and the events therein) in their calendar by selecting it from a list of all user categories stored on UniCal

4.2.3.2. A user should be able to use a local name and color for an imported category, and set a local reminder (see 4.2.5.2.8) for each event (see 4.2.5) in an imported category

4.2.4. Updating

4.2.4.1. A user should be allowed to update their calendar so that their categories are available for others to import (see 4.2.3), and a user's imported categories are updated with changes made to the events in the imported category

4.2.4.2. Users may edit their calendar while not connected to the internet

4.2.5. Adding/deleting/editing events

4.2.5.1. A user should be able to add, delete, or edit events through the user interface

4.2.5.2. A user can enter the following information for each event. All information must entered for the event to be added to a category

4.2.5.2.1. **Name:** a single line description used to identify the event (50 characters max)

4.2.5.2.2. **Start time:** the starting date and time of the event formatted as (hours)/(minutes) (am or pm) (month)/(day)/(year) using two digit numbers except for hour and month, and four digit number for the year

4.2.5.2.3. **End time:** the starting time of the event formatted as (hours)/(minutes) (am or pm) (month)/(day)/(year) using two digit numbers except for hour and month, and four digit number for the year

4.2.5.2.4. **Location:** the location of the event (using previously entered values or entering a new value)

4.2.5.2.5. **Description:** a description of the event

4.2.5.2.6. **Recurrence:** the recurrence for the event set with respect to the start time of the event (see 4.2.7)

4.2.5.2.7. **Category:** the category that contains the event (see 4.2.2)

4.2.5.2.8. **Reminder:** the reminder for the event set with respect to the start time of the event (see 4.2.8)

4.2.5.3. An event must belong to exactly one category

4.2.5.4. The end time of the event must occur after the start time of the event

4.2.5.5. The start and end time may have a minimum granularity of 1 minute

4.2.5.6. Event names do not need to be unique

4.2.6. Adding/deleting/editing tasks

4.2.6.1. A user should be able to add, delete, and edit tasks through the user interface

4.2.6.2. A user can enter the following information for each task. All information must entered for the task to be added

4.2.6.2.1. **Name:** a single line description used to identify the task (50 characters max)

4.2.6.2.2. **Description:** a description of the task

4.2.6.2.3. **Due:** the date and time the task is due formatted as (hours)/(minutes) (am or pm) (month)/(day)/(year) using two digit numbers except for hour and month, and four digit number for the year

4.2.6.2.4. **Reminder:** the reminder for the task set with respect to when the task is due (see 4.2.8)

4.2.7. Setting a recurrence

4.2.7.1. When setting a recurrence, a user should be able to select any of the following recurrence settings provided by the user interface

- 4.2.7.1.1. **No recurrence:** the event only occurs once at the specified time
- 4.2.7.1.2. **Weekly recurrence:** the user may specify which days of the week for which the event reoccurs each week
- 4.2.7.1.3. **Monthly recurrence:** the user may specify which day of the month for which the event reoccurs each month
- 4.2.7.1.4. **Yearly recurrence:** the user may specify the day of the year for which the event reoccurs each year
- 4.2.7.2. The following functionality is required for events with recurrences
 - 4.2.7.2.1. The recurrences of an event should start on or after the start date of an event
 - 4.2.7.2.2. The user must specify an end date after which the event will not occur
 - 4.2.7.2.3. Where there are conflicts between an event's information and the event's recurrence, the recurrence settings should take precedence
- 4.2.8. Setting a reminder
 - 4.2.8.1. When setting a reminder, a user may schedule the reminder at any of the following time intervals before the event: 0, 5, 10, 15, or 30 minutes, 1 to 12, or 18 hours, 1 to 6 days, or 1 to 2 weeks
 - 4.2.8.2. If the reminder is set for something that has a recurrence, then the reminder will occur for each instance of the recurrence
 - 4.2.8.3. The reminder will be triggered when the system's time and date are greater than the reminders scheduled time, and the reminder has not been dismissed
 - 4.2.8.4. When a reminder is triggered, a popup window will be displayed with all event details and a sound will be played
 - 4.2.8.5. Dismissing a reminder
 - 4.2.8.5.1. When a reminder is triggered, a user may dismiss the reminder, in which case the reminder window closes
 - 4.2.8.6. Snoozing
 - 4.2.8.6.1. When a reminder is triggered, a user may press snooze, in which case the reminder window will close and the reminder will be reschedule to 5 minutes after the time of pressing snooze
- 4.2.9. User interface
 - 4.2.9.1. Appropriate menus and toolbars should be available to access all functionality using the mouse or keyboard
 - 4.2.9.2. There should be the following three parts to the user interface arranged from left to right: the category list (see 4.2.10), the calendar view (see 4.2.11), and the task list (see 4.2.14)
 - 4.2.9.3. The category list should initially utilize 10% of the screen
 - 4.2.9.4. The calendar view should initially utilize 80% of the screen
 - 4.2.9.5. The task list should initially utilize 10% of the screen
 - 4.2.9.6. A user should be able to resize the portion of the screen that each section utilizes
 - 4.2.10. Category list
 - 4.2.10.1. This displays a list of imported (see 4.2.3) and created (see 4.2.2) category names sorted alphabetically
 - 4.2.10.2. The list should display each category name to the right of a checkbox (which indicates whether the events of the category are visible in the calendar view (see 4.2.11)). The user should be able to check or uncheck this box to include or exclude the events from the calendar view respectively.
 - 4.2.10.3. The area behind each category name should be colored according to the category color (see 4.2.2.2.2)
 - 4.2.10.4. The user should be able to edit category properties via a context menu for each category in the list
 - 4.2.11. Calendar view
 - 4.2.11.1. This view should consist of a tabbed pane which includes a tab for a monthly calendar (see 4.2.12) and a tab for a weekly calendar (see 4.2.13). The tabs are located at the top left of the view.
 - 4.2.11.2. All calendar views should start on the same day of the week, either Sunday or Monday. The user should be able to set this in system preferences.
 - 4.2.11.3. A user should be able to enter a date, and have any view focus in on that date.
 - 4.2.12. Monthly calendar view
 - 4.2.12.1. This view should display 7 days by 5 weeks using equally sized boxes to represent the scheduled events for each day.
 - 4.2.12.2. The day of the month should be displayed in the top right corner of each box
 - 4.2.12.3. The box representing the first of any month should contain the name of the month in the top left corner and the four digit year between the month name and the day of month
 - 4.2.12.4. The name of each event for each day should be listed in that days box in order of the event starting time
 - 4.2.12.5. Events that span multiple days should be included in each day that they occur
 - 4.2.12.6. The background of each event should be colored according to the color of its respective category (see 4.2.2.2.2)
 - 4.2.12.7. Scrollbars for a given box should be included in the box if all information cannot fit within the box

4.2.12.8. Double clicking on the number of any day should change the current view to the weekly calendar (see 4.2.13). The weekly calendar should include the day that was double clicked

4.2.12.9. The current day (according to the system clock) should be highlighted with a red border at the edge of the box representing that day

4.2.12.10. A day may be selected using the mouse or the arrow keys of the keyboard, the current day is selected by default

4.2.12.11. Page-up will select the date that is one month prior to the currently selected day, Page-down will work similarly

4.2.12.12. The selected day is highlighted with a black border (within the current day highlight (see 4.2.12.9) if necessary)

4.2.12.13. Adding an event to the calendar when in this view should use the currently selected date as the default starting and ending date for the event.

4.2.13. Weekly calendar view

4.2.13.1. This view should display a grid where each day of the week is represented by 7 columns and the time of the day is indicated along the left hand side

4.2.13.2. The date for each day should be displayed at the top of each column and formatted as (Week Day), (Month)/(Day)/(Year) using a the full week day name for the week day, a number for the month, a two digit number for the day, and a four digit number for the year

4.2.13.3. Time increments for the left hand side should be selected from 5, 10, 15, 30 minutes, and 1 hour, the default should be 30 minute increments

4.2.13.4. Each event for a given day should be displayed in the following way

4.2.13.4.1. A 25% translucent rectangle should indicate the starting and ending time and day of each event by covering the area indicated by the start date and time and end date and time of the event. The color of the rectangle should be the color indicated by the events category

4.2.13.4.2. The rectangle should span multiple days for multi-day events

4.2.13.4.3. The name of the event should appear at the top left corner of the translucent rectangle

4.2.13.4.4. Conflicting events should simply overlap

4.2.13.5. The current day (according to the system clock) should be highlighted with a red border at the edge of column representing that day

4.2.13.6. An event or time range may be selected using the mouse or keyboard, the selected event or time range is highlighted with a black border

4.2.13.7. Page-up will display the previous week, Page-down will display the next week

4.2.13.8. Adding an event to the calendar when in this view should use the currently selected date and time range as the default starting and ending date and time for the event

4.2.14. Task list

4.2.14.1. The task list should display each task in descending order of due date

4.2.14.2. A user should be able to mark tasks as completed

4.2.14.3. Tasks that are marked as completed should disappear in 24 hours

4.3. MISCELLANEOUS FUNCTIONALITY

4.3.1. Additional user interface requirements

4.3.1.1. All functionality should be accessible through menu items

4.3.1.2. All functionality should be accessible with or without the use of a mouse

4.3.2. Help

4.3.2.1. UniCal should have help documentation provided through the UI that describes each function in this document including an example of how to perform each function

4.3.2.2. UniCal help documentation should also include a short tutorial on how UniCal is used

4.3.3. Error Handling

4.3.3.1. All error handling is left to the designers of the UniCal system

4.3.3.2. All error handling should allow a user to correct the error with minimal effort, and when possible, allowing a user to continue their previous function rather than starting over

5. Environmental Requirements

Since most people on the UCI campus use either Microsoft Windows or the Macintosh OS, UCI has decided that UniCal should be able to run on both platforms. UCI has also hired a consultant regarding programming languages, and, based on her report recommends that UniCal be implemented in Java™, to ensure portability across platforms as well as easy maintainability. Use of the JDK 1.4 is expected.

6. Other Requirements

Users should interface with UniCal through a stand alone application on their machines. The cost of development for the UniCal system must not exceed \$399,999.98. UCI has consulted with financial analysts to determine that this is the maximum amount the system can cost without becoming unprofitable. Retro Software Inc. should carefully document all decisions made, especially decisions pertaining to changes in this

document (and subsequent documents; always per agreement with UCI). Retro Software Inc. will deliver detailed user manuals for the UniCal system.

7. Software Qualities

- **User-friendliness** – Because many of UniCal’s users are expected to be people with minimal to no time to learn a complicated new system, it is imperative that the system be as user-friendly as possible.

- **Correctness** – Because UCI wants the system to be widely used, it is important that UniCal perform all of its requirements correctly and does not result in the proliferation of inaccurate or incomplete information.

- **Reliability** – Reliability of UniCal is not essential, but nonetheless important. The accepted rate of failure is one crash per month. Any failure rate above this will create an unfavorable impression of UCI towards Retro Software.

- **Performance** – Performance is crucial to UniCal – if the system is too slow, customers may revert to sending e-mails and using word of mouth. Synchronizing events with UniCal should take no more than one second for every fifty events. All other operations must be performed nearly instantaneously.

- **Reusability** – Reusability is important because it is hoped that UniCal will eventually be adopted at all other UC campuses.

- **Extensibility** – Several future enhancements for UniCal have already been proposed, and there will undoubtedly be more forthcoming. Therefore, it is critical that UniCal can be extended with relative ease.

- **Evolvability** – UniCal is expected to undergo significant changes as UCI determines which features are most important to its population. Evolvability is central to this goal.

- **Robustness** – UniCal must not crash if a user enters incorrect or invalid data, or performs some otherwise unexpected action. A reasonable response that allows the application to resume normal operation must be given.

- **Verifiability** – Although it is preferable that the system undergo formal, thorough verification and testing before deployment, this is not feasible with the rigorous time schedule desired by UCI. However, extensive testing of the accuracy of the system’s functionality should be performed before release.

- **Maintainability** – UCI anticipates that UniCal will be used over long periods of time, eventually by large numbers of users. Due to this fact, the likelihood of several future enhancements, and the high probability of an equally tight time schedule for their development, it is imperative that UniCal be easily maintainable.

- **Reparability** – Because UniCal is being developed under such a tight time schedule, it is likely that testing will not be done as thoroughly as desired, and some faults will make it into the product. Therefore, the system must be easily repairable in order to fix these defects in a timely manner so as not to disrupt the business of UCI.

- **Safety** – Since UniCal is not a safety-critical application, there are no safety concerns.

- **Portability** – Even though UniCal will be implemented entirely in Java™, a highly portable language, portability will still need to be considered in the design and implementation of the system. It is UCI’s desire that no features of UniCal will violate commonly accepted standards for each platform.

- **Understandability** – To support evolvability, reparability, and maintainability, it is imperative that all aspects of UniCal be easily understood, even to future developers who are not currently involved in the creation of the system.

- **Interoperability** – For this current set of requirements, the interoperability of UniCal with any other application is not needed. However, it is of prime importance that UniCal interoperate with other calendaring systems in future versions.

- **Productivity** – Because UCI would like to release the UniCal application as quickly as possible, it is desirable that the productivity of all involved in the development of UniCal be supported and facilitated with quality processes and tools as much as possible. However, limited funding is available for this requirement, so it must be foregone.

- **Size** – Hard drive space and memory are abundant nowadays – size is not an issue.

- **Scalability** – It is important that UniCal be able to scale to support the entire population of UCI, and perhaps, the UC system. Therefore, scalability is an important issue to consider in the development of UniCal.

- **Timeliness** – It is imperative that all artifacts of the UniCal system be delivered on time.

- **Visibility** – Due to the rigorous time schedule, extra time and effort to make the project status visible should be forfeited.

8. Time Schedule

The development schedule for UniCal is as follows:

- Design must be completed by **February 26th, 2004 at 2:00 PM.**

- Implementation and module testing must be completed by **March 9th, 2004 at 2:00 PM.**

- System testing must be completed by **March 18th, 2004 at 2:00 PM**, however this date may be changed.

9. Potential Risks

- **Difficult to use** – UniCal has a lot of features and addresses every kind of user. Furthermore, a significant portion of its expected user base is not computer literate. Hence, it is possible that some users might find it too difficult to use UniCal.

- **Limited schedule** – The schedule according to which UniCal is being developed is extremely aggressive and may result in a product that does not adequately satisfy the needs of UCI.

- **Lack of adequate support processes and tools** – A project of this size would ideally be supported by a number of quality software engineering processes and tools. Unfortunately, UCI lacks the funds necessary to obtain these tools.

- **Privacy issues** – As all categories will be viewable by every user, some users may feel uncomfortable letting other people see their (private) events. This could cause users to refrain from using UniCal.

10. Future Changes

- **Programmatic API for administrator functionality** – In future versions, UCI may want to be able to programmatically access administrator functionality so that the creation and removal of user accounts can be done automatically by computers processing incoming and outgoing students, staff, and faculty.

- **Web & PDA interface** – In future versions, UCI may want to create Web interfaces and PDA interfaces to supplement UniCal.

- **Permissions** – In future versions, UCI may want to introduce permissions to allow users of UniCal to keep certain events private.

- **Interface with other calendaring applications** – In future versions, UCI may want UniCal to interface with other calendaring applications. Minimally this will involve sharing event information such as through importing and exporting .vcal files (see <http://www.imc.org/pdi/vcal-10.txt>). However, support of more automated techniques such as scheduling meetings may also be desired.

- **Scheduling meetings** – In future versions, UCI may want to have UniCal facilitate the process of scheduling meetings, reserving rooms, and request RSVP information.

11. Glossary

- **Administrator** – One of the two roles in the UniCal System; administrators have the ability to add, delete, and modify user accounts.

- **Category** – A grouping of events into a common theme

- **Event** – A social gathering or activity that the system will help to coordinate.

- **Java™** – A highly portable, general purpose programming language developed by Sun Microsystems in the early- to mid-1990's.

- **UCI** – University of California, Irvine. The client company contracting the development of the UniCal system.

- **User** – One of the two roles in the UniCal system; users are ordinary people who wish to schedule, share, and view different university events.

12. References

- <http://www.uci.edu>

- <http://www.java.sun.com>

- <http://www.dictionary.com>

- <http://www.imc.org/pdi/vcal-10.txt>

- <http://www.ietf.org/rfc/rfc2822.txt?number=2822>